

Python para finanzas y trading algorítmico

Aprendizaje automático, aprendizaje profundo, análisis de series temporales, gestión de riesgos y carteras para MetaTrader 5 Live Trading

Lucas Inglese

Traducido por Juan Gabriel Gomila y María Santos

incluye notebooks



incluye +10 estrategias listas para usar



Incluye curso online de regalo



2ª edición

Python para finanzas y trading algorítmico

Lucas Inglese

**Traducido por Juan Gabriel Gomila
y María Santos**

Descargo de responsabilidad: No estamos autorizados por ninguna autoridad financiera para dar consejos de inversión. Este libro tiene únicamente fines educativos. Renunciamos a toda responsabilidad por cualquier pérdida de capital por tu parte. Además, el 78.18% de los inversores particulares pierden dinero operando con CFD. El uso de la información y las instrucciones de esta obra es bajo tu propio riesgo. Cualquier muestra de código u otras tecnologías que esta obra contiene o describe están sujetas a licencias de código abierto o a derechos de propiedad intelectual de terceros. En ese caso, tu responsabilidad es asegurarte de que su uso cumple con dichas licencias y derechos. Este libro no pretende ser un asesoramiento financiero. Consulta a un profesional cualificado si necesitas asesoramiento financiero. Los resultados pasados no son indicativos de los resultados futuros.

ADVERTENCIAS: Cualquier reproducción parcial o total sin el consentimiento del autor puede dar lugar a acciones legales.

¿Tienes alguna pregunta para el autor? Sección 1.3
["Únete a la comunidad"](#)

Tabla de Contenido

¿POR QUÉ DEBERÍAS LEER ESTE LIBRO?

¿QUIENES SOMOS?

CAPÍTULO 1: EMPIEZA POR AQUÍ

1.1. ENCUENTRA EL CÓDIGO

1.2. CONVENCIONES UTILIZADAS EN ESTE LIBRO

1.3. ÚNETE A LA COMUNIDAD

CAPÍTULO 2: PRERREQUISITOS

2.1. PRERREQUISITOS MATEMÁTICOS

2.1.1. Álgebra

2.1.2. Estadística

2.1.3. Optimización

2.2. PRERREQUISITOS EN FINANZAS

2.2.1. Eficiencia del mercado

2.2.2. Fundamentos del trading

2.2.3. Fundamentos de la gestión de carteras

2.3. PRERREQUISITOS EN PYTHON

2.3.1. Librerías para data science

2.3.2. Librerías para finanzas

2.3.3. Librerías para matemáticas

PARTE 1: GESTIÓN DE CARTERAS, GESTIÓN DE RIESGOS Y BACKTESTING

CAPÍTULO 3: OPTIMIZACIÓN DE LA CARTERA ESTÁTICA

3.1. EXPLICACIÓN DEL MÉTODO DE OPTIMIZACIÓN DE CARTERAS

3.1.1. Riesgo sistémico y riesgo específico

3.1.2. Diversificación

3.2. LOS MÉTODOS TRADICIONALES DE OPTIMIZACIÓN DE CARTERAS

3.2.1. Función de utilidad de la cartera

3.2.2. Criterio de la media-varianza

3.2.3. Criterio de la media-varianza-asimetría-curtosis

3.3. LOS MÉTODOS MODERNOS DE OPTIMIZACIÓN DE CARTERAS

3.3.1. Criterio de Sharpe

3.3.2. Criterio de Sortino

CAPÍTULO 4: GESTIÓN TÁCTICA DE LA CARTERA

4.1. ¿CÓMO FUNCIONAN LOS MÉTODOS DINÁMICOS?

4.1.1. Venta en corto

4.1.2. Factor de impulso

4.1.3. Reequilibrio

4.2. ESTRATEGIA DE MEDIA MÓVIL

4.2.1. Media móvil

4.2.2. Factor de media móvil

4.2.3. Construir la estrategia

4.3. ESTRATEGIA DE SEGUIMIENTO DE LA TENDENCIA

4.3.1. Correlación

4.3.2. Factor de seguimiento de la tendencia

4.3.3. Calcular la estrategia

CAPÍTULO 5: GESTIÓN DE RIESGOS Y BACKTESTING

5.1. LAS MÉTRICAS DE BACKTESTING

5.1.1. Las métricas CAPM

5.1.2. Sharpe y Sortino

5.1.3. Drawdown

5.2. MÉTODOS DE GESTIÓN DE RIESGOS

5.2.1. Valor en riesgo (VaR)

5.2.2. Valor en riesgo condicional (cVaR)

5.2.3. Contribución al riesgo

5.3. AUTOMATIZAR EL ANÁLISIS

5.3.1. Crear una función

5.3.2. Analizar la cartera estática

5.3.3. Analizar la cartera dinámica

CAPÍTULO 6: MÉTODOS AVANZADOS DE BACKTESTING

6.1 CONSEJOS ÚTILES PARA BACKTEST

6.1.1 Backtest no es una herramienta de búsqueda

6.1.2 Los días importantes no son nuestros amigos

6.1.3 Entiende tu estrategia

6.2 CALCULAR LOS RENDIMIENTOS DE LA ESTRATEGIA UTILIZANDO TP Y SL

6.2.1 Encontrar el extremo

[6.2.2 Calcular los rendimientos](#)

[6.2.3 Analizar el backtest](#)

[6.3 HERRAMIENTAS AVANZADAS DE BACKTEST](#)

[6.3.1 Lista de métricas de backtest](#)

[6.3.2 Simulación Monte Carlo](#)

[6.3.3 Stop dinámico fácil](#)

PARTE 2: MODELOS ESTADÍSTICOS PREDICTIVOS

CAPÍTULO 7: ARBITRAJE ESTADÍSTICO

[7.1. DE LA ESTACIONARIEDAD A LA COINTEGRACIÓN](#)

[7.1.1. Estacionariedad](#)

[7.1.2. Cointegración](#)

[7.2. TRADING POR PARES](#)

[7.2.1. Cómo funciona](#)

[7.2.2. Aplicaciones](#)

CAPÍTULO 8: MODELOS DE MEDIAS MÓVILES AUTORREGRESIVAS (ARMA)

[8.1. FUNDAMENTOS DE LAS SERIES TEMPORALES](#)

[8.1.1. Tendencia, ciclo, estacionalidad](#)

[8.1.2. Propiedades de precio logarítmico](#)

[8.1.3. La regresión lineal](#)

[8.2. MODELOS AR AND MA](#)

[8.2.1. Modelo autoregresivo \(AR\)](#)

[8.2.2. MODELO DE MEDIA MÓVIL \(MA\)](#)

[8.3. MODELOS ARMA](#)

[8.3.1. Modelo ARMA](#)

8.3.2. Modelo ARIMA

CAPÍTULO 9: REGRESIÓN LINEAL Y REGRESIÓN LOGÍSTICA

9.1. REGRESIÓN Y CLASIFICACIÓN

9.1.1. Recordatorio sobre la regresión

9.1.2. Entender la clasificación

9.2. REGRESIÓN LINEAL

9.2.1. Preparación de los datos

9.2.2. Implementación del modelo

9.2.3. PREDICCIONES Y BACKTEST

9.3. REGRESIÓN LOGÍSTICA

9.3.1. PREPARACIÓN DE LOS DATOS

9.3.2. Implementación del modelo

9.3.3. Predicciones y backtest

PARTE 3: MACHINE LEARNING, DEEP LEARNING Y TRADING EN VIVO

CAPÍTULO 10: INGENIERÍA DE CARACTERÍSTICAS Y OBJETIVOS

10.1 MOTIVACIÓN E INTUICIÓN

10.1.1 Ingeniería de características

10.1.2 Ingeniería de objetivos

10.1.3 ¿Por qué es tan importante?

10.2 APLICACIÓN AL TRADING

10.2.1 Crear indicadores de trading y funciones de trading útiles

10.2.2 Etiquetado del objetivo

CAPÍTULO 11: SUPPORT VECTOR MACHINE (SVM)

11.1. PREPARACIÓN DE LOS DATOS

11.1.1. Ingeniería de características

11.1.2. Estandarización

11.2. CLASIFICACIÓN CON SUPPORT VECTOR MACHINE (SVC)

11.2.1. Intuición sobre cómo funciona un SVC

11.2.2. Cómo crear un SVC usando Python

11.2.3 Predicciones y backtest

11.3. REGRESIÓN CON SUPPORT VECTOR MACHINE (SVR)

11.3.1. Intuición sobre el funcionamiento de un SVR

11.3.2. Cómo crear un SVR usando Python

11.3.3. Predicciones and backtest

CAPÍTULO 12: MÉTODOS ENSEMBLE Y ÁRBOL DE DECISIÓN

12.1. ÁRBOL DE DECISIÓN

12.1.1. Clasificador de árbol de decisión

12.1.2. Regresor de árbol de decisión

12.1.3. Optimizar los hiperparámetros

12.2. BOSQUE ALEATORIO

12.2.1. Clasificador de bosque aleatorio

12.2.2. Regresor de bosque aleatorio

12.2.3. Optimizar los hiperparámetros

12.3. MÉTODOS ENSEMBLE

- [12.3.1. Método voting](#)
- [12.3.2. Método bagging](#)
- [12.3.3. Método stacking](#)

CAPÍTULO 13: RED NEURONAL PROFUNDA (DNN)

13.1. INTUICIÓN TRAS LA DNN

- [13.1.1. Propagación hacia delante](#)
- [13.1.2. Gradiente descendente](#)
- [13.1.3. Propagación hacia atrás](#)

13.2. DNN PARA CLASIFICACIÓN

- [13.2.1. Preparación de los datos](#)
- [13.2.2. Implementación de una DNN para una tarea de clasificación](#)
- [13.2.3. Predicción y Backtest](#)

13.3. DNN PARA REGRESIÓN

- [13.3.1. Implementación de una DNN para una tarea de regresión](#)
- [13.3.2. Función de pérdida personalizada](#)
- [13.3.3. Predicción y Backtest](#)

CAPÍTULO 14: RED NEURONAL RECURRENTE (RNN)

14.1 PRINCIPIOS DE LA RNN

- [14.1.1. Cómo funciona una RNN](#)
- [14.1.2. Neurona LSTM](#)
- [14.1.3. Célula GRU](#)

14.2. RNN PARA CLASIFICACIÓN

[14.2.1. Transformar datos 2d a datos 3d](#)

[14.2.2. Implementando el modelo](#)

[14.2.3. Predicción y backtest](#)

[14.3. REGRESOR RNN](#)

[14.3.1. Precisión sobre la estandarización](#)

[14.3.2. Implementando el modelo](#)

[14.3.3. Predicciones y backtest](#)

[CAPÍTULO 15: BONUS - RED NEURONAL CONVOLUCIONAL RECURRENTE \(RCNN\)](#)

[15.1. INTUICIÓN DE LA CNN](#)

[15.2. CREAR UNA RCNN](#)

[15.3. BACKTEST](#)

[CAPÍTULO 16: PROYECTO COMPLETO DE LA VIDA REAL](#)

[16.1. PREPARACIÓN DE LOS DATOS](#)

[16.1.1. Importar los datos](#)

[16.1.2. Ingeniería de características](#)

[16.1.3. Conjuntos de entrenamiento, test y validación](#)

[16.2. MODELANDO LA ESTRATEGIA](#)

[16.2.1. Encontrar los mejores activos](#)

[16.2.2. Combinar los algoritmos](#)

[16.2.3. Aplicar técnicas de gestión de carteras](#)

[16.3. ENCONTRAR EL TAKE PROFIT, STOP LOSS Y APALANCAMIENTO ÓPTIMOS](#)

[16.3.1. Take profit óptimo \(tp\)](#)

16.3.2. Stop loss óptimo (sl)

16.3.3. Apalancamiento óptimo

CAPÍTULO 17: DE LA NADA A TRADING EN VIVO

17.1 DIRECTRICES PARA LA CREACIÓN DE ESTRATEGIAS DE TRADING

17.1.1 El plan de trading

*17.1.2 El proceso de creación de una estrategia
de trading*

17.1.3 Diario de trading

17.2 NO PONGAS TODOS LOS HUEVOS EN LA MISMA CESTA

17.2.1 Tamaño de la apuesta

*17.2.2 ¿Por qué crear una cartera de estrategias
comerciales?*

17.3. PROCESO DE LIVE TRADING

17.3.1. La segridad ante todo

17.3.2 Fase de incubación

17.3.3 ¿Cuándo dejarlo?

ANEXO: INTERÉS COMPUESTO FRENTE A INTERÉS SIMPLE

ANEXO: GUARDAR Y CARGAR MODELOS SCIKIT-LEARN Y TENSORFLOW

ANEXO: CLASE METATRADER LECTURAS ADICIONALES

¿Por qué deberías leer este libro?

El sector financiero está experimentando una importante reestructuración. Los operadores y los gestores de carteras se están convirtiendo cada vez más en científicos de datos financieros. Los bancos, los hedge funds y las fintech están automatizando sus inversiones integrando algoritmos de Machine Learning y Deep Learning en su proceso de toma de decisiones. **Este libro presenta los beneficios de la gestión de carteras, la estadística y el aprendizaje automático aplicados al trading en vivo con MetaTrader 5.**

La primera parte está dedicada a la **gestión de carteras, la gestión de riesgos y el backtesting**. Estos capítulos nos permitirán entender cómo combinar estrategias y qué métricas mirar para entender la solidez de la estrategia.

La parte 2 trata de **los modelos estadísticos de predicción**. Discutiremos el arbitraje estadístico y el modelo de media móvil autorregresiva (ARMA) e introduciremos los algoritmos de clasificación mediante la regresión logística.

La parte 3 nos da a conocer los modelos predictivos **de Machine Learning y Deep Learning**. Veremos estos algoritmos mediante el ejemplo de estrategias de trading: máquinas de soporte vectorial (SVM), árboles de decisión, bosques aleatorios, métodos ensemble, Redes Neuronales Artificiales (ANN), Redes Neuronales Recurrentes (RNN), Redes Neuronales Convolucionales Recurrentes (RCNN).

El libro finaliza con **un proyecto concreto de la A a la Z**: importación de datos de tu broker, creación de una cartera de estrategias de trading, despliegue en el trading en vivo o uso de un screener.

¿Quiénes somos?

Lucas Inglese es un trader cuantitativo independiente especializado en Machine Learning y ciencia de datos. Es fundador de Quantreo,

una web de E-learning de trading algorítmico (www.quantreo.com).

Lucas es licenciado en matemáticas y economía por la Universidad de Estrasburgo (Francia). Ya ha ayudado a más de 56.000 estudiantes a través de sus cursos online y su canal de YouTube dedicado al trading algorítmico.

Tiene un enfoque de trading cuantitativo que combina modelos predictivos, teoría financiera y cálculo estocástico.

Para mostrarte algunos resultados realistas, puedes ver el beneficio de su última cartera de estrategias en el trading en vivo: **2.5% de rentabilidad** para un **drawdown del 0.6%** sin apalancamiento en 1 mes y medio.

Figura 1: La última señal de trading en vivo en BullTrading de Lucas



Puedes ver una de las últimas señales de Lucas en BullTrading (bulltrading.be), una plataforma de copy trader. Esta estrategia se basa en el aprendizaje automático y el análisis cuantitativo (el mismo que en el libro).

Juan Gabriel Gomila y María Santos son cofundadores de Frogames S.L. Ambos son matemáticos e instructores online en su plataforma [Frogames Formación](#), donde disponen de toda una [ruta de aprendizaje de Trading Algorítmico](#). Juntos han enseñado a más de medio millón de estudiantes de habla hispana de todo el mundo en diferentes temáticas como machine learning, inteligencia artificial, programación, matemáticas, desarrollo de videojuegos... Si quieres estar al tanto de las últimas novedades de Frogames, nos puedes encontrar en nuestras redes sociales.

Ambos se han encargado de la traducción y revisión tanto de los cursos de trading algorítmico de Quantreo como de este libro. Esperamos que lo disfrutes.

Capítulo 1: Empieza por aquí

1.1. Encuentra el código

Todos los recursos del libro están disponibles en el repositorio de GitHub (<https://github.com/joanby/libro-trading-python-es>). Si hay algún problema con este enlace o relacionado con el código, ponte en contacto con nosotros a través de la página de contacto www.quantreo.com.

Antes de leer este libro, debes descargar Jupyter Notebook usando Anaconda Navigator (<https://www.anaconda.com>). En el capítulo 1, puedes encontrar un notebook que instala todas las librerías necesarias para el libro. Todas las lecturas adicionales se pueden encontrar en el archivo README.md en el repositorio GitHub.

1.2. Convenciones utilizadas en este libro



Este recuadro ofrece precisiones, sugerencias o consejos que suelen utilizarse en finanzas, comercio o ciencias de la información.



Este recuadro ofrece precisiones, sugerencias o consejos sobre el código.

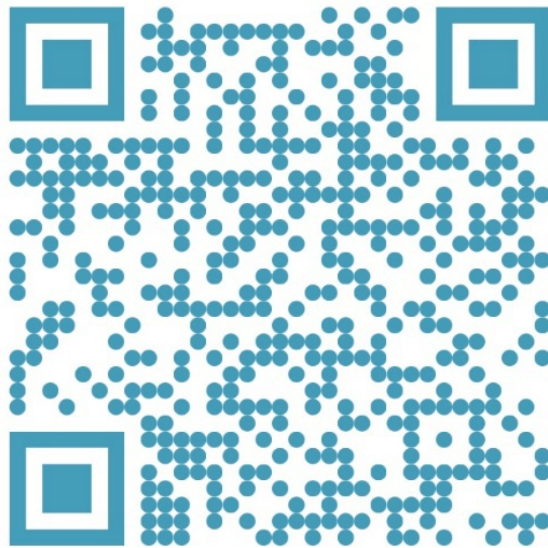


Este recuadro ofrece precisiones, sugerencias o consejos a partir de la experiencia del autor.

Contenido en cursiva: explica las figuras

1.3. Únete a la comunidad

Síguenos en las redes sociales para obtener la última información sobre trading algorítmico, consejos y estrategias listas para usar. Además, ¡te regalamos sin coste alguno el primer curso de la ruta de Trading Algorítmico! Una vez te unas, tendrás acceso a la comunidad exclusiva de la ruta de Trading Algorítmico de Frogames, donde podrás hacer cualquier pregunta sobre el libro para leerlo fácilmente.



Capítulo 2: Prerrequisitos

En este capítulo se analizan los prerrequisitos necesarios para entender a fondo este libro. Primero, discutiremos algunos fundamentos de matemáticas, estadística, álgebra y optimización. Luego, las principales teorías financieras y los fundamentos de Python son obligatorios para implementar nuestras estrategias de trading.

2.1. Prerrequisitos matemáticos

Esta sección pretende detallar muchos términos y conceptos matemáticos necesarios para la comprensión total del libro. Sin embargo, no se trata de matemáticas completas para el curso de finanzas. Cubriremos una introducción al mundo del álgebra, la estadística y la optimización.

2.1.1. Álgebra

El álgebra es un campo importante que hay que conocer cuando trabajamos en finanzas. De hecho, trabajaremos con matrices todo el tiempo porque es el corazón del álgebra. Por lo tanto, es necesario entender lo básico sobre ellas.

Hay muchas teorías sobre las matrices en matemáticas, algunas más complejas que otras, pero para nosotros será sencillo: una matriz será un conjunto de datos. Veamos un ejemplo de una matriz A con dimensión (n,m) , donde m es el número de columnas y n el número de filas de la matriz.

$$A = \begin{pmatrix} 1 & 0.7 \\ -1 & -3 \\ 0.6 & 0.1 \end{pmatrix}$$

Podemos imaginar que la matriz A con dimensión $(3,2)$ nos dará la rentabilidad diaria de algunos activos. Normalmente, las filas serán los días y las columnas representan los activos.

Así, la matriz tiene dos activos y tres rendimientos diarios para cada uno.

Hay muchas operaciones esenciales que podemos aplicar a una matriz. Veamos algunas de ellas:

- **Suma:** necesitamos matrices con la misma dimensión para sumarlas. Entonces, debemos sumar los coeficientes de la matriz A con los de la matriz B, uno a uno. Pongamos un ejemplo para explicarlo.

$$\begin{pmatrix} 1 & 1 \\ 2 & 3 \end{pmatrix} + \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 + 1 & 1 + 1 \\ 2 + 1 & 3 + 1 \end{pmatrix}$$

- **Resta:** necesitamos matrices con la misma dimensión para restarlas. Entonces, debemos restar los coeficientes de la matriz B de los coeficientes de la matriz A, uno por uno. Pongamos un ejemplo para explicarlo.

$$\begin{pmatrix} 1 & 1 \\ 2 & 3 \end{pmatrix} - \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 - 1 & 1 - 1 \\ 2 - 1 & 3 - 1 \end{pmatrix}$$

- **Multipliación por un escalar:** multiplicamos cada coeficiente por el escalar (sólo un número). Pongamos un ejemplo para explicarlo.

$$3 * \begin{pmatrix} 1 & 1 \\ 2 & 3 \end{pmatrix} = \begin{pmatrix} 3 * 1 & 3 * 1 \\ 3 * 2 & 3 * 3 \end{pmatrix}$$

Las tres operaciones anteriores son las operaciones básicas entre matrices, pero existen operaciones más complejas como la inversa, la matriz adjunta y la matriz transpuesta. Afortunadamente, sólo necesitamos conocer la matriz transpuesta para el resto de este libro. Supongamos que cada coeficiente de la matriz se anota $a_{i,j}$ donde i es el número de fila y j el número de columna del coeficiente. Todos los coeficientes de la matriz A de la forma $a_{i,j}$ se convierten en $a_{j,i}$

en la matriz transpuesta, denotada por A^t . Pongamos un ejemplo para explicarlo.

$$\begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}^t = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

2.1.2. Estadística

La estadística es obligatoria cuando trabajamos en finanzas cuantitativas. Desde el cálculo de los rendimientos hasta el cálculo de las probabilidades, veremos los conocimientos necesarios para trabajar en un proyecto financiero con tranquilidad.

En primer lugar, veremos algunas métricas estadísticas, pero no te preocupes si no las entiendes al 100%, porque las discutiremos más adelante en el libro. Para calcular un ejemplo de cada métrica, trabajaremos con estos vectores (los vectores son matrices con dimension $(1,n)$) $u = (0,1,3)$ y $v = (1,2,3)$.

- **Media:** Es el estadístico más detallado. Es la suma de todos los valores dividida por el número de valores.

$$\bar{v} = \frac{1}{N} \sum_{i=1}^N v_i = \frac{1}{3} * (1 + 2 + 3) = 2$$

- **Varianza:** La varianza es una medida de la dispersión de los valores en una muestra. Esta métrica permite comprender cuánto se dispersan los valores en torno a la media.

$$\text{Var} = \frac{1}{N} \sum_{i=1}^N (v_i - \bar{v})^2 = \frac{1}{3} * ((-1)^2 + 0^2 + 1^2) = \frac{2}{3} = 0.67$$

- **Desviación estándar:** Es la raíz cuadrada de la varianza. En finanzas, nos permite calcular la volatilidad de un activo.

$$\sigma = \sqrt{\text{Var}} = \sqrt{\frac{2}{3}}$$

- **Suma acumulada:** Esta métrica devuelve un vector, no sólo un número real. Una suma acumulada es una secuencia de sumas parciales de una secuencia dada.

$$v_j = \sum_{i=1}^j v_i = (1, 1 + 2, 1 + 2 + 3) = (1, 3, 6)$$

- **Producto acumulado:** Esta métrica devuelve un vector, no sólo un número real. Hace el producto de todos los valores anteriores para cada coeficiente.

$$v_j = \prod_{i=1}^j v_i = (1, 1 * 2, 1 * 2 * 3) = (1, 2, 6)$$

- **Error cuadrático medio (MSE):** Esta métrica se utiliza para calcular la pérdida de un algoritmo que nos permite entrenarlo (pero hablaremos de ello más adelante). Para penalizar el error significativo, calcula todas las diferencias entre las predicciones y los valores reales. Necesitamos dos vectores o matrices con la misma dimensión.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (v_i - u_i)^2 = \frac{1}{3}(1^2 + 1^2 + 0^2) = \frac{2}{3}$$

- **Error absoluto medio (MAE):** Esta métrica también se utiliza para calcular la pérdida de un algoritmo. Calculará todas las diferencias entre las predicciones y tomará el valor absoluto de la resta. Necesitamos dos vectores o matrices con la misma dimensión.

$$MAE = \frac{1}{N} \sum_{i=1}^N |v_i - u_i| = \frac{1}{3}(1 + 1 + 0) = \frac{2}{3}$$

Donde $|-x| = |x| = x$. Significa que el valor absoluto sólo toma los valores, no el signo.

Ahora, hablemos de las pruebas estadísticas. Hay muchas, pero en este libro sólo veremos una que es esencial entender: el test de Dick y Fuller aumentado. Sin embargo, primero vamos a explicar cómo funciona generalmente un test estadístico. Normalmente, hay dos hipótesis: H_0 y H_1 . El objetivo del test es comprobar la hipótesis H_0 .

Se trata de un campo amplio y apasionante que puede abarcarse en un libro entero (o, en nuestro caso, en un curso completo de Frogames). Por ello, intentaremos simplificarlo rápidamente. Cuando realicemos un test estadístico, sólo tendremos dos posibilidades: rechazar H_0 o no poder rechazar H_0 . Para saber en qué situación nos encontramos, utilizaremos el p-valor (un valor que mide la probabilidad entre 0 y 1). Si tomamos un umbral de error s y el p-valor de una prueba al que llamaremos p , tenemos estas dos posibilidades:

- **$p > s$:** No podemos rechazar H_0
- **$p < s$:** Podemos rechazar H_0

Por último, tenemos que hablar de las leyes de probabilidad. Son esenciales en finanzas porque nos permiten entender cómo se distribuyen las observaciones. La ley más utilizada en finanzas es la ley normal. Así, definiremos los siguientes conceptos tomándola de referencia. Hay dos funciones esenciales en una ley de probabilidad:

- **Función de distribución acumulada (CDF):** Esta función devuelve la probabilidad de que la variable aleatoria X esté por debajo del valor x . La CDF toma todos los valores absolutos y devuelve un valor entre 0

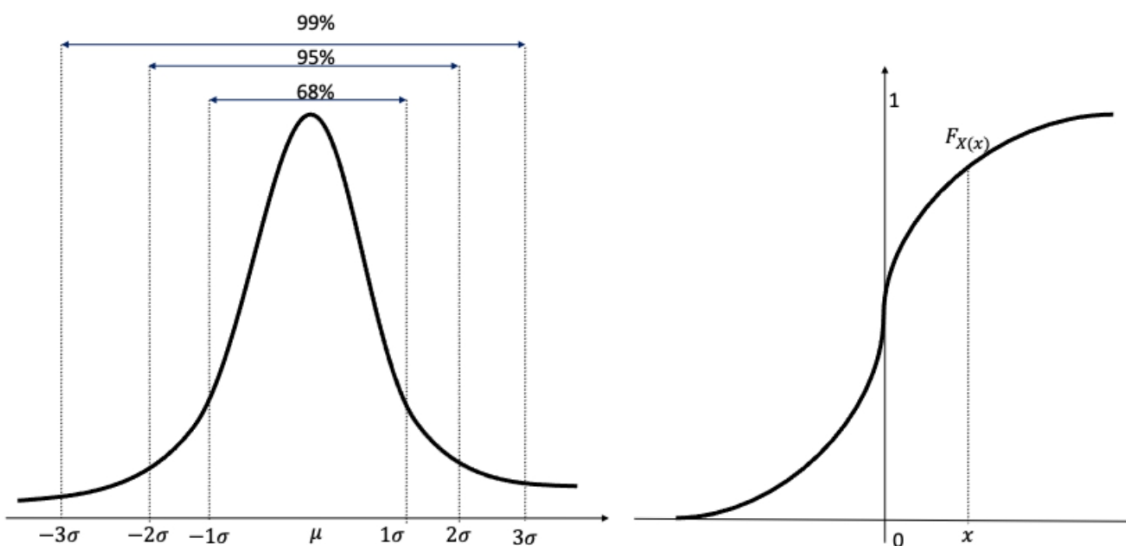
y 1, porque es una probabilidad. Formalicemos un poco esto.

$$F_X(x): \mathbb{R} \rightarrow [0,1]$$

$$F_X(x) = P(X < x)$$

- **Función de densidad de probabilidad (PDF):** Esta función nos permite comprender las distribuciones de las observaciones. Veamos en la figura 2.1 la diferencia entre CDF y PDF.

Figura 2.1: CDF y PDF para la ley normal



Podemos ver la CDF a la derecha y la PDF a la izquierda de la figura. Sin embargo, el porcentaje entre el intervalo de desviación estándar sólo está disponible con una distribución normal, no para los demás.

2.1.3. Optimización

Cuando hablamos de gestión de carteras, siempre implica hablar de optimización. Los métodos de optimización nos

permiten minimizar o maximizar una función bajo restricciones. Por ejemplo, en la gestión de carteras, maximizamos la utilidad de la cartera bajo la restricción de que el peso de la cartera debe ser igual al 100%. Tomemos un ejemplo de un problema de optimización explicando cada parte.

$$\begin{array}{ll} \max & U(w) \\ \sum w_i = 100\% \end{array}$$

La función a maximizar es $U(w)$ y queremos maximizarla. Sin embargo, tenemos una restricción, $\sum w_i = 100\%$, que significa que queremos invertir todo nuestro capital. El vector de pesos w es la variable que nos permite optimizar la función.

2.2. Prerrequisitos en finanzas

Esta sección explicará algunas teorías que hay que conocer obligatoriamente antes de crear algunas estrategias de trading algorítmico: eficiencia del mercado, fundamentos del trading y teoría de la gestión de carteras.

2.2.1. Eficiencia del mercado

La teoría de la eficiencia del mercado implica que no podemos predecir el comportamiento de las acciones porque es un paseo aleatorio. Sin embargo, es la teoría. En la práctica, es posible predecir el comportamiento de las acciones. Veamos los diferentes niveles de eficiencia para explicarlo.

- **Eficiencia débil del mercado:** Esta parte implica que es imposible predecir el precio de las acciones utilizando los precios del pasado. Por tanto, el precio de las acciones es un paseo aleatorio. Sin embargo, podemos predecir el precio futuro de las acciones utilizando técnicas de Machine Learning. Este libro sólo utilizará modelos cuantitativos de predicción para

demostrar que la eficiencia débil no siempre se respeta.

- **Eficiencia semifuerte del mercado:** Esta parte implica que toda la información se considera siempre en los precios de las acciones. Significa que no podemos obtener beneficios utilizando las noticias. En efecto, el mercado ya debería considerar las noticias antes de que se produzcan utilizando las previsiones de los inversores. Sin embargo, es posible demostrar que no siempre se respeta el mercado de eficiencia semifuerte utilizando el Procesamiento del Lenguaje Natural (NLP). Por ejemplo, mediante algoritmos de Machine Learning podemos utilizar los tweets, las noticias y la televisión para encontrar oportunidades.
- **Eficiencia fuerte del mercado:** Esta parte implica que toda la información está disponible para cualquiera. Sin embargo, es incorrecto porque el director general de la empresa puede acceder a cierta información que los inversores privados no tienen. Puede utilizarla para obtener beneficios en el mercado. Por lo tanto, esto demuestra que el mercado de fuerte eficiencia no siempre se respeta. Sin embargo, no podemos utilizar algoritmos para predecirlo.

2.2.2. Fundamentos del trading

Cuando hablamos de trading en este libro, hablamos sólo de trading especulativo. El objetivo es obtener beneficios utilizando **CFD (Contrato por Diferencias)**.

Un Contrato por Diferencias (CFD) es un derivado que sigue la misma variación que su activo subyacente. La ventaja del CFD es que podemos apostar por la bajada de la acción. Es lo que se llama **ir en corto** con un activo (lo discutiremos más adelante).

Cuando compremos un CFD, pagaremos unas tasas. Estas tasas son la comisión o el spread. El **spread** es la diferencia entre el precio de compra y el precio de venta. Además, las **comisiones** son una cantidad fija que pagaremos al broker **para entrar en posiciones y salir de ellas**.

El último concepto que debemos conocer es el **apalancamiento**. Una herramienta financiera disponible cuando abrimos una cuenta con el broker permite multiplicar los rendimientos de la estrategia. Por lo tanto, es bueno cuando ganamos dinero y malo cuando lo perdemos. El apalancamiento es una herramienta poderosa que puede ser un destructor de nuestro capital en las manos equivocadas (recordatorio: un gran poder conlleva una gran responsabilidad).

Normalmente, el apalancamiento se toma por cuenta y no por cada operación. Como hemos dicho antes, el apalancamiento aumenta el riesgo de perder dinero. Sin embargo, permite que personas con poco capital puedan invertir en el mercado. Así que, en la práctica, es muy recomendable trabajar en cuentas con cobertura de capital, lo que significa que el broker cierra nuestras posiciones antes de que nos quedemos sin capital para no endeudarnos con el broker, aunque durante movimientos extremadamente volátiles, podemos deberle dinero al broker si no ha podido cerrar las posiciones a tiempo.

Hay otra variable que debemos considerar: el **swap**. Se trata de un interés que se paga al broker por mantener la posición abierta durante la noche. Como el swap es una diferencia de tipos de interés, puede ser negativo o positivo.

Cuando hemos abierto operaciones, podemos establecer un **Stop Loss** (SL) o un **Take Profit** (TP). El stop loss es un umbral que es el precio del activo asociado a la peor pérdida que queremos aceptar. Significa que si no queremos perder más del 1% de nuestro capital, fijamos nuestro stop loss en el precio del activo, lo que hace que perdamos sólo el 1% del capital. Por ejemplo, tenemos 100\$ de capital y queremos

limitar la pérdida en esta posición al 1%. Si estamos en una posición extendida, fijaremos el stop loss en 99\$. Si el precio baja de 99\$, la posición se cierra automáticamente. Nos permite limitar el riesgo de la inversión. El take profit es lo mismo, pero en el sentido inverso. Cerraremos la posición cuando ganemos la cantidad deseada.

2.2.3. Fundamentos de la gestion de carteras

El objetivo de la gestión de carteras es invertir utilizando técnicas para reducir el riesgo de la cartera (diversificación). Los fundamentos de la gestión de carteras fueron establecidos por M.Markovitz (1952). La teoría de la cartera de media-varianza fue la base de la técnica de gestión de carteras. De hecho, la primera teoría se centra más en la estadística que en los demás parámetros. Será la base de nuestro análisis estático de carteras.

2.3. Prerrequisitos en Python

En esta parte, hablaremos de las principales funciones de las librerías de ciencia de datos, matemáticas y finanzas que utilizaremos.

2.3.1. Librerías para data science

Numpy

<code>np.array(obj)</code>	Crear un array
<code>np.zeros([n, m])</code>	Crear una matriz de dimensión (n,m)
<code>np.ones([n, m])</code>	Crear una matriz entera de unos
<code>np.mean(array, axis =)</code>	Devolver la media del array
<code>np.std(array, axis =)</code>	Devolver la desviación estándar del array
<code>np.var(array, axis =)</code>	Devolver la varianza del array
<code>np.sum(array, axis =)</code>	Devolver la suma del array
<code>np.cumprod(array, axis =)</code>	Devolver el product acumulado del

	array
<code>np.where(condition, value_true, value_false)</code>	Aplicar una condición a un array de numpy
<code>np.concatenate([arr,...], axis =)</code>	Concatenar arrays

Pandas

<code>pd.DataFrame(obj, index = [], columns = [])</code>	Crear un data frame
<code>df.columns = [col, ..., col]</code>	Cambiar las columnas de un data frame
<code>pd.Series(obj, index = [])</code>	Crear un objeto series

Matplotlib

<code>plt.figure()</code>	Modificar el objeto figure
<code>plt.plot(x, y)</code>	Representar una recta
<code>plt.fill_between(x, y, condition)</code>	Representar un área
<code>plt.title("title")</code>	Dar un título
<code>plt.xlabel("title")</code>	Dar una etiqueta al eje x
<code>plt.ylabel("title")</code>	Dar una etiqueta al eje y
<code>plt.legend([col, ..., col])</code>	Dar nombre a la leyenda
<code>plt.xlim([a, b])</code>	Redimensionar los límites del eje x
<code>plt.show()</code>	Mostrar la figure

2.3.2. Librerías para finanzas

Yfinance

<code>yf.download(ticker, start =, end =)</code>	Descargar el precio de las acciones
--	--

Datetime

<code>datetime.now()</code>	Dar una fecha en formato datetime
<code>datetime.now().strftime('%Y-%m-%d')</code>	Convertir datetime a string
<code>datetime.now().week()</code>	Encontrar el día de la semana

Time

<code>time.sleep(n)</code>	Poner el ordenador en una pausa de n segundos
----------------------------	--

2.3.3. Librerías para matemáticas

Scipy

<code>scipy.optimize.minimize(criterion, x, args = ())</code>	Minimizar una función
---	------------------------------

StatsModels

<code>statsmodels.tsa.stattools.adfuller(arr)</code>	Calcular el test de adfuller
<code>statsmodels.api.stat.OLS(x, y).fit()</code>	Entrenar el modelo
<code>model.resid()</code>	Devolver los residuos
<code>statsmodels.tsa.arima_model.ARIMA(arr)</code>	Calcular el modelo ARIMA
<code>statsmodels.graphics.tsaplots.plot_acf(arr)</code>	Representar el acf
<code>statsmodels.graphics.tsaplots.plot_pacf(arr)</code>	Representar el pacf

Parte 1: Gestión de carteras, gestión de riesgos y backtesting

En esta parte hablaremos de los métodos de optimización de carteras estáticas, como la optimización de la media-varianza y la optimización del ratio de Sharpe. También veremos algunos métodos de optimización de carteras dinámicas, como los criterios de impulso. Una vez que hayamos creado algunas carteras, analizaremos el riesgo de estas carteras y las someteremos a pruebas retrospectivas.

Resumen:

Capítulo 3: Optimización de la cartera estática

[Capítulo 4: Gestión táctica de la cartera](#)

Capítulo 5: Gestión de riesgos y backtesting

[Capítulo 6: Métodos avanzados de backtesting](#)

Capítulo 3: Optimización de la cartera estática

En este capítulo se analizarán los métodos de optimización estática de carteras. En primer lugar, veremos la intuición que hay detrás de la optimización de carteras y por qué es tan valiosa para la gestión de carteras. Después, estudiaremos los métodos tradicionales de optimización de carteras como el criterio de la media-varianza o el criterio de la media-varianza de la asimetría-curtosis. Además, estudiaremos cómo hacer nuestro criterio de optimización de carteras.

3.1. Explicación del método de optimización de carteras

Esta sección explicará por qué la optimización de la cartera es tan valiosa en las finanzas. En primer lugar, veremos la diferencia entre los riesgos sistémicos y los específicos. A continuación, destacaremos las ventajas de la diversificación.

3.1.1. Riesgo sistémico y riesgo específico

Cuando hablamos del riesgo de un activo, pensamos en la volatilidad del mismo. En cambio, podemos descomponer el riesgo de un activo de dos maneras: el riesgo sistémico y el riesgo específico.

El riesgo específico de un activo es el riesgo asociado a un activo que no depende de las condiciones del mercado. Por ejemplo, una emisión en una fábrica de Tesla puede reducir el precio de las acciones de Tesla, pero sólo de las acciones de Tesla porque no hay razones para que esta emisión afecte al precio de las acciones de Netflix.

El riesgo sistémico es un riesgo macroeconómico sobre todos los activos de la economía. Por ejemplo, supongamos que hay una recesión importante causada por la inestabilidad

geopolítica. En ese caso, todos los precios de las acciones bajarán porque la gente no quiere consumir por la incertidumbre del futuro. Es un riesgo sistémico porque afecta a todos los activos de la economía.

La gran diferencia entre el riesgo específico y el sistémico es que el riesgo sistémico no depende de las acciones de la empresa. Por lo tanto, el objetivo de los métodos de optimización de carteras es reducir el riesgo específico en la cartera. Para ello, utilizamos la diversificación.

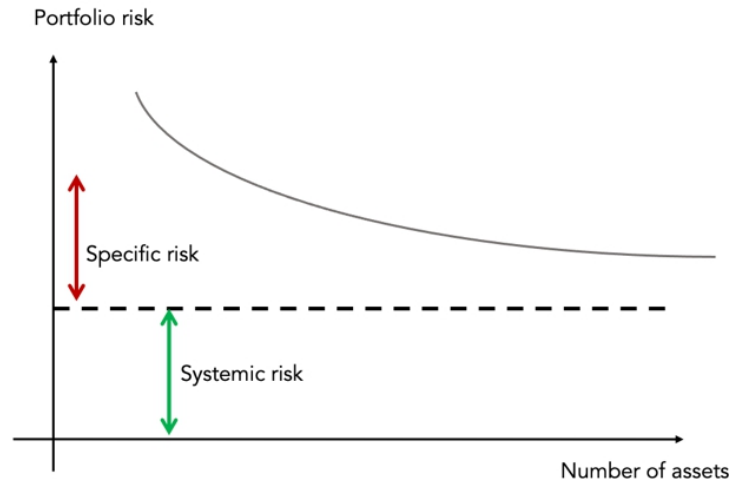


Este libro sólo ve cómo reducir el riesgo específico, pero es posible reducir el riesgo sistémico utilizando derivados.

3.1.2. Diversificación

La diversificación es el proceso central de la gestión de carteras. Permite reducir el riesgo específico de la cartera, como se ha explicado anteriormente. La estrategia consiste en aumentar el número de activos de la cartera para que el riesgo específico de cada activo sea insignificante. La explicación se muestra en la figura 3.1.

Figura 3.1: Riesgo de la cartera



En esta figura, vemos la descomposición del riesgo entre el riesgo específico y el riesgo sistémico. Cuanto más aumenta el número de activos, más nos acercamos al riesgo sistémico de la cartera.

La figura muestra que el aumento del número de activos permite reducir el riesgo de la cartera al riesgo sistémico de la cartera, porque el riesgo específico de cada activo disminuye con el número de activos. Así pues, la figura pone de manifiesto el poder de la diversificación para disminuir el riesgo de una cartera.



La diversificación disminuye el riesgo específico hasta un nivel teórico de 0, pero en la práctica, el coste de la transacción o el impuesto obliga a limitar un poco la diversificación y a encontrar una asignación razonable teniendo en cuenta el coste de la transacción.

3.2. Los métodos tradicionales de optimización de carteras

En esta sección, aprenderemos los métodos tradicionales de optimización de carteras. Este libro no se centrará en la teoría de la función de utilidad de la cartera, sino sólo en este cálculo y sus ventajas^[1]. A continuación, utilizaremos una derivada de

la función de utilidad de la cartera para crear criterios de diversificación avanzados.

3.2.1. Función de utilidad de la cartera

En las siguientes partes explicaremos algunos resultados interesantes sobre la función de utilidad de la cartera y el proceso que seguiremos. La función de utilidad de la cartera es el punto principal de la teoría de gestión de carteras. En efecto, con esta función podemos cuantificar la satisfacción que nos proporciona la cartera. Naturalmente, utilizaremos esta función para maximizar la utilidad del inversor.

La función de utilidad es difícil de calcular en su forma original. Por lo tanto, para evitar este problema, utilizaremos la aproximación de Taylor de esta función para encontrar la media-varianza y el criterio de asimetría-curtosis de la media-varianza.

La ecuación $U_{MV}(w)$ es el criterio de la media-varianza, y la $U_{SK}(w)$ es el criterio de la media-varianza-asimetría-curtosis.

En la siguiente ecuación, μ es la media de los rendimientos de la cartera, σ la volatilidad de la cartera, s la asimetría de la cartera y k la curtosis de la cartera. Además, λ es el nivel de aversión al riesgo (lo fijamos en 3 porque es una aversión al riesgo habitual), w la riqueza libre de riesgo (fijamos 1.0025), y w la riqueza de la cartera.

$$U_{MV}(w) = \frac{w^{1-\lambda}}{1-\lambda} + \frac{w^{-\lambda}}{1-\lambda} \mu - \frac{\lambda}{2} w^{-1-\lambda} w^2 \sigma^2$$

$$U_{SK}(w)$$

$$= U_{MV}(w) + \frac{\lambda(\lambda+1)}{6} w^{-2-\lambda} w^3 s - \frac{\lambda(\lambda+1)(\lambda+2)}{24} w^{-3-\lambda} w^4 k$$

No te preocupes, te mostraremos cómo calcularlo en la siguiente sección.

3.2.2. Criterio de la media-varianza

En esta subsección, implementaremos una optimización de la media-varianza. Es necesario descargar una base de datos con las cotizaciones ajustadas al cierre de algunos activos. Para hacer la optimización de la cartera, utilizaremos estos activos: Facebook, Netflix y Tesla. Para la importación, utilizaremos la librería *yfinance*, que nos permite importar los datos muy rápidamente. Además, transformamos los datos en variación diaria para poner todos los activos en la misma escala.

Código 3.1: Importación de datos

```
# Importación de datos
list_tickers = ["FB", "NFLX", "TSLA"] 1
database = yf.download(list_tickers) 2

# Toma sólo el precio ajustado de las acciones
database = database["Adj Close"] 3

# Eliminar los valores faltantes
data = database.dropna().pct_change(1).dropna() 4
```

- 1 Crea una lista de teletipos utilizando la notación de Yahoo.
- 2 Usa la función *download()* de *yfinance* para importar el dataset.
- 3 Selecciona sólo los precios de cierre ajustados.
- 4 Crea retornos y elimina los valores faltantes.

Figura 3.2: Extracto de la base de datos

Date	05-21	05-22	05-23	05-24	05-25	05-29
Facebook	-0.110	-0.089	0.032	0.032	-0.34	-0.096
Netflix	0.025	-0.056	0.062	-0.023	-0.001	-0.011
Tesla	0.044	0.071	0.007	-0.024	-0.016	0.063

Extracto de la base de datos del código 3.1, que importa los datos de Facebook (FB), Netflix (NFLX) y Tesla (TSLA) desde 2012 hasta 2021.

Ahora, tenemos que definir la función *MV_criterion* para calcular la utilidad de la función y encontrar la mejor distribución entre los tres activos. Para ello, vamos a calcular la ecuación en la parte 3.2.1 $U_{MV}(w)$.

Código 3.2: función de criterio de media-varianza

```
def MV_criterion(weights, data):  
    """  
    -----  
    | Output: optimización del criterio de la cartera |  
    -----  
    | Inputs: -weights (type ndarray numpy): Pesos de la cartera |  
    |          -data (type ndarray numpy): Retornos de los activos |  
    -----  
    """  
  
    # Parámetros 1  
    Lambda = 3  
    W = 1  
    Wbar = 1 + 0.25 / 100 2  
  
    # Calcular la rentabilidad de la cartera  
    portfolio_return = np.multiply(data, np.transpose(weights)) 3
```



```

portfolio_return = portfolio_return.sum(axis = 1) 4

# Calcular la media y la volatilidad de la cartera
mean = np.mean(portfolio_return, axis = 0) 5
std = np.std(portfolio_return, axis = 0) 6

# Calcular el criterio
criterion = Wbar ** (1 - Lambda) / (1 + Lambda) + \
            Wbar ** (-Lambda) * W * mean - \
            Lambda / 2 * Wbar ** (-1 - Lambda) * \
            W ** 2 * std ** 2 7

criterion = -criterion 8
return criterion

```

- 1 Establece los parámetros del modelo anterior.
- 2 La riqueza sin riesgo es $1 +$ el índice sin riesgo (0.25%).
- 3 Multiplica las columnas por su coeficiente para mantener una matriz con dimensión (n, m).
- 4 Suma todas las columnas para tener la cartera (shape = (n, 1)).
- 5 Calcula la media de los rendimientos diarios de la cartera (utilizando axis = 0 para hacerlo por filas).
- 6 Calcula la volatilidad diaria de la cartera (la desviación estándar).
- 7 Calcula $U_{MV}(w)$ usando la formula anterior.
- 8 Devuelve el opuesto del criterio para minimizarlo.



Maximizar una función es lo mismo que minimizar la opuesta de dicha función. Así, utilizaremos la función minimize en el siguiente código porque MV_criterion nos devuelve $-U_{MV}(w)$.

Una vez que tenemos una función a minimizar (maximizar la inversa de $U(w)$), necesitamos configurar los límites para el siguiente problema de optimización.

$$\max_{\sum \alpha_i = 1} U_{MV}(w)$$

Significa que maximizaremos la utilidad bajo la restricción de utilizar todo el capital porque la suma de α_i debe ser igual al 100%. Por lo tanto, tenemos que utilizar todo nuestro capital. Así, los límites para cada activo son (0, 1).

También tenemos que establecer un peso para el inicio de la optimización. Además, sólo realizaremos la optimización en el conjunto de entrenamiento (70% de los datos) y analizaremos el rendimiento del conjunto de test (30% de los datos).

Código 3.3: Implementación del problema de optimización de la cartera

```
split = int(0.7 * len(data)) 1
train_set = data.iloc[:split, :]
test_set = data.iloc[split:, :]

# Encontrar el número de activos
n = data.shape[1] 2

# Inicialización de los pesos
x0 = np.ones(n) 3
```

```

# Restricciones del problema de optimización
cons = ({'type': 'eq', 'fun': lambda x: sum(abs(x)) - 1}) 4

# Establecer los límites
Bounds = [(0, 1) for i in range(0, n)] 5

# Resolución del problema de optimización
res_MV = minimize(MV_criterion, x0, method = "SLSQP",
                  args = (train_set), bounds = Bounds,
                  constraints = cons, options = {'disp':
True}) 6

# Resultado
X_MV = res_MV.x 7

```

- 1 La variable split es un número entero que representa el valor del 70% de los datos. Es un truco para seleccionar los conjuntos de entrenamiento y prueba.
- 2 n es el número de activos, por lo que utilizamos el comando `.shape[1]` para tener el número de columnas, que es el número de activos.
- 3 Inicializa el valor del vector de pesos. Es un vector de unos con dimensión (n,).
- 4 Define las restricciones de la optimización. Aquí queremos que el inversor utilice todo su capital. Por lo tanto, deseamos que la suma del peso sea igual al 100%. (Hay valor absoluto si deseamos también a corto).
- 5 Define el límite de la optimización. Definimos los límites de 0 a 1 porque queremos una estrategia de sólo largo. Si creamos una estrategia largo-corto, los límites serán (-1, 1).
- 6 Minimiza el opuesto de $U_{MV}(w)$ usando la función *minimize* de *scipy*.

7 Extrae el peso óptimo para nuestra cartera.

Con el código anterior hemos encontrado la mejor asignación para este activo y el vector de las ponderaciones está disponible en la figura 3.3.

Figura 3.3: Asignación de pesos de la cartera con el criterio de MV

Asset	Facebook	Netflix	Tesla
Weight	33.35%	33.35%	33.30%

Como vemos, la optimización con el criterio MV se lleva la acción de Facebook por un 33.35%, la de Netflix por un 33.35% y la de Tesla por un 33.30%.

Ahora tenemos los pesos de cada activo en nuestra cartera. Por lo tanto, tenemos que mostrar los rendimientos de la cartera en el conjunto de test para ver si la optimización de la cartera es buena o no. Aunque los rendimientos acumulados de la cartera no pueden decir por sí solos si es una buena optimización o no, es una buena indicación.

Código 3.4: Comprobando el rendimiento en el conjunto de test

```
# Calcular la rentabilidad acumulada de la cartera (CM)
portfolio_return_MV = np.multiply(test_set,np.transpose(X_MV)) 1
portfolio_return_MV = portfolio_return_MV.sum(axis = 1) 2

# Representar el CM
plt.figure(figsize = (15, 8)) 3

plt.plot(np.cumsum(portfolio_return_MV) * 100,
         color = "#035593", linewidth = 3) 4
plt.ylabel("Cumulative return %", size = 15,
```

```
fontweight = "bold") 5
plt.xticks(size = 15, fontweight = "bold")
plt.yticks(size = 15, fontweight = "bold")
plt.title("Cumulative return of the mean variance potfolio",
          size = 20)
plt.axhline(0, color = "r", linewidth = 3) 6
plt.show() 7
```

- 1 Tenemos varias columnas por su coeficiente. Así, siempre tenemos una matriz de dimensión (n, m).
- 2 Calculamos la suma de cada columna para tener la rentabilidad de la cartera.
- 3 Cambiamos el tamaño de la figura para tener una mejor visualización.
- 4 Imprimimos la suma acumulada de la cartera y la ponemos en porcentajes.
- 5 Ponemos un nombre y aumentamos el tamaño de las y-labels.
- 6 Dibujamos una línea horizontal en el 0 para resaltar este umbral.
- 7 Mostramos el gráfico usando *plt.show()*.

En la figura 3.4, podemos ver el gráfico de la rentabilidad acumulada en el conjunto de test. Es bueno ver la rentabilidad de la cartera, pero en el próximo capítulo aprenderemos a completar el análisis de riesgo y el backtest.

De hecho, veremos muchas métricas como los ratios de Sharpe y Sortino, el drawdown y la métrica CPAM, por poner sólo un ejemplo.

Figura 3.4: Rendimiento acumulado de la cartera en el conjunto de test



Como podemos ver, la rentabilidad acumulada de la cartera se estanca en el primer año. Sin embargo, después de este periodo, la rentabilidad de la cartera tiene un crecimiento considerable.

3.2.3. Criterio de la media-varianza-asimetría-curtosis

En la última subsección, hemos visto el criterio de la media-varianza. En esta subsección, vamos a ver una extensión del criterio de la media-varianza: el criterio de la media-varianza-asimetría-curtosis (criterio SK).

Vamos a tomar precisamente los mismos datos para comparar las optimizaciones. En este caso, utilizaremos como criterio el $U_{SK}(w)$ y no $U_{MV}(w)$. Por lo tanto, vamos a ver la diferencia en el peso con diferentes criterios.

Código 3.6: Implementación del criterio SK

```
def SK_criterion(weights, data):
```

```
    """
```

```
    -----
```

```
    | Output: optimización del criterio de la cartera |
```

```

-----
| Inputs: -weights (type ndarray numpy): Pesos de la cartera |
| -data (type ndarray numpy): Retornos de los activos |
-----

"""

from scipy.stats import skew, kurtosis

# Parámetros 1
Lambda = 3
W = 1
Wbar = 1 + 0.25 / 100

# Calcular la rentabilidad de la cartera 2
portfolio_return = np.multiply(data, np.transpose(weights))
portfolio_return = portfolio_return.sum(axis = 1)

# Calcular la media, la volatilidad, el sesgo y la curtosis de
# la cartera
mean = np.mean(portfolio_return, axis = 0)
std = np.std(portfolio_return, axis = 0)
skewness = skew(portfolio_return, 0) 3
kurt = kurtosis(portfolio_return, 0) 4

# Calcular el criterio
criterion = Wbar ** (1 - Lambda) / (1 + Lambda) + Wbar ** (-Lambda) \
    * W * mean - Lambda / 2 * Wbar ** (-1 - Lambda) * W ** 2 * std ** 2 \
    + Lambda * (Lambda + 1) / (6) * Wbar ** (-2 - Lambda) * W ** 3 * skewness \
    - Lambda * (Lambda + 1) * (Lambda + 2) / (24) * Wbar ** (-3 - Lambda) * \
    W ** 4 * kurt 5

criterion = -criterion 6
return criterion

```

- 1 Establece los parámetros de los modelos (*lambda*=3 es una aversión al riesgo típica. Cuanto más alto sea el lambda, más

se odia el riesgo).

- 2 Multiplica cada activo por su coeficiente y haz la suma para tener la rentabilidad de la cartera.
- 3 Calcula la asimetría de la función de densidad de probabilidad (PDF).
- 4 Calcula la curtosis, que representa la "cola" de la función de densidad de probabilidad (PDF).
- 5 Calcula $U_{SK}(w)$.
- 6 Calcula el opuesto del criterio porque lo minimizaremos.

Siguiendo el mismo proceso que el criterio MV, haremos la optimización con el criterio SK.

Código 3.7: Optimización con el criterio SK

```
# Encontrar el número de activos
n = data.shape[1]

# Inicialización de los pesos
x0 = np.ones(n)

# Restricciones del problema de optimización
cons = ({'type': 'eq', 'fun': lambda x: sum(abs(x)) - 1})

# Establecer los límites
Bounds = [(0, 1) for i in range(0, n)]

# Resolución de problemas de optimización
res_SK = minimize(SK_criterion, x0, method = "SLSQP",
                  args = (train_set), bounds = Bounds,
                  constraints = cons, options = {'disp': True})

# Resultado de los cálculos
X_SK = res_SK.x
```

Figura 3.5: Asignación de pesos de la cartera mediante SK

Asset	Facebook	Netflix	Tesla
Weight	32.75%	21.06%	46.19%

La optimización con el criterio SK se lleva las acciones de Facebook en un 32.75%, las de Netflix en un 21.06% y las de Tesla en un 46.19%.

Las ponderaciones de la cartera son diferentes a las de la optimización MV. El objetivo es trazar la rentabilidad de la cartera de optimización MV y la rentabilidad de la cartera de optimización SK para comparar lo mejor en esta situación (Figura 3.6).

Figura 3.6: Rendimiento acumulado de las carteras en el conjunto de test



Podemos ver que las dos carteras siguen la misma tendencia, pero la cartera SK es un poco mejor que la cartera MV.

3.3. Los métodos modernos de optimización de carteras

En esta sección, tratamos el método moderno de optimización de carteras. Veremos cómo crear su criterio utilizando el ejemplo de los criterios de Sharpe y Sortino.

3.3.1. Criterio de Sharpe

En esta subsección, hablaremos del criterio de Sharpe. En primer lugar, debemos definir el ratio de Sharpe para entender los beneficios de esta optimización.

El ratio de Sharpe es la más conocida de las métricas financieras. De hecho, es una métrica de referencia en el sector. Nos permite entender los beneficios adicionales por un 1% más de riesgo. Podemos calcular el ratio de Sharpe con la siguiente fórmula:

$$\text{Sharpe} = \frac{\mu - r_f}{\sigma}$$

μ es la media de los rendimientos de la cartera (anualizada), σ es la volatilidad de los rendimientos de la cartera (anualizada)

y r_f es el activo libre de riesgo (fijamos 0 para el activo libre de riesgo porque actualmente está en torno a este valor).

Así, el objetivo es crear un criterio de optimización de la cartera con esta métrica. Para ello, seguiremos el mismo proceso que el método tradicional del apartado anterior. Así que tenemos que crear una función `SR_criterion()`, que devuelva el opuesto al ratio de Sharpe porque maximizar el ratio de Sharpe es igual a minimizar el opuesto al ratio de Sharpe.

Código 3.8: Función del criterio SR

```

def SR_criterion(weight, data):
    """
    -----
    | Output: El opuesto del ratio de Sharpe para minimizarlo. |
    -----
    | Inputs: -weight (type ndarray numpy): Peso de la cartera |
    |         -data (type dataframe pandas): Retornos de los activos |
    -----
    """

    # Calcular la rentabilidad de la cartera
    portfolio_return = np.multiply(data, np.transpose(weight))
    portfolio_return = portfolio_return.sum(axis = 1)

    # Calcular la media y la volatilidad de la cartera
    mean = np.mean(portfolio_return, axis = 0)
    std = np.std(portfolio_return, axis = 0)

    # Calcular el opuesto al ratio de Sharpe
    Sharpe = mean / std
    Sharpe = -Sharpe
    return Sharpe

```

Ahora, podemos hacer el problema de optimización como en la sección anterior. Sin embargo, representaremos los resultados para comparar los métodos de optimización en la figura 3.8 y las ponderaciones de la cartera en la figura 3.7.

Figura 3.7: Asignación de pesos de la cartera con el criterio de SR

Asset	Facebook	Netflix	Tesla
Weight	24.08%	48.01%	27.91%

Como podemos ver, la optimización con el criterio SR se lleva las acciones de Facebook en un 24.08%, las de Netflix en un 48.01% y las de Tesla en un 27.91%.

Figura 3.8: Rendimiento acumulado de las carteras en el conjunto de test



Podemos hacer la misma interpretación que antes. La cartera sigue la misma tendencia, pero con un poco de variación.

3.3.2. Criterio de Sortino

En el último subapartado hemos aprendido el criterio del ratio de Sharpe máximo. Sin embargo, el ratio de Sharpe tiene un gran punto negativo: calcula la volatilidad como la suma de la volatilidad al alza y a la baja. Sin embargo, ¿estamos realmente en contra de la volatilidad alcista significativa cuando somos un inversor? No, por supuesto, porque es sinónimo de un beneficio considerable. Por lo tanto, el enemigo es sólo la volatilidad a la baja.

El ratio de Sortino es una excelente métrica porque deriva del ratio de Sharpe y sólo considera la volatilidad a la baja. Por lo tanto, nos permite entender los beneficios adicionales por un 1% más de riesgo (sólo bajo riesgo). Podemos calcular el ratio de Sortino con la siguiente fórmula:

$$\text{Sortino} = \frac{\mu - r_f}{\sigma_{\text{downward}}}$$

Donde μ es la media de los rendimientos de la cartera (anualizada), σ_{downward} es la volatilidad a la baja de los rendimientos de la cartera (anualizada) y r_f es el activo libre de riesgo (fijamos 0 para los activos libres de riesgo porque en realidad está alrededor de este valor).

Así que, como anteriormente, necesitamos calcular una función para minimizar. En la siguiente parte de código podemos encontrar la función de criterio SOR (Sortino).

Código 3.9: Criterio de la función SOR

```
def SOR_criterion(weight, data):
    """
    -----
    | Output: Opuesto del ratio de Sortino para minimizarlo |
    -----
    | Inputs: -weight (type ndarray numpy): Peso de la cartera |
    |         -data (type dataframe pandas): Retornos de los activos |
    -----
    """

    # Calcular la rentabilidad de la cartera
    portfolio_return = np.multiply(data, np.transpose(weight))
    portfolio_return = portfolio_return.sum(axis = 1)

    # Calcular la media y la volatilidad de la cartera
    mean = np.mean(portfolio_return, axis = 0)
```

```
std = np.std(portfolio_return[portfolio_return < 0],
              axis = 0) 1
```

```
# Calcular el opuesto al ratio de Sortino
```

```
Sortino = mean / std
```

```
Sortino = -Sortino
```

```
return Sortino
```

- 1 Para calcular la volatilidad a la baja, tomamos todos los rendimientos negativos y calculamos su desviación estándar.

Ahora, podemos hacer el problema de optimización como en la sección anterior. Sin embargo, representaremos los resultados para comparar los métodos de optimización en la figura 3.10 y las ponderaciones de la cartera en la figura 3.9.

Figura 3.9: Asignación de pesos de la cartera según el criterio SOR

Asset	Facebook	Netflix	Tesla
Weight	21.87%	49.85%	28.28%

Como podemos ver, la optimización con el criterio SOR se lleva las acciones de Facebook en un 21.87%, las de Netflix en un 49.85% y las de Tesla en un 28.28%.



El peso también es casi el mismo. Por lo tanto, la cartera también será muy similar a las demás porque la correlación entre estos tres activos es alta.

Figura 3.10: Rendimiento acumulado de las carteras en el conjunto de test



Podemos hacer la misma interpretación que antes. La cartera sigue la misma tendencia, pero con un poco de variación.

Resumen

- El riesgo específico está relacionado con un solo valor. En cambio, el riesgo sistémico es un riesgo macroeconómico que puede afectar a todos los activos de la economía.
- Tradicionalmente, utilizamos la aproximación de la función de utilidad para encontrar los criterios a maximizar. Por ejemplo, el criterio de la media-varianza y el criterio de la media-varianza-asimetría-curtosis.
- Maximizar una función es igual a minimizar el opuesto de la función.
- El ratio de Sharpe cuantifica los beneficios adicionales por un 1% más de riesgo en el coste de la volatilidad total.
- El ratio Sortino cuantifica los beneficios adicionales por un 1% más de riesgo en el coste de la baja volatilidad.

- La clave de una cartera diversificada son los activos con correlación negativa (lo trataremos en el siguiente capítulo sobre la correlación).

Capítulo 4: Gestión táctica de la cartera

Las optimizaciones estáticas de la cartera son una forma excelente de optimizar una inversión a largo plazo. Sin embargo, a veces tenemos que desviarnos de nuestra optimización de cartera estática óptima utilizando la gestión de carteras dinámicas. Además, los métodos de carteras dinámicas pueden poner en corto algunos activos y puede ser ventajoso obtener beneficios si el mercado sube o baja. Para explicar estos métodos, veremos cómo funciona una cartera dinámica, la estrategia de media móvil y la estrategia de correlación.

4.1. ¿Cómo funcionan los métodos dinámicos?

Esta sección explicará cómo funciona la **asignación táctica de activos** (TAA). Para ilustrar esta noción, explicaremos qué es la venta en corto, cómo crear un factor de impulso y cómo reequilibrar una cartera dinámica.

4.1.1. Venta en corto

Normalmente, en las **estrategias de cartera estática** (SAA), compramos las acciones en proporciones adecuadas. Sin embargo, en los métodos dinámicos (TAA), también podemos apostar por la bajada de las acciones, y se llama venta en corto. La venta en corto suele utilizarse sólo con el TAA porque el TAA tiene un intervalo de reequilibrio minúsculo: por ejemplo un mes, a diferencia del SAA, que tiene un intervalo de reequilibrio muy largo, como por ejemplo tres años. Por lo tanto, la TAA puede captar los beneficios cuando el precio aumenta o disminuye en lugar de la SAA, que nos permite apostar sólo por el aumento. Así, cuando compramos un

contrato de venta en corto, sólo apostamos por la bajada de la acción.

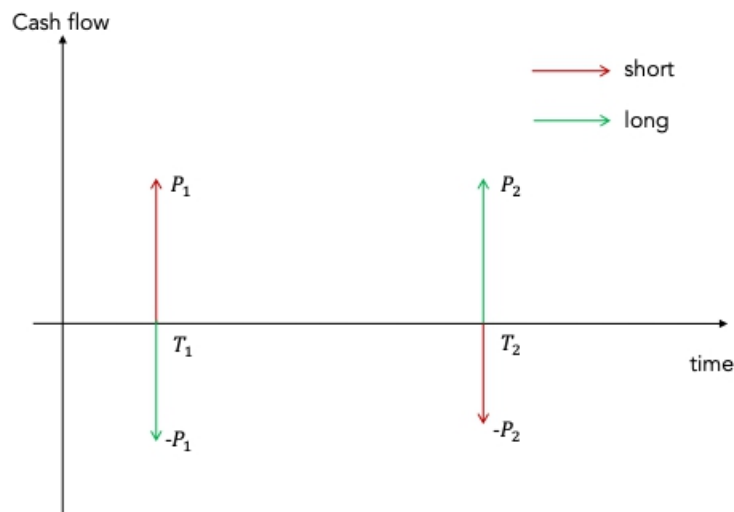


Es poco frecuente ponerse en corto con una acción directamente. Por lo general, utilizaremos un Contrato por Diferencia (CFD) para apostar por bajar la acción.

Recordatorio: No tenemos acceso al dividendo con el CFD.

En la figura 4.1, podemos ver los flujos de caja de un contrato largo (apuesta por la subida) y de un contrato corto (apuesta por la bajada). P_1 es el flujo de caja en el momento 1, al comprar el contrato. Para un largo, compramos la acción en P_1 . Así, el flujo de caja es el opuesto al precio de la acción (sin costes de transacción), y cuando cerramos la posición, tenemos el precio real de la acción P_2 . Para el contrato corto, es precisamente lo contrario. En efecto, cuando compramos el contrato de venta, vendemos un contrato largo que no tenemos ya, pero con la obligación de comprar la acción más adelante (en nuestro ejemplo T_2). Así, los beneficios de un contrato largo son $P_2 - P_1$ y para el contrato corto $P_1 - P_2$.

Figura 4.1: Flujo de caja para contratos largos y cortos



Esta figura muestra el flujo de caja de un contrato largo y corto desde el momento T_1 hasta el momento T_2 . En la figura, podemos aprender los siguientes cálculos, los beneficios de un contrato largo son $P_2 - P_1$ y para el contrato corto $P_1 - P_2$.

4.1.2. Factor de impulso

Al principio de una asignación táctica de activos, debemos elegir un factor de impulso. Es un valor que podemos construir en la estrategia. Por ejemplo, en la siguiente sección, crearemos un factor de impulso con la autocorrelación del activo. Este factor es el pilar de la estrategia. De hecho, si es una buena elección, podemos obtener beneficios, y si es errónea, perderemos dinero.

Debemos entender que el factor de impulso puede ser el que queramos. Por ejemplo, podemos crear el momentum utilizando el crecimiento de los tipos de interés, la inflación del país, el porcentaje de rentabilidad de los últimos 12 meses, etc.

Establecemos un factor vectorial λ para esta parte para hacer todas las transformaciones necesarias. Cuando tenemos este factor, solemos crear un z-score, la normalización del factor. Pondremos la fórmula de un z-score para resaltar el proceso.

$$z_{\lambda_i} = \frac{\lambda_i - \mu_{\lambda}}{\sigma_{\lambda}}$$

Donde λ_i es un valor del vector λ , μ_{λ} es la media del vector λ , y σ_{λ} es la desviación estándar del vector λ .



En esta situación, la normalización de los datos no es necesaria. Sin embargo, es una buena práctica a tener en cuenta porque transforma nuestra distribución en una media 0 y una desviación estándar de 1.

Cuando tengamos nuestro vector z-score, tenemos que calcular la mediana del vector. Entonces, tomaremos una posición larga para todos los activos con un z-score superior a la mediana y una posición corta para todos los activos con un z-score inferior a la mediana. (Esta es nuestra estrategia, pero podemos tomar el umbral que queramos).

4.1.3. Reequilibrio

El reequilibrio de una TAA es algo esencial. En efecto, es en este momento cuando se determinan las ponderaciones de cada activo de la cartera. En los siguientes ejemplos, utilizaremos una asignación con los mismos pesos para todos los activos para facilitar las cosas. La dinámica de la estrategia será únicamente el signo (positivo si estamos largos y negativo si estamos cortos).



Normalmente, tomamos el mismo peso para cada activo y luego combinamos las carteras SAA y TAA siguiendo esta ecuación^[2]: $\text{portfolio} = \alpha * \text{SAA} + (1 - \alpha) * \text{TAA}$

con $\alpha \in [0,1]$. Naturalmente, las técnicas avanzadas cambiaron el peso de cada mes, pero no se verá en el libro.

Figura 4.2: Asignación de pesos de la cartera TAA

Activos	Facebook	Netflix	Tesla
Pesos	-33% / +33%	-33% / +33%	-33% / +33%

Los pesos de los activos son los mismos, la única diferencia es que el contrato puede ser largo o corto cada mes. Así, el peso -33% o +33%.

4.2. Estrategia de media móvil

En esta sección se explicará cómo calcular una cartera táctica basada en un factor de media móvil. Explicaremos cómo funcionan las medias móviles y cómo hacer una cartera dinámica utilizando un factor de media móvil.

4.2.1. Media móvil

En esta subsección se calcularán algunas medias móviles para crear el factor de impulso en la segunda parte. La media móvil es el indicador técnico más fácil de entender. De hecho, es una media, pero en lugar de hacer la media en todas las muestras, crearemos un vector de una media de los n últimos días para cada día seleccionado.

Nos permite tener una mejor comprensión del comportamiento de las acciones. Podemos construir el vector de medias móviles con la siguiente fórmula.

$$mv_i = \frac{1}{n} \sum_{\Delta = i - n}^i x_{\Delta}$$

Donde mv_i es la media móvil en el índice i , n es la ventana (el número de días que deseas tomar para hacer la media móvil), x es el valor del activo.

Ahora, calcularemos una media móvil sobre los tres activos anteriores para una ventana de 15 días con este código.

Además, mostraremos los resultados en la figura 4.3.

Código 4.1: Cálculo de la media móvil

```
data["SMA15 FB"] = data["FB"].rolling(15).mean().shift(1)
data["SMA15 NFLX"] = data["NFLX"].rolling(15).mean().shift(1)
data["SMA15 TSLA"] = data["TSLA"].rolling(15).mean().shift(1)
```

Figura 4.3: Activos con su media móvil simple (SMA 15)



En el código 4.1, es necesario poner un desplazamiento de 1. Si no se pone, pronosticaremos el valor en $t+15$ en $t+14$ pero usando $t+15$ datos. Lo explicaremos más adelante.

Date	FB	NFLX	TSLA	SMA 15 FB	SMA 15 NFLX	SMA 15 TSLA
06-11	27.01	9.00	5.82	29.84	9.63	5.89
06-12	27.40	9.00	5.93	29.01	9.56	5.91
06-13	27.27	8.97	5.95	28.65	9.48	5.93

La media móvil suele utilizarse para comprender la tendencia del activo. La figura 4.4 muestra el precio de las acciones con la SMA (media móvil simple) de 15 y la SMA de 60. Así, puede ser fácil entender las tendencias al alza y a la baja.

Figura 4.4: Precio de las acciones de Facebook con SMA 15 y SMA 60



Esta figura muestra el precio de las acciones de Facebook con la SMA 15 y la SMA 60. Cuando la tendencia es alcista, vemos que la SMA rápida está por encima de la SMA lenta y lo contrario cuando la tendencia es bajista.

4.2.2. Factor de media móvil

En esta subsección, vamos a construir el factor de media móvil. De hecho, utilizaremos la SMA de tres meses y la SMA de 12 meses de cada acción (Facebook, Netflix, Tesla) para crear una señal mensual para reequilibrar nuestra cartera.

Utilizaremos las mismas muestras que en el capítulo anterior. Utilizaremos el 70% de los datos para crear la estrategia. Luego, la probaremos en el conjunto de test (el otro 30%). La diferencia entre el capítulo anterior y éste es que mantenemos el precio en valor absoluto y no en variaciones. Entonces, podemos calcular el factor que marca la diferencia entre la SMA corta y la larga. Además, estamos calculando el z-score

en la normalización del factor. Podemos ver cómo hacerlo en el código 4.2 y los resultados en la figura 4.5.

Código 4.2: Crear los z-scores

```
list_tickers = ["FB", "NFLX", "TSLA"]
# Hacemos un bucle para crear los SMA de cada activo
for col in list_tickers:
    data[f"pct {col}"] = data[col].pct_change(1)
    data[f"SMA3 {col}"] = data[col].rolling(3).mean().shift(1)
    data[f"SMA12 {col}"] = data[col].rolling(12).mean().shift(1)
    data[f"Momentum factor {col}"] = data[f"SMA3 {col}"] - \
        data[f"SMA12 {col}"]

# Normalización del zscore
split = int(0.7 * len(data))
train_set = data.iloc[:split, :]
test_set = data.iloc[split:, :]

# Encontrar los vectores mean y std
columns = [f"Momentum factor {col}" for col in list_tickers]
train_set_mean = train_set[columns].mean()
train_set_std = train_set[columns].std()

# Crear los zscores
train_set[columns] = (train_set[columns]-train_set_mean) /
    train_set_std
test_set[columns] = (test_set[columns]-train_set_mean) /
    train_set_std 1

# Encontrar las medianas
median = train_set[columns].median()
```

- 1 Cálculo de los z-scores del conjunto de test utilizando la media y la std del conjunto de entrenamiento para evitar interferencias en los datos. Normalmente, no podemos conocer la media y la std de este conjunto porque representan el futuro.

Figura 4.5: z-scores de los activos

Fecha	Momentum FB	Momentum NFLX	Momentum TSLA
06-01	-1.38	-0.36	0.16
07-01	-1.35	-0.39	0.68
08-01	-1.10	-0.42	1.23

En esta figura, podemos ver el factor de impulso z-score para cada activo de la cartera (Facebook, Netflix y Tesla).

4.2.3. Construir la estrategia

Esta subsección nos mostrará cómo construir la estrategia de cartera táctica utilizando el factor de media móvil. Lo haremos utilizando los z-scores calculados anteriormente.

En primer lugar, tenemos que calcular la señal de cada activo si tomamos una posición larga o corta en el próximo mes. Supongamos que la mediana del z-score es inferior al mes del z-score. En ese caso, tomamos una posición corta el mes siguiente, y si la mediana del z-score es superior, tomamos una posición larga en el activo.

Código 4.3: Cálculo de la señal y el beneficio para la estrategia SMA

```
# Calcular las señales y los beneficios
for i in range(len(columns)): 1

    # Inicializar una nueva columna para la señal
    test_set[f"signal {columns[i]}"] = 0 2

    # La señal es -1 si factor < median
    test_set.loc[test_set[f"{columns[i]}"] < median[i],
                  f"signal {columns[i]}"] = -1

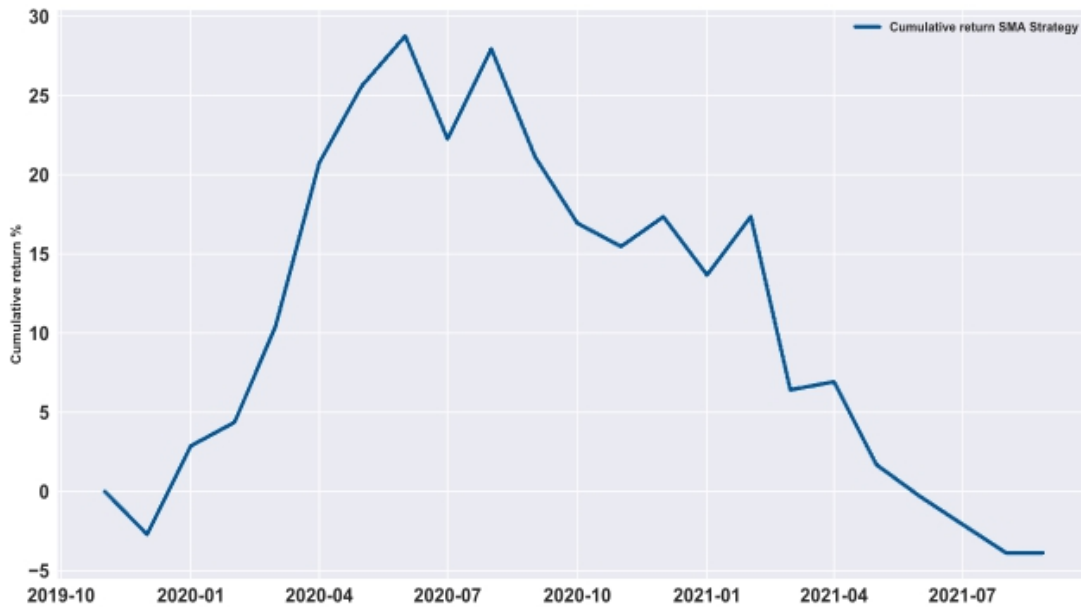
    # La señal es 1 si factor > median
    test_set.loc[test_set[f"{columns[i]}"] > median[i],
                  f"signal {columns[i]}"] = 1
```

```
# Calcular el beneficio
Test_set[f"profit {columns[i]}"] =
    (test_set[f"signal {columns[i]}"].shift(1)) * \
    test_set[f"pct {list_tickers[i]}"]
```

- 1 Bucle para calcular la señal y el beneficio de cada activo.
- 2 Crea una nueva columna para la señal.
- 3 Añade un desplazamiento a la señal porque la señal nos dice que entremos en posición. La ganancia señala los rendimientos de hoy y mañana porque abrimos la posición al cierre del activo. Si tomamos $signal_t * return_t$ tendremos resultados excelentes pero erróneos.

También nos hemos desplazado a la señal porque vemos el mes siguiente si es rentable cuando tenemos una señal. Si no nos desplazamos, obtendremos un beneficio masivo porque predeciremos el pasado con el futuro, no a la inversa. Ahora, veamos en la figura 4.6 los resultados de esta estrategia.

Figura 4.6: Rendimiento acumulado de la estrategia de cartera táctica



Esta figura muestra que esta estrategia no es muy rentable a largo plazo. A lo largo del periodo, tenemos un drawdown de alrededor del 30% para una rentabilidad del -3%.

4.3. Estrategia de seguimiento de la tendencia

En esta sección se hablará de la estrategia de seguimiento de la tendencia. En primer lugar, tenemos que explicar la correlación estadística de Pearson. Luego, encontraremos el mejor activo para esta estrategia. A continuación, crearemos una estrategia con nuestro factor de seguimiento de la tendencia.

4.3.1. Correlación

En esta sección se explicará en detalle cómo funciona la correlación y por qué es esencial en esta estrategia. En efecto, la correlación nos permitirá comprender si existe alguna relación entre las dos series temporales.

La correlación es una herramienta esencial en las finanzas. Nos permite entender la relación entre los activos. Sin

embargo, mucha gente la utiliza de forma equivocada. Por eso tenemos toda una parte dedicada a esta noción. Existen muchos tipos de correlación, pero la correlación de Pearson es la más útil. Calcula la correlación lineal entre las series temporales x e y con esta fórmula:

$$\rho_{x,y} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Donde x_i es una observación de la serie temporal x , y_i es una observación de la serie temporal y . $\bar{x}$ e $\bar{y}$ son respectivamente la media de las series temporales x e y .



En términos sencillos, la correlación capta cómo varían los valores de las dos series temporales con respecto a sus medias. Si el valor de las series temporales x e y está por debajo de su media o por encima simultáneamente con frecuencia, están correlacionadas.

Suele ser bueno tomarse el tiempo de analizar un poco esta ecuación. En efecto, como podemos ver, esta fórmula calcula la distancia de la media a la observación. Así, como muchos activos tienen una tendencia al alza, la correlación entre el precio absoluto del activo es positiva. Al mismo tiempo, la tendencia representa el comportamiento del activo. Para resolver este problema, necesitamos correlacionar el porcentaje de variación del activo.

Veamos algunos ejemplos para asegurarnos de que se entiende la noción.

En la figura 4.7, podemos ver la diferencia de valor entre la correlación del valor absoluto y la correlación del porcentaje de variación.

Figura 4.7: Correlación del valor absoluto y del porcentaje de retorno

Correlación del valor absoluto

	Facebook	Netflix	Tesla
Facebook	1		
Netflix	0.94	1	
Tesla	0.76	0.74	1

Correlación del porcentaje de variación

	Facebook	Netflix	Tesla
Facebook	1		
Netflix	0.31	1	
Tesla	0.29	0.19	1

Podemos ver que la diferencia entre el valor absoluto y el porcentaje de variación de la correlación es enorme. Algunas casillas están vacías porque la matriz es simétrica, lo que significa que la correlación entre Facebook y Netflix es igual a la correlación entre Netflix y Facebook.

Veamos ahora cómo interpretar la correlación de Pearson:

- **$-1 < \text{corr} < 0$** : Existe una correlación negativa entre los dos activos. Esto significa que si el primero aumenta, el segundo tiene más posibilidades de disminuir. Cuanto más se acerque la correlación de Pearson a -1, mayor será la probabilidad de que disminuya.
- **$\text{Corr} = 0$** : No hay correlación. Eso significa que si uno aumenta o disminuye, no podemos decir nada sobre el comportamiento del otro activo.
- **$0 < \text{corr} < 1$** : Existe una correlación positiva entre los dos activos. Eso significa que si el primero aumenta, el segundo tiene más posibilidades de aumentar. Cuanto

más cerca está la correlación de Pearson de 1, la probabilidad de aumentar es mayor.



Si los activos tienen una sólida correlación, a menudo se secarán en la misma dirección, pero **no necesariamente con la misma intensidad**.

4.3.2. Factor de seguimiento de la tendencia

En esta subsección, tenemos que cambiar un poco los activos. Probablemente nos preguntemos por qué, para esta última estrategia, debemos cambiar el activo en su lugar. Hemos dicho que tomamos sólo el mismo para comparar la estrategia en el próximo capítulo. Es cierto, pero esta estrategia tiene un proceso diferente de los demás.

Esta estrategia utiliza 12 de los rendimientos del mes pasado para crear una inversión de factor. Además, la posición se reequilibra cada mes. Sin embargo, para encontrar el mejor activo, tenemos que utilizar la correlación en muchos activos para encontrar los mejores. Así que, naturalmente, los mejores activos no son los mismos que los anteriores.

Para encontrar los mejores activos, tomaremos diez activos, pero podemos tomar más para tener una mejor elección. Además, calculamos la autocorrelación entre los 12 del mes pasado y los del mes siguiente para encontrar el mejor activo para la estrategia en el conjunto de entrenamiento. Esto significa que buscamos los activos con la mejor correlación entre el periodo de revisión y el periodo de mantenimiento (también se puede encontrar el mejor periodo de revisión y mantenimiento con esta técnica).

Código 4.4: La mejor selección de activos

```
# Calcular el periodo de espera y el periodo de retención
for col in list_:
    data[f"pct+1 {col}"] = data[f"{col}"].pct_change(-1)
```

```

data[f"pct-12 {col}"] = data[f"{col}"].pct_change(12) 2
# Normalizar el zscore
split = int(0.7 * len(data))
train_set = data.iloc[:split, :]
test_set = data.iloc[split:, :]

# Calcular la correlación
corr = []
for col in list_:
    cor = train_set[[f"pct-12 {col}",
                    f"pct+1 {col}"]].corr().values[0][1]

    corr.append(cor)

correlation = pd.DataFrame(corr, index=list_, columns = ["Corr"]) 3
correlation.sort_values(by = "Corr", ascending = False)

```

- 1 Cálculo de la rentabilidad del periodo de retención (1 mes).
- 2 Cómputo de la rentabilidad del periodo de retrospección (12 meses).
- 3 List_ es una lista que contiene todos los activos que queremos probar para la estrategia.

Figura 4.8: Mejores activos con la anterior correlación de activos

Asset	INTC	GOOG	PYPL	TSLA	FB	NFLX
Corr	0.29	0.24	0.19	0.17	0.09	0.07

Este gráfico muestra la correlación entre el periodo de espera y el de retención de los tres mejores activos y el anterior.

Ahora, tenemos tres activos (Intel, Google y PayPal) para calcular una estrategia. En primer lugar, tenemos que calcular el z-score para encontrar la señal de inversion. Entonces, como antes, tomaremos una posición corta si el z-score está

por debajo de la mediana y tomaremos una posición larga si el z-score está por encima de la mediana.

Código 4.5: Crear los z-scores

```
best = ["PYPL", "INTC", "GOOG"]

# Encontrar los vectores mean y std
columns = [f"pct-12 {col}" for col in best]
train_set_mean = train_set[columns].mean()
train_set_std = train_set[columns].std()

# Crear los zscores
train_set[columns] = (train_set[columns] - train_set_mean) /
    train_set_std
test_set[columns] = (test_set[columns] - train_set_mean) /
    train_set_std

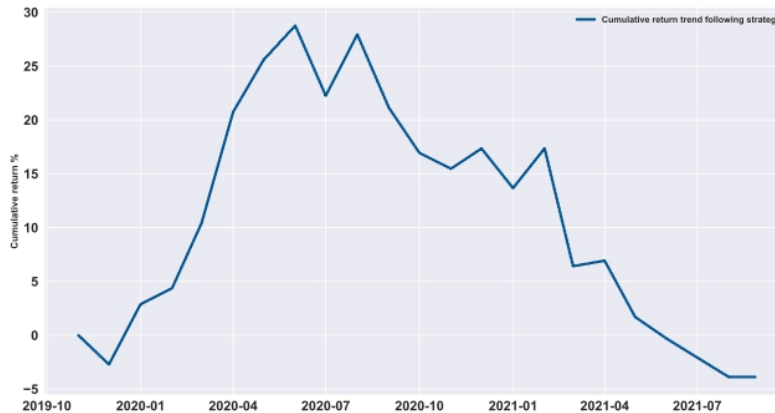
# Encontrar las medianas
median = train_set[columns].median()
```

4.3.3. Calcular la estrategia

En esta parte, calcularemos la estrategia. Para hacerlo, es precisamente el mismo código que para la estrategia de media móvil. Ahora, veamos en la figura 4.9 los resultados de esta estrategia.

A diferencia de la estrategia SMA después de la crisis del COVID, podemos ver una disminución en el rendimiento acumulado. Puede explicarse por el hecho de que la crisis desvía los rendimientos de los 12 últimos meses.

Figura 4.9: Rendimiento de la estrategia de retorno de la tendencia



En esta figura, podemos ver el beneficio de la estrategia de retorno de la tendencia.

Resumen

- Una posición corta nos permite apostar por el descenso de una acción.
- Las estrategias tácticas de cartera son una excelente manera de obtener beneficios cuando el mercado sube o baja
- La cartera TAA es útil para corregir las imperfecciones de la SAA. Por lo tanto, es necesario combinar ambas cosas para tener una buena cartera.
- La media móvil es una forma excelente de tener una media más significativa.
- La correlación es una forma excelente de entender la relación entre dos acciones de comportamiento.

Capítulo 5: Gestión de riesgos y backtesting

En este capítulo se realizará un backtest de algunas estrategias de inversión anteriores utilizando métricas de gestión del riesgo y otros indicadores. Comenzaremos con las métricas de backtest más conocidas, como beta, alfa, ratio de Sharpe, ratio de Sortino y drawdown. A continuación, explicaremos cómo realizar el análisis de riesgos mediante el VaR, el cVaR y las contribuciones al riesgo. Además, automatizaremos este análisis para aplicarlo a estrategias posteriores.

5.1. Las métricas de backtesting

En esta sección hablaremos de las métricas CPAM para entender la relación entre nuestra estrategia y el mercado. A continuación, calcularemos el drawdown de nuestras estrategias para entender cuánto tiempo perdemos dinero durante el periodo en el que seguimos una estrategia.

5.1.1. Las métricas CAPM

Primero, hablaremos sobre las métricas CAPM^[3]. De hecho, el CAPM es el modelo más famoso para la gestión de carteras y nos proporciona algunas métricas interesantes: la beta y el alfa.

La métrica beta de una acción pone de manifiesto la relación entre el mercado representado por un índice bursátil como el S&P 500 y la cartera. Por ejemplo, una beta de 1.15 implica que si el mercado tiene una variación del 1%, la cartera varía al 1.15%. Puede considerarse una métrica del riesgo sistémico de la cartera. Se calcula mediante la siguiente fórmula:

$$\beta_s = \frac{\text{Cov}(r_s, r_m)}{\text{Var}(r_m)}$$

r_s es la rentabilidad del activo y r_m es la rentabilidad del índice de mercado.

Cuando calculemos la beta, nos encontraremos en una de estas situaciones:

- $\beta_s < 1$: Significa que la cartera es menos volátil que el mercado, lo que es bueno porque no tiene un riesgo sistémico significativo.
- $\beta_s = 1$: La cartera se mueve en la misma proporción del mercado. Supongamos que el mercado se mueve al 1%, y la cartera se mueve al 1%.
- $\beta_s > 1$: La cartera es más volátil que el mercado y tiene un importante riesgo sistémico.

Ahora, vamos a crear una función para calcular la beta de una cartera. (En este ejemplo, el índice de referencia será el S&P 500 y su símbolo de Yahoo es "^GSPC").

Código 5.1: función de coeficiente beta

```
def beta_function(portfolio, ben = "^GSPC"): 1
    """
    -----
    | Output: Métrica Beta CAPM                |
    -----
    | Inputs: - portfolio (type dataframe pandas): Retornos de la cartera |
    |          - ben (type string): Nombre del benchmark                |
    -----
    """
    # Importar el benchmark
```

```

benchmark = yf.download(ben)["Adj Close"].pct_change(1).dropna()

# Concatenar el active y el benchmark

join = pd.concat((portfolio, benchmark), axis = 1).dropna() 2

# Covarianza entre el activo y el benchmark

cov = np.cov(join, rowvar = False)[0][1] 3

# Calcular la varianza del benchmark

var = np.cov(join, rowvar = False)[1][1]

return cov/var

```

- 1 La configuración por defecto del índice de referencia es S&P 500 (^GSPC), pero podemos poner todos los índices que queramos en función del mercado que queramos tomar.
- 2 Creación de un dataframe con los rendimientos de la estrategia y del índice de referencia para eliminar los días que no están en ambas series temporales.
- 3 Es esencial especificar `rowvar = False` porque, sin eso, Python considerará que cada fila representa una serie temporal en lugar de cada columna. Por lo tanto, si no establecemos este parámetro, tendremos un mal resultado.



Hemos calculado la varianza de referencia utilizando la matriz de covarianza porque los valores diagonales son la varianza de los activos.

Con esta función, hemos obtenido una beta de 1.08 para la cartera utilizando el criterio de media-varianza. Esto significa que cuando el mercado (S&P 500) varía un 1%, la cartera varía un 1.08%.

Hablemos ahora de la métrica alfa. Nos permite entender si una cartera supera o no al índice de referencia. Por ejemplo, supongamos que tenemos una cartera con un alfa de 0.0115. En ese caso, la cartera supera al índice de referencia en un

1.15%, teniendo en cuenta la rentabilidad y el riesgo de la cartera. Puede calcularse mediante la siguiente fórmula:

$$\alpha_s = E[r_s] - r_f - \beta_s(E[r_m] - r_f)$$

Donde $E[r_s]$ es la media de la rentabilidad del activo y $E[r_m]$ es el rendimiento del índice de mercado, r_f el rendimiento del activo sin riesgo y β_s la beta del activo.



Cuando calculamos una media de rentabilidad, volatilidad, ratio de Sharpe, etc. necesitamos anualizar los valores (siempre). Porque si queremos comparar algunas estrategias con diferentes marcos temporales, puede ser muy difícil.



El activo libre de riesgo está muy cerca del 0%, incluso negativo en algunas situaciones. Por ello, se prefiere fijarlo en el 0% para facilitar el cálculo y la comprensión.

Cuando calculemos el alfa, estaremos en una de estas situaciones:

- $\alpha_s < 0$: Eso significa que la cartera tiene un rendimiento inferior al del índice de referencia. Esto no es bueno porque habría sido una mejor inversión si hubiéramos comprado el índice de referencia.
- $\alpha_s = 0$: Esto significa que la cartera no rinde ni por debajo ni por encima del índice de referencia, lo que no es bueno porque significa que si acabáramos de comprar el índice de referencia, es la misma pareja riesgo-rendimiento.

- $\alpha_s > 0$: Esto significa que la cartera supera al índice de referencia, y esto es algo bueno. Significa que hemos invertido mejor que el índice de referencia.

Código 5.2: función de coeficiente alpha

```
def alpha_function(portfolio, ben = "^GSPC", timeframe = 252):
    """
    -----
    | Output: Métrica Alpha CAPM                               |
    -----
    | Inputs: - portfolio (type dataframe pandas): Retornos de la cartera |
    |         - ben (type string): Nombre del benchmark                |
    |         - timeframe (type int): Factor de anualización          |
    -----
    """

    # Importar el benchmark
    benchmark = yf.download(ben)["Adj Close"].pct_change(1).dropna()

    # Concatenar el active y el benchmark
    join = pd.concat((portfolio, benchmark), axis = 1).dropna()

    # Calcular la beta
    beta = beta_function(portfolio_return_MV, ben = ben) 1

    mean_stock_return = join.iloc[:,0].mean() * timeframe
    mean_market_return = join.iloc[:,1].mean() * timeframe
    return mean_stock_return - beta * mean_market_return
```

- 1 Uso de la función beta_function creada anteriormente para calcular la beta.



Hemos establecido un nuevo parámetro denominado timeframe para poder gestionar la anualización de los datos. En efecto, no podemos multiplicar por el mismo factor si tenemos rendimientos mensuales y diarios. En

este caso, el marco temporal es 252 porque hay rendimientos diarios.

Hemos obtenido un alfa del 35% para la cartera utilizando el criterio de media-varianza con esta función. Esto significa que la cartera supera en un 35% al benchmark.

5.1.2. Sharpe y Sortino

En esta parte, hablaremos de algunas métricas que ya hemos visto. De hecho, estas métricas se utilizan en el capítulo 3: los ratios de Sharpe y Sortino. Sin embargo, vamos a profundizar en el tema para entender estas métricas.

Hablemos del ratio de Sharpe. Esta métrica es muy utilizada en finanzas; como dijimos antes, nos permite entender los beneficios adicionales de un 1% más de riesgo.

$$\text{Sharpe} = \frac{\mu - r_f}{\sigma}$$

Donde μ es la media de los rendimientos de la cartera (anualizada), σ es la volatilidad de los rendimientos de la cartera (anualizada) y r_f es el activo libre de riesgo (fijamos 0 para el activo libre de riesgo porque en realidad está en torno a este valor).



Para anualizar la volatilidad, tenemos que multiplicar la volatilidad por la raíz cuadrada del factor de anualización. Por ejemplo, si queremos anualizar la volatilidad diaria, hacemos $\sigma_{\text{daily}} * \sqrt{252}$.

Cuando calculemos el ratio de Sharpe, nos encontraremos en una de estas situaciones:

- **sharpe < 0**: No es una buena inversión. Asumimos algunos riesgos, pero tenemos un rendimiento esperado negativo en nuestra cartera.
- **0 < sharpe < 1**: Esta es una buena inversión. Tenemos un rendimiento esperado positivo, pero el riesgo que asumimos es mayor que el rendimiento.
- **sharpe > 1**: Esta es una excelente inversión. Tenemos una rentabilidad esperada positiva, pero con un riesgo asumido menor que el rendimiento.

Código 5.3: Función del ratio de Sharpe

```
def sharpe_function(portfolio, timeframe = 252):
    """
    -----
    | Output: Métrica del ratio Sharpe |
    -----
    | Inputs: - portfolio (type dataframe pandas): Retornos de la cartera |
    |         - timeframe (type int): Factor de anualización |
    -----
    """
    mean = portfolio.mean() * timeframe
    std = portfolio.std() * np.sqrt(timeframe)

    return mean/std
```

Utilizando el criterio de media-varianza con esta función, hemos obtenido un ratio de Sharpe de 1.37 para la cartera. Se trata de una cartera excepcional, ya que un 1% de riesgo adicional da un 1.37% de rentabilidad.

Ahora, hablemos del ratio de Sortino. El ratio de Sortino es una métrica excelente porque deriva del ratio de Sharpe, que sólo considera la volatilidad a la baja. Por lo tanto, nos permite entender los beneficios adicionales por un 1% más de bajo

riesgo. Podemos calcular el ratio de Sortino con la siguiente fórmula:

$$\text{Sortino} = \frac{\mu - r_f}{\sigma_{\text{downward}}}$$

Donde μ es la media de los rendimientos de la cartera (anualizada), σ_{downward} es la volatilidad a la baja de los rendimientos de la cartera (anualizada) y r_f es el activo libre de riesgo (fijamos 0 para los activos libres de riesgo porque en realidad está en torno a este valor).



El ratio de Sortino puede ser mejor que el ratio de Sharpe porque sólo es sensible a la volatilidad a la baja. Por lo tanto, es más representativo de lo que desea el inversor.

Cuando calculemos el ratio de Sortino, nos encontraremos en una de estas situaciones:

- **sortino < 0**: No es una buena inversión. Asumimos un riesgo bajo, pero tenemos un rendimiento esperado negativo en nuestra cartera.
- **0 < sortino < 1**: Se trata de una buena inversión. Tenemos un rendimiento esperado positivo, pero el bajo riesgo que asumimos es mayor que el rendimiento.
- **sortino > 1**: Esta es una excelente inversión. Tenemos un rendimiento esperado positivo, además de un riesgo bajo asumido menor que los rendimientos.



La única diferencia entre los ratios de Sharpe y Sortino es la forma de calcular el riesgo. Uno elige la volatilidad y otro la volatilidad a la baja.

Código 5.4: Función de ratio Sortino

```
def sortino_function(portfolio, timeframe = 252):  
    """  
    -----  
    | Output: Métrica del ratio Sortino |  
    -----  
    | Inputs: - portfolio (type dataframe pandas): Retornos de la cartera |  
    |           - timeframe (type int): Factor de anualización |  
    -----  
    """  
  
    # Tomar valores a la baja  
    portfolio = portfolio.values  
    downward = portfolio[portfolio < 0]  
  
    mean = portfolio.mean() * timeframe  
    std = downward.std() * np.sqrt(timeframe)  
  
    return mean/std
```

Con esta función, hemos obtenido un ratio de Sortino de 1.87 para la cartera utilizando el criterio de media-varianza. Se trata de una cartera excelente y un 1% de riesgo adicional a la baja da un 1.87% de rentabilidad.

5.1.3. Drawdown

En esta parte, hablaremos del drawdown. Es una de las mejores métricas para un backtest. De hecho, nos permite entender la pérdida significativa que sufrimos. Nos da la pérdida más destacada si entramos en una posición en el peor momento de este periodo. Ahora bien, puede ser complicado de entender, pero es algo sencillo. Veamos. Primero hay que explicar la fórmula para calcular el drawdown:

$$\delta_i = \frac{\varphi_i}{\phi_i} - 1$$

Donde δ_i es el drawdown en el momento i , φ_i el producto de los rendimientos acumulados hasta el momento i y ϕ_i es el máximo de los productos anteriores hasta el momento i para i en *[fecha de inicio de la cartera: fecha de finalización de la cartera]*.

Código 5.5: función del vector de reducción de la demanda

```
def drawdown_function(portfolio):
    """
    -----
    | Output: Drawdown                                |
    -----
    | Inputs: - portfolio (type dataframe pandas): Retornos de la cartera |
    -----
    """

    # Calcular el producto acumulado de los rendimientos
    cum_rets = (portfolio+1).cumprod() 1

    # Calcular el máximo producto acumulado
    running_max = np.maximum.accumulate(cum_rets.dropna()) 2
    running_max[running_max < 1] = 1

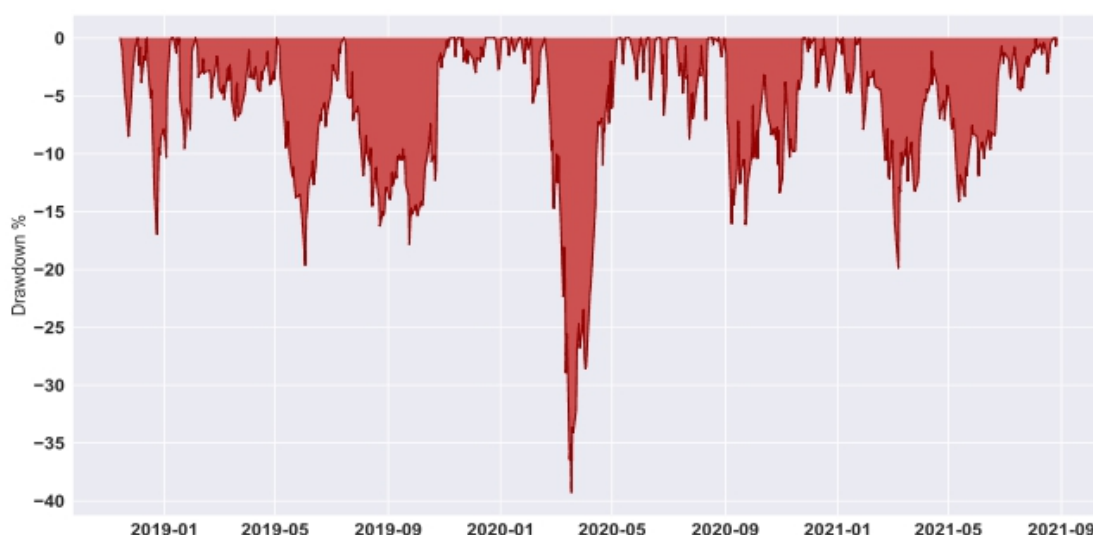
    # Calcular el drawdown
    drawdown = ((cum_rets)/running_max - 1) 3

    return drawdown
```

- 1 Cálculo de la rentabilidad acumulada utilizando el interés compuesto (puedes leer el anexo sobre este tema) porque es más sensible a las pérdidas.
- 2 Cálculo del valor máximo acumulado.
- 3 Cálculo del drawdown, que es la relación entre la rentabilidad acumulada y el máximo acumulado.

A continuación, podemos calcular el drawdown de la cartera de media-varianza. Podemos visualizar el drawdown de la cartera en la figura 5.1.

Figura 5.1: Drawdown de la cartera de media-varianza



Esta figura muestra la drawdown de la cartera de varianza media calculada en el capítulo 3 de este libro.

Como podemos ver, la reducción máxima es de alrededor del 40% durante la crisis del COVID. Esto se debe a que se trata de una estrategia de sólo largo plazo. Excepto en este

periodo, el drawdown es de alrededor del 15%. Esto significa que si entramos en posición con esta cartera justo antes de la crisis, hemos perdido el 40% de nuestro capital en 1 mes. Sin embargo, después de mayo, hemos recuperado nuestro capital al 100%.



Podemos calcular la rentabilidad acumulada de la cartera utilizando la suma acumulada o la suma del producto. La suma acumulada se utiliza para la estrategia de lote fijo y el producto acumulado para la estrategia de lote automático.

5.2. Métodos de gestión de riesgos

En esta sección hablaremos de las métricas de gestión de riesgos. Veremos la métrica de gestión de riesgos más conocida, el valor en riesgo (VaR). Luego veremos un derivado de esta métrica: el valor en riesgo condicional (cVaR). Además, aprenderemos a calcular la contribución al riesgo de cada activo de la cartera.

5.2.1. Valor en riesgo (VaR)

En este subapartado hablaremos del Valor en Riesgo (VaR). Se trata de una excelente métrica del riesgo. En efecto, nos permite comprender cuánto perdemos en el peor de los casos, considerando un umbral de error. En primer lugar, veamos una fórmula para calcular el VaR (hay muchas y sólo veremos una).

$$\text{VaR}_\theta = \mu + \sigma \phi^{-1}(\theta)$$

Donde μ es la rentabilidad de la cartera, σ la volatilidad de la cartera, θ el umbral de error y ϕ la CDF de la distribución $N(0,1)$.

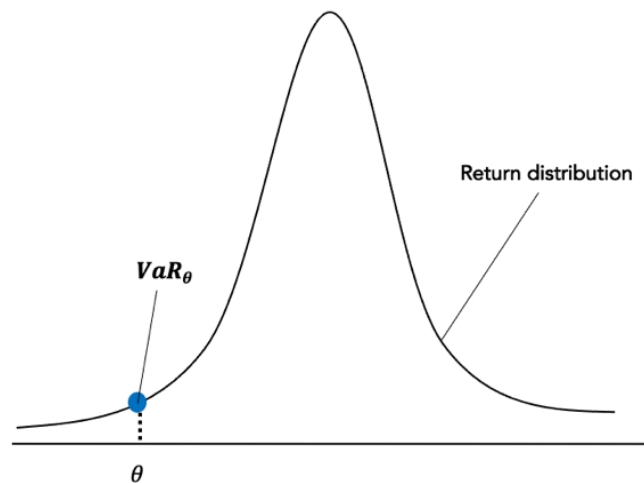
El VaR depende del marco temporal que utilicemos.



Tendremos una peor pérdida diaria si utilizamos la distribución de rentabilidad diaria. Por lo tanto, si queremos la menor pérdida anual, tenemos que anualizar los rendimientos y la volatilidad antes.

La reflexión con la inversa de la función CDF es un poco complicada. Podemos explicar el concepto más fácilmente en la figura 5.2.

Figura 5.2: Explicación del VaR



El objetivo del VaR es encontrar el valor que tiene $\theta\%$ de los valores por encima de él. Por ejemplo, si theta es igual al 5%, el VaR es el valor con el 5% de los rendimientos ordenados por encima de él.

Código 5.6: función VaR

```
def VaR_function(theta, mu, sigma):  
    """  
    -----  
    | Output: VaR                               |  
    -----  
    | Inputs: - theta (type float): umbral del error en %           |  
    |           - mu (type float): retorno esperado de la cartera    |  
    |           - sigma (type float): volatilidad de la cartera      |  
    """
```

```

"""
# Número de simulaciones
n = 100000

# Encontrar los valores para el umbral de error theta
t = int(n * theta)

# Crear un vector con n simulaciones de la ley normal
vec = pd.DataFrame(np.random.normal(mu, sigma, size = (n, )),
                    columns = ["Simulations"])

# Ordena los valores y encuentra el valor theta%
var = vec.sort_values(by = "Simulations").iloc[t].values[0]

return var

```



Para calcular el VaR más fácilmente, hemos utilizado la ley de los grandes números. En efecto, para hallar el valor $\theta\%$, hemos simulado 100.000 simulaciones d $N(\mu, \sigma)$ y hemos tomado el valor $\theta\%$ del vector ordenado.

Utilizando esta función, podemos hallar el VaR para la cartera de varianza media. De hecho, para esta cartera, encontramos un VaR del 3.87% para un día, del 13.61% para un mes y del 7.02% para un año. Significa que en el 5% del peor de los casos, podemos perder más del 3.87% por día, el 13.61% por mes y el 7.02% por año con esta estrategia.

El VaR es una métrica interesante. Sin embargo, no tiene en cuenta los extremos (sean muchos o no) de los valores inferiores al VaR. Para ello, necesitamos calcular el cVaR.

5.2.2. Valor en riesgo condicional (cVaR)

En esta subsección se analizará el Valor en riesgo condicional (cVaR). Se trata de un derivado del VaR. De hecho, esta

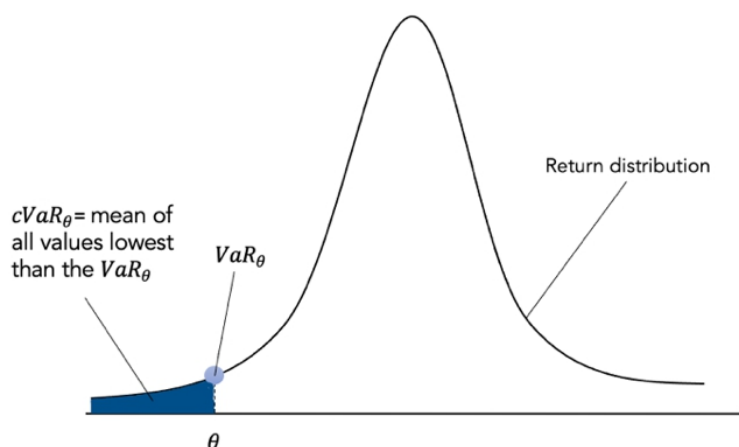
métrica está en la misma línea que el VaR, pero tiene en cuenta los valores extremos, a diferencia del VaR ordinario.

En efecto, el cVaR hace una media de cada valor por debajo del VaR. Pongamos la fórmula del cVaR ante un esquema en la figura 5.3 para entenderlo más fácilmente:

$$cVaR_{\theta} = \mu + \sigma \frac{\varphi(\Phi^{-1}(\theta))}{1 - \theta}$$

Donde μ es la rentabilidad de la cartera, σ la volatilidad de la cartera, θ el umbral de error, φ la PDF (Función de Distribución Parcial) de $N(0,1)$ y Φ la CDF (Función de Distribución Acumulada) de $N(0,1)$.

Figura 5.3: Explicación de cVaR



En esta figura, podemos ver cómo calcular el cVaR. En efecto, es la media de los valores inferiores al VaR.

Código 5.7: Función para cVaR

```
def cVaR_function(theta, mu, sigma):
```

```
    """
```

```
    -----
```

```
    | Output: cVaR
```

```
    |
```



```

-----
| Inputs: - theta (type float): umbral del error en %          |
|           - mu (type float): retorno esperado de la cartera |
|           - sigma (type float): volatilidad de la cartera   |
|-----|
"""

# Número de simulaciones
n = 100000

# Encontrar los valores para el umbral de error theta
t = int(n*theta)

# Crear un vector con n simulaciones de la ley normal
vec = pd.DataFrame(np.random.normal(mu, sigma, size = (n, )),
                   columns = ["Simulations"])
# Ordena los valores y encuentra el valor theta%
cvar = vec.sort_values(by = "Simulations").iloc[0:t, :].
       mean().values[0]

return cvar

```

Utilizando esta función, podemos hallar el cVaR para la cartera de varianza media. De hecho, para esta cartera, encontramos un cVaR del 4.87% para un día, del 18.28% para un mes y del 24.20% para un año. Esto significa que en el 5% del peor de los casos, podemos perder más del 4.87% por día, el 18.28% por mes y el 24.20% por año con esta estrategia.



Estoy utilizando el cVaR. Es muy interesante porque tiene en cuenta las pérdidas monetarias significativas por debajo del VaR.

5.2.3. Contribución al riesgo

La contribución al riesgo es una métrica interesante porque nos permite conocer el porcentaje de riesgo de cada activo.

De hecho, esta métrica nos permite saber qué activo añade más riesgo a la cartera.

Para calcular la contribución al riesgo, necesitamos la beta de los activos y el peso de cada activo en la cartera. Veamos cómo calcular con la fórmula:

$$\lambda_i = \beta_i * \omega_i$$

Donde λ_i es la contribución al riesgo de cada activo, β_i es la beta de los activos y ω_i es el peso del activo i en la cartera.



Para calcular el porcentaje de cada activo de riesgo en la cartera tenemos que dividir el λ_i entre la suma ($\sum \lambda_i$) para tener una base del 100%.

Código 5.8: Función de contribución al riesgo

```
def CR_function(weights, database, ben = "^GSPC"):
    """
    | Output: Métrica de la contribución al riesgo |
    |-----|
    | Inputs: - weights (type 1d array numpy): pesos de la cartera |
    |         - database (type dataframe pandas): Retornos de los activos |
    |         - ben (type string): Nombre del benchmark |
    |-----|
    """

    # Encontrar el número de activos en la cartera
    l = len(weights)

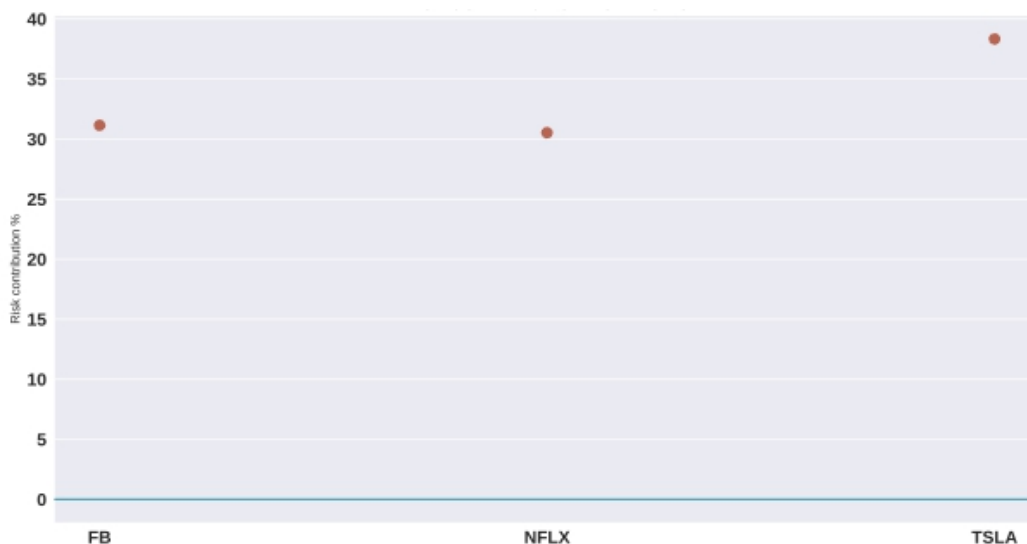
    # Calcular la contribución al riesgo de cada activo
    crs = []
    for i in range(l):
        cr = beta_function(data.iloc[:,i]) * weights[i]
        crs.append(cr)
```

```
# Normalizar por la suma de la contribución al riesgo
```

```
return crs / np.sum(crs)
```

Utilizando esta función, podemos hallar la contribución al riesgo de cada activo. En efecto, Facebook representa el 31.04% del riesgo de la cartera, Netflix el 30.61% y Tesla el 38.34%. También podemos representar estos valores en un gráfico como el de la figura 5.4.

Figura 5.4: Gráfico de contribución al riesgo



En esta figura, podemos ver la contribución de cada activo al riesgo de la cartera.

Ahora, tenemos las métricas necesarias para el backtest de una estrategia. En la siguiente sección, vamos a crear una función para aplicar fácilmente a cada estrategia.

5.3. Automatizar el análisis

Esta sección creará una función de backtesting y haremos backtesting de todas las estrategias anteriores.

5.3.1. Crear una función

En esta subsección, retomaremos las secciones 5.1 y 5.2 para crear una función, que se utilizará en el próximo capítulo. Veamos el código y expliquemos algunos detalles.

Código 5.9: Función para backtesting

```
def backtest(weights, database, ben = "^GSPC",
             timeframe = 252, CR = False):
    """
    -----
    | Output: Métrica Beta CAPM                               |
    -----

    | Inputs: - weights (type 1d array numpy): pesos de la cartera |
    |         - database (type dataframe pandas): Retornos de los activos |
    |         - ben (type string): Nombre del benchmark           |
    |         - timeframe (type int): Factor de anualización      |
    -----
    """

    # Calcular la cartera
    portfolio = np.multiply(database, np.transpose(weights))
    portfolio = portfolio.sum(axis = 1)
    columns = database.columns
    columns = [col for col in columns]

    ##### CALCULAR LA BETA #####
    # Importar el benchmark
    benchmark = yf.download(ben)["Adj Close"].pct_change(1).dropna()

    # Concatenar el activo y el benchmark
    join = pd.concat((portfolio, benchmark), axis = 1).dropna()

    # Covarianza entre el activo y el benchmark
    cov = np.cov(join, rowvar = False)[0][1]

    # Calcular la varianza del benchmark
    var = np.cov(join, rowvar = False)[1][1]
```

```
beta = cov/var
```

```
##### CALCULAR EL ALPHA #####
```

```
# Media de los rendimientos del activo
```

```
mean_stock_return = join.iloc[:, 0].mean() * timeframe
```

```
# Media de los rendimientos del mercado
```

```
mean_market_return = join.iloc[:, 1].mean() * timeframe
```

```
# Alpha
```

```
alpha = mean_stock_return - beta * mean_market_return
```

```
##### CALCULAR EL SHARPE #####
```

```
mean = portfolio.mean() * timeframe
```

```
std = portfolio.std() * np.sqrt(timeframe)
```

```
Sharpe = mean/std
```

```
##### CALCULAR EL SORTINO #####
```

```
downward = portfolio[portfolio < 0]
```

```
std_downward = downward.std() * np.sqrt(timeframe)
```

```
Sortino = mean / std_downward
```

```
##### CALCULAR EL DRAWDOWN #####
```

```
# Calcular el producto acumulado de los retornos
```

```
cum_rets = (portfolio + 1).cumprod()
```

```
# Calcular el máximo de los productos anteriores
```

```
running_max = np.maximum.accumulate(cum_rets.dropna())
```

```
running_max[running_max < 1] = 1
```

```
# Calcular el drawdown
```

```
drawdown = ((cum_rets) / running_max - 1)
```

```
min_drawdon = -drawdown.min()
```

```
##### CALCULAR EL VaR #####
```

```
theta = 0.01
```

```
# Número de simulaciones
```

```
n = 100000
```

```

# Encontrar los valores para el umbral de error theta
t = int(n * theta)

# Crear un vector con n simulaciones de la ley normal
vec = pd.DataFrame(np.random.normal(mean, std, size = (n, )),
                    columns = ["Simulations"])

# Ordena los valores y encuentra el valor theta%
VaR = -vec.sort_values(by = "Simulations").iloc[t].values[0]

##### CALCULAR EL cVaR #####
cVaR = -vec.sort_values(by = "Simulations").iloc[0:t, :].mean() \
        .values[0]

##### CALCULAR EL RC #####
if CR:
    # Encontrar el número del activo en la cartera
    l = len(weights)

    # Calcular la contribución al riesgo de cada activo
    crs = []
    for i in range(l):
        cr = beta_function(data.iloc[:, i]) * weights[i]
        crs.append(cr)

    # Normalizando por la suma de la contribución al riesgo
    crs = crs / np.sum(crs)

##### PLOT DE LOS RESULTADOS #####
print(f""" -----
Portfolio: {columns}

-----

Beta : {np.round(beta, 3)} \t Alpha: {np.round(alpha, 3)} \t \
Sharpe: {np.round(Sharpe, 3)} \t Sortino: {np.round(Sortino, 3)}

-----

VaR : {np.round(VaR, 3)} \t cVaR: {np.round(cVaR, 3)} \t \

```

```

VaR/cVaR: {np.round(cVaR/VaR, 3)}

"""

plt.figure(figsize = (10, 6))
plt.plot(portfolio.cumsum())
plt.title("CUMULATIVE RETURN", size = 15)
plt.show()

plt.figure(figsize = (10, 6))
plt.fill_between(drawdown.index, drawdown * 100,
                 0, color = "#E95751")
plt.title("DRAWDOWN", size = 15)
plt.show()

if CR:
    plt.figure(figsize = (10, 6))
    plt.scatter(columns, crs, linewidth = 3, color = "#B96553")
    plt.axhline(0, color = "#53A7B9")
    plt.grid(axis = "x")
    plt.title("RISK CONTRIBUTION PORTFOLIO", size = 15)
    plt.xlabel("Assets")
    plt.ylabel("Risk contribution")
    plt.show()

```



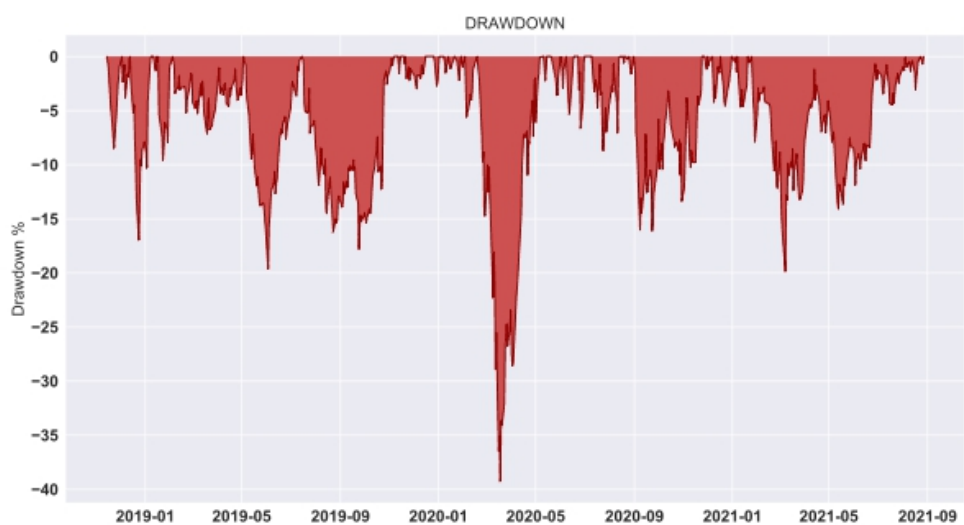
Incorporar un parámetro que muestre o no determinadas métricas evita tener que crear muchas funciones.

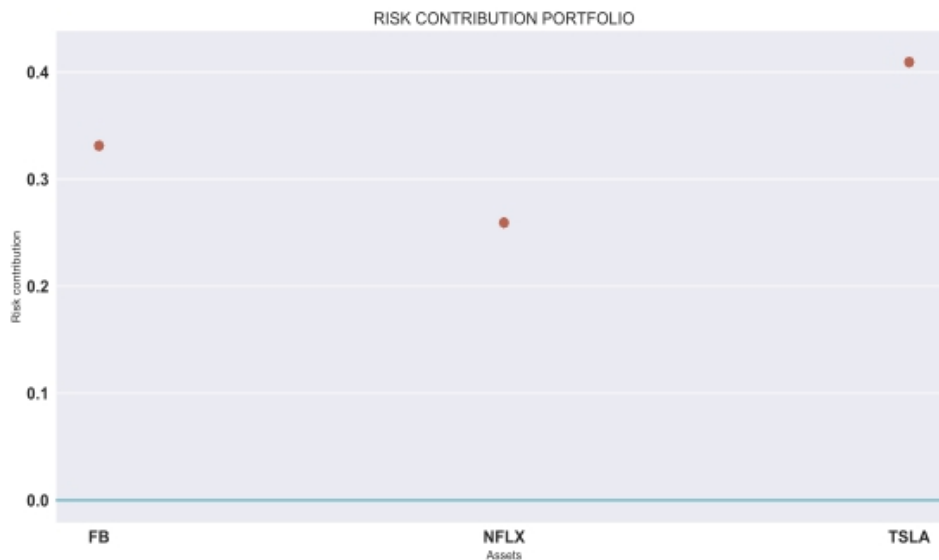
Figura 5.5: Backtest para la cartera de media-varianza

Portfolio: ['FB', 'NFLX', 'TSLA']

Beta: 1.066 Alpha: 35.45 % Sharpe: 1.51 Sortino: 2.034

VaR: 30.74 % cVaR: 42.98 % VaR/cVaR: 1.398 drawdown: 39.27 %





Esta figura muestra el backtest de la estrategia media-varianza con métricas y algunos gráficos para comprender mejor el comportamiento de la cartera.

5.3.2. Analizar la cartera estática

Con la función de backtesting, ahora podemos comparar carteras estáticas entre sí.



Todas las estrategias son casi iguales porque sólo permiten la posición larga y la correlación entre estos activos es muy alta.



Sería útil encontrar activos con una correlación negativa de rendimientos. Proporcionaría una excelente diversificación de la cartera.

Figura 5.6: Métricas para la cartera de media-varianza (Recordatorio)

Portfolio: ['FB', 'NFLX', 'TSLA']

Beta: 1.066 Alpha: 35.45 % Sharpe: 1.51 Sortino: 2.034

VaR: 30.74 % cVaR: 42.98 % VaR/cVaR: 1.398 drawdown: 39.27 %

Esta figura muestra que esta estrategia es perfecta porque el alfa es igual al 35.45%. Sin embargo, un VaR enorme implica un riesgo importante en esta estrategia.

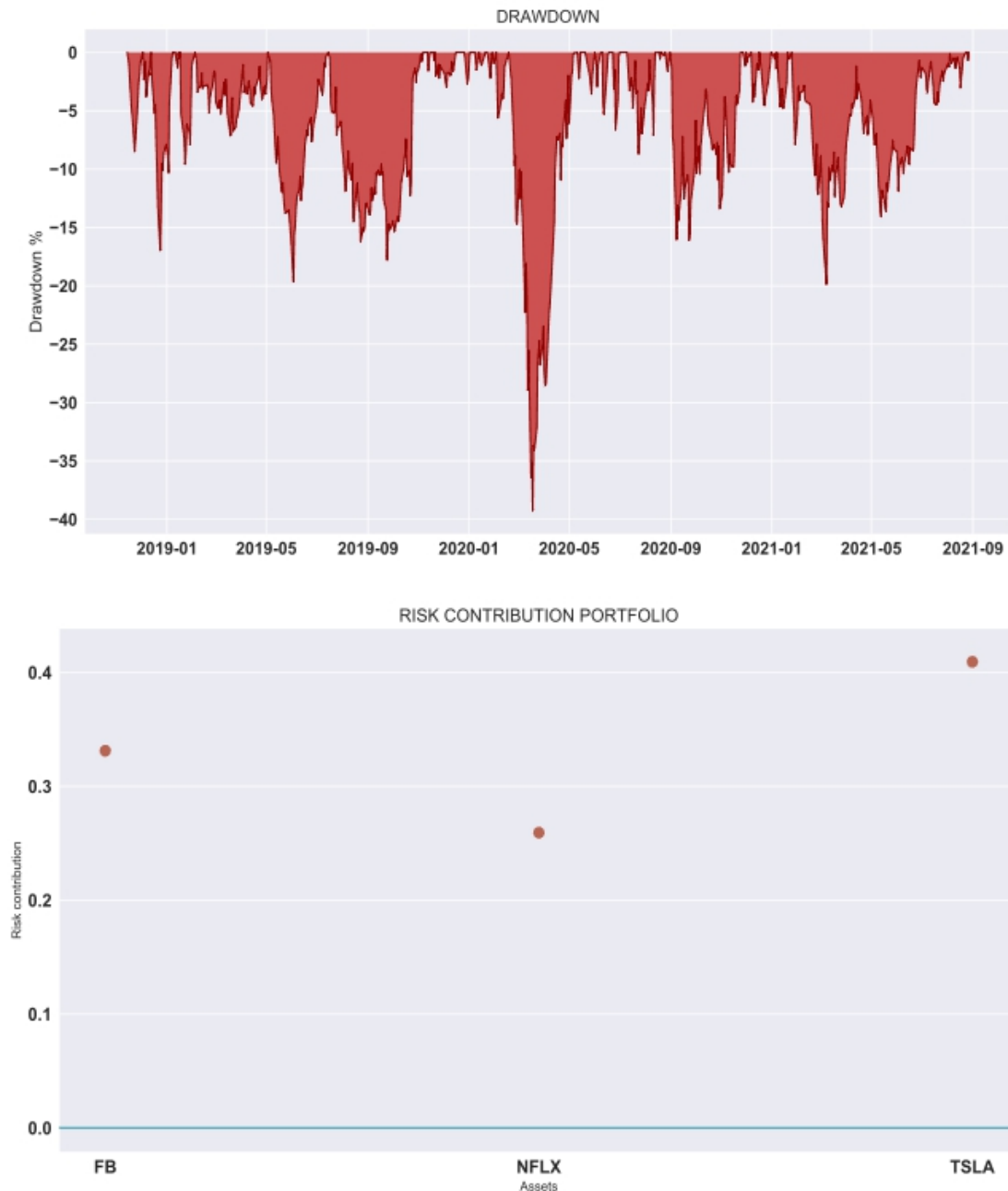
Figura 5.7: Métricas de la cartera media-varianza-asimetría-curtosis

Portfolio: ['FB', 'NFLX', 'TSLA']

Beta: 1.136 Alpha: 47.35 % Sharpe: 1.547 Sortino: 2.105

VaR: 33.36 % cVaR: 47.59 % VaR/cVaR: 1.427 drawdown: 44.24 %





Esta figura muestra que esta estrategia es buena porque el alfa es igual al 47.35%. Sin embargo, un VaR masivo implica un riesgo importante en esta estrategia y un drawdown al 44.24%.

Ahora, comparamos estas dos estrategias porque son estrategias en la misma línea. Como podemos ver en la media-varianza, todas las métricas implican que la estrategia

es muy rentable, pero muy arriesgada. Al mismo tiempo, la cartera media-varianza-asimetría-curtosis también tiene una mejor rentabilidad con un riesgo mayor.



Con estas métricas, no podemos decir qué estrategia es la mejor. Esta elección depende del inversor. En efecto, si le gusta la inversión de riesgo, elegirá la segunda, en caso contrario, la primera.



Las únicas diferencias son el riesgo de contribución de cada activo. Por lo tanto, si tenemos algunas preferencias en un activo, también podemos elegir la mejor cartera.

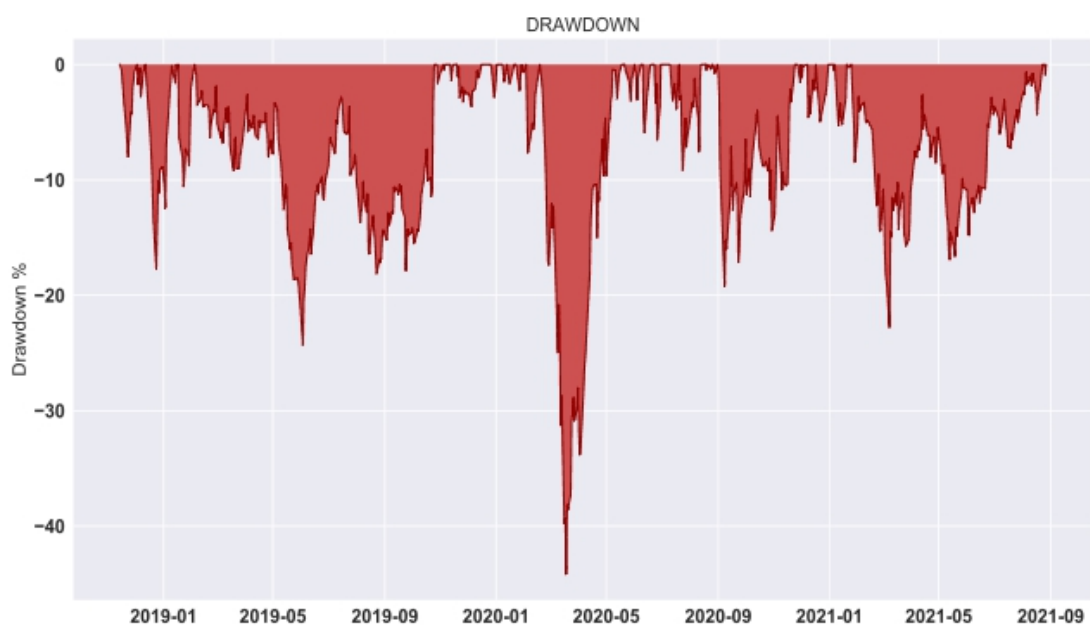
Analicemos las otras dos estrategias estáticas, las carteras Sharpe y Sortino. Podemos ver en las figuras 5.8 y 5.9 las métricas de la estrategia respectiva.

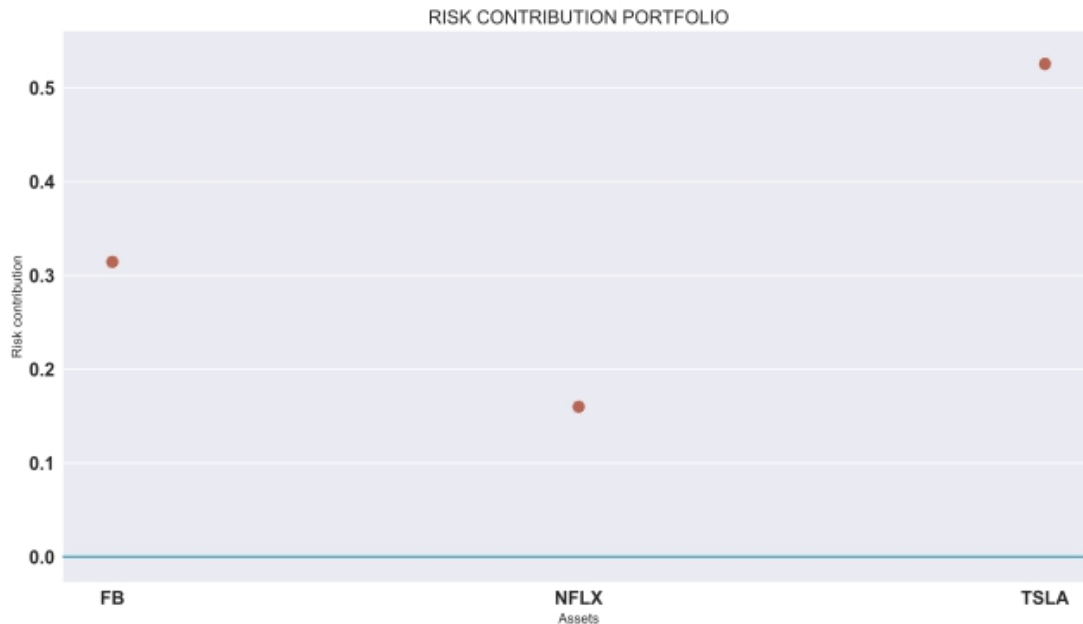
Figura 5.8: Métricas para la optimización de la cartera de Sharpe

Portfolio: ['FB', 'NFLX', 'TSLA']

Beta: 1.048 Alpha: 31.28 % Sharpe: 1.266 Sortino: 1.772

VaR: 40.37 % cVaR: 53.62 % VaR/cVaR: 1.328 drawdown: 35.52%



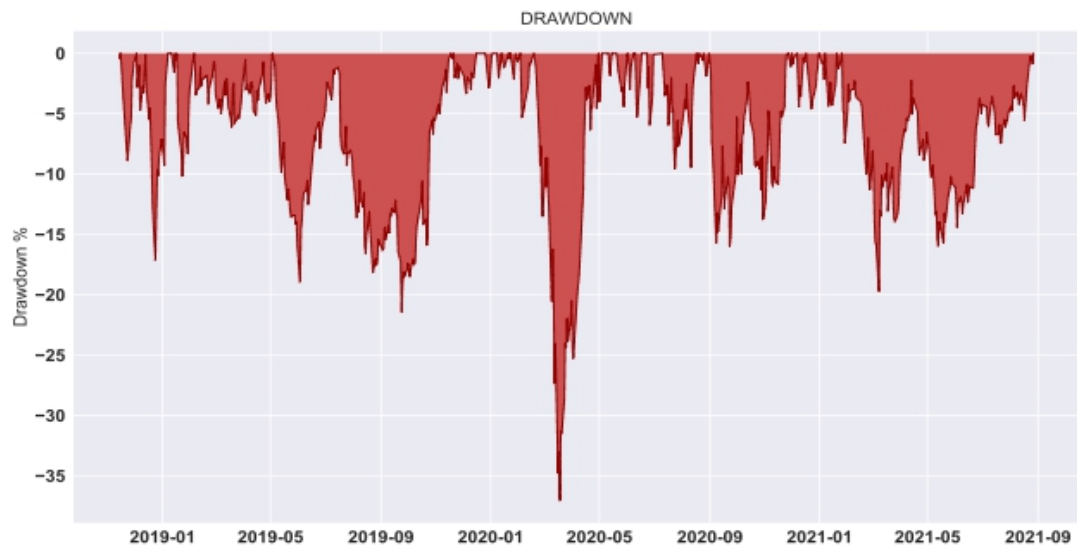


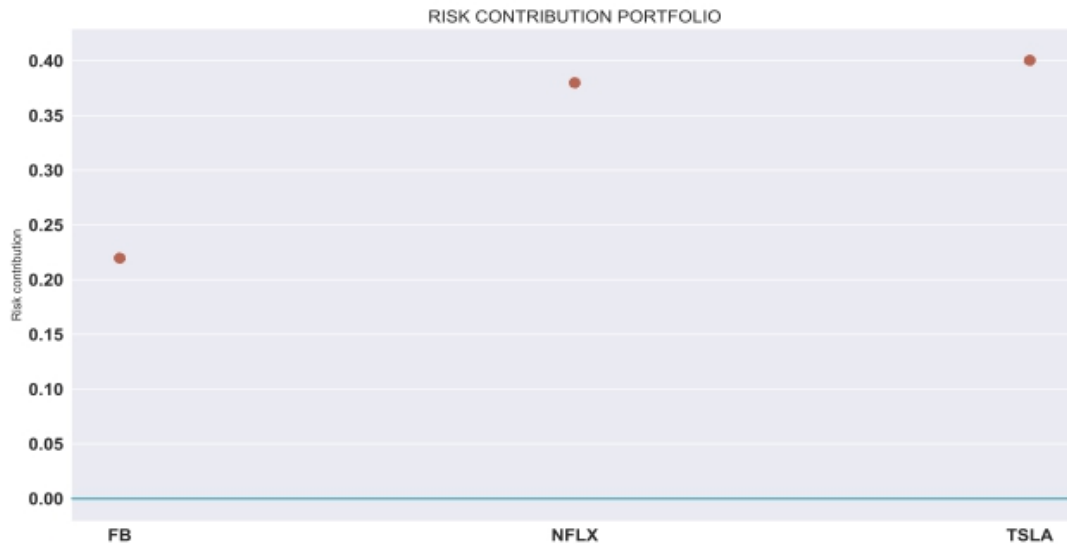
En esta figura, podemos decir que esta estrategia es excelente porque el alfa es igual al 40.37%. Sin embargo, un VaR masivo implica un riesgo importante en esta estrategia.

Figura 5.9: Métricas para la optimización de la cartera Sortino

Portfolio: ['FB', 'NFLX', 'TSLA']

Beta: 1.042	Alpha: 30.28 %	Sharpe: 1.241	Sortino: 1.739
VaR: 41.13 %	cVaR: 54.7 %	VaR/cVaR: 1.33	drawdown: 34.92 %





Esta cifra muestra que esta estrategia es excelente porque el alfa es igual al 41.13%. Sin embargo, un VaR masivo implica un riesgo importante en esta estrategia.

Como podemos ver, las dos estrategias tienen casi las mismas métricas. No podemos decir si una u otra es mejor. La única diferencia es el riesgo en cada activo. Si preferimos invertir en acciones de Netflix y creemos en este valor, elegimos la cartera Sortino. Sin embargo, si preferimos Tesla, elegimos la cartera Sharpe, por ejemplo.

5.3.3. Analizar la cartera dinámica

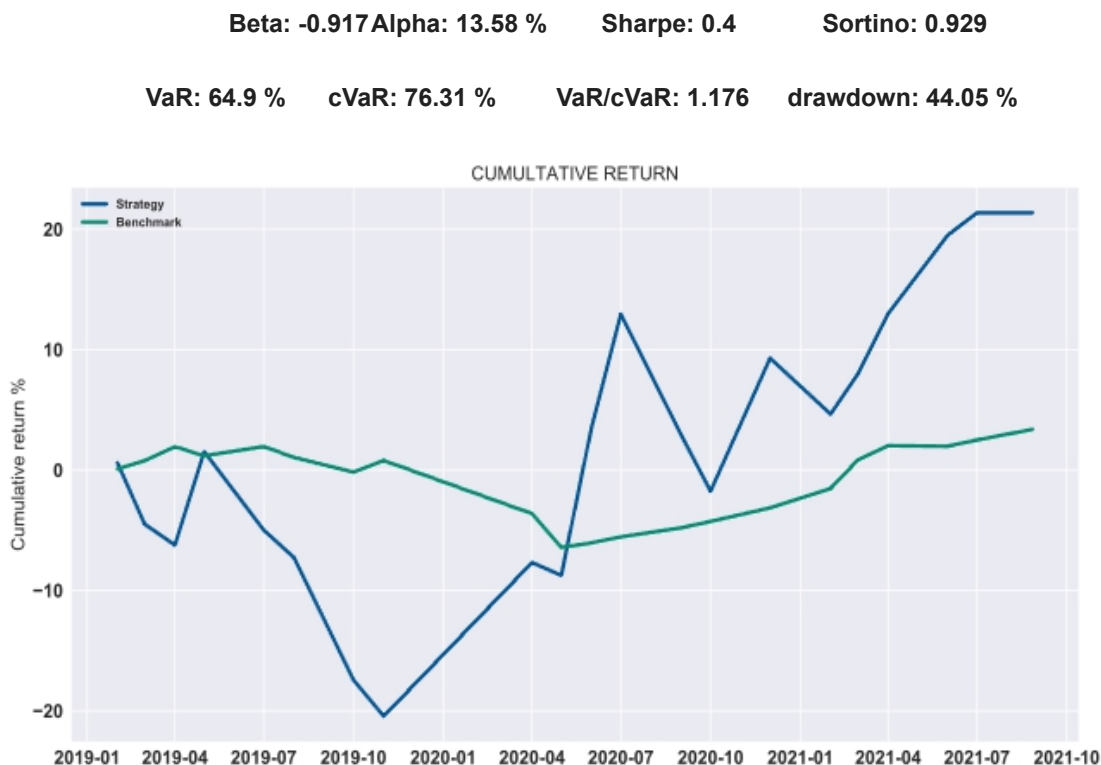
En esta parte, hablamos del análisis de las carteras dinámicas. Efectivamente, eran pésimas en comparación con la cartera estática.

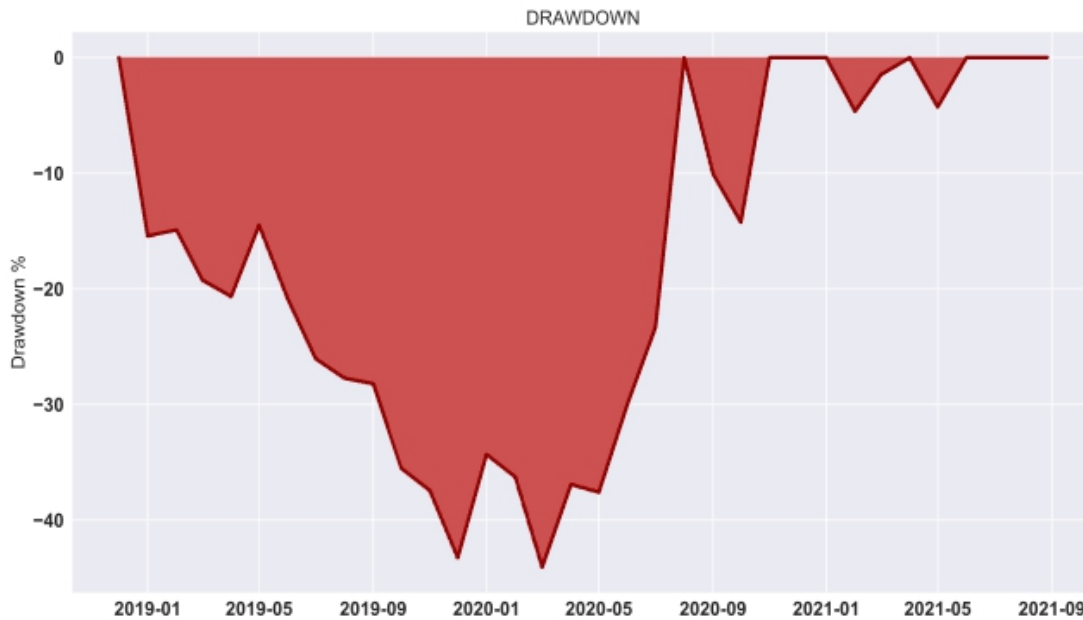
Pero, ¿por qué? Hay varias respuestas a esta pregunta. En primer lugar, era fundamental comprender que nuestra cartera dinámica es una estrategia fundamental.

Además, no hicimos selección de activos para la primera estrategia con el SMA. Las estrategias también son fundamentales, pero el objetivo era entrar sin problemas en el campo de las estrategias long-short. Por lo tanto, veremos en

los siguientes capítulos cómo crear estrategias de trading algorítmico a corto plazo que funcionen mucho mejor que éstas. Veamos los resultados de estas estrategias en las figuras 5.10 y 5.11.

Figura 5.10: Métricas para la cartera de optimización del SMA

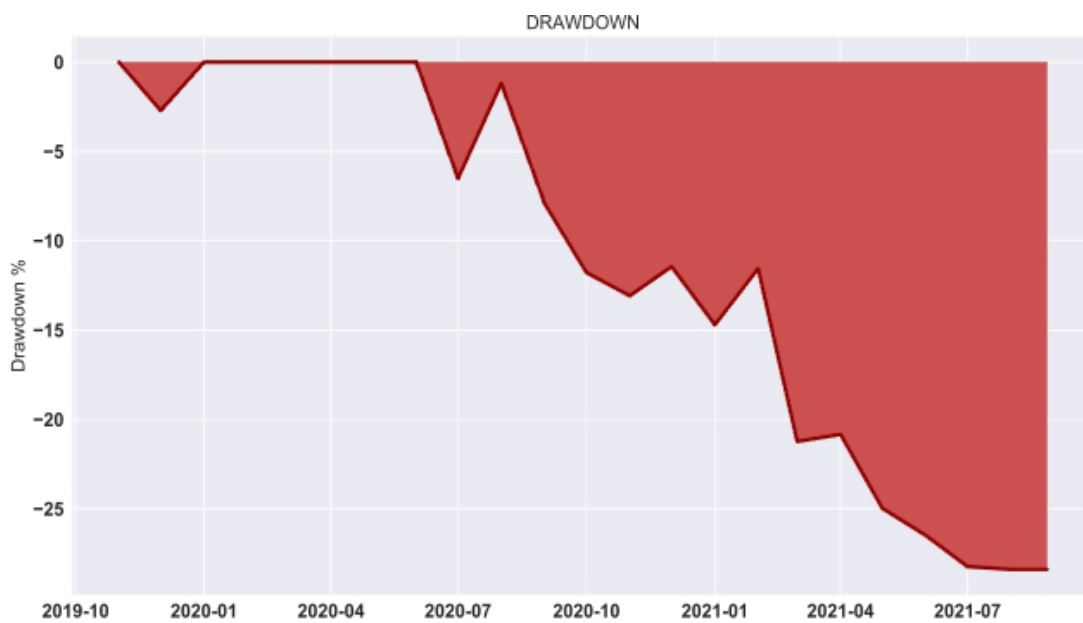
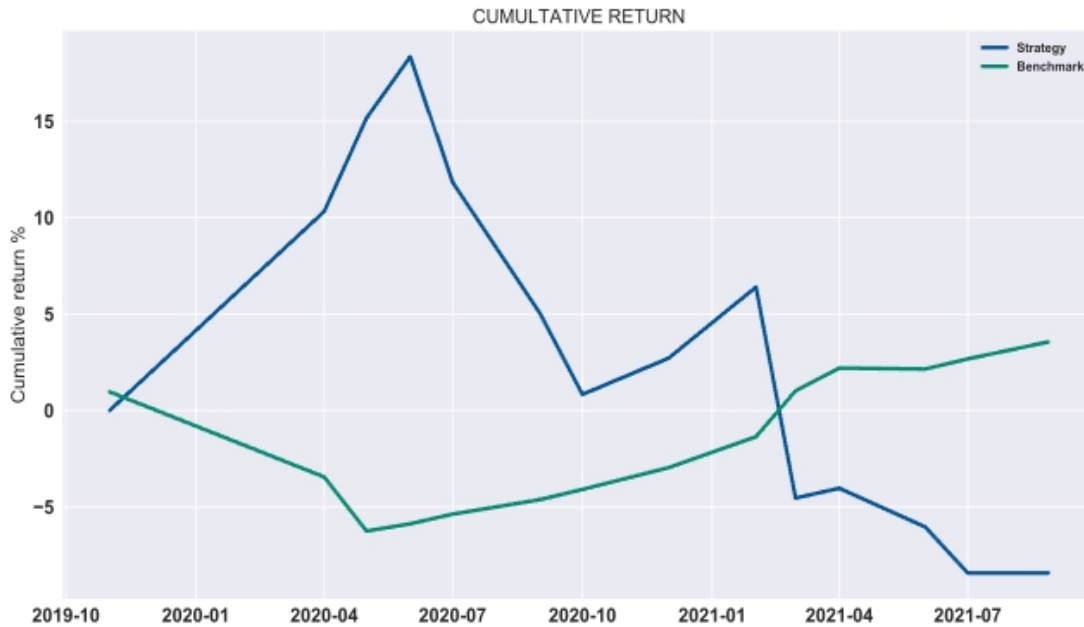




Este gráfico muestra que esta estrategia no es buena porque el alfa es igual al 13.58% y el VaR se sitúa en el 64% anual. El ratio de Sharpe es escaso (0.40).

Figura 5.11: Métricas para la cartera de optimización de los rendimientos tendencias

Beta: -2.118	Alpha: -1.31 %	Sharpe: -0.058	Sortino: -0.092
VaR: 40.74 %	cVaR: 46.4 %	VaR/cVaR: 1.134	drawdown: 27.7%



Este gráfico muestra que esta estrategia no es buena porque el valor de alfa es igual a -1.53%, pero el VaR está en el 40% y el ratio de Sharpe es escaso (0.031).

Resumen

- Las métricas CAPM, el alfa y la beta son fascinantes para comprender los vínculos entre la cartera y el benchmark.

- El drawdown nos da la pérdida más importante que podemos tener en un periodo si entramos en posición en el peor momento de este periodo.
- Los ratios de Sharpe y Sortino nos permiten comprender la ganancia de rentabilidad por un 1% de asunción de riesgo. Sin embargo, la forma de computar el riesgo es diferente.
- El VaR y el cVaR nos permiten especular sobre las peores pérdidas que se pueden tener utilizando la estrategia.

Capítulo 6: Métodos avanzados de backtesting

En este capítulo se analizan técnicas avanzadas de backtest, como el take-profit, el stop-loss, métricas interesantes y el trailing stop loss. Nos dará más ideas sobre las medidas utilizadas en finanzas y trading.

6.1 Consejos útiles para backtest

Ya hemos visto cómo hacer un backtest, pero no hemos hablado de los errores más frecuentes entre los más de 60000 alumnos.

6.1.1 Backtest no es una herramienta de búsqueda

La primera y más importante regla a seguir es "Backtest no es una herramienta de búsqueda". Marco López de Prado dijo en uno de sus libros: "Hacer backtest mientras se investiga es como beber y conducir".

Tomaremos un pequeño ejemplo de los pasos que siguen casi todos los estudiantes para poner de relieve la cuestión en su proceso:

1. Encontrar algunos datos, crear nuevas características y dividir el conjunto de datos en un conjunto de entrenamiento y un conjunto de test.
2. Crear una estrategia utilizando sólo el conjunto de entrenamiento.
3. Backtest de la estrategia en la prueba.
4. Repetir hasta que encontremos una estrategia rentable en el conjunto de test.

¡EL PASO 4 ES EL QUE DEBEMOS EVITAR!

¿Por qué? Porque obtendremos un sobreajuste oculto de la estrategia. Sin darnos cuenta, utilizaremos un conjunto de datos que se supone que representa al máximo la realidad para desarrollar nuestra estrategia. Sin embargo, en la realidad, no tenemos acceso al precio futuro: ¡ese es el problema! Utilizaremos el futuro para entrenar nuestros algoritmos, por lo que tendremos rendimientos muy diferentes en el trading en vivo.

Ahora, permíteme mostrarte los pasos a seguir para evitar encontrar una estrategia "afortunada" o una estrategia sobreajustada como en nuestro ejemplo anterior:

1. Encontrar los datos, crear nuevas características y dividir el conjunto de datos.
2. Crea una estrategia y OPTIMIZARLA en el conjunto de entrenamiento
3. Realizar una prueba retrospectiva de la estrategia en el conjunto de test: mantenla si es buena o detente aquí. No cambies los parámetros de la estrategia; la estrategia no es rentable. No importa; ¡probaremos con otra!

Debemos comprender que cuanto más toquemos nuestro conjunto de test para adaptar la estrategia, más malos resultados obtendremos en el futuro (en operaciones reales).

6.1.2 Los días importantes no son nuestros amigos

Obviamente, los días de grandes beneficios son nuestros amigos, así que ¿por qué la sección del título dice lo contrario? La segunda regla a seguir cuando backtestamos una estrategia es no tener en cuenta el gran beneficio.

De hecho, cuanto más infrecuente es el evento en el backtest, más baja es la probabilidad de volver a enfrentarse a él. Entonces, supongamos que basamos la rentabilidad de

nuestra estrategia en estos 2 o 3 grandes beneficios. En ese caso, la probabilidad de ser rentable en operaciones reales es muy baja.

Figura 6.1: Estrategia con escasa estabilidad de rendimiento



Vemos que la estrategia será una estrategia perdedora sin estas tres grandes ganancias. Esto significa que es una estrategia arriesgada, por lo que, en mi opinión, es mejor no seleccionarla.

Para ilustrar este propósito, veamos la Figura 6.1 y la Figura 6.2, en las que vemos dos estrategias con la misma rentabilidad a lo largo del periodo, pero con un comportamiento muy diferente. La primera pierde una parte importante de las operaciones pero es buena gracias a las tres ganancias significativas. Por el contrario, la segunda estrategia (Figura 6.2) es mucho más interesante porque el crecimiento del capital es más estable.

Figura 6.2: Estrategia con una excelente estabilidad de rendimiento



Esta estrategia es mucho más estable porque el beneficio proviene de gran parte de la operación. No tenemos grandes ganancias ni pérdidas significativas en los rendimientos, lo que es esencial para ver crecer nuestro capital de forma segura.



No olvidemos nunca que el primer objetivo del trading es preservar nuestro capital y el segundo hacerlo crecer, ¡y no al contrario!

6.1.3 Entiende tu estrategia

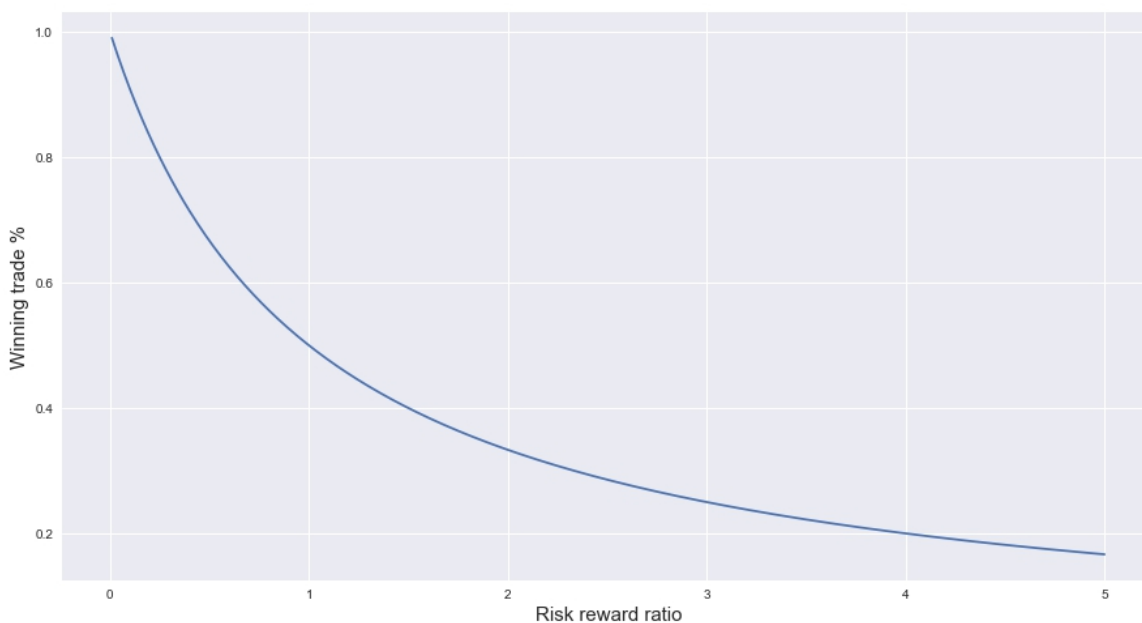
Hacer un backtest no es interesante para predecir el comportamiento futuro de los rendimientos de la estrategia. El precio sólo ha seguido un camino entre un número infinito de caminos posibles. Entonces, ¿por qué debemos hacer un backtest? Porque nos permitirá entender los puntos fuertes y débiles de la estrategia, es esencial antes de ponerla en el comercio en vivo.

Lo primero que hay que comprobar es el ratio HIT, el porcentaje de operaciones ganadoras: mantener una

estrategia con un ratio HIT adaptado a nuestros objetivos es esencial. Por ejemplo, supongamos que queremos crear una señal de trading para ponerla en copy trading (como en BullTrading). En ese caso, deberíamos tener el mayor ratio HIT posible porque la gente tendrá más confianza si ve muchos beneficios, aunque sean pequeños. Sin embargo, podemos aceptar el ratio HIT más bajo si desarrollamos algoritmos sólo para nuestra inversión.

Por otro lado, el ratio HIT no es nada sin un ratio riesgo-recompensa, que es la relación entre la recompensa prevista (a menudo el beneficio obtenido) y el riesgo que asumimos (a menudo el stop-loss). Debemos recordar siempre que los ratios HIT y riesgo-recompensa son las dos caras de una misma moneda.

Figura 6.3 : La relación HIT-riesgo-recompensa se sitúa en el punto de equilibrio



Deberíamos encontrar un equilibrio entre estas dos métricas porque nunca obtendremos el 95% de las operaciones ganadoras con una R (relación riesgo-recompensa) superior a 1. Además, está bien porque en la Figura 6.3, con una relación HIT del 95%, sólo necesitamos una R de 0.052, lo que

significa que si tenemos un riesgo del 1%, necesitamos un beneficio del 0.052% cuando ganemos una operación (con una tasa de ganancia del 95%) para estar en el punto de equilibrio.

Estas dos métricas son adecuadas para clasificar la estrategia y entender si son adecuadas para nuestro perfil de inversor o no. Por otra parte, tenemos que mirar más métricas como la vida de la operación y el tiempo bajo el agua (cuando el drawdown de la estrategia es inferior a 0, pero lo explicaremos en detalle en la siguiente sección).

No hay un buen valor para estos valores; aunque cuanto más pequeño sea, mejor. Sin embargo, depende del objetivo de nuestra estrategia: si queremos una estrategia de scalping. Tenemos un tiempo máximo bajo el agua de 2 meses; no es bueno, pero si utilizamos una estrategia de swing, puede ser aceptable.



Adapta siempre el análisis de tu backtest a tu estrategia objetivo. Por ejemplo, no esperes una duración de 10 minutos si utilizas una estrategia swing.

6.2 Calcular los rendimientos de la estrategia utilizando TP y SL

Esta sección nos mostrará cómo calcular los rendimientos de la estrategia utilizando una estrategia TP-SL, adaptar los datos a este método de backtest y analizar los resultados.

6.2.1 Encontrar el extremo

En el capítulo anterior, utilizamos el backtesting vectorizado. Sin embargo, aquí es imposible porque necesitamos comprobar para cada posición cuándo hemos tocado el take-profit o el stop-loss. Sin embargo, si utilizamos los ticks, el

backtest tardará mucho tiempo (horas, tal vez días si no tenemos un ordenador potente).

Por lo tanto, para solucionar este problema, vamos a utilizar un pequeño consejo. De hecho, ¿por qué necesitamos los ticks? Primero tocamos el take-profit o el stop-loss para encontrarlo para cada posición. Por lo tanto, para cada vela, necesitamos saber qué fue primero, ¿el mínimo o el máximo!

Utilizar sólo un marco temporal será imposible. Sin embargo, supongamos que utilizamos la base de datos del marco temporal más alto (por ejemplo: horario) y la base de datos del marco temporal más bajo (por ejemplo: 15 min). En ese caso, saber cuál se toca primero será posible. La ventaja de esta estrategia es que mantenemos el mismo número de datos con una nueva fila atractiva, que se llamará "Primero" y nos dirá si hemos tocado primero el máximo o el mínimo de esta vela. Tenemos que mirar en el repositorio de Github (enlace en el capítulo 1) para encontrar la base de datos utilizada en el código.

Código 6.1: Averigua el precio de primer toque: Bajo o Alto

```
def find_timestamp_extremum(data, df_lowest_timeframe):  
    """  
    :params: data(mayor timeframe OHLCV),  
            df_lowest_timeframe (menor timeframe OHLCV)  
    :return: data con tres nuevas columnas: Low_time (TimeStamp),  
            High_time (TimeStamp),  
            High_first (Boolean)  
    """  
  
    # Establecer nuevas columnas  
    data["Low_time"] = np.nan  
    data["High_time"] = np.nan  
    data["First"] = np.nan
```

```

# Bucle para saber cuál de los Take Profit y Stop loss aparece
# primero
for i in tqdm(range(len(data) - 1)): 1
    # Extraer valores del marco temporal más bajo
    start = data.iloc[i:(i + 1)].index[0]
    end = data.iloc[(i + 1):(i + 2)].index[0]
    row_lowest_timeframe = df_lowest_timeframe.loc[start:end]
                                .iloc[:-1]

    # Extraer la marca de tiempo del máximo y el mínimo del
    # periodo (marco temporal más alto)
    try:
        high = row_lowest_timeframe["high"].idxmax()
        low = row_lowest_timeframe["low"].idxmin()

        data.loc[start, "Low_time"] = low
        data.loc[start, "High_time"] = high

    except Exception as e:
        print(e)
        data.loc[start, "Low_time"] = start
        data.loc[start, "High_time"] = start

# Averiguar cuál aparece primero
data.loc[data["High_time"] > data["Low_time"], "First"] = 1
data.loc[data["High_time"] < data["Low_time"], "First"] = 2
data.loc[data["High_time"] == data["Low_time"], "First"] = 0 2

# Verificar el número de filas sin TP y SL al mismo tiempo
percentage_garbage_row =
    len(data.loc[data["First"] == 0].dropna()) / len(data) * 100

if percentage_garbage_row < 95:
    print(f"WARNINGS: Garbage row: {'%.2f' %

```

```
percentage_garbage_row} %") 3
```

```
# Transformar las columnas en columnas datetime
```

```
data.High_time = pd.to_datetime(data.High_time)
```

```
data.Low_time = pd.to_datetime(data.Low_time)
```

```
# Eliminamos la última fila porque no podemos encontrar el
```

```
# extremo
```

```
data = data.iloc[:-1]
```

```
# Específicos de los datos actuales
```

```
if "timestamp" in data.columns:
```

```
    del data["timestamp"]
```

```
return data
```

- 1 Itera en cada fila del marco temporal más alto y selecciona las filas de marco temporal más bajo correspondientes para encontrar el punto del marco temporal más alto del primer hit utilizando el precio mínimo y máximo del marco temporal más bajo.
- 2 A veces, tendrás alguna fila indeterminada si el máximo y el mínimo del marco temporal más alto es un hit en la misma fila del marco temporal más bajo.
- 3 Calcula el número de filas en las que no podemos encontrar qué extremo se halla antes.

En nuestro ejemplo tenemos un 0.86% de filas indeterminadas, por lo que es aceptable. Sin embargo, si tenemos más de un 5% de filas basura, es mejor tomar un plazo inferior.

6.2.2 Calcular los rendimientos

Una vez que tenemos una base de datos completa con nuestros datos OHLCV y tenemos una nueva fila que nos dice qué extremo se halla antes, podemos calcular los rendimientos de nuestra estrategia utilizando una estrategia de salida TP-SL.

Para poner en práctica nuestro backtest, necesitamos una señal: -1 si queremos tomar una posición de venta, 0 si mantenemos y 1 si tomamos una posición de compra. Sin embargo, si no disponemos de estas señales, podemos simularlas utilizando el siguiente código.

Código 6.2: Calcular señales aleatorias

```
# Crear señales aleatorias
np.random.seed(70)
values = [-1, 0, 1]
df["Signal"] = [np.random.choice(values,
    p=[0.10, 0.80, 0.10]) for _ in
range(len(df))]
```

Será muy sencillo. Para calcular los rendimientos utilizando una estrategia de salida Take-profit / Stop-loss, tenemos que considerar cuatro casos:

1. Abrimos una posición de compra y tocamos el TP primero (Precio alto)
2. Abrimos una posición de compra y tocamos el SL primero (Precio bajo)
3. Abrimos una posición de venta y tocamos el TP primero (Precio bajo)
4. Abrimos una posición de venta y tocamos el SL primero (Precio alto)

Es un poco fastidioso, pero el razonamiento es sencillo: tenemos una señal y en la apertura de la vela siguiente,

abrimos una operación (de compra o de venta, según el caso). Luego, para cada máximo y mínimo, comprobamos la variación desde la apertura hasta el máximo y mínimo actuales para ver si hemos cruzado o no el TP o el SL.

Código 6.3: Backtest usando TP y SL

```
def run_tp_sl(data, leverage = 1, tp = 0.015, sl = -0.015,
              cost = 0.00):
    """
    :params (mandatory): data(tiene que contener las columnas
                          High_time y Low_time)
    :params (optional): leverage = 1, tp = 0.015, sl = -0.015,
                          cost=0.00
    :return: data con tres nuevas columnas: Low_time (TimeStamp),
                          High_time (TimeStamp),
                          High_first (Boolean)

    """

    # Establecer algunos parámetros
    buy = False
    sell = False
    data["duration"] = 0

    for i in range(len(data)):

        # Extraer los datos
        row = data.iloc[i]

        ##### ABRIR COMPRA #####
        if buy == False and row["Signal"] == 1:
            buy = True
            open_buy_price = row["open"]
            open_buy_date = row.name

        # VERIFICAR
```

EL

```
if buy:
    var_buy_high = (row["high"] - open_buy_price) /
                    open_buy_price
    var_buy_low = (row["low"] - open_buy_price) /
                  open_buy_price
    # VERIFICAR PARA TP Y SL EN LA MISMA VELA
    if (var_buy_high > tp) and (var_buy_low < sl): 1

        # SI TP / SL EN EL MISMO TIMESTAMP, ELIMINAMOS

        # RETORNO DE COMERCIO
        if row["Low_time"] == row["High_time"]:
            pass

        elif row["First"] == 2:
            data.loc[row.name, "returns"] = (tp-cost) *
                                            leverage
            data.loc[row.name, "duration"] = row.High_time -
                                            open_buy_date

        elif row["First"] == 1:
            data.loc[row.name, "returns"] = (sl-cost) *
                                            leverage
            data.loc[row.name, "duration"] = row.Low_time -
                                            open_buy_date

    buy = False
    open_buy_price = None
    var_buy_high = 0
    var_buy_low = 0
    open_buy_date = None

elif var_buy_high > tp:
    data.loc[row.name, "returns"] = (tp-cost) * leverage
    buy = False
```



```

open_buy_price = None
var_buy_high = 0
var_buy_low = 0
data.loc[row.name, "duration"] = row.High_time -
                                open_buy_date
open_buy_date = None

elif var_buy_low < sl:
    data.loc[row.name, "returns"] = (sl-cost) * leverage
    buy = False
    open_buy_price = None
    var_buy_high = 0
    var_buy_low = 0
    data.loc[row.name, "duration"] = row.Low_time -
                                open_buy_date
    open_buy_date = None

##### ABRIR VENTA #####
if sell == False and row["Signal"] == -1:
    sell = True
    open_sell_price = row["open"]
    open_sell_date = row.name

# VERIFICAR
if sell: 2
    var_sell_high = -(row["high"] - open_sell_price) /
                    open_sell_price
    var_sell_low = -(row["low"] - open_sell_price) /
                    open_sell_price

    if (var_sell_low > tp) and (var_sell_high < sl):

        if row["Low_time"] == row["High_time"]:
            pass
        elif row["First"] == 1: # INVERTIR PARA LA COMPRA

```

```

        data.loc[row.name, "returns"] = (tp-cost) *
            leverage
        data.loc[row.name, "duration"] = row.Low_time -
            open_sell_date

    elif row["First"] == 2:
        data.loc[row.name, "returns"] = (sl-cost) *
            leverage
        data.loc[row.name, "duration"] = row.High_time -
            open_sell_date

    sell = False
    open_sell_price = None
    var_sell_high = 0
    var_sell_low = 0
    open_sell_date = None

    elif var_sell_low > tp:
        data.loc[row.name, "returns"] = (tp-cost) * leverage
        sell = False
        open_sell_price = None
        var_sell_high = 0
        var_sell_low = 0
        data.loc[row.name, "duration"] = row.Low_time -
            open_sell_date
        open_sell_date = None

    elif var_sell_high < sl:
        data.loc[row.name, "returns"] = (sl-cost) * leverage
        sell = False
        open_sell_price = None
        var_sell_high = 0
        var_sell_low = 0
        data.loc[row.name, "duration"] = row.High_time -
            open_sell_date

```

```
open_sell_date = None

# Poner 0 cuando tengamos valores faltantes
data["returns"] = data["returns"].fillna(value = 0)
return data
```

- 1 Debemos comprobar primero si tanto el TP como el SL se cruzan en la misma fila para mirar en las columnas Primera para ver cuál se halla antes (si First = 0, no contamos la operación).
- 2 Tenemos que tener mucho cuidado cuando calculamos el beneficio de las posiciones de venta porque tenemos que utilizar el tiempo bajo para encontrar el TP y el tiempo alto para encontrar el SL.
- 3 Sería útil sustituir todos los valores Nan por ceros para calcular las métricas de backtest de forma eficiente.

Ya hemos realizado el backtest, pero no sabemos cómo interpretarlo. De hecho, tenemos una serie de rendimientos sin información clara. Ese es el objetivo de la siguiente sección.

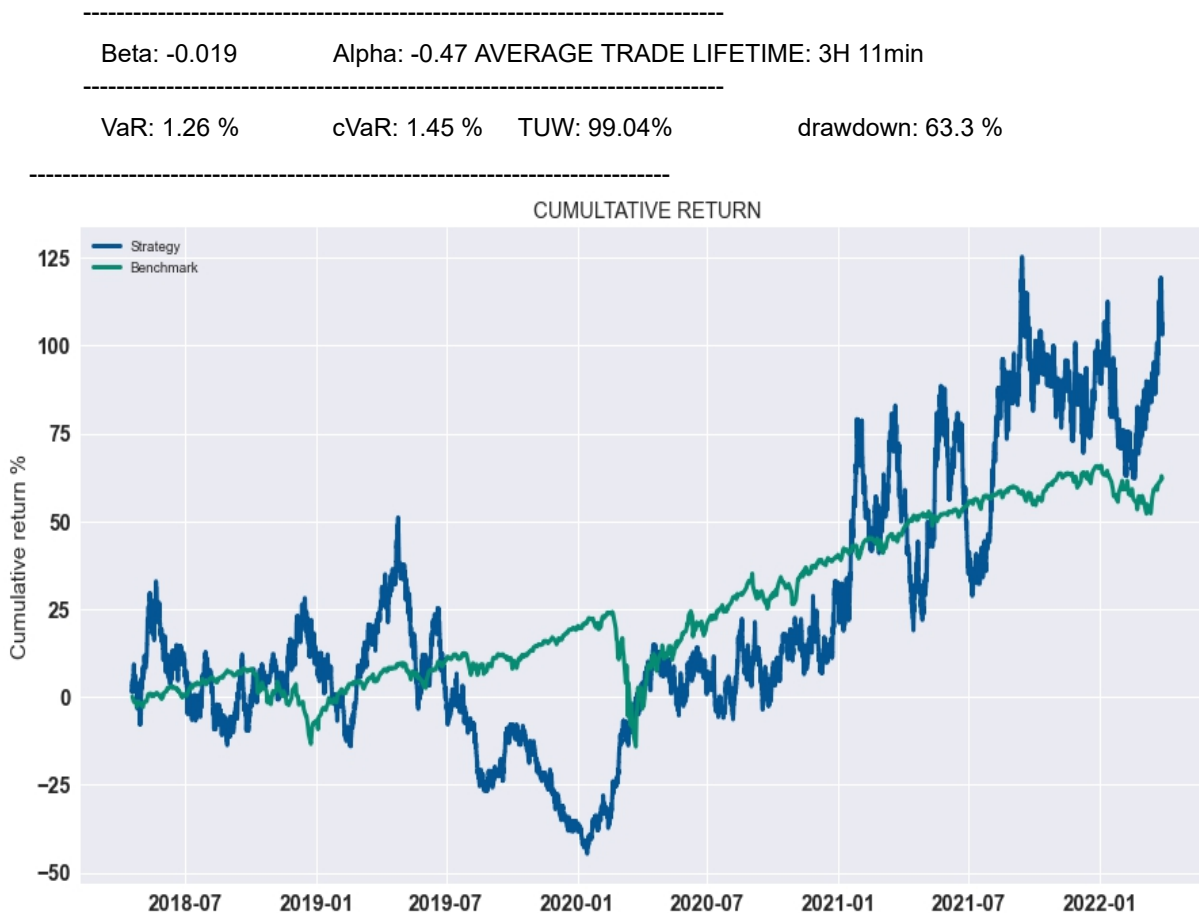
6.2.3 Analizar el backtest

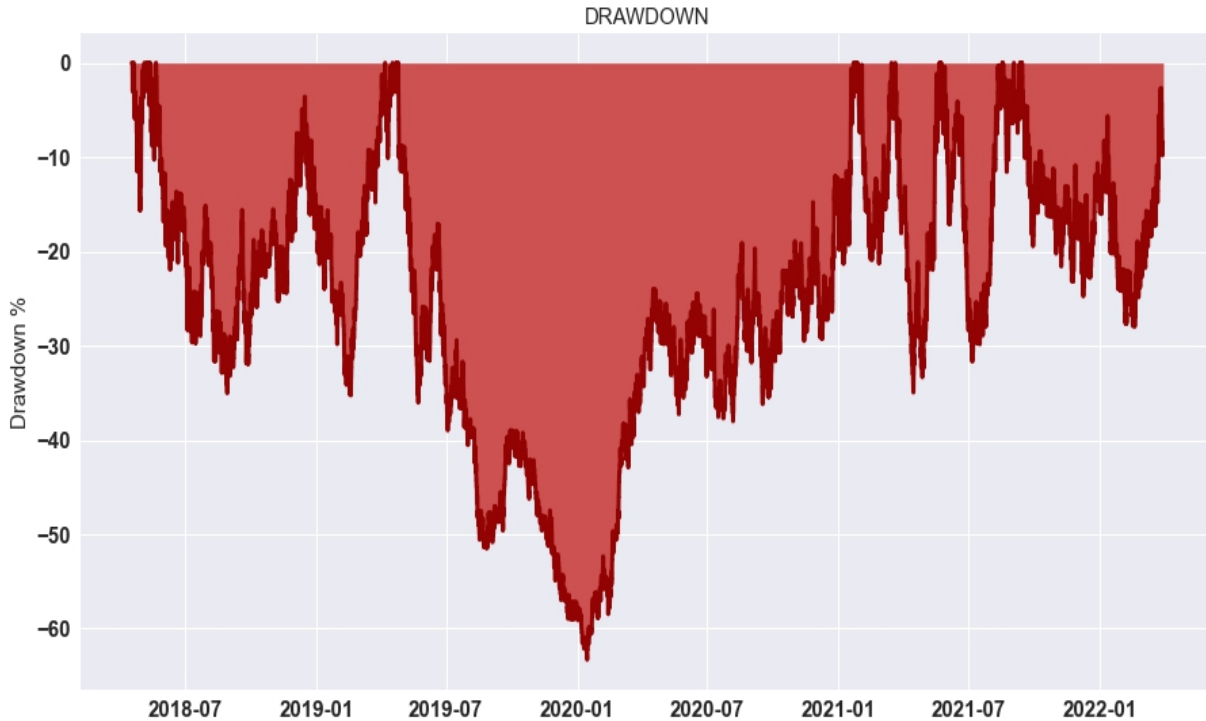
Calcular los rendimientos no es suficiente; necesitamos calcular diferentes métricas para tener una visión rápida del rendimiento de nuestra estrategia. Para ello, cambiaremos un poco nuestra función de backtest anterior.

La modificación más significativa en la salida es que no tenemos el ratio de Sharpe y Sortino en este backtest. ¿Por qué? Porque el TP y el SL son fijos, no hay utilidad para calcular la volatilidad, especialmente la volatilidad bajista, porque todas las pérdidas son iguales al stop-loss menos las comisiones. Por lo tanto, podemos sustituirlo por la vida media de la operación y el ratio VaR sobre cVaR por el TUW (time

underwater) porque el ratio será cercano a 1 debido al stop-loss.

Figura 6.4: Backtest de señal aleatoria





Tómate un minuto para analizar el backtest. Aquí, la estrategia irá directamente a la papelera. ¿Por qué? Por el riesgo que asume. El TUW es igual a 99.04%, con un drawdown máximo de 63.3%.

Además, también podemos mostrar los rendimientos mensuales, que pueden ser una indicación interesante. Sepamos cómo hacerlo en el código siguiente y veamos el resultado en la Figura 6.5.

Código 6.4: Calcular la rentabilidad mensual

```
# Bonus
def profitable_month_return(p):
    total = 0
    positif = 0

    r=[]
    # Bucle para cada año
```

```

for year in p.index.strftime("%y").unique():
    e = []
    nbm = p.loc[p.index.strftime("%y") == year].
        index.strftime("%m").unique()
    # Bucle en cada mes diferente
    for mois in nbm:

        monthly_values = p.loc[p.index.strftime("%y:%m")
==
                                f"{year}:{mois}"]
        sum_ = monthly_values.sum()

    # Comprobación de que hay al menos un 75% de los
valores
    if len(monthly_values) > 15:

        # Cálculo de la suma de rentabilidad
        s = monthly_values.sum()

        if s > 0:
            positif += 1

        else:
            pass

        total+=1

    else:
        pass
    e.append(sum_)
    r.append(e)
r[0] = [0 for _ in range(12-len(r[0]))] + r[0]
r[-1]= r[-1] + [0 for _ in range(12 - len(r[-1]))]

```

```
def heatmap(data):  
    htm = profitable_month_return(data["returns"]) * 100  
    htm.index.name = "Year"  
    htm.index = [f"20{idx}" for idx in htm.index]  
  
    plt.figure(figsize = (20, 8))  
    pal = sns.color_palette("RdYlGn", n_colors = 15)  
    sns.heatmap(htm, annot = True, cmap = pal,  
                vmin = -100, vmax = 100)  
  
    plt.title("Heatmap Monthly returns")  
    plt.show()
```



Es una forma muy interesante de mostrar visualmente la estabilidad del rendimiento de la estrategia.

6.3 Herramientas avanzadas de backtest

Una vez que tenemos un backtest clásico, podemos ajustar las métricas y simular diferentes trayectorias. Además, podemos realizar fácilmente un backtest de una estrategia de trailing stop loss.

6.3.1 Lista de métricas de backtest

En esta sección, recapitularemos varios indicadores de backtest (algunos que ya hemos visto en el libro, pero los pondremos aquí de nuevo para tener una lista completa de indicadores). Esta lista no es exhaustiva.

1. **Tiempo bajo el agua (time underwater):** el porcentaje de tiempo que tenemos un drawdown por debajo de 0. Nos permite entender el porcentaje de tiempo que ganaremos dinero. Es fundamental entender que el tiempo no da la intensidad de la pérdida: podemos tener un 1% de tiempo bajo el agua y un drawdown máximo del 100%, por lo que perderemos todo nuestro capital. Esta métrica debe combinarse siempre con el drawdown máximo.
2. **Vida comercial:** el tiempo medio de una posición comercial. Cuanto más scalping sea la estrategia, más esencial será la duración de la operación..
3. **Activos gestionados:** es el valor en dólares de nuestra cartera en cada momento (es un vector). Solemos calcular el AUM (activos gestionados) medio.

4. **Ratio largo – corto:** Número de posiciones largas sobre el número de posiciones vendedoras. Así, si trabajamos con una estrategia long-short, cuanto más se acerque el valor a 0.5, mejor será.
5. **Número de operaciones:** es muy importante conocer el número de operaciones que realizas y ver si se ajusta a tu plan de trading: una estrategia con 12 operaciones a lo largo de un año no es un backtest significativo, pero 500 sí lo es.
6. **Rentabilidad anualizada:** Es esencial calcular la rentabilidad anualizada para poder comparar varias estrategias entre sí.
7. **Correlación subyacente:** La correlación de Pearson suele ser entre los rendimientos de la estrategia y los rendimientos subyacentes. Cuanto más se acerque el valor a 1, más larga será la estrategia para el activo, y cuanto más se acerque la correlación a -1, más corta será la estrategia para el activo. Por lo tanto, la estrategia no añade mucho valor si la correlación es cercana a -1 o 1.
8. **Ratio HIT:** porcentaje de operaciones ganadoras. Debe asociarse a la relación riesgo-recompensa.
9. **Relación riesgo-recompensa:** esencial para comprender cuántos son nuestros riesgos en comparación con nuestra recompensa objetivo. De nuevo, los ratios HIT y riesgo-recompensa (ratios R) son las dos caras de la misma moneda.
10. **Ratio de Sharpe:** métrica financiera esencial, permite comprender los beneficios de la rentabilidad de un porcentaje más de riesgo. Debe anualizarse.

- 11. Ratio de Sortino:** misma métrica que el ratio de Sharpe, pero calculamos el riesgo utilizando la volatilidad a la baja en lugar de la volatilidad clásica para el ratio de Sharpe.
- 12. Beta:** nos dará algunas indicaciones sobre el grado de correlación de la estrategia con el mercado (SP500, por ejemplo) (más explicaciones en el capítulo 5).
- 13. Alpha:** nos dice cómo la estrategia rinde más o menos que el mercado. (Más explicaciones en el capítulo 5).
- 14. Ratio de información:** nos permitirá comparar la pareja riesgo-recompensa de la estrategia con la pareja riesgo-recompensa de referencia.
- 15. Rendimiento de los activos sin riesgo:** considera los rendimientos de los activos sin riesgo a lo largo del periodo (anualizados) para compararlos con los rendimientos anualizados de la estrategia.
- 16. VaR:** nos dará la peor pérdida que podemos tener con un umbral de error del 5%. Podemos calcularlo sobre el periodo que queramos: la peor pérdida por día, mes o año, por ejemplo. (Más explicaciones en el capítulo 5).
- 17. cVaR:** como el VaR, pero con algunas diferencias en el cálculo. (Más explicaciones en el capítulo 5).



Utilizar 20 indicadores de backtest no hará que tu estrategia sea mejor. En mi opinión, 5 buenos indicadores son claramente suficientes para tener una visión rápida.

6.3.2 Simulación Monte Carlo

Hasta ahora, hemos analizado el rendimiento de nuestra estrategia en un único camino posible. De hecho, el pasado es sólo un camino posible entre un infinito.

Para analizar distintos caminos posibles, podemos utilizar simulaciones de Monte Carlo. Para crear una simulación de Montecarlo, tomaremos aleatoriamente los rendimientos de la estrategia y reorganizaremos los datos.

Código 6.4: Calcular los rendimientos mensuales

```
def monte_carlo(data, method = "simple"): 1
    random_returns = []
    data["returns"] = data["returns"].fillna(value = 0)

    for _ in tqdm(range(100)):
        returns = data["returns"] - 10 ** -100 2
        np.random.shuffle(returns)
        random_returns.append(returns)

    if method == "simple":
        df_ret = pd.DataFrame(random_returns).
            transpose().cumsum() * 100
        cur_ret = data["returns"].cumsum() * 100
    else:
        df_ret = ((1 + pd.DataFrame(random_returns).
            transpose()).cumprod() - 1) * 100
        cur_ret = ((1 + data["returns"]).cumprod() - 1) * 100

    p_90 = np.percentile(df_ret, 99, axis = 1)
    p_50 = np.percentile(df_ret, 50, axis = 1)
    p_10 = np.percentile(df_ret, 1, axis = 1)

    plt.figure(figsize = (20, 8))
```

```

plt.plot(df_ret.index, p_90, color = "#39B3C7")
plt.plot(df_ret.index, p_50, color = "#39B3C7")
plt.plot(df_ret.index, p_10, color = "#39B3C7")

plt.plot(cur_ret, color = "blue", alpha = 0.60,
         linewidth = 3, label = "Current returns")

plt.fill_between(df_ret.index, p_90, p_10,
                 p_90 > p_10, color = "#669FEE",
                 alpha = 0.20, label = "Monte carlo area")

plt.ylabel("Cumulative returns %", size = 13)
plt.title("MONTE CARLO SIMULATION", size = 20)

plt.legend()
plt.show()

```

- 1 Podemos elegir el método de capitalización. Los resultados serán diferentes si utilizamos capitalización simple o compuesta.
- 2 Para cada simulación, tenemos que cambiar un poco los valores para barajar los datos de nuevo (es sólo una restricción de programación), pero no va a cambiar nada a los resultados.

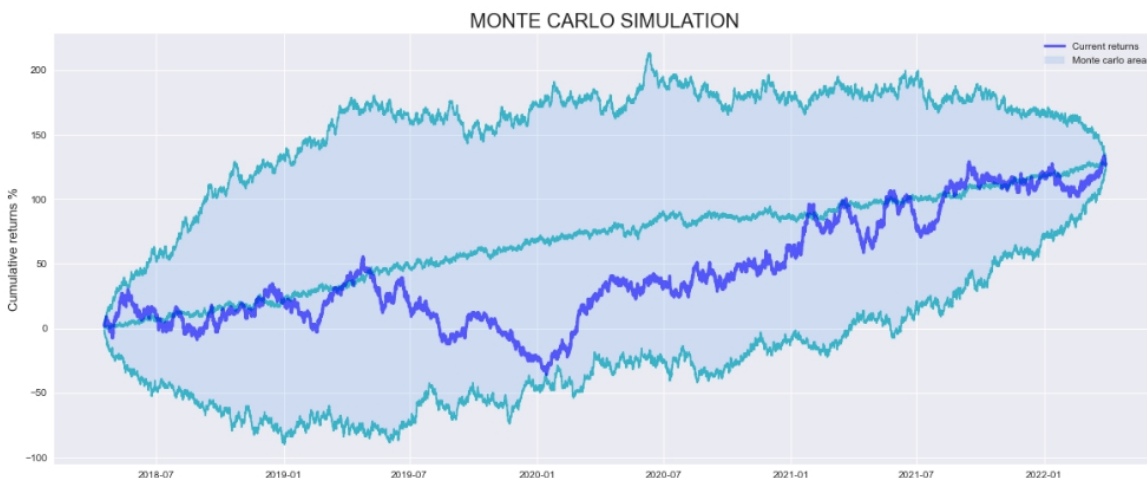
Una vez que tengamos varias simulaciones, tenemos que trazarlas de forma inteligente. Para destacar los distintos caminos posibles, crearemos un área de Monte-Carlo (entre el 1er centil dinámico y el 99º centil) y pondremos la mediana dinámica con los rendimientos como en la Figura 6.6.

Es esencial hacer este análisis para comprender la pérdida que hemos tenido en el peor de los casos (curva del 1er centil). En nuestro caso, podemos ver que hemos perdido casi todo nuestro capital en el primer año. Esta estrategia es demasiado volátil porque incluso si se sigue la distribución de

la estrategia, es posible haber perdido el 90% de nuestro capital en el primer año.

Cuando tenemos nuestra área de Monte-Carlo, lo mejor es ver un área corta cerca de una línea de tendencia alcista. Cuanto más pequeña sea el área, menor será la volatilidad de la estrategia.

Figura 6.6: Simulación Monte-Carlo con capitalización simple de intereses



Vista la banda inferior del Monte-Carlo, podemos ver que en uno de los peores caminos, podemos perder mucho tras el lanzamiento de nuestra estrategia. Además, el área de Monte-Carlo es demasiado grande.

6.3.3 Stop dinámico fácil

El stop loss dinámico requiere ticks para ser calculado correctamente. Sin embargo, aquí hay un consejo rápido para encontrar el peor caso posible utilizando una estrategia de stop loss dinámico. Así, si utilizamos la función backtest que encontrará el peor de los casos, veremos si, en el peor de los casos, somos rentables o no. Si es así, es una buena cosa porque significa que todos los grandes beneficios de la pérdida de stop dinámico serán un bono.

Hay muchos otros tipos de stop loss dinámico, como el stop loss dinámico clásico y los objetivos de doble beneficio. En nuestro ejemplo, utilizaremos el stop loss dinámico de umbral. Una vez que hemos tocado el TP, comenzamos el Stop Loss Dinámico (TSL, del inglés Trailing Stop Loss) con un margen del 0.1%. Esto significa que cuando tocamos el TP, nos aseguramos un 1.4% con un TP del 1.5%, y luego utilizamos un TSL clásico: para cada aumento del precio, aumentamos el stop loss para asegurarnos más y más beneficios. El objetivo será encontrar la rentabilidad. Si nunca obtenemos beneficios adicionales del stop loss dinámico, comprueba la rentabilidad de la estrategia para aumentar la probabilidad de rentabilidad en el trading en vivo.

Código 6.5: Calcular la rentabilidad mensual

```
def run_tsl(data, leverage = 1, tp = 0.015, sl = -0.015,
           tsl = 0.001, cost = 0.00):
    """
    :params (obligatorio): data(Debe contener las columnas
                           High_time y Low_time)
    :params (opcional): leverage = 1, tp = 0.015, sl = -0.015,
                       cost = 0.00
    :return: data con tres nuevas columnas: Low_time (TimeStamp),
                           High_time (TimeStamp),
                           High_first (Boolean)

    """
    tpl = tp - tsl 1
```

- 1 La única diferencia con la función run_tpsl es que necesitamos tener el margen TSL.



A veces calcular el peor caso de una estrategia o un indicador es mucho más fácil que calcular todo el proceso. Utilízalo en tu beneficio.

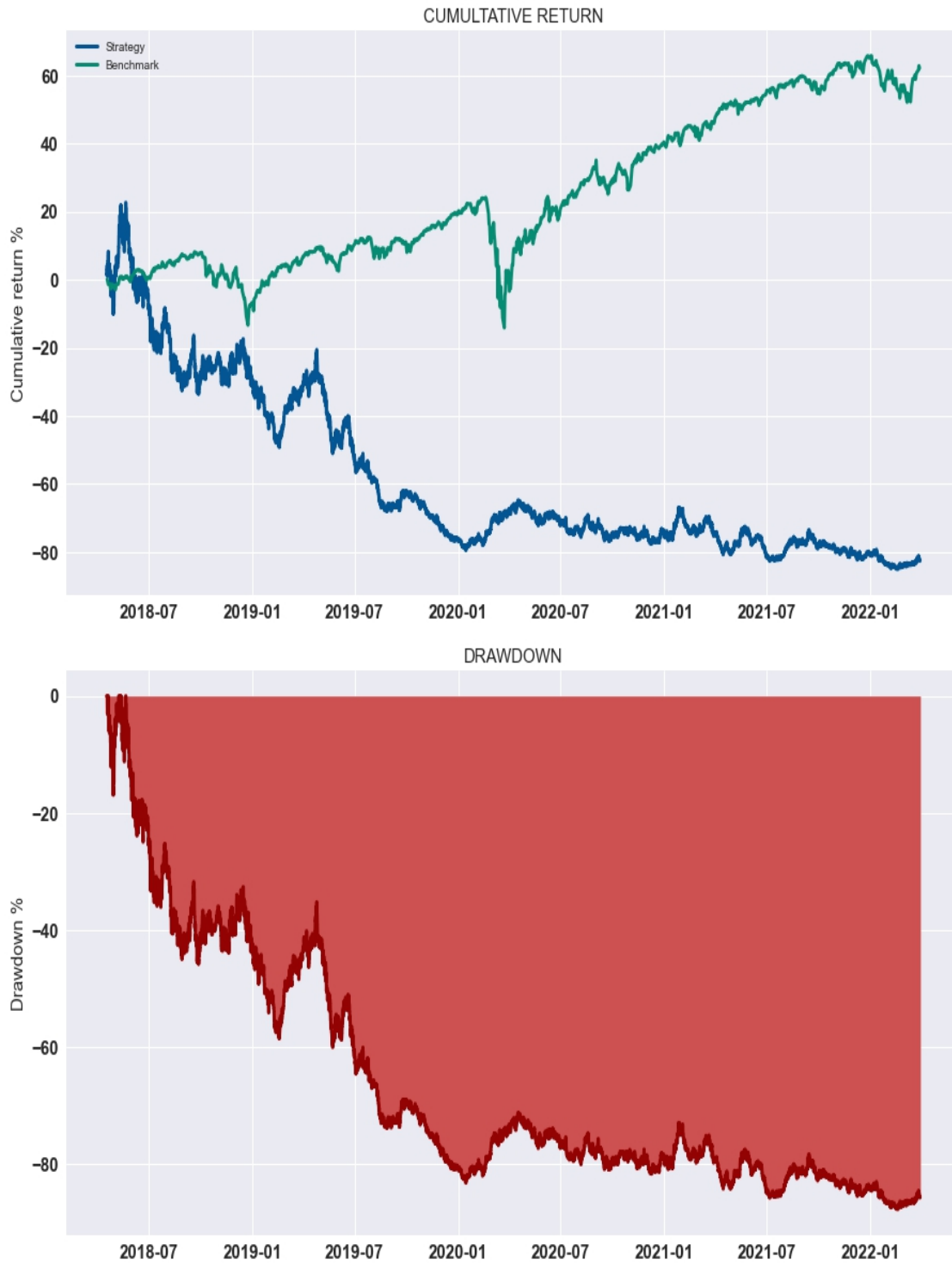
Como se muestra en la figura 6.7, la señal aleatoria con TSL en el peor de los casos no es rentable. Esto se debe a la escasa solidez de la estrategia con un subyacente de alta volatilidad.

Lógicamente, el peor caso de TSL no es muy bueno. Sin embargo, el objetivo será demasiado delicado, algo con rendimientos aceptables, lo que aumentará la probabilidad de ser rentable en el futuro. Será mucho más difícil en la creación, pero mucho más fácil cuando estamos en el comercio en vivo, y tenemos que saber cuando queremos sufrir.

Figura 6.7: Peor caso TSL con una señal aleatoria

Beta: -0.018	Alpha: -3.13 %	AVERAGE TRADE LIFETIME: 3H	
11min			

VaR: 1.32 %	cVaR: 1.5 %	TUW: 99.71%	drawdown: 87.8 %



Resumen

- Comprender el proceso para crear un Backtest lo más limpio posible.

- Cómo crear un backtest long-short utilizando TP y SL como señales de salida.
- Hay muchas métricas para entender nuestro backtest (17 en el capítulo, pero una infinidad que se puede utilizar). ¡No olvides que hacer 20 métricas backtest no hará nuestra estrategia mejor!
- Utiliza la simulación Monte-Carlo para comprender las diferentes trayectorias previas posibles de tus datos
- Cómo calcular fácilmente un Stop Loss Dinámico (TSL)

Parte 2: Modelos estadísticos predictivos

En esta parte se analiza la técnica clásica utilizada en el sector financiero. En primer lugar, aprenderemos la operativa de arbitraje estadístico con un ejemplo: la operativa de pares. A continuación, hablaremos del modelo ARMA para predecir los rendimientos de las acciones. Y para terminar, profundizaremos en la noción de regresión lineal y en el concepto de clasificación con la regresión logística.

Resumen:

Capítulo 7: Arbitraje estadístico

[Capítulo 8: Modelos de medias móviles autorregresivas \(ARMA\)](#)

Capítulo 9: Regresión lineal y regresión logística

Capítulo 7: Arbitraje estadístico

En este capítulo hablaremos del arbitraje estadístico en finanzas. La noción se aprenderá a través de la estrategia de trading de pares. En primer lugar, veremos la idea de estacionariedad y cointegración. A continuación, explicaremos cómo funciona la estrategia de trading de pares.

Antes de empezar el capítulo, hablemos del arbitraje estadístico. En este capítulo sólo veremos uno de los muchos ejemplos de la estrategia de arbitraje estadístico.

Teóricamente, obtener beneficios a largo plazo con una estrategia de trading es imposible porque el mercado debería estar en equilibrio y ser eficiente. Sin embargo, en la realidad es un poco diferente. De hecho, hay muchos desequilibrios en el mercado financiero, y las estrategias de arbitraje estadístico convierten este desequilibrio en un beneficio. Por ejemplo, una de las estrategias más sencillas consiste en encontrar errores en las cotizaciones de Forex.

Por ejemplo, tenemos los siguientes pares de divisas: $\text{NZD/AUD}=1$, $\text{AUD/CAD}=1.5$ y $\text{NZD/CAD}=1$. Tenemos CAD y queremos aprovechar una oportunidad de arbitraje estático para obtener beneficios. Usando el tercer par de divisas, podemos comprar 100NZD porque $100\text{NZD}=100\text{AUD}$ y $100\text{AUD}=150\text{CAD}$. Es una oportunidad estadística porque tenemos 100CAD. Podemos tener 150CAD usando un **desequilibrio** en el mercado sin riesgo.

7.1. De la estacionariedad a la cointegración

En esta sección se estudiará la estacionariedad de una serie temporal y cómo utilizarla para encontrar cointegración entre dos series temporales.

7.1.1. Estacionariedad

La estacionariedad de una serie temporal es esencial porque es un punto crucial del comportamiento de una serie temporal.

Hay estacionariedad si la serie temporal es **plana en torno a la media y sin tendencia**. Además, debe tener una **varianza constante en el tiempo**.^[4]. (Hay muchos supuestos que comprobar, pero estos son los más importantes).



Una serie temporal tiende a volver a una media constante si es estacionaria. Por lo tanto, es una buena cosa si deseas utilizar el comercio de arbitraje estadístico.

Podemos comprobar la estacionariedad de una serie temporal mediante el test de Dick y Fuller. La hipótesis nula de la prueba (H_0) es que la serie temporal no es estacionaria. Entonces, supongamos que tenemos un p-valor inferior en el umbral de error α . En ese caso, no se acepta la hipótesis H_0 , y se trata de una serie temporal estacionaria. Por ejemplo, si queremos saber a un umbral de error del 10% (si la serie temporal es estacionaria), calcularemos un test de Dick y Fuller aumentado y comprobaremos si el p-valor es inferior o superior al 10%. Así pues, existen las siguientes posibilidades:

- **$p > 0.10$:** H_0 es aceptada, y la serie temporal no es estacionaria. Tenemos que hacer algunas modificaciones si queremos que se convierta en estacionaria.
- **$p < 0.10$:** H_0 no es aceptada, y la serie temporal es estacionaria.

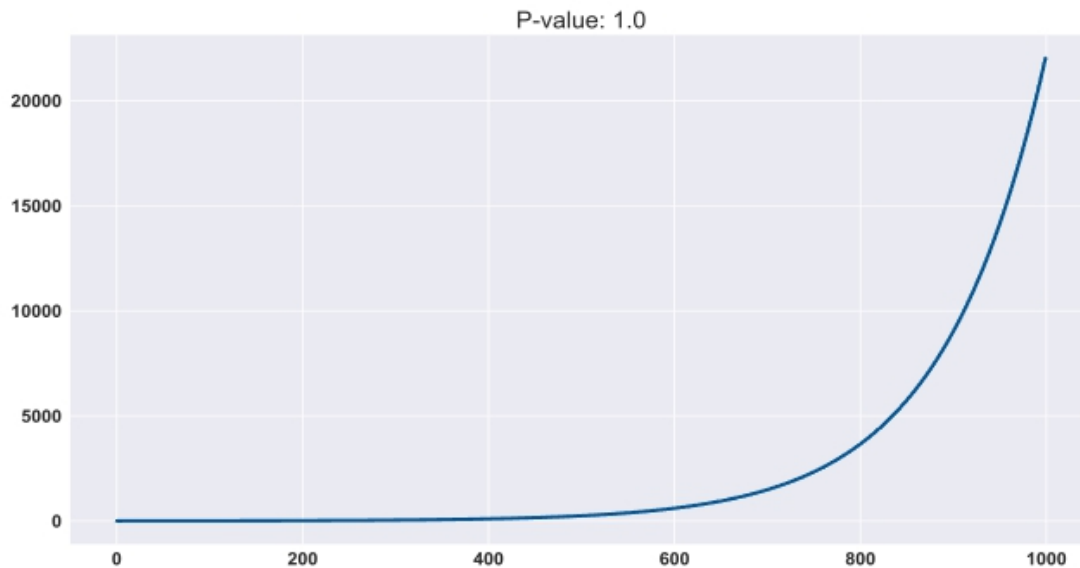


Existen muchas pruebas para determinar si una serie temporal es estacionaria. Sin embargo, hay que tener cuidado porque todas las pruebas tienen hipótesis nulas diferentes, ya que una es la estacionariedad y la otra es la no estacionariedad.

A continuación se muestran algunas imágenes de series temporales estacionarias y no estacionarias. En la figura 7.1 y

la figura 7.2, podemos ver una serie temporal no estacionaria y estacionaria, respectivamente.

Figura 7.1: Serie no estacionaria



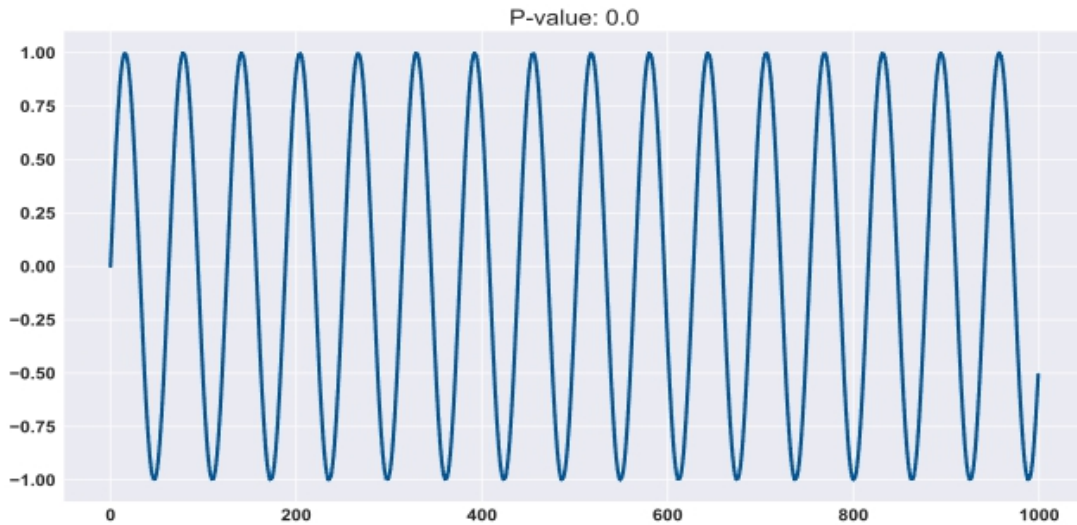
Como podemos ver en el título, el p-valor de esta serie temporal es 1.0. Esto significa que no hay duda de que esta serie no es estacionaria.



Cuanto más se acerque el p-valor a 0, mayor será la probabilidad de que la serie temporal sea estacionaria.

La figura 7.2 muestra el comportamiento de una serie temporal estacionaria. Es el dato más estacionario que podemos tener. Así, podemos encontrar las diferencias entre la figura 7.1 y la figura 7.2.

Figura 7.2: Serie temporal estacionaria



Como podemos ver en el título, el p-valor de esta serie temporal es 0.0. Esto significa que no hay duda de que esta serie es estacionaria.

7.1.2. Cointegración

Muchas veces, los precios de las acciones no son estacionarios. Entonces, ¿cómo podemos aplicar modelos que necesitan datos estacionarios? Para ello, vamos a hablar de cointegración. **La cointegración nos permite encontrar una serie temporal estacionaria combinando series temporales no estacionarias.** ^[5]



Para saber si dos series temporales están cointegradas, es necesario encontrar una relación como la siguiente (donde Y y X son dos series temporales)

$$Y = \alpha + \beta * X + \varepsilon,$$

donde ε es estacionario (el término de error) y X e Y son dos series temporales no estacionarias.

Por lo tanto, para crear una prueba de cointegración, necesitamos calcular la regresión lineal. Aprenderemos cómo funciona en detalle en uno de los capítulos siguientes. Ahora, lo único que necesitamos saber es que el propósito de la regresión es encontrar relaciones entre los datos. Como

hemos visto antes en el cuadro de consejos, la regresión lineal nos da el alfa y el beta para aproximar el Y, sólo conociendo el X y el ϵ es el término de error.

Código 7.1: test de cointegración

```
import statsmodels.api as sm
import statsmodels.tsa.stattools as ts

def cointegration(x, y):
    ols = sm.OLS(x, y).fit() 1
    adf_results = ts.adfuller(ols.resid) 2
    if adf_results[1] <= 0.1: 3
        return 'Cointegration'
    else:
        return 'No Cointegration'
```

- 1 Ajuste de la regresión lineal con Statsmodels.
- 2 *ols.resid* nos permite tomar los residuos fácilmente.
- 3 El umbral de aceptabilidad de la prueba es 0.1. Es decir, el 10%.

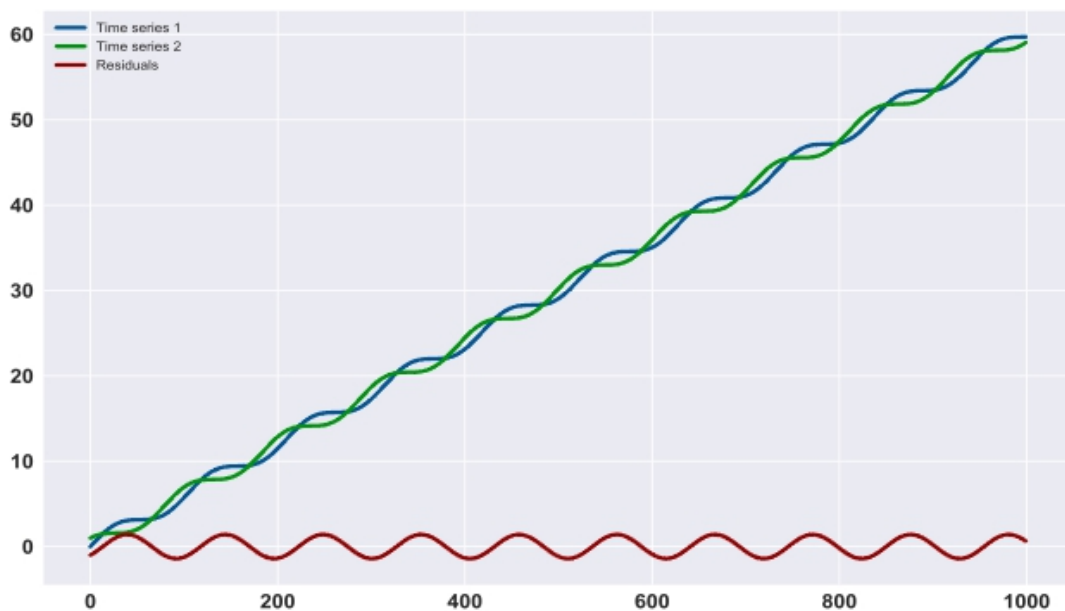


Con Python, es más fácil trabajar con la mejor librería para cada situación. **StatsModels** es mejor cuando necesitas hacer tareas de estadística con regresión lineal, y si quieres hacer una regresión lineal para aprendizaje automático, usarás **Scikit-learn**.

La figura 7.3 muestra la diferencia entre dos series temporales cointegradas. En efecto, en la figura se observa el residuo entre ambas series temporales. En este ejemplo, ambas series no son estacionarias. De hecho, el p-valor del test de Dick y Fuller aumentado para las dos series temporales es igual a 1.

Además, el p-valor del test de Dick y Fuller aumentado para los residuos es 0. Por lo tanto, los residuos son estacionarios. Además, se puede ver que en la figura sin cálculos.

Figura 7.3: Cointegrar dos series temporales



Como podemos ver, los residuos de esta serie temporal son estacionarios, y esto implica que las dos series están cointegradas porque también es fácil ver que las dos series temporales no son estacionarias.

7.2. Trading por pares

En la sección anterior hemos visto algunas nociones de estadística. En esta sección, las aplicaremos a un contexto financiero con una estrategia de trading por pares [\[6\]](#).

7.2.1. Cómo funciona

En esta parte, hablaremos de cómo funciona teóricamente el trading en pares. Nos permitirá entender cómo elegir los mejores activos y el proceso para hacer esta estrategia:

- **Debemos realizar un análisis cualitativo de los dos pares:** necesitamos activos del mismo sector. Por ejemplo, BTC/USD y ETH/USD. Entonces, es menos crítico, pero necesitamos encontrar activos con una capitalización de mercado similar y volúmenes diarios negociados equivalentes.
- **Analizamos si las series temporales están cointegradas:** esta es la parte esencial del proceso. Necesitamos tener dos series temporales no estacionarias con residuos estacionarios para encontrar una relación lineal entre los dos activos.
- **Analizar la correlación:** si tenemos un conjunto de activos y demasiados con cointegración, podemos tomar el n-ésimo activo mejor correlacionado (en rendimientos).

Una vez hecho esto, ya tenemos el par de activos para hacer la estrategia de trading por pares. Es necesario calcular los z-scores utilizando las diferencias entre las dos series temporales para la cartera dinámica (si no recuerdas cómo hacerlo, puedes ir a la sección 4.1).

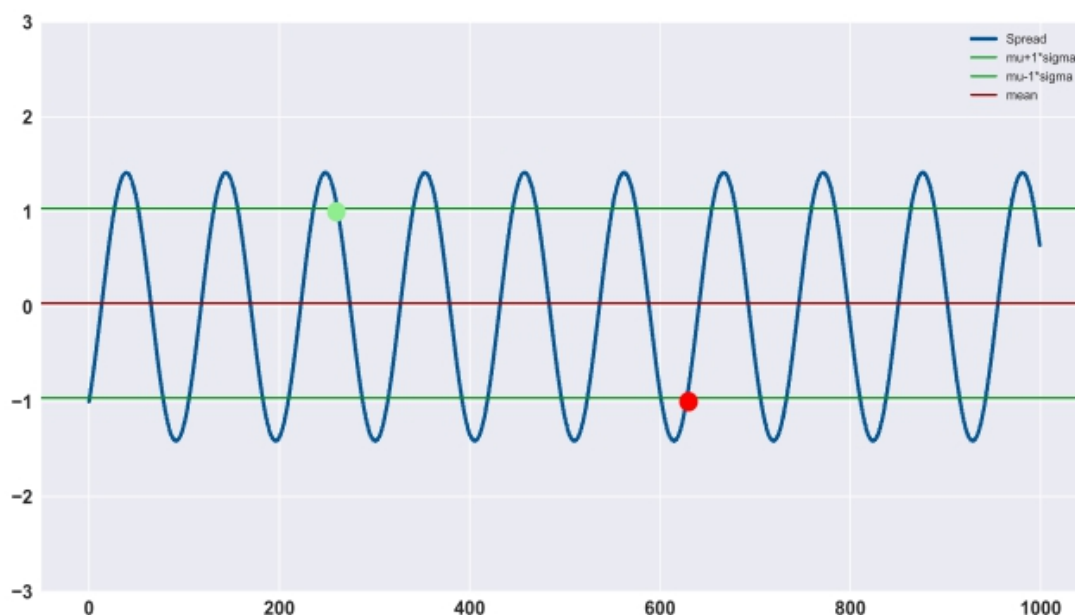


Para la estrategia de trading de pares, puedes calcular los z-scores utilizando la diferencia de la diferencia logarítmica. Es una elección personal.

Ahora que tenemos la puntuación z, tenemos que definir las señales largas y cortas. Hay muchas formas de hacerlo. Una de ellas es entrar en posiciones cuando pasamos un umbral de desviación estándar y salir de las posiciones en la media de la dispersión.

La figura 7.4 muestra el diferencial de la figura 7.3 con la desviación típica y la media de los valores. El círculo verde representa una posición corta para la serie temporal 1 y una posición larga para la serie temporal 2. De hecho, el diferencial es serie temporal 1 menos serie temporal 2. Necesitamos una disminución de la serie temporal 1 o un aumento de la serie temporal 2 para volver a la media del diferencial. El círculo rojo representa la inversa de las posiciones porque el diferencial es negativo.

Figura 7.4: Spread con umbrales



Esta figura muestra el diferencial con la desviación típica y la media del diferencial y la señal de posición de entrada.



Es fundamental entrar en posición cuando el diferencial cruza el umbral por segunda vez y no antes porque el diferencial puede seguir aumentando la primera vez.

7.2.2. Aplicaciones

En esta parte, vamos a calcular una estrategia de trading de pares. En primer lugar, vamos a utilizar una función para

encontrar los pares cointegrados y la correlación entre los dos activos para elegir el mejor par.

Tenemos una lista de 10 divisas, y necesitamos encontrar todas las combinaciones de 2 divisas de estas diez divisas. Para encontrar el número de combinaciones, podemos utilizar la siguiente fórmula:

$$C_r^n = \frac{n!}{r!(n-r)!}$$

Donde n es el número de elementos y r el número de elementos que elegimos por combinación. Aquí $r=2$ y $n = 10$ porque para cada combinación elegimos 2 activos de los 10.



$n!$ es el product de todos los números de 1 a n . Por ejemplo,

$10! = 1 * 2 * 3 * 4 * 5 * 6 * 7 * 8 * 9 * 10$. Se denomina factorial de n .

Una vez que sabemos que tendremos 45 combinaciones posibles, las computamos en una lista con el código 7.2.

Código 7.2: Encontrar las combinaciones

```
# Necesitamos encontrar el número de combinaciones de 2 elementos de un total de 10
# 10! / (2! * 8!) = 45 1

# Inicializar las variables
nbc = 0
list_com = []

while nbc < 45:
    # Tomar los activos para el par al azar
    c1 = np.random.choice(currencies)
    c2 = np.random.choice(currencies) 2

    # Añadir la lista de los dos activos
```



```

# Añadir los resultados de la correlación
row.append(train_set[com].pct_change(1).corr().values[0][1])

# Añadir cada fila para hacer una lista de listas
resume.append(row)

# Crear un dataframe para una mejor visualización
sum = pd.DataFrame(resume, columns = ["Asset1", "Asset2",
                                     "Cointegration", "Cor"])

# Filtrar la fila por el par cointegrado
sum.loc[sum["Cointegration"] == "Cointegration"]

```



Utilizaremos la estructura de lista de listas porque no depende del número de pares. Por lo tanto, podemos aplicar este código a cualquier lista de pares.

Figura 7.5: Resumen de las métricas de cada par

ç	Activo 2	Cointegración	Correlación
NZDUSD	AUDNZD	True	-0.27
AUDNZD	EURUSD	True	-0.01
USDCHF	EURUSD	True	-0.55
EURAUD	USDCHF	True	-0.15

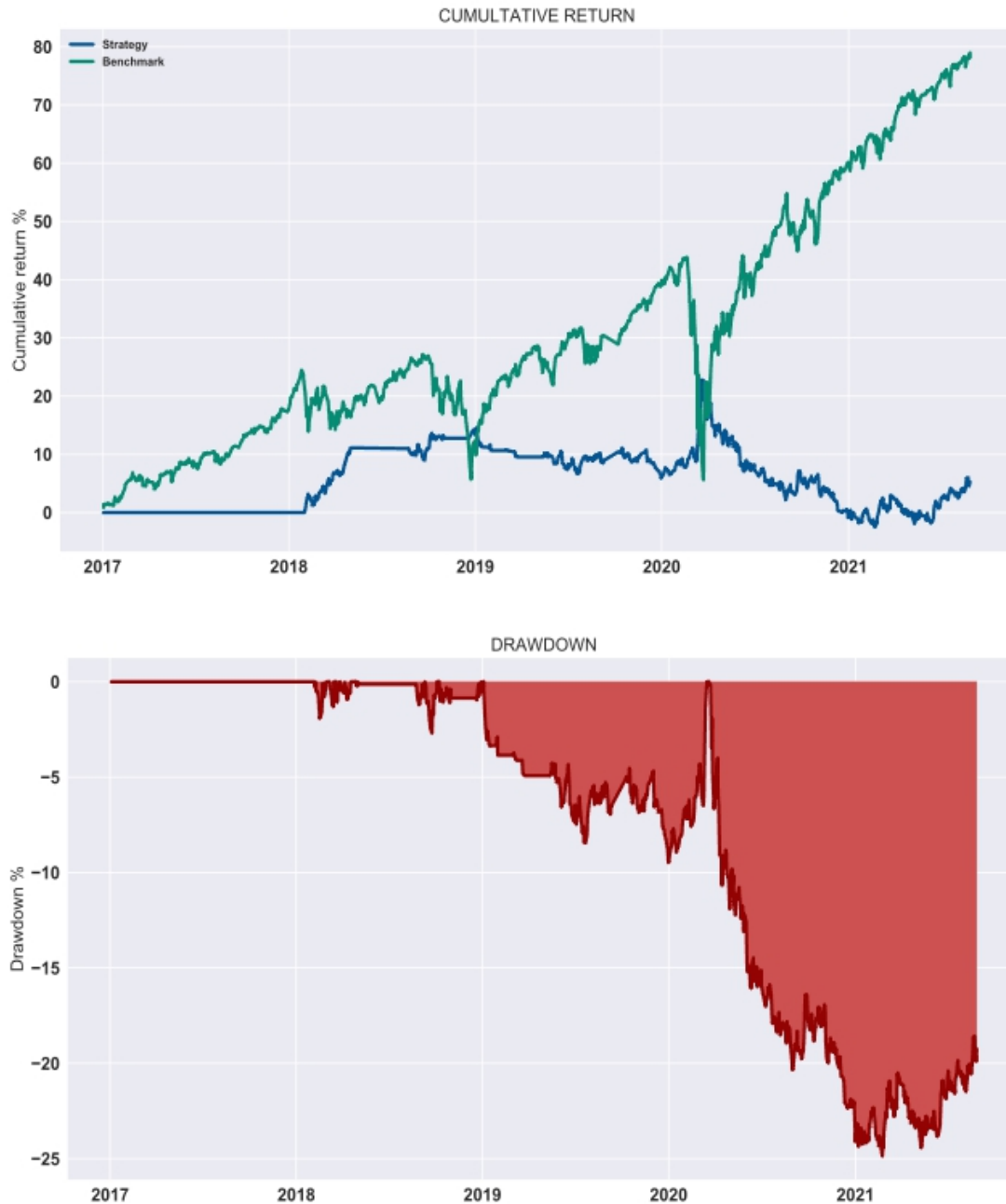
En esta figura, podemos ver los pares cointegrados de las 45 combinaciones posibles.

Instintivamente, cuanto menor sea la correlación negativa entre el rendimiento del activo, mejor funcionará la estrategia porque tomamos posiciones inversas. Mientras que (USDCHF, EURUSD) y (NZDUSD, AUDZND) no toman posiciones con esta estrategia (se puede verificar con el código). Por lo tanto, tomamos el par (EURAUD, USDCHF). Podemos ver el resultado de la estrategia en la figura 7.6.

Figura 7.6: Rendimiento de la estrategia en (EUR/AUD,USD/CHF)

Beta: -0.016 Alpha: 1.41 % Sharpe: 0.047 Sortino: 0.056

VaR: 17.6 % cVaR: 20.01 % VaR/cVaR: 1.137 drawdown: 24.86 %



Esta figura muestra el backtest de la estrategia de trading de pares EURAUD USDCHF. El backtest no es emocionante porque los rendimientos acumulados no son muy altos. Tenemos un ratio de Sharpe pequeño; sobre todo, el drawdown no deja de crecer durante 3 años.



La trading de pares tiene un gran supuesto, que es el hecho de que la situación del mercado no debe cambiar. Sin embargo, la crisis del COVID ha provocado un desequilibrio considerable del mercado, que no ha recuperado el equilibrio rápidamente.

Resumen

- El arbitraje estadístico se produce cuando podemos obtener beneficios del mercado financiero sin riesgo gracias a un desequilibrio en el mercado.
- Una serie temporal es estacionaria cuando es plana en torno a la media, sin tendencia y con varianza temporal constante.
- La cointegración es útil en finanzas para crear una serie temporal estacionaria a partir de dos series temporales no estacionarias.

Aplicación Trading en vivo y Screener

Este código se basa en la clase creada en el anexo: MetaTrader.

Código 7.7: Aplicación Trading / Screener para trading por pares

```
import warnings
warnings.filterwarnings("ignore")
from MT5 import *
import numpy as np
import time

def pair_trading(ts1_symbol, ts2_symbol, pair = 1):
    mt5.initialize()
```

Importar los datos

```
ts1_data = MT5.get_data(ts1_symbol, 3500)["close"]
ts2_data = MT5.get_data(ts2_symbol, 3500)["close"]
train_set = pd.concat((ts1_data, ts2_data), axis = 1).dropna()
train_set.columns = [ts1_symbol, ts2_symbol]

train_set["spread"] = train_set[ts1_symbol]-train_set[ts2_symbol]

train_set["z-score"] = (train_set["spread"] -
                        train_set["spread"].mean()) /
                        train_set["spread"].std()

std_high = train_set["z-score"].mean() + train_set["z-score"].std()
std_low = train_set["z-score"].mean() - train_set["z-score"].std()
mean = train_set["z-score"].mean()
```

Ayer

```
yts1 = train_set[ts1_symbol].values[-2]
yts2 = train_set[ts2_symbol].values[-2]
```

Hoy

```
pts1 = train_set[ts1_symbol].values[-1]
pts2 = train_set[ts2_symbol].values[-1]
```

Datos de hoy

```
spread = pts1 - pts2
zscore = (spread-train_set["spread"].mean()) /
          train_set["spread"].std()
```

Ayer

```
yspread = yts1 - yts2
yzscore = (yspread-train_set["spread"].mean()) /
          train_set["spread"].std()
```



```
# Serie temporal 1
short_ts1 = False
long_ts1 = False

if zscore < std_high and yzscore > std_high:
    short_ts1 = True
    long_ts1 = False

elif zscore > std_low and yzscore < std_low:
    short_ts1 = False
    long_ts1 = True

else:
    pass

# Serie temporal 2
short_ts2 = False
long_ts2 = False

if zscore < std_high and yzscore > std_high:
    short_ts2 = False
    long_ts2 = True

elif zscore > std_low and yzscore < std_low:
    short_ts2 = True
    long_ts2 = False

else:
    pass

# Posiciones
if pair == 1:
    buy, sell = long_ts1, short_ts1
```

```

else:

    buy, sell = long_ts2, short_ts2

return buy, sell

# True = Trading en vivo and False = Screener
live = True

if live:

    current_account_info = mt5.account_info()

    print("-----")
    print("Date: ", datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
    print(f"Balance: {current_account_info.balance} USD, \t"
          f"Equity: {current_account_info.equity} USD, \t"
          f"Profit: {current_account_info.profit} USD")
    print("-----")

# Inicializar los inputs
symbols = ["EURUSD", "EURGBP"]
lots = [0.01, 0.01]
pairs = [1, 2]

start = datetime.now().strftime("%H:%M:%S") #"23:59:59"
while True:

    # Verificación para el lanzamiento
    if datetime.now().weekday() not in (5, 1):

        is_time = datetime.now().strftime("%H:%M:%S") == start

    else:

        is_time = False

    # Lanzar el algoritmo
    if is_time:

```

```
for symbol, lot, pair in zip(symbols, lots, pairs):

    # Crear las señales
    buy, sell = pair_trading(symbols[0], symbols[1],
                             pair = pair)

    # Ejecutar el algoritmo
    if live:
        MT5.run(symbol, buy, sell, lot)

    else:
        print(f"Symbol: {symbol}\n"
              f"Buy: {buy}\n"
              f"Sell: {sell}")

    time.sleep(1)
```



El parámetro live establece el modo de trading. Este será directo si live = True o bien el modo será screener si establecemos live = False.

Capítulo 8: Modelos de medias móviles autorregresivas (ARMA)

En este capítulo, explicaremos cómo funcionan los modelos ARMA. En la primera sección, exploraremos los conceptos de series temporales. A continuación, explicaremos el modelo AR y el modelo MA. Además, explicaremos el modelo ARMA y sus derivados.

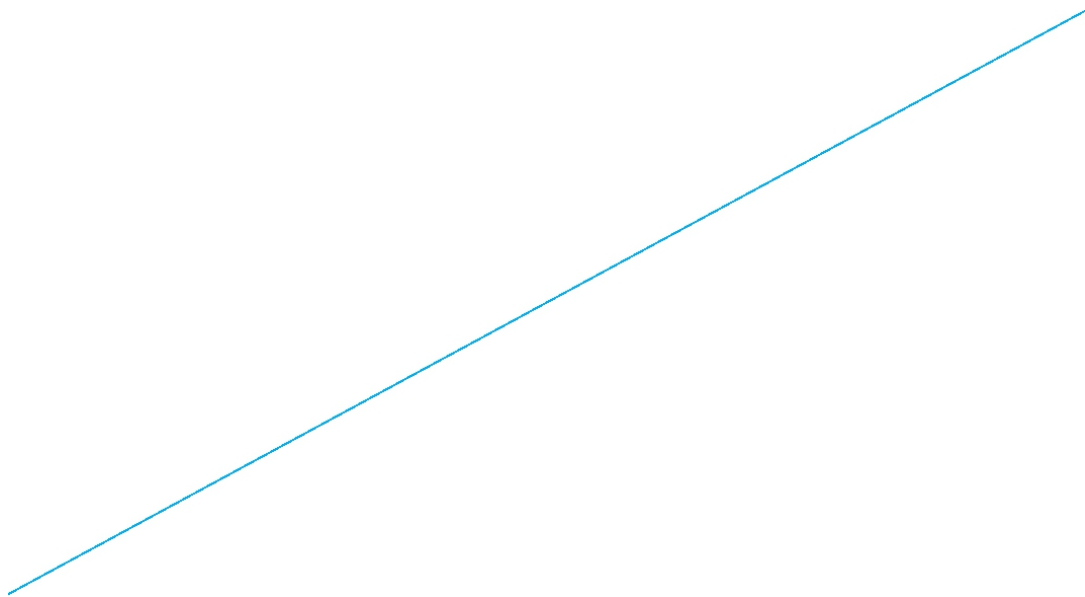
8.1. Fundamentos de las series temporales

En esta sección se explican algunos requisitos previos del concepto de serie temporal. Explicaremos la idea de tendencias, ciclos y estacionalidad. A continuación, explicaremos algunas cosas interesantes sobre las distintas formas de calcular los rendimientos. A continuación, explicaremos el modelo de predicción más fácil de entender: la regresión lineal.

8.1.1. Tendencia, ciclo, estacionalidad

Las series temporales tienen tres conceptos esenciales: tendencia, ciclo y estacionalidad. Empecemos por el más sencillo. La tendencia es un comportamiento a largo plazo seguido por la serie temporal. Podemos ver un ejemplo en la figura 8.1.

**Figura 8.1: Tendencia al alza del precio de las acciones de Google
(desde 2007 hasta 2021)**



Trend

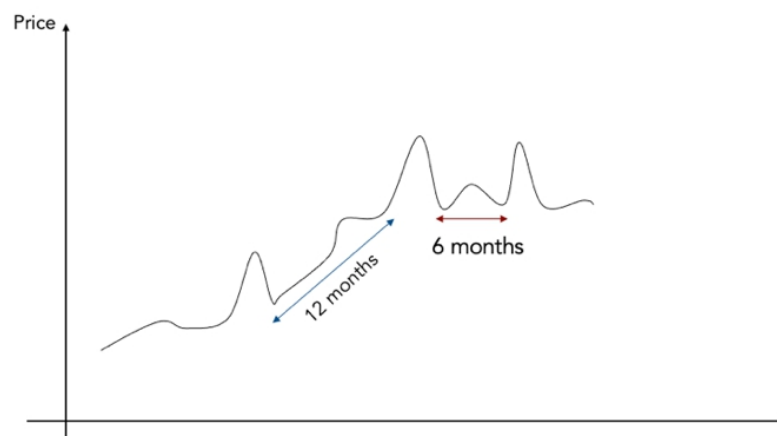
Esta figura muestra una tendencia exponencial al alza del precio de las acciones de Google desde 2007 hasta 2021. De

hecho, podemos ver que el precio de las acciones aumenta incluso si hay algún período a la baja.

También existe la posibilidad de una tendencia bajista en el precio de una acción, pero sólo es interesante si podemos ponernos cortos en la acción.

Hablemos ahora del ciclo en las series temporales. Para explicarlo, crearemos una serie temporal ficticia para resaltar algunos puntos. Una serie temporal es cíclica si existe un esquema repetido en el precio con un tiempo no determinado entre dos repeticiones de este comportamiento. Veamos un ejemplo en la figura 8.2

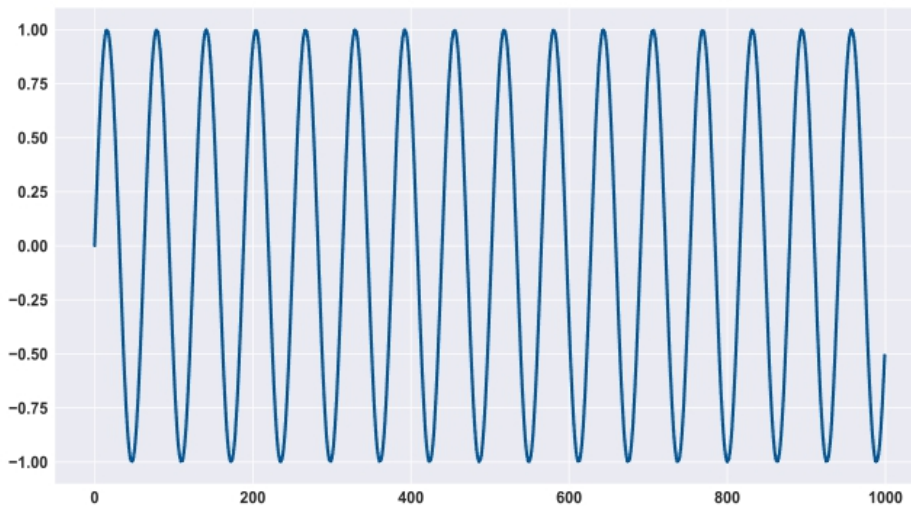
Figura 8.2: Serie temporales cíclica



Vemos que a veces se repite el mismo comportamiento, pero con un tiempo no determinado entre cada repetición.

Veamos ahora cómo hallar una serie temporal estacional. La estacionalidad es como la ciclicidad, pero hablamos de estacionalidad con igual tiempo entre cada repetición. Veamos la diferencia entre ambos conceptos en la figura 8.3.

Figura 8.3: Serie temporal estacional



En esta figura, podemos ver el mismo tiempo entre dos repeticiones del comportamiento. Por lo tanto, se trata de una serie temporal estacional.

8.1.2. Propiedades de precio logarítmico

Antes de profundizar, debemos aprender algunas propiedades del precio logarítmico. A veces utilizamos el precio logarítmico porque nos permite comprender mejor la variación de la rentabilidad. Supongamos que queremos encontrar el mejor periodo de rentabilidad (para las acciones de Google). En ese caso, comprobaremos el precio de Google e intentaremos ver el mejor aumento en un gráfico. Sin embargo, el precio de una acción es un valor absoluto y no pone de relieve la evolución relativa de la acción. Expliquémoslo con un ejemplo en la figura 8.4.



La elección entre rendimientos y rendimientos logarítmicos es una opción personal, pero casi siempre pruebo ambos para optimizar mis algoritmos.

Figura 8.4: Precio logarítmico y precio de las acciones de Google

Precio absoluto de las acciones



Precio logarítmico de las acciones



Este gráfico muestra que el periodo de 2005 a 2008 es mejor que el periodo de 2017 a 2020 para una estrategia long-only gracias al gráfico de precios logarítmicos porque sitúa el precio en un rango mínimo.

8.1.3. La regresión lineal

En esta subsección, crearemos un modelo de regresión lineal. Explicaremos cómo funciona la regresión lineal porque este concepto es necesario para entender los modelos ARMA.

La regresión lineal es un algoritmo de predicción. El objetivo de este algoritmo es permitirnos predecir algunos datos. Por ejemplo, supongamos que conocemos la relación entre el oro y el S&P 500. Podemos utilizar la regresión lineal para predecir el precio de las acciones del S&P 500 utilizando el precio del oro.

Veamos cómo calcular una regresión lineal utilizando un problema de optimización y expliquemos más profundamente la intuición utilizando la figura 8.5.

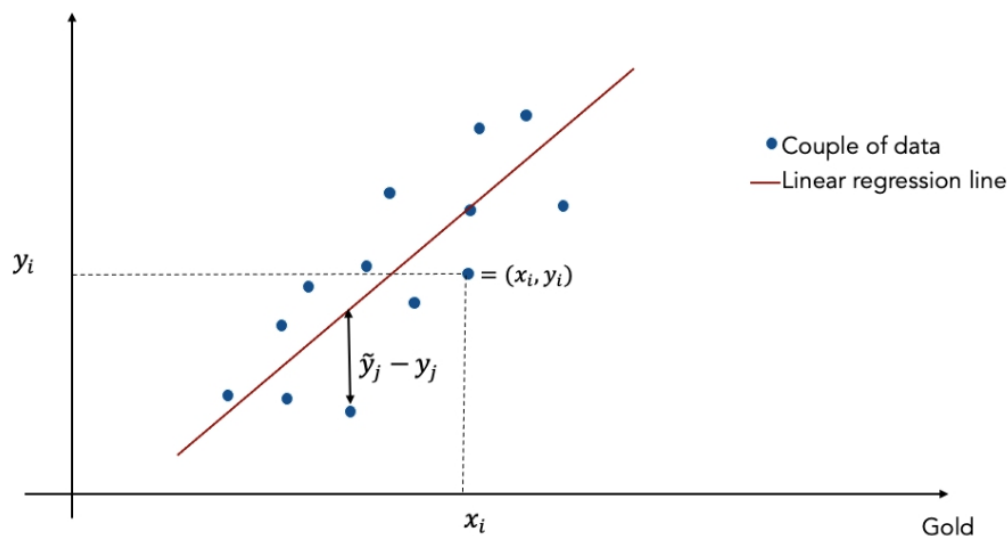
$$\min_{\beta} \sum (\tilde{y}_i - y_i)^2$$

Donde $\tilde{y}_i$ es el valor predicho, y_i el valor real y β son los parámetros del modelo.



La intuición de la regresión lineal puede ser retomada por un algoritmo que trata de encontrar la mejor manera de minimizar la distancia entre los valores predichos y reales.

Figura 8.5: Intuición de la regresión lineal



Esta figura muestra cómo funciona la regresión lineal con un modelo que toma el oro y predice el precio del S&P 500.

Es importante entender que para un conjunto de datos X que tiene m filas y n columnas $\tilde{y}_i = \beta_0 + \beta_1 x_{i,1} + \dots + \beta_n x_{i,n}$. En nuestro ejemplo con el S&P 500 y el oro $\tilde{y}_i = \beta_0 + \beta_1 \text{gold price}_i$ (β_0 a β_n son los parámetros del modelo).

8.2. Modelos AR and MA

En esta sección, aprenderemos dos algoritmos de predicción prácticos: el modelo autorregresivo (AR) y el modelo de media móvil (MA).

8.2.1. Modelo autoregresivo (AR)

La intuición que subyace al modelo autorregresivo es sencilla. Es una regresión lineal, pero utiliza el valor anterior de la acción para predecir el valor de hoy o de mañana.

Por ejemplo, si volvemos a tomar el modelo con el S&P 500 y el oro, predeciremos el S&P 500 utilizando el oro en la regresión lineal. Ahora, predeciremos el S&P 500 en el momento t utilizando el S&P 500 en el momento $t-1$, que puede ampliarse al S&P 500 en el momento $t-p$. Veamos la ecuación del modelo $AR(p)$, donde p es el número de datos previos que se toman.

$$AR(1): SP500_t = \phi_0 + \phi_1 SP500_{t-1}$$

$$AR(p): SP500_t = \phi_0 + \phi_1 SP500_{t-1} + \dots + \phi_p SP500_{t-p}$$

Donde de ϕ_0 a ϕ_p son los parámetros del modelo y cada parámetro debe tener un valor comprendido entre 0 y 1.

Veamos ahora cómo hacer un modelo AR utilizando Python sobre la divisa EURUSD.

Código 8.1: Implementación del modelo AR

```
def AR_predict_value(train_set):  
    """  
    Función para predecir el valor de mañana utilizando el modelo AR  
    """  
  
    # Definir el modelo  
    p = 1  
    model = ARIMA(train_set, order = (p, 0, 0))  
  
    # Ajustar el modelo  
    model_fit = model.fit(dispatch = 0)  
  
    # Hacer previsiones  
    forecast = model_fit.forecast() 1  
  
    return forecast[0][0]  
  
def AR(df, returns = True):  
    """  
    Función para predecir el conjunto de entrenamiento con  
    Un modelo AR  
    """  
  
    # Encontrar la división  
    split = int(len(df)*0.70)  
  
    # Predecir el precio utilizando la función AR  
    df['predicted_value'] = df['Adj Close'].rolling(split).\  
        apply(AR_predict_value) 2
```

```

# Calcular la rentabilidad y la señal del activo
if returns:
    df['returns'] = df['Adj Close'] 3
    df['signal'] = np.where(df.predicted_value > 0, 1, -1) 4
else:
    df['returns'] = df['Adj Close'].pct_change(1) 5
    df['signal'] = np.where(df.predicted_value >
                           df['Adj Close'], 1, -1) 6

# Calcular el rendimiento de la estrategia
df['strategy'] = df.signal.shift(1) * df.returns 7

return df

```

- 1 Predecir el siguiente valor utilizando el modelo ajustado.
- 2 Aplicar una función a todo el conjunto de datos mediante *rolling()* de Pandas.
- 3 Si utilizamos los rendimientos para ajustar el modelo, la columna "returns" es igual a la columna "Adj Close".
- 4 Si la rentabilidad prevista es positiva, compramos, y si es negativa, vendemos.
- 5 Calcula el porcentaje de rentabilidad del activo porque "Adj Close" es el precio absoluto.
- 6 Crea la señal utilizando la condición: si el valor pronosticado está por debajo del cierre ajustado, mañana compramos o vendemos.
- 7 Calcula los rendimientos de la estrategia como la señal de ayer multiplicada por los rendimientos.

En la función AR, hemos tenido un parámetro `return = True`. Esto nos permite trabajar con rendimientos de



acciones o precios de acciones.

Podemos responder por qué hacer un modelo AR con los rendimientos es mejor que con el precio. La respuesta necesita algunos conocimientos del capítulo anterior. De hecho, es mejor trabajar con el rendimiento de las acciones que con el precio de las acciones porque necesitamos datos estacionarios (véase la sección 7.1).

Normalmente, con el modelo AR o ARMA, utilizamos los métodos de diferenciación para transformar los datos (pt - pt-1). Aún así, nos resulta difícil utilizar estos datos porque, en finanzas, no sólo queremos predecir un valor, sino crear una estrategia con este valor, y necesitamos el porcentaje de variación para calcular los rendimientos de la estrategia. Puede ser fastidioso calcular los rendimientos utilizando diferentes pasos, mientras que existe una función para hacerlo más accesible.



En finanzas, es mejor utilizar los rendimientos para tener datos estacionarios porque será más fácil calcular los rendimientos de la estrategia y comparar los activos entre sí.

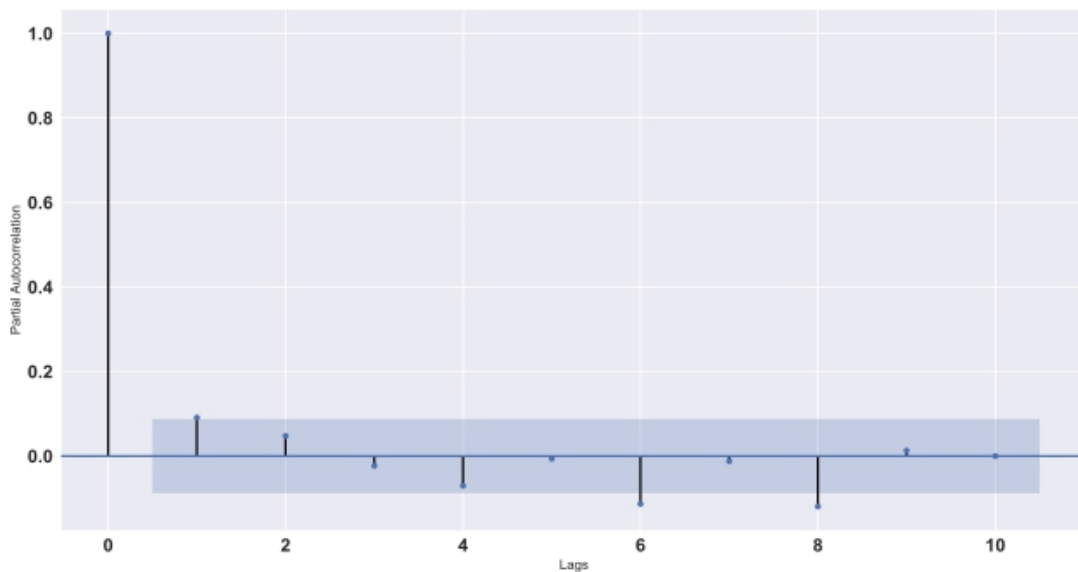
En el código 8.1, podemos ver que la p del modelo AR se fija en 1 (número de retardos del modelo autorregresivo). Pero, ¿por qué no 3, 8 ó 10? Existen muchas formas de encontrar el mejor valor para p . La primera es probar el error del modelo utilizando MSE o MAE (consulta la sección 1.1) para p de 1 a 15, por ejemplo, y elegir el error más bajo. Sin embargo, puede llevar mucho tiempo calcular todos los modelos.

La otra forma es utilizar un gráfico de autocorrelación parcial para encontrar el mejor número teórico de retardos. Sin embargo, ¿qué es la autocorrelación parcial? Una autocorrelación parcial resume la relación entre una observación de una serie temporal con observaciones

anteriores eliminando las relaciones de las observaciones intermedias.

Veamos cómo interpretar un gráfico de autocorrelación parcial (puedes encontrar el código para hacerlo en el cuaderno asociado al capítulo) en la figura 8.6.

Figura 8.6: Gráfico de autocorrelación parcial



Vemos el gráfico de autocorrelación de los rendimientos del EURUSD. La zona azul es el intervalo en el que la autocorrelación parcial no es significativa, lo que significa que todo el retardo mayor o menor que el azul es significativo. Así, podemos ver que los lag 1, 6 y 8 son significativos y tienen mejores resultados. Debemos elegir un AR(8) para estimar teóricamente el modelo EURUSD.

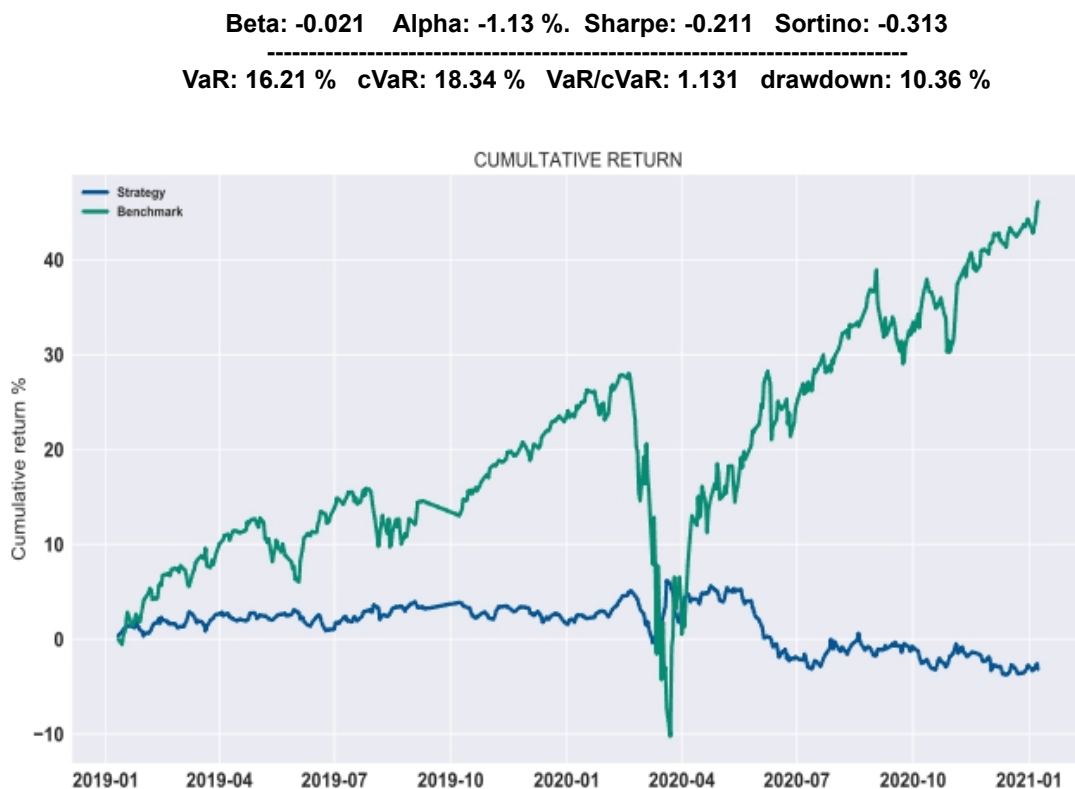


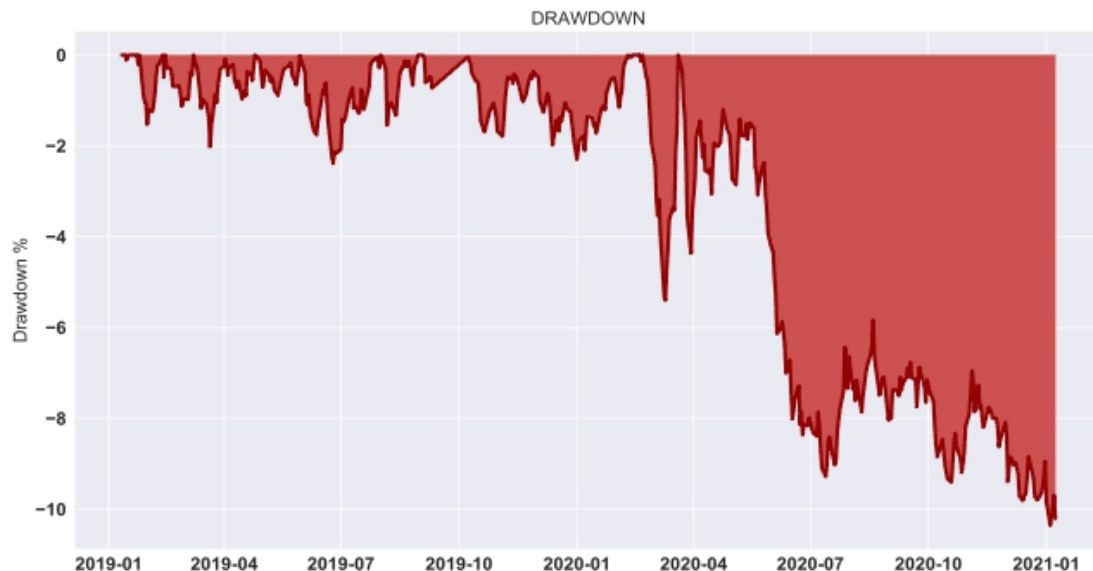
En finanzas, siempre se aplica un pequeño desfase para evitar el sobreajuste del algoritmo. Un desfase máximo de 3 puede ser una buena idea porque si tomamos varios desfases, por ejemplo, 150, el modelo será perfecto en los datos de entrenamiento y no se podrá utilizar en el conjunto de test. Necesitamos un modelo para predecir el valor de un conjunto de test.

Hablemos ahora del rendimiento de la estrategia con el modelo AR. En la figura 8.7, podemos ver los rendimientos acumulados de un método que utiliza el modelo AR en EURUSD. Podríamos decir que el modelo funciona bastante bien porque, en 1 año, ha obtenido un 8% de rentabilidad aunque tuviera un riesgo elevado pero, no es el punto destacado aquí.

Durante la crisis del COVID, la situación del mercado era diferente de la normal. Mientras que el modelo AR es un algoritmo predictivo que utiliza datos pasados para predecir el valor (algoritmo regresor), los datos de entrenamiento pueden tener propiedades muy diferentes como otras dos series temporales.

Figura 8.7: Backtest del modelo AR para EURUSD





La figura muestra que el backtest de la estrategia no es bueno. En efecto, alfa, Sharpe y Sortino son negativos. En cambio, vemos una excelente capacidad para predecir la rentabilidad antes de la crisis corona, aunque no sea excelente. Desde la crisis corona, el drawdown ha tenido un crecimiento constante. Por lo tanto, la estrategia no es buena y no la utilizamos así. Tenemos que buscar otro activo con una mayor autocorrelación parcial.

8.2.2. Modelo de media móvil (MA)

Hablemos ahora del modelo de media móvil. Este modelo no utilizará la media móvil como nombre, sino el error pasado del modelo. Es una parte esencial del modelo ARMA porque nos permite aprender no del valor del conjunto de entrenamiento sino del error del algoritmo. Veamos cómo funciona el modelo MA con su ecuación:

$$\text{MA}(1): \text{SP500}_t = \theta_0 + \theta_1 \varepsilon_{t-1}$$

$$\text{MA}(q): \text{SP500}_t = \theta_0 + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q}$$

Donde de θ_0 a θ_q son los parámetros del modelo y $\varepsilon_i = y_{\text{real}_i} - y_{\text{predict}_i}$.

Código 8.2: Implementación de la media móvil (MA)

```
def MA_predict_value(train_set):  
    """  
    Función para predecir el valor de mañana utilizando el modelo AR  
    """  
  
    # Definir el modelo  
    q = 1  
    model = ARIMA(train_set, order = (0, 0, q))  
  
    # Ajustar el modelo  
    model_fit = model.fit(dispatch = 0)  
  
    # Hacer previsiones  
    forecast = model_fit.forecast()  
  
    return forecast[0][0]  
  
def MA(df, returns = True):  
    """  
    Función para predecir el conjunto de test con un modelo MA  
    """  
  
    # Encontrar la división  
    split = int(len(df) * 0.70)  
  
    # Predecir el precio utilizando la función AR  
    df['predicted_value'] = df['Adj Close'].rolling(split).  
        apply(MA_predict_value)
```

```

# Calcular la rentabilidad del activo
if returns:
    df['returns'] = df['Adj Close']
    df['signal'] = np.where(df.predicted_value > 0, 1, -1)
else:
    df['returns'] = df['Adj Close'].pct_change(1)
    df['signal'] = np.where(df.predicted_value >
                           df['Adj Close'], 1, -1)

df['strategy'] = df.signal.shift(1) * df.returns

return df

```

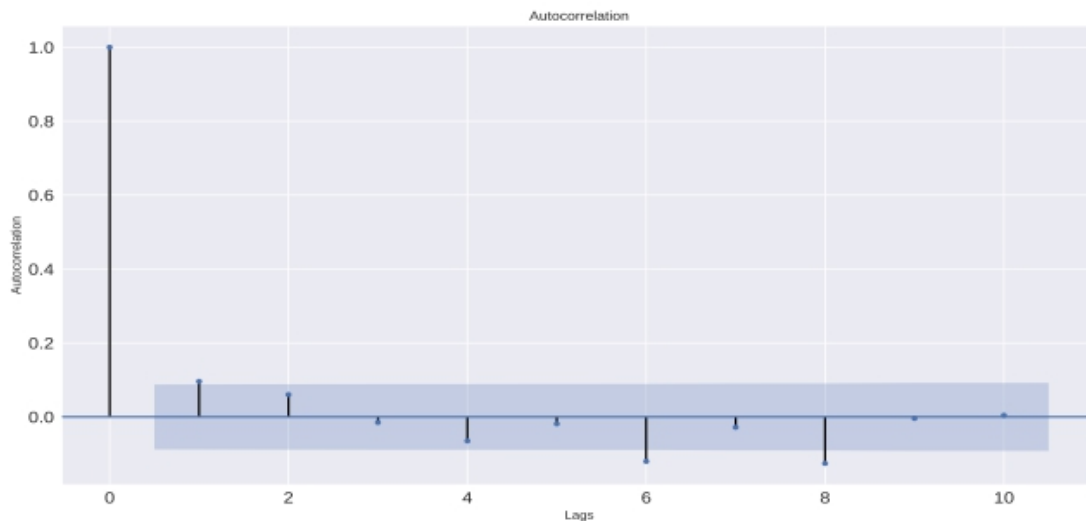
- 1 La única diferencia entre los modelos AR y MA son los parámetros de la función ARIMA de StatModels.



Para calcular el modelo de media móvil (MA), hemos utilizado la misma función que el modelo AR, la función ARIMA de Statsmodels. Sin embargo, hay una diferencia en los parámetros de la función, por lo que ésta se explicará mejor en la sección siguiente.

¿Cómo elegir la q? También podemos utilizar la técnica que prueba todas las q de 1 a 20 para ver cuál tiene el error más bajo en el conjunto de entrenamiento. Alternativamente, podemos elegir el mismo método que la autocorrelación parcial pero utilizando la autocorrelación absoluta. Veamos el gráfico de autocorrelación de los rendimientos del EURUSD en la figura 8.8.

Figura 8.8: Gráfico de autocorrelación de los rendimientos del EURUSD



Vemos que la autocorrelación es insignificante desde el retardo 1 hasta el retardo 10, excepto para los retardo 6 y 8, pero es muy pequeña..

Elegimos tomar un retardo de 1 para este modelo porque no hay autocorrelación. En la figura 8.9, podemos ver el rendimiento del MA(1), que es muy similar al AR(1). Sin embargo, hay una pérdida menor que el AR(1) durante la crisis del COVID porque el error de ayer se ajusta a él.

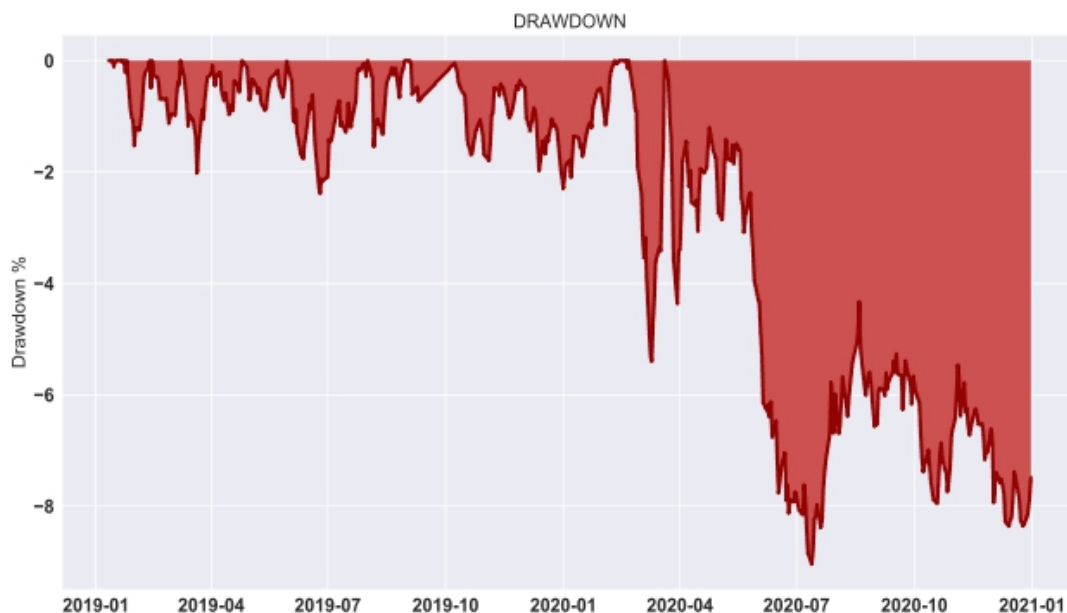


Cuando no hay autocorrelación o autocorrelación parcial significativa, se aconseja no utilizar un modelo AR o MA. De hecho, a veces lo mejor es no hacer nada.

Figura 8.9: Backtest del modelo MA para EURUSD

Beta: -0.017. Alpha: 0.68 % Sharpe: 0.167 Sortino: 0.259

VaR: 13.76 % cVaR: 15.92 % VaR/cVaR: 1.157 drawdown: 8.01 %



Los ratios alfa, Sharpe y Sortino son positivos. Sin embargo, en cuanto a la estrategia anterior, la estrategia ya no funciona después de la crisis corona. Lo mejor es que el cVaR es igual al 16%, por lo que no deberíamos perder más del 16% en un año.

8.3. Modelos ARMA

En esta sección se explica el modelo ARMA^[7], que combina los modelos MA y AR. A continuación, daremos respuesta a

uno de los mayores problemas que plantea el modelo ARMA: ¿cómo utilizar un modelo ARMA en datos no estacionarios?

8.3.1. Modelo ARMA

En esta parte explicaremos el modelo de medias móviles autorregresivas (ARMA). Este modelo es una combinación del modelo AR y el modelo MA mostrados anteriormente.

El modelo es la combinación de la ecuación AR y la ecuación MA. Vamos a mostrar la ecuación del modelo ARMA. Así, un modelo ARMA tiene dos parámetros, q y p. El parámetro q es el número de rezagos para la parte de media móvil y p es para la parte autorregresiva.

$$\text{ARMA}(1,1): y_t = \phi_0 + \theta_0 + \phi_1 y_{t-1} + \theta_1 \varepsilon_{t-1}$$

$$\begin{aligned} \text{ARMA}(p,q): y_t \\ = \phi_0 + \theta_0 + \phi_1 y_{t-1} + \theta_1 \varepsilon_{t-1} + \dots + \phi_p y_{t-p} + \theta_q \varepsilon_{t-q} \end{aligned}$$

Donde de θ_0 a θ_q son los parámetros de la parte MA y $\varepsilon_i = y_{\text{real}_i} - y_{\text{predict}_i}$, de ϕ_0 a ϕ_p son los parámetros de la parte AR y cada uno de ellos debe ser un valor entre 0 y 1.



Recuerda que los datos deben ser estacionarios al utilizar un modelo ARMA.

Código 8.4: Implementación del modelo ARMA

```
def ARMA_predict_value(train_set):  
    """  
    Función para predecir el valor de mañana utilizando el modelo AR  
    """  
  
    # Definir el modelo  
    p = 1  
    q = 1
```

```
model = ARIMA(train_set, order=(p, 0, q)) 1
```

```
# Ajustar el modelo
```

```
model_fit = model.fit(dispatch = 0)
```

```
# Hacer previsiones
```

```
forecast = model_fit.forecast()
```

```
return forecast[0][0]
```

```
def ARMA(df, returns = True):
```

```
    """
```

```
    Función para predecir el conjunto de test con un modelo ARMA
```

```
    """
```

```
# Encontrar la división
```

```
split = int(len(df) * 0.70)
```

```
# Predecir el precio utilizando la función AR
```

```
df['predicted_value'] = df['Adj Close'].rolling(split).\\\n    apply(ARMA_predict_value)
```

```
# Desplazar el precio previsto 1 periodo
```

```
df['predicted_value'] = df['predicted_value'].shift(1)
```

```
# Calcular la rentabilidad del activo
```

```
if returns:
```

```
    df['returns'] = df['Adj Close']
```

```
    df['signal'] = np.where(df.predicted_value > 0, 1, -1)
```

```
else:
```

```
    df['returns'] = df['Adj Close'].pct_change(1)
```

```
    df['signal'] = np.where(df.predicted_value >\\\n        df['Adj Close'], 1, -1)
```

```
df['strategy'] = df.signal.shift(1) * df.returns
```

return df

- 1 La única diferencia entre los modelos anteriores son los parámetros de la función ARIMA de StatModels. Ahora, combina los parámetros AR y MA.



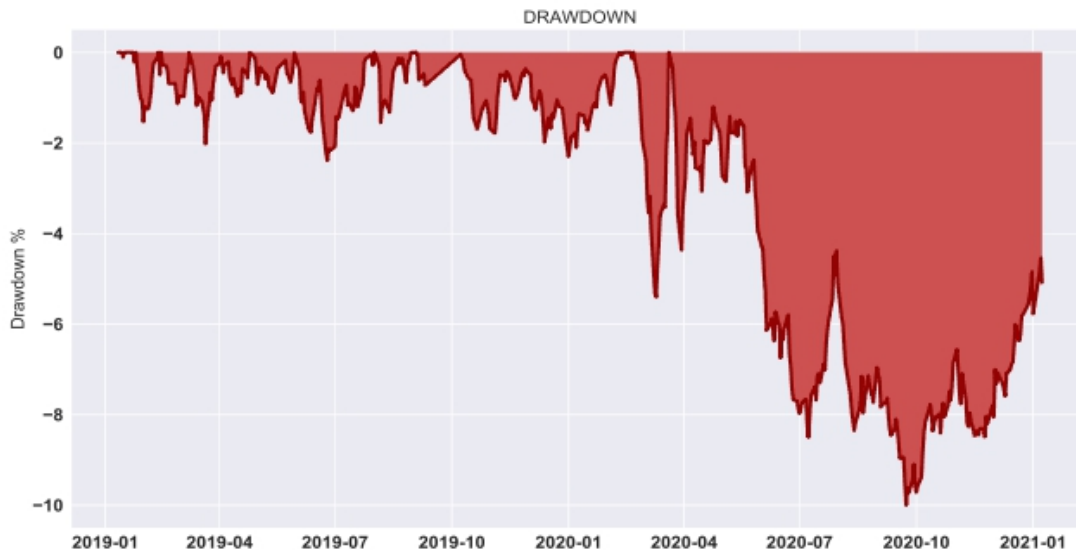
Ahora entiendes dos de los tres parámetros de la función ARIMA: la p del modelo AR y la q del modelo MA.

Figura 8.10: Backtest del modelo ARMA para EURUSD

Beta: -0.023 Alpha: 1.75 % Sharpe: 0.227 Sortino: 0.341

VaR: 13.45 %. cVaR: 15.55 % VaR/cVaR: 1.156 drawdown: 10.0 %





ARMA ha sido una combinación de dos modelos perdedores en estos datos. Tenemos un mal rendimiento utilizando el modelo ARMA en este ejemplo.

8.3.2. Modelo ARIMA

Ya hemos visto suficientes modelos ARMA en este capítulo. Sin embargo, necesitamos entender el parámetro d del modelo ARIMA. Si recuerdas bien la sección anterior, hemos visto que los datos tienen que ser estacionarios para utilizar el modelo ARMA.

Sin embargo, en finanzas, la mayoría de las veces, las series no son estacionarias. Por lo tanto, necesitamos transformar los datos en una serie temporal estacionaria. Para ello, utilizamos la diferenciación. La diferenciación es del precio $t-1$ al precio. La función ARIMA nos permite diferenciar con el parámetro d . Normalmente, con las series temporales financieras, $d=1$ si ajustamos el modelo ARIMA utilizando el precio o $d=0$ si utilizamos los rendimientos del activo.



Es esencial comprender que el modelo ARIMA es un modelo ARMA que utiliza datos no estacionarios y es la única diferencia.

Código 8.5: Implementación del modelo ARIMA

```
def ARIMA_predict_value(train_set):  
    """  
    Función para predecir el valor de mañana utilizando el modelo AR  
    """  
  
    # Definir el modelo  
    p = 1  
    q = 1  
    d = 1  
    model = ARIMA(train_set, order = (p, d, q))  
  
    # Ajustar el modelo  
    model_fit = model.fit(dispatch = 0)  
  
    # Hacer 10 demostros  
    forecast = model_fit.forecast()  
  
    return forecast[0][0]  
  
def ARIMA_model(df, returns = True):  
    """  
    Función para predecir el conjunto de test con un modelo ARIMA  
    """  
  
    # Encontrar la 10 demostros  
    split = int(len(df) * 0.70)  
  
    # Predecir el precio utilizando la función AR  
    df['predicted_value'] = df['Adj Close'].rolling(split).\  
        apply(ARIMA_predict_value)
```

```

# Desplazar el precio previsto 1 periodo
df['predicted_value'] = df['predicted_value'].shift(1)

# Calcular la rentabilidad del 1odemo
if returns:
    df['returns'] = df['Adj Close']
    df['signal'] = np.where(df.predicted_value > 0, 1, -1)
else:
    df['returns'] = df['Adj Close'].pct_change(1)
    df['signal'] = np.where(df.predicted_value >
        df['Adj Close'], 1, -1)

df['strategy'] = df.signal.shift(1) * df.returns

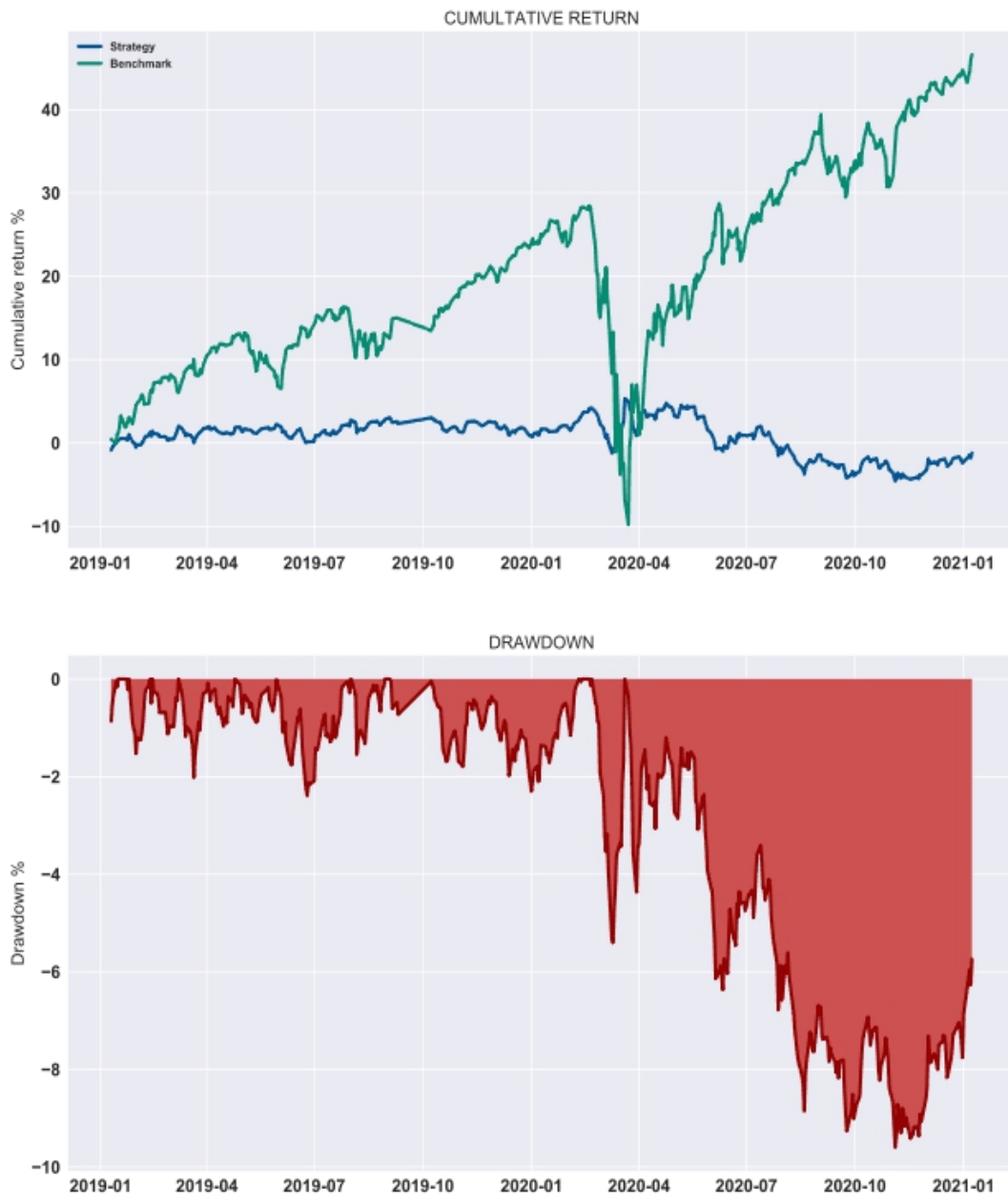
return df

```

- 1 Advertencia: no 1odemos llamar a esta función ARIMA porque habrá interferencias entre ésta y la función ARIMA de StatModels.

Figura 8.11: Backtest del modelo ARIMA para EURUSD

Beta: -0.016 Alpha: -0.25 % Sharpe: 0.103 Sortino: 0.16
 VaR: 14.16 %. cVaR: 16.43 % VaR/cVaR: 1.16 drawdown: 9.6 %



Misma interpretación que para el modelo ARMA

Resumen

- Las propiedades de los precios logarítmicos son fascinantes para comprender la variación natural de los activos.

- La regresión lineal encuentra la recta que minimiza la distancia entre todas las observaciones.
- El modelo autorregresivo es una regresión lineal de los datos anteriores. Podemos encontrar los retardos óptimos utilizando el gráfico de autocorrelación parcial.
- El modelo de media móvil utiliza el término de error pasado para ajustar el modelo. Podemos encontrar el desfase óptimo utilizando un gráfico de autocorrelación.
- El modelo ARMA es una combinación de los modelos AR y MA. Podemos encontrar los mejores parámetros utilizando los mismos criterios que para los modelos AR y MA.
- El modelo ARMA necesita datos estacionarios. Podemos poner el precio de las acciones en rentabilidad para utilizarlo o utilizar el modelo ARIMA, que nos permite añadir diferenciación en el modelo para tener datos estacionarios.

Aplicación Trading en vivo y Screener

Este código se basa en la clase creada en el anexo: MetaTrader.

Código 8.6: Aplicación Trading / Screener para el modelo ARIMA

```
from MT5 import *  
import numpy as np  
import pandas as pd  
import warnings  
warnings.filterwarnings("ignore")  
from statsmodels.tsa.arima_model import ARIMA  
import time
```

```

def sig_ARIMA_model(symbol):
    """
    Función para predecir el valor de mañana utilizando el modelo ARIMA
    """

    train_set = MT5.get_data(symbol, 3500)["close"]

    # Definir el modelo
    p = 1
    q = 1
    d = 1
    model = ARIMA(train_set, order = (p, d, q))

    # Ajustar el modelo
    model_fit = model.fit(dispatch = 0)

    # Hacer previsiones
    forecast = model_fit.forecast()

    value_forecasted = forecast[0][0]
    buy = train_set.iloc[-1] < value_forecasted
    sell = not buy

    return buy, sell

# True = Trading en vivo y False = Screener
live = True

if live:
    current_account_info = mt5.account_info()
    print("-----")
    print("Date: ", datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
    print(f"Balance: {current_account_info.balance} USD, \t"
          f"Equity: {current_account_info.equity} USD, \t")

```

```

        f"Profit: {current_account_info.profit} USD")

print("-----")

info_order = {
    "Euro vs USdollar": ["EURUSD", 0.01]
}

start = datetime.now().strftime("%H:%M:%S") # "23:59:59"

while True:

    # Verificación para el lanzamiento
    if datetime.now().weekday() not in (5, 6):
        is_time = datetime.now().strftime("%H:%M:%S") == start
    else:
        is_time = False

    # Lanzar el algoritmo
    if is_time:

        # Abrir las transacciones
        for asset in info_order.keys():

            # Inicializar los inputs
            symbol = info_order[asset][0]
            lot = info_order[asset][1]

            # Crear las señales
            buy, sell = sig_ARIMA_model(symbol)

            # Ejecutar el algoritmo
            if live:
                MT5.run(symbol, buy, sell, lot)

```

```
else:  
    print(f"Symbol: {symbol}\\t"  
          f"Buy: {buy}\\t"  
          f"Sell: {sell}")  
time.sleep(1)
```



El parámetro live establece el modo de trading en vivo (live = True) o el modo screener (live = False).

Capítulo 9: Regresión lineal y regresión logística

En este capítulo se analiza el modelo de regresión. En primer lugar, profundizaremos en la regresión lineal simple. Después, veremos otra clase de modelos predictivos, los clasificadores.

9.1. Regresión y clasificación

En primer lugar, en esta sección, aprenderemos la diferencia entre modelos de regresión y de clasificación.

9.1.1. Recordatorio sobre la regresión

En el capítulo anterior, hemos hecho un pequeño resumen de la regresión lineal. Aquí, vamos a recordar y explicar cómo funciona la regresión para cualquier algoritmo de regresión.

- Todos los modelos de regresión quieren predecir un valor continuo. Por ejemplo, si un modelo predice el porcentaje de variación del stock, la predicción estará contenida en el intervalo $[-30\%, +30\%]$, y puede tomar todos los valores de este intervalo (por ejemplo, 1.57%, 6.78%, 13.39%).
- Intuitivamente, la idea de la regresión es encontrar un modelo que se acerque más a cada valor de un conjunto de datos. Podemos calcular la suma de la distancia entre el valor predicho $\tilde{y}_i$ y el valor real y_i utilizando el error cuadrático medio (MSE). Matemáticamente, $MSE = \sum (\tilde{y}_i - y_i)^2$
- Todos los algoritmos de regresión tienen su método para ser optimizados. Sin embargo, todos pueden probarse con el MSE para saber si es un buen modelo o no. En comparación, la regresión lineal es la única que optimiza el MSE para encontrar los mejores parámetros.

9.1.2. Entender la clasificación

En esta parte, hablaremos de los modelos de clasificación. Para entenderlo rápidamente, seguiremos la misma pauta que el recordatorio anterior sobre la regresión lineal para destacar las diferencias.

- Cuando trabajamos con un algoritmo de clasificación, el objetivo no es un valor continuo. En realidad, es un valor definido. Por ejemplo, podemos predecir si el precio de las acciones aumentará o disminuirá mañana, pero no predecir un valor. En clasificación, el vector objetivo es 0 para el día que baja y 1 para el día que sube.
- Naturalmente, la función de entropía cruzada es la más habitual para encontrar los mejores parámetros de este algoritmo. Es un nombre complicado para algo sencillo. Esta función calcula el porcentaje de la predicción excelente.
- Las métricas más utilizadas para conocer el rendimiento de un modelo de clasificación son la precisión y la matriz de confusión.

Ahora bien, supongamos que hemos comprendido perfectamente todos estos puntos. En ese caso, podemos preguntarnos: "¿Por qué se ha utilizado la entropía cruzada para optimizar el algoritmo de clasificación y no sólo la precisión?". Utilizamos la función de entropía cruzada para optimizar porque, en la optimización, necesitamos una función continua como $f(x) = x$. Aún así, la precisión no es continua, por lo que necesitamos encontrar otras soluciones y encontramos esta solución utilizando la entropía cruzada. La función no tiene importancia pero se pone para el que la quiera conocer.

$$CE = - \sum_{c=1}^M \sum_{o=1}^N y_{o,c} \log(p_{o,c})$$

Donde M es el número de clases, N es el número de observaciones, $y_{o,c}$ es la observación verdadera y $\log(p_{o,c})$ es la probabilidad prevista de observación. Más adelante profundizaremos en la noción de probabilidad predicha.

Hablemos ahora de las métricas de rendimiento de las pruebas de clasificación. La métrica más conocida es la precisión. Se calcula dividiendo el número de predicciones verdaderas entre el número de predicciones.

La matriz de confusión se considera una de las mejores métricas de rendimiento de las pruebas. Es una matriz con verdaderos positivos, falsos positivos, falsos negativos y verdaderos negativos. Es interesante entender en qué se equivoca el algoritmo. Veamos en la figura 9.1 una matriz de confusión teórica.

Figura 9.1: Matriz de confusion teórica

Real\Predicción	True	False
True	Verdadero Positivo	Falso negativo
False	Falso positivo	Verdadero negativo

Además, la utilización de la matriz de confusión depende del contexto. Supongamos que trabajamos en un algoritmo de predicción de terremotos. En ese caso, el peor valor es el verdadero negativo porque significa que no predecimos el terremoto, pero ocurrió. Los falsos positivos son erróneos, pero no verdaderos porque esperábamos el terremoto y no ocurrió. Supongamos que el modelo predice la tendencia alcista de una empresa al año siguiente. En ese caso, los verdaderos negativos son insignificantes porque no invertimos en esa empresa, por lo que no perdemos dinero, a diferencia de los falsos positivos, que significan que hemos invertido en la empresa equivocada.

9.2. Regresión lineal

En esta sección aprenderemos a realizar una regresión lineal sobre el precio de las acciones. Aprenderemos a preparar los datos, a realizar una regresión lineal y a realizar una prueba retrospectiva.

9.2.1. Preparación de los datos

En primer lugar, tenemos que importar los datos. Hemos elegido la divisa EURUSD para poder comparar con el rendimiento del modelo ARIMA.



Trabajo sólo con el porcentaje de variación porque no necesitamos normalizar los datos en muchos casos y las variaciones son más representativas que el precio de las acciones (explicaremos este punto en la parte 9.3.1).

En los datos, añadimos una nueva columna con el precio de ayer. Será la variable que utilizaremos para predecir el precio de hoy. Veámoslo en el código 9.1.

Código 9.1: Importar los datos

```
# Importar los datos
df = yf.download("EURUSD=X", end = "2021-03-01")[["Adj Close"]].\
    pct_change(1)

# Crear una X
df["Adj Close t-1"] = df["Adj Close"].shift(1) 1

# Descartar valores faltantes
df = df.dropna()
```

- 1 Añadir un desplazamiento para poner la X (devolución de ayer) y la y (devolución de hoy) del mismo día en la misma fila.

En estadística y machine learning, los datos se dividen en X e y. X son todas las variables que utilizamos para predecir la variable y. En nuestro caso, X es la rentabilidad de ayer, e y es la rentabilidad de hoy. Además, tenemos que dividir un conjunto de entrenamiento y un conjunto de test. Esta descomposición nos permite entrenar el modelo en el conjunto de entrenamiento y probar las prestaciones en datos desconocidos, el conjunto de test. Normalmente, utilizamos el 80% de los datos para el conjunto de entrenamiento y el 20% para el conjunto de test.

Código 9.2: Creación de los conjuntos de entrenamiento y testing

```
# Porcentaje del conjunto de entrenamiento
split = int(0.80 * len(df))

# Creación del conjunto de entrenamiento
X_train = df[["Adj Close t-1"]].iloc[:split]
y_train = df[["Adj Close"]].iloc[:split]

# Creación del conjunto de test
X_test = df[["Adj Close t-1"]].iloc[split:]
y_test = df[["Adj Close"]].iloc[split:]
```



Puedes usar la función `train_test_split` de `scikit-learn` para hacer lo mismo. Asegúrate de que entiendes la función porque hay muchas sutilezas en esta función, como la posibilidad de hacer un barajado que no es bueno en finanzas (no queremos predecir el pasado con el futuro).

9.2.2. Implementación del modelo

En esta parte, implementamos el modelo de regresión lineal. Antes de empezar el código, es interesante tener aquí la

ecuación de regresión lineal porque es una ecuación directa (no hacemos eso con los otros algoritmos).

$$\tilde{y}_i = \beta_0 + \beta_1 x_{i,1} + \dots + \beta_n x_{i,n}$$

Donde los β_i son parámetros del modelo, $x_{i,j}$ las observaciones del dataset e $\tilde{y}_i$ la predicción. En nuestro caso, la ecuación es:

$$r_{\text{EURUSD}, t} = \beta_0 + \beta_1 r_{\text{EURUSD}, t-1}$$

Donde $r_{\text{EURUSD}, t}$ es el retorno de EURUSD en el tiempo t .

Código 9.3: Implementación de la regresión lineal

```
# Importar la clase
from sklearn.linear_model import LinearRegression 1

# Inicializar la clase
lr = LinearRegression() 2

# Ajustar el modelo
lr.fit(X_train, y_train) 3
```

- 1 Importar la clase *LinearRegression* de *scikit-learn*.
- 2 Instanciar la clase (Interesante cuando tenemos dos modelos sobre datos diferentes).
- 3 Usar el comando *fit()* para entrenar los coeficientes de la regresión lineal.



Cuando el modelo está entrenado, podemos utilizarlo para tener los parámetros (sólo para la regresión lineal y algunos otros algoritmos) y hacer predicciones.

Usando los comandos *lr.coef_* and *lr.intercept_* Podemos encontrar los parámetros de la ecuación de la regression lineal:

$$r_{\text{EURUSD}, t} = 3.53e^{-05} + -0.21_1 r_{\text{EURUSD}, t-1}$$

9.2.3. Predicciones y backtest

En esta parte utilizaremos el modelo implementado en la última parte para hacer predicciones y backtestar la estrategia.

Utilizaremos el comando *predict* de la clase *lr* para hacer predicciones. A continuación, tenemos que encontrar la manera de codificar nuestra estrategia. Es fácil. Compramos si la rentabilidad prevista es positiva y vendemos si es negativa. Hacemos el backtest usando la función del capítulo 5.

Código 9.4: Predicciones y backtesting

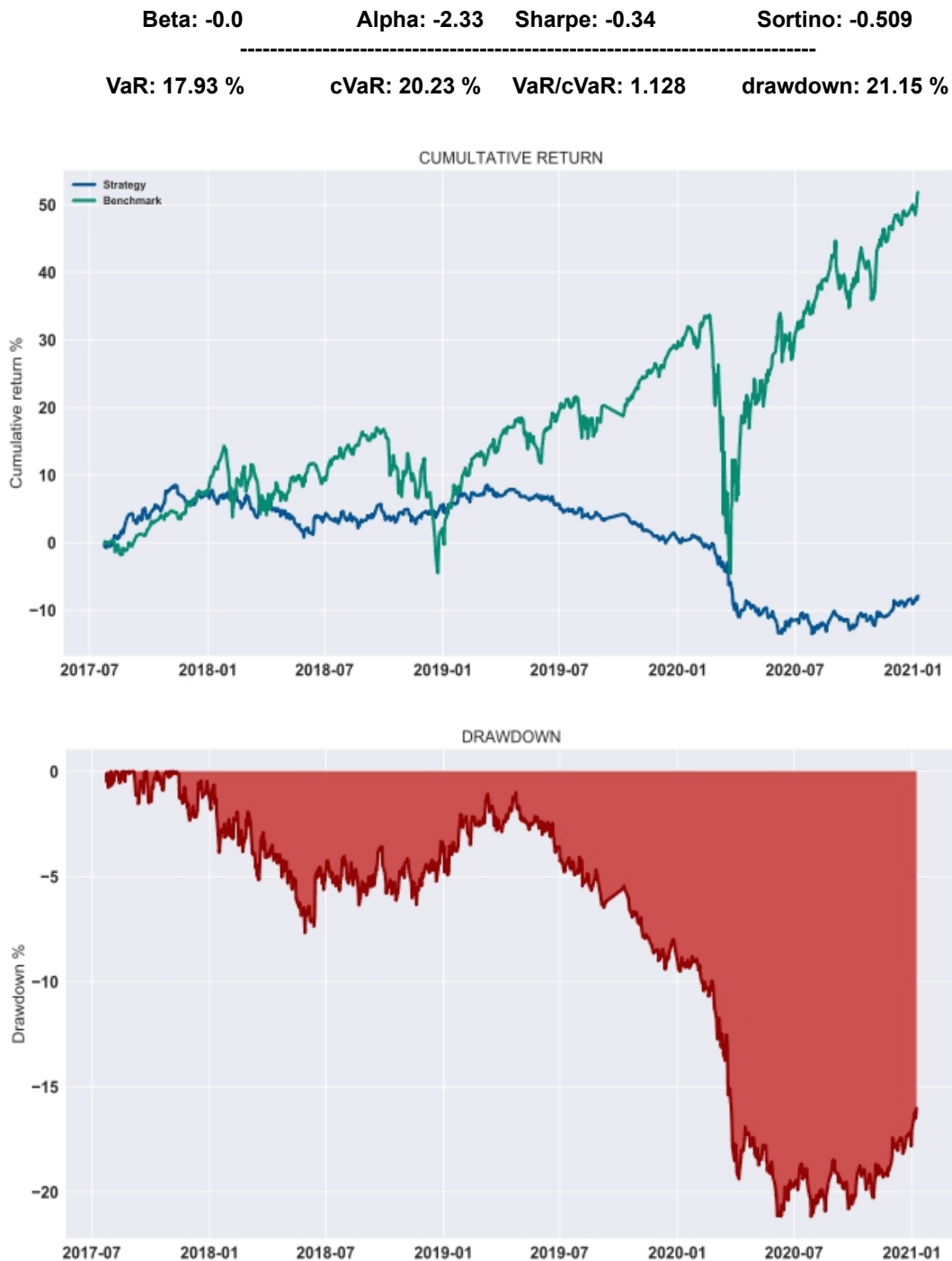
```
# Crear predicciones para todo el conjunto de datos
df["prediction"] = lr.predict(df[["Adj Close t-1"]]) 1

# Calcular la estrategia
df["strategy"] = np.sign(df["prediction"]) * df["Adj Close"] 2

# Backtest
backtest_dynamic_portfolio(df["strategy"].iloc[split:]) 3
```

- 1 Usar el comando *predict()* de la clase *lr* para predecir variaciones futuras.
- 2 No necesitamos poner un *shift(1)* porque se pone en la X.
- 3 Backtest sólo para los valores posteriores a la división (conjunto de test).

Figura 9.2: Estrategia de backtesting creada mediante regresión lineal en EURUSD



Este gráfico muestra que el rendimiento es malo. De hecho, hay un ratio de Sharpe negativo, lo que significa que

perdemos dinero si elegimos esta estrategia. Además, perdemos dinero desde el principio: es fácil de ver utilizando el gráfico de reducción de pérdidas.

9.3. Regresión logístitca

En esta sección se utilizará la regresión logística para crear una estrategia de trading. Sin embargo, necesitamos dar una pequeña explicación de la regresión logística antes de empezar.

La regresión logística es matemáticamente como la regresión lineal, pero con un paso añadido. Para crear una regresión logística, necesitamos utilizar una sigmoidea. La función sigmoide sólo devuelve valores entre 0 y 1 (hay probabilidades). Luego, necesitamos un umbral. Normalmente es el 50%. Si la probabilidad es superior a 0.5, la predicción es uno, y es 0 si es inferior.

$$\tilde{y}_{i, \text{ continuous}} = \beta_0 + \beta_1 x_{i,1} + \dots + \beta_n x_{i,n}$$

$$\tilde{y}_{i, \text{ categorical}} = \frac{1}{1 + e^{-\tilde{y}_{i, \text{ continuous}}}}$$

Donde los β_i son los parámetros del modelo, $x_{i,j}$ las observaciones del dataset, $\tilde{y}_{i, \text{ continuous}}$ la predicción continua e $\tilde{y}_{i, \text{ categorical}}$ la predicción categórica. Veámoslo en la figura 9.3.

Figura 9.3: Explicación de la regression logística



9.3.1. Preparación de los datos

En esta parte, prepararemos los datos para un algoritmo de clasificación. Es casi lo mismo que un algoritmo de regresión. La diferencia es el objetivo, que debe ser una variable categórica.

Hay muchas maneras de procesar para crear una variable categórica (o ficticia). La más útil es utilizar la función `where` de `numpy`, que nos permite reemplazar algún valor utilizando condiciones.

Veremos en la siguiente sección cómo utilizar `np.where`. Es un poco complicado, ya que trabajamos con rentabilidades de acciones. La acción ha subido si la rentabilidad ha sido superior a 0 y ha bajado si la rentabilidad ha sido inferior a 0. Así que supongamos que sumamos 0.5 a cada valor y redondeamos al entero más cercano. En ese caso, tenemos 1 para el valor creciente y 0 para el valor decreciente.

Código 9.5: Preparación de los datos para un clasificador

```
# Porcentaje del conjunto de entrenamiento
split = int(0.75 * len(df)) 1

# Creación de los conjuntos de entrenamiento
X_train = df[["Adj Close t-1"]].iloc[:split]
y_train = np.round(df[["Adj Close"]].iloc[:split] + 0.5)
```

```
# Creación de los conjuntos de testing
X_test = df[["Adj Close t-1"]].iloc[split:]
y_test = np.round(df[["Adj Close"]].iloc[split:] + 0.5)
```

- 1 Podemos elegir el 75% de los datos para el conjunto de entrenamiento. La regla que hay que respetar es que el conjunto de entrenamiento debe contener al menos el 70% y como máximo el 80% de los datos.

9.3.2. Implementación del modelo

La clase de regresión logística es de la librería scikit-learn también. La sintaxis es la misma que para la regresión lineal. Por lo tanto, necesitamos inicializar la clase y ajustarla usando los datos `X_train` y `y_train`.

Código 9.6: Implementación de la regression logística

```
# Importar la clase
from sklearn.linear_model import LogisticRegression

# Inicializar la clase
lr = LogisticRegression()

# Entrenar el modelo
lr.fit(X_train, y_train)
```

Salida de los parámetros del modelo:

```
LogisticRegression(C = 1, class_weight = None, dual = False, fit_intercept = True,
                    intercept_scaling = 1, l1_ratio = None, max_iter = 100,
                    multi_class = 'auto', n_jobs = None, penalty = 'l2',
                    random_state = None, solver = 'lbfgs', tol = 0.0001,
                    verbose = 0, warm_start = False)
```

No hay muchos parámetros para la regresión lineal. Por lo tanto, es mejor analizar algunos parámetros de este modelo. Es esencial entender que cada modelo puede ser personalizado cambiando los

parámetros de los modelos. Supongamos que creamos una regresión logística sobre los mismos datos. En ese caso, los resultados pueden ser diferentes si cambiamos los parámetros. Se aconseja consultar la documentación de la librería para entender todos los parámetros. Sin embargo, vamos a explicar algunos parámetros esenciales:

- **fit_intercept:** puede ser verdadero o falso. En efecto, tenemos un β_0 en la ecuación de regresión lineal. Si es falso, no tenemos uno.
- **penalty:** si es 'l2', la función de error (la función que queremos minimizar) es un MSE. Si es 'l1', la función de error es el MAE (error medio absoluto).
- **verbose:** puede ser 0 o 1. Si es 0, no podemos ver detalles sobre la optimización del algoritmo. Si es 1, tenemos más información sobre el entrenamiento.



Es interesante cambiar algunos parámetros para mejorar tu modelo, pero si lo adaptas demasiado lejos de los datos, el modelo puede resultar inutilizable con otros datos. Este problema se denomina sobreajuste, muy activo en finanzas debido a la cambiante situación del mercado.

9.3.3. Predicciones y backtest

En esta parte analizaremos la estrategia creada mediante el algoritmo de regresión logística. Será igual que la regresión lineal, pero después compararemos ambos algoritmos.

Código 9.6: Predicciones de la regresión lineal

```
# Crear las predicciones
df["prediction"] = lr.predict(df[["Adj Close t-1"]])

# Crear las posiciones
```

```
df["prediction"] = np.where(df["prediction"] == 0, -1, 1)

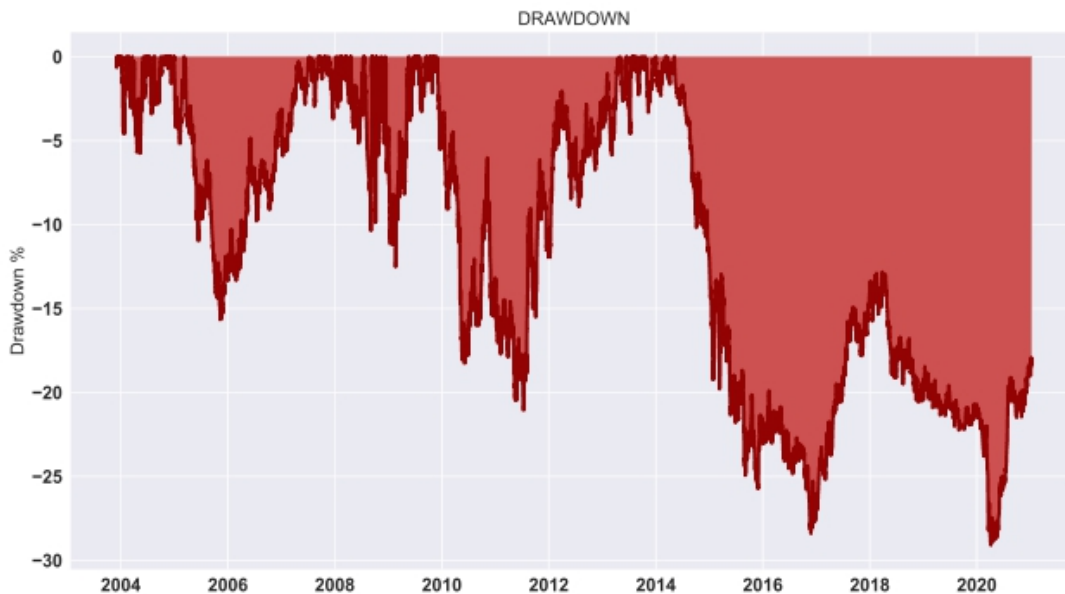
# Calcular la rentabilidad de la estrategia
df["strategy"] = np.sign(df["prediction"]) * df["Adj Close"]
```



Hemos utilizado la función de numpy para dar un ejemplo de potencia de esta función.

Figura 9.4: Rentabilidad acumulada de la estrategia mediante regresión logística





Es un buen backtest. Tenemos buenas métricas, pero un drawdown elevado (normalmente ante una combinación de estrategias). El punto negativo es que el crecimiento de los rendimientos acumulados se ha detenido desde 2010. Los problemas pueden resolverse ajustando de nuevo el modelo porque hay seis años sin otro ajuste (no es muy realista).

Resumen

- La regresión lineal es el algoritmo de regresión más fácil de entender. Se entrena utilizando el MSE.
- La regresión logística es el algoritmo de clasificación derivado de la regresión lineal. Para encontrar la clase, utiliza la función sigmoidea.
- Si queremos ajustar demasiado el algoritmo a los datos de entrenamiento, será inutilizable para los datos de prueba. Este problema se denomina sobreajuste.

Aplicación Trading en vivo y Screener

Este código se basa en la clase creada en el anexo: MetaTrader.

Código 9.7: Aplicación Trading / Screener para regresión logística

```
from MT5 import *
import numpy as np
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
from sklearn.linear_model import LogisticRegression
import time
import pickle
from joblib import dump, load
import os

path = "" # Ex: C:/Desktop/Python_for_finance_and_algorithmic_trading/ChapterN/Models

def create_model_weights(symbol):
    """
    Ponderaciones para la regresión lineal sobre el cambio porcentual
    """
    # Importar los datos
    data = MT5.get_data(symbol, 3500)[["close"]].pct_change(1)

    # Crear una nueva variable
    data["close t-1"] = data[["close"]].shift(1)

    # Dividir los datos
    data = data.dropna()
    split = int(0.80*len(data))

    # Creación del conjunto de entrenamiento
```

```

X_train = data[["close t-1"]].iloc[:split]
y_train = np.round(data[["close"]].iloc[:split] + 0.5)

# Crear el modelo
alg = LogisticRegression()

# Ajustar el modelo
alg.fit(X_train, y_train)

# Guardar el modelo
alg_var = pickle.dumps(alg)
alg_pickel = pickle.loads(alg_var)

dump(alg_pickel, os.path.join(path, f"Models/{symbol}.joblib"))

def log_reg_sig(symbol):
    """
    Función para predecir el valor de mañana utilizando el modelo ARIMA
    """

    # Crear los pesos si no hay en la carpeta
    try:
        alg = load(os.path.join(path, f"Models/{symbol}.joblib"))
    except:
        create_model_weights(symbol)
        alg = load(os.path.join(path, f"Models/{symbol}.joblib"))

    # Tomar el último porcentaje de cambio
    data = MT5.get_data(symbol, 3500)[["close"]].pct_change(1)
    data["close t-1"] = data[["close"]].shift(1)

    X = data["close t-1"].iloc[-1].reshape(-1, 1)

    # Encontrar la señal

```

```

prediction = alg.predict(X)
prediction = np.where(prediction == 0, -1, 1)
buy = prediction[0][0] > 0
sell = not buy

return buy, sell

# True = Trading en vivo y False = Screener
live = True

if live:
    current_account_info = mt5.account_info()

    print("-----")
    print("Date: ", datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
    print(f"Balance: {current_account_info.balance} USD, \t"
          f"Equity: {current_account_info.equity} USD, \t"
          f"Profit: {current_account_info.profit} USD")
    print("-----")

info_order = {
    "Euro vs USdollar": ["EURUSD", 0.01]
}

start = datetime.now().strftime("%H:%M:%S") #"23:59:59"
while True:
    # Verfication for launch
    if datetime.now().weekday() not in (5, 6):
        is_time = datetime.now().strftime("%H:%M:%S") == start
    else:
        is_time = False

    # Lanzar el algoritmo
    if is_time:

```



```
# Abrir las transacciones
for asset in info_order.keys():

    # Inicializar los inputs
    symbol = info_order[asset][0]
    lot = info_order[asset][1]

    # Crear las señales
    buy, sell = log_reg_sig(symbol)

    # Ejecutar el algoritmo
    if live:
        MT5.run(symbol, buy, sell, lot)

    else:
        print(f"Symbol: {symbol}\n"
              f"Buy: {buy}\n"
              f"Sell: {sell}")

    time.sleep(1)
```



El parámetro live establece el modo trading en vivo (live = True) o el modo screener (live = False).

Parte 3: Machine learning, deep learning y trading en vivo

En esta parte se tratará la implementación de los algoritmos de aprendizaje automático (Machine Learning) más famosos para el trading: máquina de vectores soporte, árbol de decisión, bosque aleatorio, etc. A continuación, aprenderemos a crear algunos de los algoritmos más potentes: red neuronal profunda, red neuronal recurrente y red neuronal convolucional recurrente. Además, implementaremos un proyecto completo utilizando gestión de carteras, estadística y algoritmos de aprendizaje automático.

Resumen:

Capítulo 10: Ingeniería de características y objetivos

Capítulo 11: Support vector machine (SVM)

Capítulo 12: Métodos ensemble y árbol de decisión

Capítulo 13: Red neuronal profunda (DNN)

Capítulo 14: Red neuronal recurrente (RNN)

Capítulo 15: Bonus - Red neuronal convolucional recurrente (RCNN)

Capítulo 16: Proyecto completo de la vida real

Capítulo 17: De la nada al trading en vivo

Capítulo 10: Ingeniería de características y objetivos

Los datos clásicos como OHLCV son como un coche desmontado, es decir, no podemos ir a ninguna parte. Por eso necesitamos ensamblar el coche (ingeniería de características) para poder conducir. Para encontrar el destino, utilizaremos la ingeniería de objetivos.

10.1 Motivación e intuición

La ingeniería de características y objetivos es una de las partes esenciales. Aunque tengamos el algoritmo más potente, si le damos una entrada basura, tendremos un resultado basura.

10.1.1 Ingeniería de características

Pongamos un ejemplo sencillo para entender mejor qué es la ingeniería de características. Imaginemos que queremos predecir el tiempo que tardaremos en ir del punto A al punto B, y las únicas características que tenemos en este paso son la longitud y la latitud de los dos puntos. Así, en nuestra base de datos, tenemos 5 columnas: `longitude_start`, `latitude_start`, `longitude_end`, `latitude_end` y `travel_time`.

La pregunta es: ¿cómo podemos ayudar a nuestro algoritmo a predecir mejor el tiempo de viaje con los datos de longitud y latitud? Un ejemplo de ello será calcular la distancia en kilómetros entre el punto inicial y el final de cada viaje. Encontraremos una excelente relación entre la nueva característica (distancia) y el objetivo (tiempo de viaje).

La ingeniería de características también puede permitirnos encontrar las mejores características para nuestro problema observando la varianza de las características. Sin embargo, estos métodos no son relevantes en el trading porque

crearemos cuidadosamente nuevas variables para dar más información a nuestros algoritmos sobre tendencias, movimientos a corto plazo, volatilidad, etc.

La mayoría de las nuevas características en el trading son indicadores técnicos, cifras de la acción del precio o métricas cuantitativas. He aquí algunas métricas que darán algunas ideas para proyectos:

- **Características cuantitativas:** La variación desde el periodo N anterior hasta ahora, correlación móvil entre dos columnas. Nos permitirá comprender la tendencia a corto y largo plazo.
- **Patrones de acción del precio:** Detectar algunos patrones que nos den información sobre las fuerzas bajistas y alcistas del mercado, como el engulfing y el doji.
- **Indicadores técnicos:** Son muy versátiles y pueden ayudarnos a comprender la volatilidad, la variación del volumen y el impulso.

En conclusión, la ingeniería de características es el núcleo de nuestro problema de predicción comercial, porque los datos son esenciales en la ciencia de datos. Por lo tanto, ¡tenemos que elegir las características con cuidado!



La mejor manera de crear nuevas funciones es añadiendo tu propia experiencia. Elige indicadores que sepas que son importantes para el problema.

10.1.2 Ingeniería de objetivos

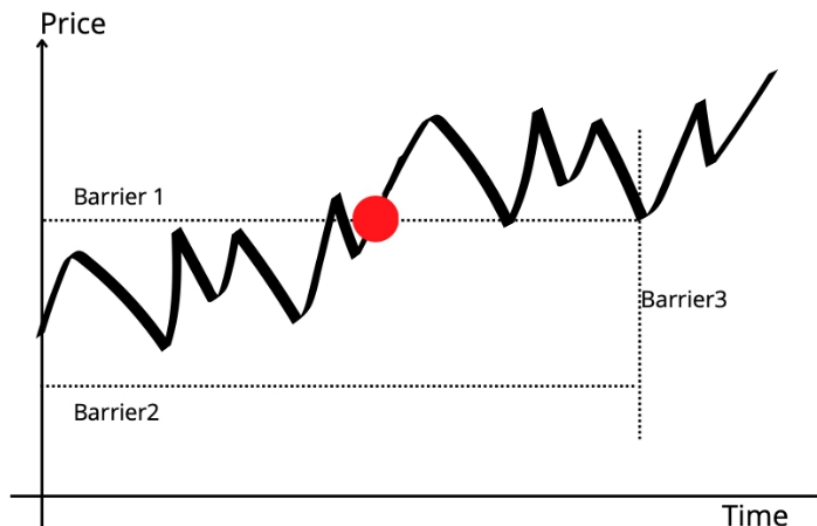
La ingeniería de características es el coche, pero no iremos a ninguna parte sin un objetivo claro. Por eso necesitamos un objetivo claro. Así que tenemos que hacer ingeniería de objetivos.

Entender que las mejores características no son nada si no conocemos el objetivo es esencial. Retomando nuestro ejemplo del coche, si queremos ir a un fantástico paraje de montaña, deberíamos coger un vehículo 4x4 en lugar de un coche de carreras porque el vehículo 4x4 está más adaptado al objetivo (terreno accidentado y sinuoso).

En el trading ocurre lo mismo; no podemos crear un conjunto de características para todos nuestros algoritmos porque el objetivo de cada uno de ellos no es el mismo. Así que, antes de crear las características, tenemos que elegir el objetivo. Veamos algunos ejemplos de ingeniería de objetivos para comprenderlos mejor:

- **Método de la triple barrera:** Destacado por Marco López de Prado es una técnica común para crear una variable ficticia que contenga -1 si tocamos la barrera 2, 0 si tocamos la barrera 3 y 1 si tocamos la barrera 1 (Figura 10.1). Si trabajamos con una posición vendedora, tenemos que invertir las barreras 1 y 2.

Figura 10.1: Método de triple barrera con la barrera 1 cruzada



- **Umbral de variación:** miramos las próximas N velas y calculamos la variación desde ahora hasta dentro de N velas para obtener un objetivo regresivo.

10.1.3 ¿Por qué es tan importante?

Aunque ya lo hayamos dicho muchas veces, las características y la ingeniería de objetivos son esenciales. Aquí crearemos el "problema" para definir qué variable queremos predecir (el objetivo) utilizando qué características.

Una vez que tenemos buenas características, encontrar un buen modelo predictivo es sencillo. Sin embargo, si no tenemos buenos datos, podemos crear el algoritmo más complejo que podamos, pero no será capaz de predecir nada.

En nuestro ejemplo del coche, el modelo es como los neumáticos del coche; aunque mantengamos los neumáticos de verano en invierno, podremos ir a donde queramos. Tardaremos más tiempo y aumentará el riesgo de accidente. Así pues, encontrar el modelo adaptado a nuestro problema es esencial. Sin embargo, será fácil encontrarlo con buenas características y un objetivo claro.

En efecto, supongamos que tenemos una importación de datos de mala calidad. En ese caso, aunque tengamos una ingeniería de características perfecta, no será fácil obtener buenos resultados y lo mismo ocurre en la etapa siguiente.

Además, como la ingeniería de características y de objetivos son una de las primeras etapas de un proyecto, pero no por ello menos importante, ya que dedicamos el 80% del tiempo de un proyecto a esta etapa, es esencial hacerla con cuidado.

10.2 Aplicación al trading

En esta sección aplicaremos la teoría de la que hemos hablado anteriormente. Veremos cómo crear algunas características interesantes y algunos objetivos relevantes.

10.2.1 Crear indicadores de trading y funciones de trading útiles

Dividiremos esta subsección en características cuantitativas, patrones de acción del precio e indicadores técnicos. La lista que se dará no es exhaustiva, así que siéntete libre de probar muchos otros para encontrar el mejor.

Características cuantitativas

N días previos: es una de las variables más accesibles pero también una de las más importantes porque nos permitirá comprender la variación anterior a largo o corto plazo según el periodo que tomemos.

Código 10.1: Cálculo de la variación de los N días previous

```
# Variación de los N días previos
n = 10
df[f"var_{n}"] = df["Adj Close"].pct_change(n)
```

Correlación móvil: Para este ejemplo, optaremos por calcular la correlación móvil entre las variaciones de los 10 días anteriores y las variaciones de los 200 días anteriores. Lo hacemos para comprender la relación entre la tendencia a largo plazo y el comportamiento a corto plazo. Es una característica interesante para un algoritmo de aprendizaje automático (según mi experiencia).

Código 10.2: Cálculo de la correlación móvil

```
# Correlación móvil
col_1 = "var_200"
col_2 = "var_10"
df["moving_correlation"] = df[col_1].rolling(50).corr(df[col_2])
```

Patrones de acción del precio

Doji: En general, decimos que los doji expresan indecisión. Por eso no clasificamos los doji como doji crecientes o doji decrecientes como en la siguiente figura. Es útil para el algoritmo entender cuando los inversores están indecisos porque nos puede mostrar el final o el inicio de una tendencia de trading.

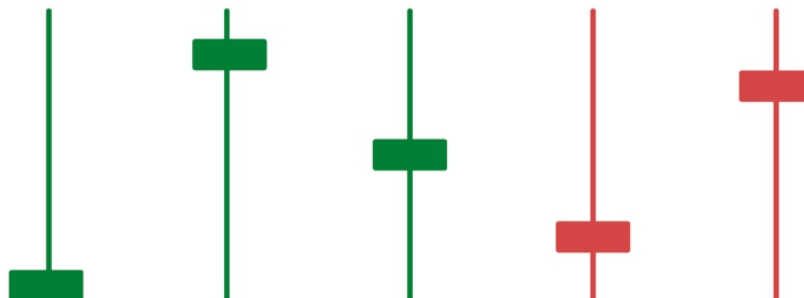
Código 10.3: Cálculo del Doji

```
# DOJI
df["DOJI"] = ta.CDLDOJI(df["Open"], df["High"],
                        df["Low"], df["Close"])
```



Para detectar el Doji, utilizamos la librería TA-lib de Python porque siempre es mejor utilizar la librería: minimizará el riesgo de error y optimizará tu código.

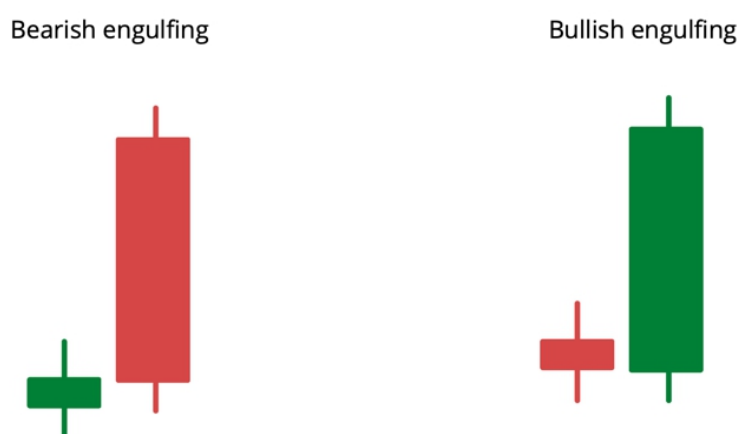
Figura 10.2 Algunos ejemplos de Doji



Podemos ver que el signo de la vela no significa nada para el Doji porque nos da movimiento de indecisión.

Patrón envolvente: Existen dos patrones envolventes diferentes: el envolvente alcista (bullish engulfing) y el envolvente bajista (bearish engulfing). Podemos ver gráficamente ambos en la Figura 10.3 y cómo calcularlos sin librería en el Código 10.4.

Figura 10.3: Patrones envolventes



Gráficamente hablamos de envolvente cuando el precio de cierre de la vela anterior es casi igual al de apertura de la vela actual. Además, la vela actual debe ser mayor que la anterior.

Código 10.4: Cálculo de la envolvente

```
# Envolvente
df["candle_way"] = -1
df.loc[(df["Open"] - df["Close"]) < 0, "candle_way"] = 1

# Amplitud
df["amplitude_abs"] = np.abs(df["Close"] - df["Open"])
df["Engulfing"] = 0

df.loc[
```

```

# Ayer vela roja y hoy aumento
(df["candle_way"].shift(1) == -1) &\
(df["candle_way"] == 1) &\

# Cierre de la última vela decreciente = Apertura de la vela creciente de hoy
(df["Close"].shift(1) < df["Open"] * (1 + 0.5 / 100)) &\
(df["Close"].shift(1) > df["Open"] * (1 - 0.5 / 100)) &\

# La última vela decreciente es menos fuerte que la vela creciente de hoy
(df["amplitude_abs"].shift(1) * 1.5 < df["amplitude_abs"]),
"Engulfing"] = 1

df.loc[
    # Ayer vela verde y hoy disminución
    (df["candle_way"].shift(1) == 1) &\
    (df["candle_way"] == -1) &\

    # Cierre de la última vela decreciente = Apertura de la vela decreciente de hoy
    (df["Close"].shift(1) < df["Open"] * (1 + 0.5 / 100)) &\
    (df["Close"].shift(1) > df["Open"] * (1 - 0.5 / 100)) &\

    # La última vela decreciente es menos fuerte que la vela de hoy
    (df["amplitude_abs"].shift(1) * 1.5 < df["amplitude_abs"]),
    "Engulfing"] = -1

```

Como vemos, esta sencilla figura tiene varias líneas de código. Por eso es mucho mejor utilizar una librería o crear nuestra función si queremos encontrar un patrón concreto.

Indicadores técnicos

Resistencia: La resistencia puede calcularse de muchas formas distintas. Este libro utilizará el valor máximo de los últimos 150 periodos. El soporte es lo mismo pero utilizando el mínimo.

Código 10.5: Cálculo de la resistencia

```
# Valor máximo en los últimos n días  
n = 150  
df["resistance"] = df["Close"].rolling(n).max()
```

Índice de fuerza relativa (RSI): Nos permitirá entender si el mercado está en sobrecompra o en sobreventa. Por lo tanto, es ideal tener estas características; por ejemplo, la combinación de un doji puede predecir mejor el retroceso.

Código 10.6: Cálculo del RSI

```
# RSI  
n = 15  
df["RSI"] = ta.RSI(df["Close"], timeperiod = n)
```



Para crear indicadores técnicos, puedes utilizar la librería TA. Te permitirá crear cerca de 100 indicadores para tus estrategias de trading.

10.2.2 Etiquetado del objetivo

Como hemos dicho antes, las características no son nada si no creamos una etiqueta interesante. En esta subsección, crearemos objetivos clasificadores y regresores. Todos los objetivos se derivarán del porcentaje de variación, pero obviamente, podemos crear el objetivo que queramos; por ejemplo, podemos predecir una figura del gráfico e intentar detectar si, después de esta figura, se produce un aumento o una disminución del precio.

En primer lugar, calcularemos la variación futura de N días. Necesitamos desplazar los datos para alinear las características y los objetivos en la misma fila, pero esto lo explicaremos con más detalle en la sección 11.1.

Código 10.7: Variación de los N días futuros

```
""" Variación de los siguientes N días """  
n = 1  
df[f"target_var_{n}"] = df["Close"].pct_change(n).shift(n) 1
```

- 1 Usamos el método *shift(n)* para no crear interferencias en los datos (como predecir el pasado utilizando el futuro)

Una vez que tengamos esta métrica, crear una variable ficticia para utilizar un algoritmo clasificador será fácil. Por ejemplo, podemos crear una variable ficticia con sólo 2 valores: -1 si la rentabilidad futura en N días es negativa y 1 si es positiva.

Código 10.8: Variación de los futuros N días (variable ficticia)

```
""" Variación de los siguientes N días (dummy)"""  
n = 1  
df[f"target_var_{n}"] = df["Close"].pct_change(n).shift(n)  
  
df["target_dummy"] = 1  
df.loc[df[f"target_var_{n}"] < 0, "target_dummy"] = -1
```

Sin embargo, con esta técnica no es fácil obtener beneficios cuando el mercado es amplio, porque la variación será positiva, pero la variación será mínima. Por lo tanto, para evitar este problema, podemos crear una variable ficticia con tres valores: -1, 0 y 1.

Para ello, calcularemos los percentiles 33 y 67. Sin embargo, es esencial calcular los percentiles utilizando el conjunto de entrenamiento para no crear interferencias en el futuro (conjunto de test). Entonces, supongamos que el valor está por debajo del percentil 33. En ese caso, lo clasificamos como -1, si está entre el percentil 33 y el percentil 67 como 0, y 1 si es superior al percentil 67.

Código 10.9: Variación de los futuros N días (variable ficticia continuación)

```

""" Clasificar las variaciones """
# Encontrar las variaciones
n = 1
df[f"target_var_{n}"] = df["Close"].pct_change(n).shift(n)

# Encuentra los percentiles 33 y 67 en el conjunto de entrenamiento
split = int(0.80 * len(df))
centile_33 = np.percentile(df[f"target_var_{n}"].
                           iloc[:split].dropna(), 33)
centile_67 = np.percentile(df[f"target_var_{n}"].
                           iloc[:split].dropna().values, 67)

# Variable dummy
df["target_dummy"] = 0
df.loc[df[f"target_var_{n}"] > centile_67, "target_dummy"] = 1
df.loc[df[f"target_var_{n}"] < centile_33, "target_dummy"] = -1

```

Resumen

- Los datos OHLCV son como un coche desmontado, por lo que es necesario transformarlos.
- La ingeniería de características ayuda al algoritmo a comprender mejor la relación entre los datos (es como montar el coche).
- Etiquetado del objetivo es la variable que queremos predecir (es donde queremos llegar).

Capítulo 11: Support vector machine (SVM)

En este capítulo se analiza uno de los algoritmos más utilizados en finanzas, la máquina de soporte vectorial (support vector machine). Es ventajoso en finanzas porque no necesita muchos datos para entrenarse. Antes de aprender SVM, haremos una puntualización sobre el preprocesamiento de datos. A continuación, aprenderemos a utilizar un regresor de máquina de soporte vectorial (SVR) y un clasificador de máquina de soporte vectorial (SVC).

11.1. Preparación de los datos

En esta sección, prepararemos los datos para nuestra SVM y recapitularemos lo que ya sabemos sobre la preparación de datos. Luego profundizaremos en el concepto explicando la normalización de datos.

11.1.1. Ingeniería de características

En esta sección, recapitularemos rápidamente la preparación de los datos. De hecho, haremos lo mismo que en los capítulos anteriores. Así, tenemos que definir conjuntos de entrenamiento y conjuntos de pruebas. Además, mejoraremos las características de nuestro modelo. Para ello, añadiremos algunos indicadores técnicos. En el libro no explicamos los indicadores técnicos que no utilizaremos. Si deseas encontrar alguna documentación sobre esto, podemos encontrarla en internet y en libros. Hacemos aquí un recordatorio de ingeniería de características para entender realmente cómo incorporarlo a nuestro proyecto.



La ingeniería de características ayuda al algoritmo a encontrar el mejor patrón en los datos. Por ejemplo, el precio de las acciones si tiene un activo con un precio entre 50\$ y 5000\$, los parámetros del modelo no pueden

ajustarse correctamente. En lugar de esto, utiliza la variación porcentual que convierte todos los activos en el mismo rango. Es muy útil para el algoritmo.

Para crear indicadores técnicos, hay dos maneras. En primer lugar, los podemos crear por nosotros mismos.

También podemos utilizar la librería de análisis técnico de Python (ta). En esta parte, crearemos los indicadores necesarios por nosotros mismos.

Para este ejemplo, hemos creado una media de los rendimientos y la volatilidad de los rendimientos. Estos indicadores no se pueden encontrar en la librería ta, por lo que tenemos que calcularlos nosotros mismos. Además, hemos puesto dos marcos temporales en cada indicador.

Código 11.1: Ingeniería de características

```
# Ingeniería de características
df["returns t-1"] = df[["returns"]].shift(1) 1

# Media de los rendimientos
df["mean returns 15"] = df[["returns"]].rolling(15).mean().shift(1)
df["mean returns 60"] = df[["returns"]].rolling(60).mean().shift(1)

# Volatilidad de los rendimientos
df["volatility returns 15"] = df[["returns"]].rolling(15).
    std().shift(1)
df["volatility returns 60"] = df[["returns"]].rolling(60).
    std().shift(1)
```

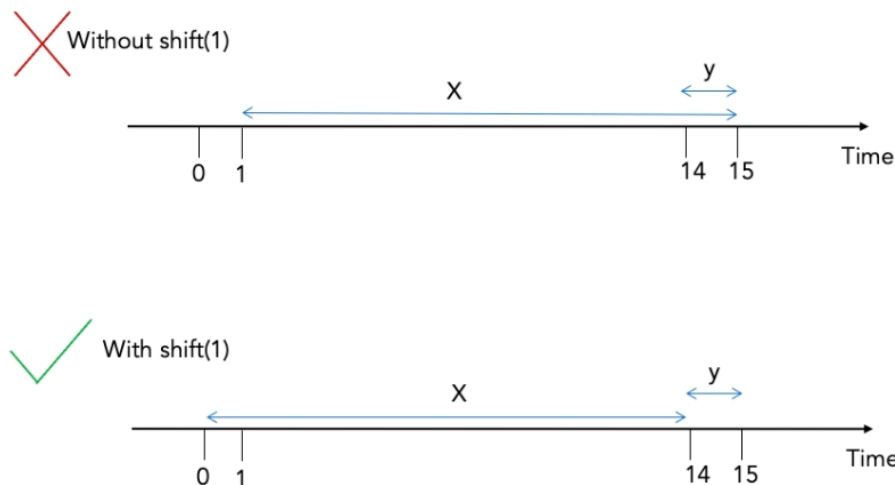
- 1 Es de esperar como este paso porque lo hemos visto muchas veces, pero el desplazamiento a las columnas de

características nos permite poner la X y la y del mismo día en la misma fila.



No olvides añadir un desplazamiento a estos indicadores para no interferir en los datos. Veamos por qué en la figura 11.1

Figura 11.1: Cómo evitar interferencias en los datos



Esta figura muestra que si no desplazamos los datos, tendremos interferencias porque predecimos el día 15 mientras que ya tenemos el día 15 en las características. No puede ser exacto, y podemos perder mucho dinero si cometemos este error.

11.1.2. Estandarización

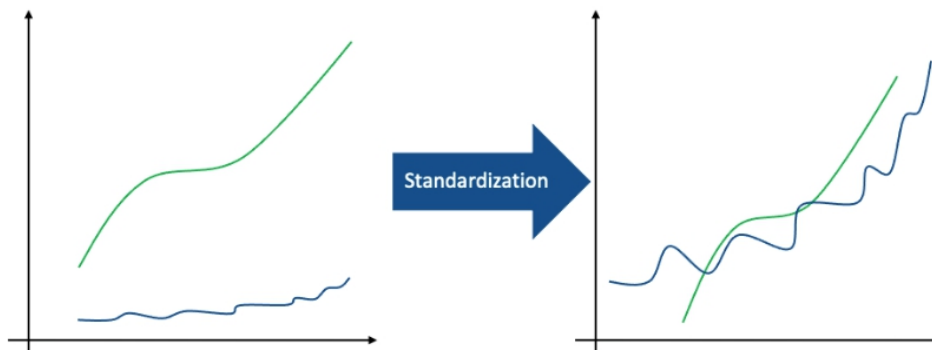
En esta subsección se explicará el concepto de normalización. Vemos esto en el capítulo SVM porque es un algoritmo geométrico y es necesario normalizar los datos para estos algoritmos. Aún así, podemos aplicar este método a otro algoritmo.



La estandarización permite hacer el cálculo más rápido. Así, es interesante para el algoritmo que exige muchos recursos.

A veces, como ocurre con este conjunto de datos, los datos no están a la misma escala. Por ejemplo, podemos tener una volatilidad del 60% con un rendimiento del 1.75%. Por lo tanto, el algoritmo tiene muchas dificultades para trabajar con eso. Necesitamos estandarizar los datos para ponerlos en la misma escala. Veamos gráficamente por qué necesitamos normalizar los datos en la figura 11.2.

Figura 11.2: ¿Cómo funciona la estandarización?



Esta figura muestra que nos resulta difícil comprender el patrón entre las dos líneas antes de la normalización. También lo es para el algoritmo. Por lo tanto, debemos normalizar los datos para comprenderlos mejor.

Ahora, veamos la fórmula para normalizar los datos y normalizar los datos utilizando Python.

$$z_i = \frac{x_i - \mu_x}{\sigma_x}$$

Donde z_i es el valor estandarizado, x_i es el valor de la observación, μ_x es la media del vector $\mathbf{x}$ y σ_x la volatilidad del vector $\mathbf{x}$.

Código 11.2: Estandarizar los datos

```
# Importar la clase
from sklearn.preprocessing import StandardScaler

# Inicializar la clase
sc = StandardScaler()

# Estandarizar los datos
X_train_scaled = sc.fit_transform(X_train)
X_test_scaled = sc.transform(X_test)
```



En cuanto a los demás algoritmos, sólo necesitamos ajustarnos al conjunto de entrenamiento porque en la vida real no podemos conocer la media y la desviación típica del conjunto de test.



Nunca se normaliza la y porque no interviene en los cálculos. Sólo se utiliza para calcular el error del modelo excepto en algunos modelos específicos.

11.2. Clasificación con Support Vector Machine (SVC)

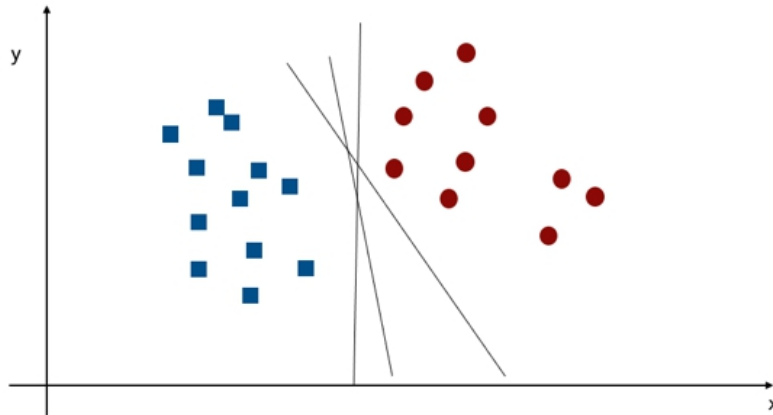
En esta sección se explicará cómo funciona un SVC, luego cómo crear un SVC utilizando Python y hacer predicciones con el SVC para crear una estrategia de trading.

11.2.1. Intuición sobre cómo funciona un SVC

En esta parte, veremos la intuición que hay detrás del clasificador de la máquina de soporte vectorial.

Para ello, vamos a empezar con un pequeño gráfico para entender lo que el SVC quiere hacer.

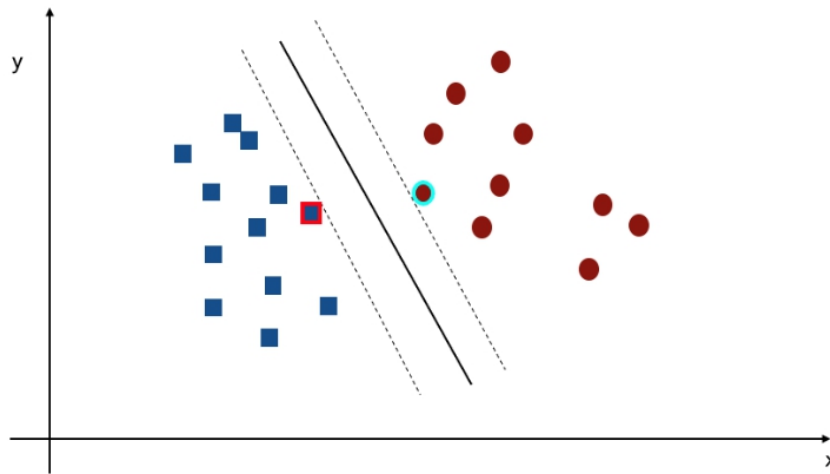
Figura 11.3: Funcionamiento de un SVC



En esta figura, podemos ver la pregunta a la que responde el SVC: "¿Cómo se pueden separar los grupos de puntos de forma óptima?".

Utilizamos márgenes para encontrar la línea óptima entre los dos grupos. Estos márgenes quieren maximizar la distancia entre los dos grupos. Entonces, tomaremos el punto medio de los dos márgenes para encontrar la delimitación de los dos grupos. Además, veremos que los puntos de los márgenes se llaman apoyos, y estos puntos son los únicos importantes en el algoritmo. Si eliminamos todos los demás puntos, el rendimiento del algoritmo será el mismo.

Figura 11.4: Optimizar la distancia



En esta figura podemos ver el funcionamiento de un SVC y los soportes.

El problema es que si un cuadrado está en el grupo del círculo, el algoritmo no puede ejecutarse. Esto se debe a que el algoritmo tiene márgenes duros. Afortunadamente, es posible crear un SVC con márgenes suaves para evitar este problema.

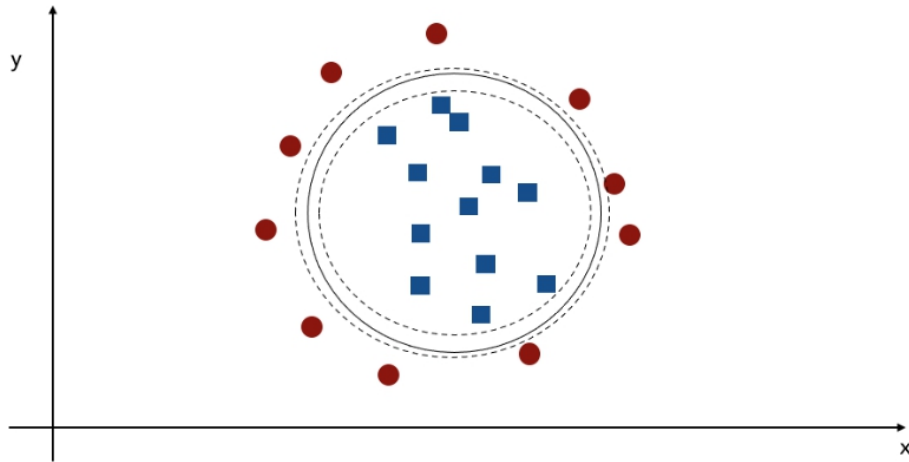
Con scikit-learn, es posible gestionar la invasión de los márgenes. Supongamos que ponemos una C con un valor alto; no permitiremos que el modelo tenga mucha invasión de márgenes. Sin embargo, cuanto más bajo sea el valor, más significativa será la invasión del margen.

Hablemos ahora del kernel del modelo. El núcleo es la forma en que nuestro algoritmo está entrenando. Hay muchas maneras de optimizar un SVC, pero el más utilizado es el kernel gaussiano o el kernel lineal en finanzas.

Hay otros kernels disponibles en scikit-learn como el kernel sigmoide. Sin embargo, sólo veremos el lineal y el gaussiano. Una línea como la de la figura 11.4 esquematiza el lineal, pero

el gaussiano puede mantener un patrón no lineal como en la figura 11.5.

Figura 11.5: Esquematización del kernel gaussiano



En esta figura, podemos ver el funcionamiento de un SVC utilizando un kernel gaussiano.

11.2.2. Cómo crear un SVC usando Python

En esta parte calcularemos un clasificador de máquina de soporte vectorial (SVC) utilizando Python. Es fácil porque sigue la misma sintaxis que la regresión logística. Tenemos que cambiar la función para utilizar la función SVC de scikit-learn.

Código 11.2: Implementación de un SVC

```
# Importar la clase
from sklearn.svm import SVC

# Inicializar la clase
svc = SVC()
```

```
# Ajustar el modelo
```

```
svc.fit(X_train_scaled, y_train_cla)
```



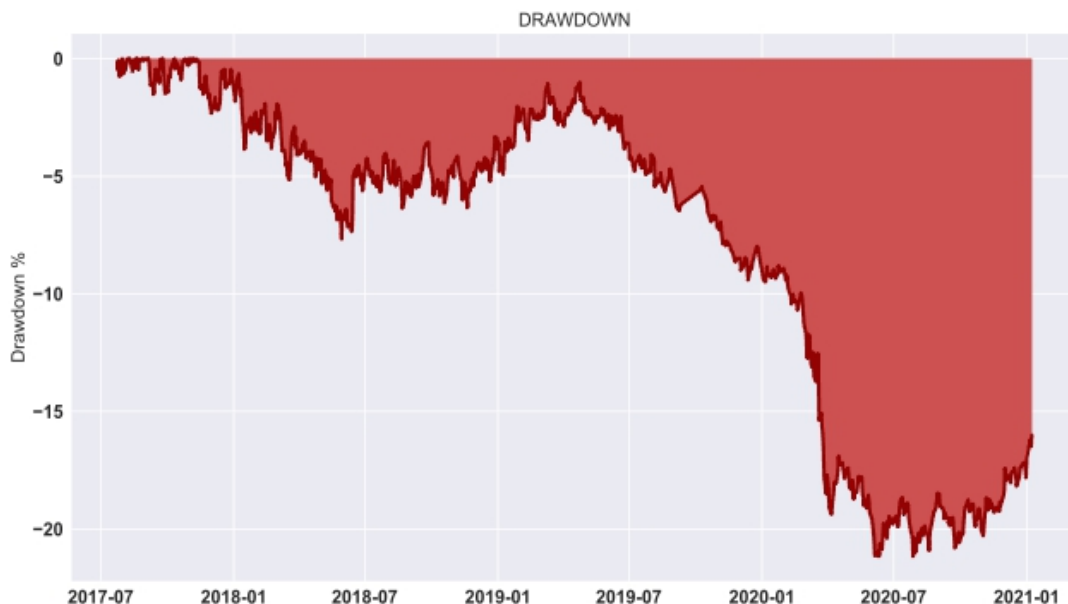
Utilizaremos un SVC con un kernel RBF, el kernel gaussiano en scikit-learn.

11.2.3 Predicciones y backtest

Como podemos ver en la figura 11.6, los rendimientos de la estrategia son terribles porque tenemos un drawdown del 67% (conferir el código) y una pérdida del 60%. Por lo tanto, si estamos buscando un algoritmo para poner en producción, no necesitamos profundizar más. Ayudaría si exploráramos otra combinación entre datos, modelo y estrategia.

Figura 11.6: Backtest de una estrategia utilizando predicciones SVC





El crecimiento constante del drawdown es lo peor que le puede pasar a una estrategia. Por lo tanto, es una estrategia muy mala.

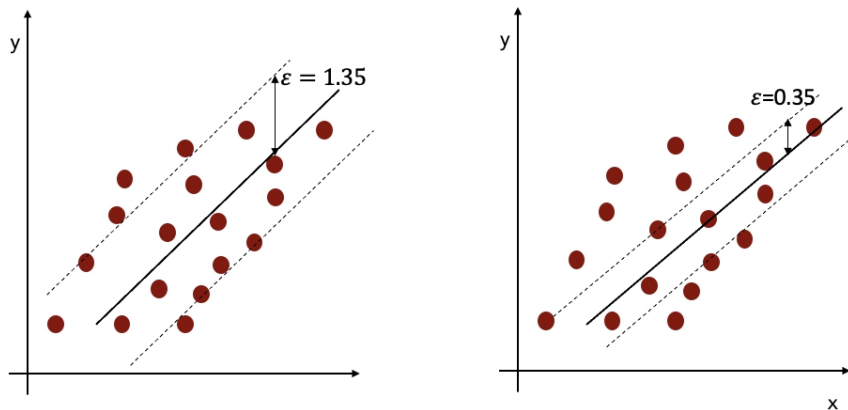
11.3. Regresión con Support Vector Machine (SVR)

En esta sección, vamos a explicar la intuición de una regresión usando una máquina de soporte vectorial (SVR). A continuación, crearemos el modelo y haremos una estrategia de trading utilizando la predicción de nuestro SVR.

11.3.1. Intuición sobre el funcionamiento de un SVR

Será fácil de entender porque el SVR sigue casi el mismo proceso que el SVC. Tenemos que plantear el problema de otra manera. En lugar de maximizar la distancia entre dos grupos, el SVR intenta maximizar el número de observaciones entre los márgenes o minimizar el número de observaciones fuera de los márgenes. Veamos un ejemplo en la figura 11.7.

Figura 11.7: Intuición sobre SVR



En esta figura, podemos ver la intuición de funcionamiento del SVR. El parámetro más útil es el ϵ porque gestiona el recorrido del SVR.

El parámetro epsilon es la tolerancia del modelo. Se puede cambiar utilizando el hiperparámetro de la función scikit-learn. Muchas funciones nos permiten encontrar la mejor combinación entre los hiperparámetros de los modelos (veremos una técnica en el próximo capítulo), pero no olvides que cuanto más optimices el algoritmo para ajustarlo a nuestros datos, más aumenta el riesgo de sobreajuste.

11.3.2. Cómo crear un SVR usando Python

En esta parte, queremos crear un modelo SVR utilizando scikit-learn. Para ello, vamos a utilizar la clase SVR de scikit-learn.

Código 11.3: Implementación de un SVR

```
# Importar la clase
from sklearn.svm import SVR

# Inicializar la clase
svr = SVR()
```



```
# Ajustar el modelo
```

```
svr.fit(X_train_scaled, y_train_reg)
```

11.3.3. Predicciones and backtest

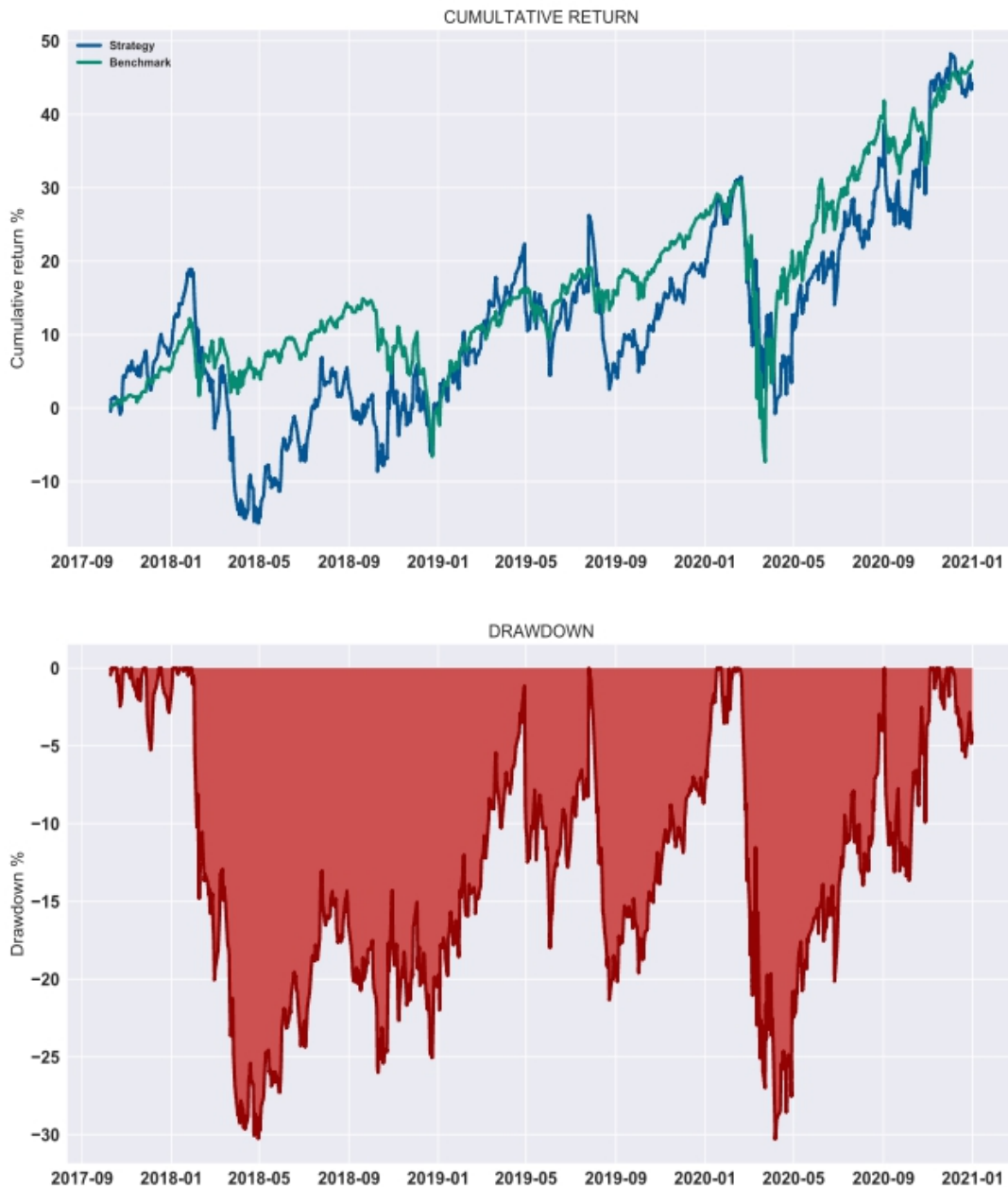
Podemos mencionar que las características son las mismas para ambos. En comparación, el rendimiento de los modelos es muy diferente. Veamos la figura 11.8. Se debe al etiquetado del objetivo (esta vez, el regresor es mejor).

Como podemos ver, el modelo regresor gana un 35% a la vez que el modelo clasificador pierde un 60% del capital inicial. Sin embargo, ¿por qué casi el mismo algoritmo presenta algunas diferencias significativas? La verdad es que no podemos decir por qué. Mientras que, para evitar tener algunos algoritmos con un rendimiento horrible como el modelo SVC, podemos hacer un backtest profundo para encontrar los mejores algoritmos que se pueden utilizar para minimizar el riesgo de perder dinero. También podemos crear una cartera de estrategias para reducir el riesgo y obtener rendimientos más estables.

Figura 11.8: Rendimiento acumulado de la estrategia utilizando las predicciones SVR

Beta: 0.787	Alpha: 2.15 %	Sharpe: 0.455	Sortino: 0.584

VaR: 56.1 %	cVaR: 66.0 %	VaR/cVaR: 1.176	drawdown: 30.32 %



La estrategia tiene un ratio de Sharpe muy bueno porque empieza a estar razonablemente cerca de 0.5 (sharpe = 0.45). No está significativamente correlacionada con el mercado (beta = 0.787). Sin embargo, muchas fluctuaciones en los rendimientos acumulados implican un riesgo considerable de pérdidas (cVaR = 66%). Es una estrategia excelente para una cartera de estrategias para disminuir el riesgo porque ahora

esta estrategia es demasiado arriesgada, teniendo en cuenta el rendimiento excesivo del mercado.

Resumen

- La normalización es obligatoria para las SVM porque se basan en un proceso geométrico.
- El clasificador de la máquina de soporte vectorial busca optimizar la distancia entre los dos grupos.
- Podemos cambiar el valor de la invasión del margen utilizando los parámetros C cuando trabajamos con scikit-learn.
- La regresión con una máquina de soporte vectorial intenta crear una ruta con el máximo de observaciones.
- Podemos cambiar la anchura de la ruta utilizando el parámetro epsilon en scikit-learn.
- Los kernels más utilizados en finanzas tanto para SVC como para SVR son el kernel lineal y el kernel gaussiano.

Aplicación Trading en vivo y Screener

Este código se basa en la clase creada en el anexo: MetaTrader.

Código 11.4: Aplicación Trading / Screener para SVR

```
from MT5 import *
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
from sklearn.linear_model import LogisticRegression
import time
```

```

import pickle
from joblib import dump, load
import os

path = ""
# Ex: C:/Desktop/Python_for_finance_and_algorithmic_trading/ChapterN/

def create_model_weights(symbol):
    """ Pesos para la regresión lineal sobre el cambio porcentual """
    # Importar los datos
    data = MT5.get_data(symbol, 3500)[["close"]].pct_change(1)

    # Crear la nueva variable
    data["close t-1"] = data[["close"]].shift(1)

    # Dividir los datos
    data = data.dropna()
    split = int(0.80 * len(data))
    # Creación del conjunto de entrenamiento
    X_train = data[["close t-1"]].iloc[:split]
    y_train = np.round(data[["close"]].iloc[:split] + 0.5)

    # Crear el modelo
    alg = LogisticRegression()

    # Ajustar el modelo
    alg.fit(X_train, y_train)

    # Guardar el modelo
    alg_var = pickle.dumps(alg)
    alg_pickel = pickle.loads(alg_var)

    dump(alg_pickel, os.path.join(path, f"Models/{symbol}.joblib"))

```

```

def log_reg_sig(symbol):
    """
    Función para predecir el valor de mañana utilizando el modelo ARIMA
    """

    # Crear los pesos si no hay en la carpeta
    try:
        alg = load(os.path.join(path, f"Models/{symbol}.joblib"))
    except:
        create_model_weights(symbol)
        alg = load(os.path.join(path, f"Models/{symbol}.joblib"))

    # Tomar el último porcentaje de cambio
    data = MT5.get_data(symbol, 3500)[["close"]].pct_change(1)
    data["close t-1"] = data[["close"]].shift(1)

    X = data["close t-1"].iloc[-1].reshape(-1, 1)

    # Encontrar la señal
    prediction = alg.predict(X)
    prediction = np.where(prediction == 0, -1, 1)
    buy = prediction[0][0] > 0
    sell = not buy

    return buy, sell

# True = Trading en vivo y False = Screener
live = True

if live:
    current_account_info = mt5.account_info()
    print("-----")

```

```

print("Date: ", datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
print(f"Balance: {current_account_info.balance} USD, \t"
      f"Equity: {current_account_info.equity} USD, \t"
      f"Profit: {current_account_info.profit} USD")
print("-----")

info_order = {

    "Euro vs USdollar": ["EURUSD", 0.01]
}

start = datetime.now().strftime("%H:%M:%S")#"23:59:59"
while True:

    # Verificación para lanzamiento

    if datetime.now().weekday() not in (5, 6):

        is_time = datetime.now().strftime("%H:%M:%S") == start

    else:

        is_time = False

    # Lanzar el algoritmo

    if is_time:

        # Abrir las transacciones

        for asset in info_order.keys():

            # Inicializar los inputs

            symbol = info_order[asset][0]

            lot = info_order[asset][1]

            # Crear las señales

            buy, sell = log_reg_sig(symbol)

            # Ejecutar el algoritmo

            if live:

                MT5.run(symbol, buy, sell, lot)

```

```
else:  
    print(f"Symbol: {symbol}\\t"  
          f"Buy: {buy}\\t"  
          f"Sell: {sell}")  
time.sleep(1)
```



El parámetro live establece el modo trading en vivo (live = True) o el modo screener (live = False).

Capítulo 12: Métodos ensemble y árbol de decisión

En este capítulo veremos muchos algoritmos de aprendizaje automático. En primer lugar, veremos el árbol de decisión, después el bosque aleatorio y algunos otros métodos ensemble de utilidad en finanzas. Todos los algoritmos de este capítulo se aplicarán a la cotización de las acciones de Google.

12.1. Árbol de decisión

En esta sección estudiaremos el algoritmo del árbol de decisión. Este algoritmo estadístico de aprendizaje automático es muy útil en finanzas porque entiende la relación no lineal entre los datos.

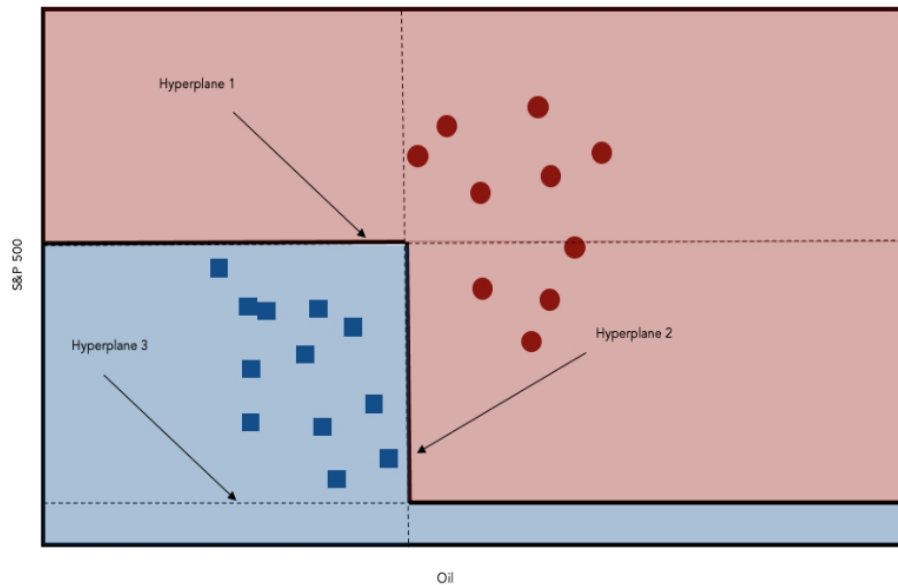
12.1.1. Clasificador de árbol de decisión

En esta subsección comenzaremos con el clasificador de árbol de decisión. Antes de programar, explicaremos cómo funciona un clasificador de árbol de decisión.

Matemáticamente, cuando tenemos un espacio con grupos de puntos, el árbol de decisión cortará el espacio con hiperplanos para separar los grupos. Un hiperplano es una recta en un espacio 2d, así que si no sabemos lo que significa, recordemos que es una recta en nuestro ejemplo.

Además, cada hiperplano es ortogonal a los demás. ¿Qué significa esto? En nuestro caso, cada hiperplano es perpendicular a los demás. Es fundamental saberlo para entender el comportamiento del algoritmo. En la siguiente figura, veremos cómo funciona un árbol de decisión de una forma más sencilla.

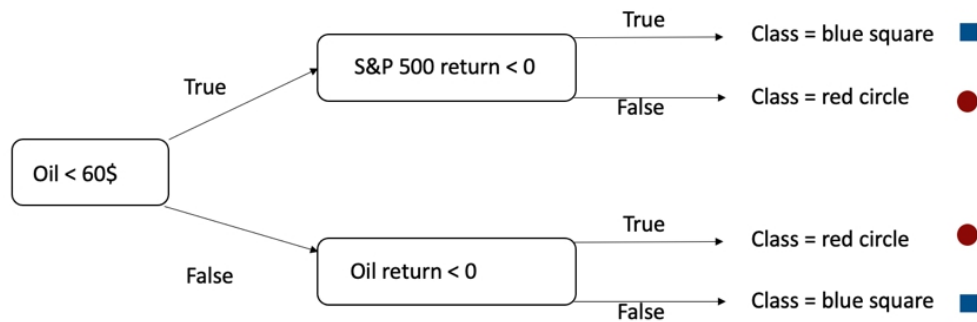
Figura 12.1: Cómo funciona gráficamente un árbol de decisión



En esta figura, podemos ver los distintos hiperplanos del árbol de decisión. En este ejemplo, hay tres hiperplanos, lo que significa que la profundidad de este árbol de decisión es 3.

Vamos a esquematizar el funcionamiento de un árbol de decisión para estar seguros de que hemos entendido lo que hace un clasificador de árbol de decisión. El árbol de decisión funciona con condiciones, veamos en la figura 12.2.

Figura 12.2: Clasificador de árbol de decisión



Además, es un modelo muy interpretable, lo que resulta interesante en finanzas. Los árboles de decisión no necesitan normalización para obtener buenos resultados. Además, es un algoritmo perfecto cuando se tienen muchas variables ficticias en las características.

Ahora veremos cómo implementar un clasificador de árbol de decisión con Python. Para ello, importamos la clase `DecisionTreeClassifier` de `scikit-learn`, y utilizaremos la misma sintaxis que para los algoritmos anteriores.

Código 12.1: Clasificador de árbol de decisión

```
# Importar la clase
from sklearn.tree import DecisionTreeClassifier

# Inicializar la clase
trc = DecisionTreeClassifier(max_depth = 6)

# Ajustar el modelo
trc.fit(X_train, y_train_cla)
```

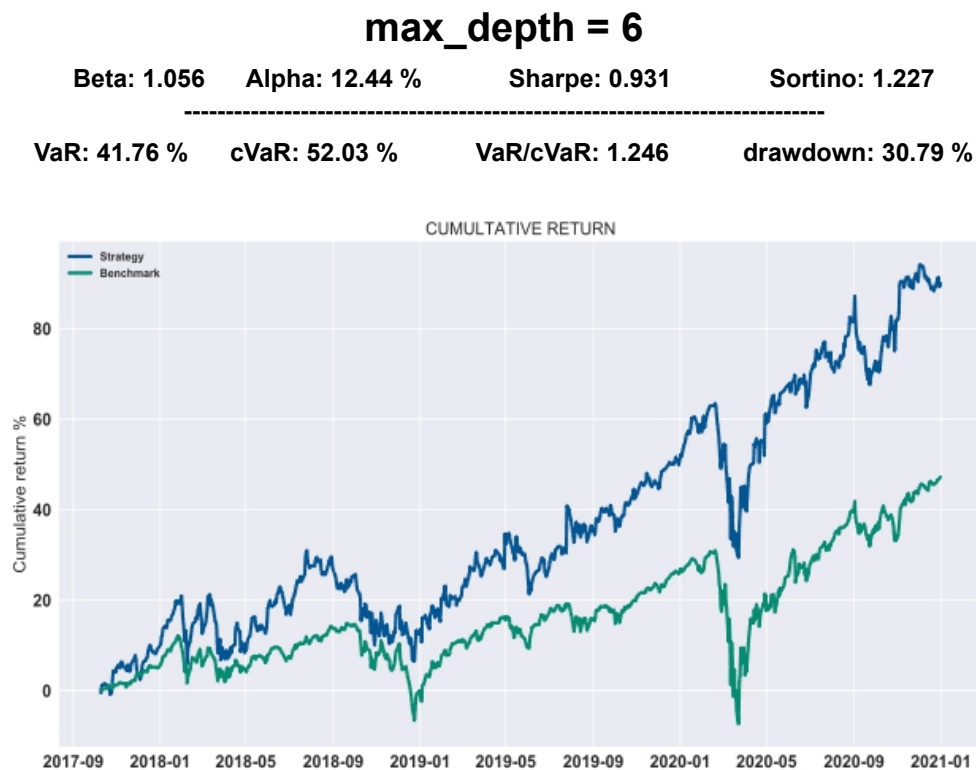


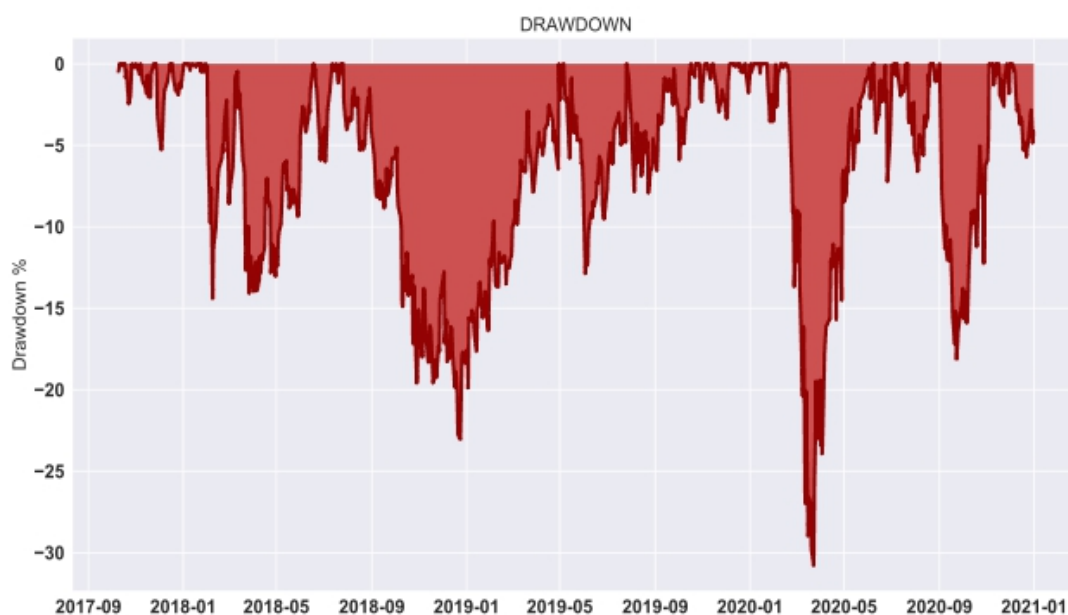
El hiperparámetro `max_depth` se fija en 6. Sin embargo, es una elección aleatoria. Se trata simplemente de no dejar que el árbol sea demasiado profundo porque no es

bueno; más adelante veremos por qué. Veremos cómo encontrar la mejor profundidad en la siguiente parte.

Entonces, tenemos el modelo, podemos hacer predicciones y backtest de la estrategia. Como se muestra en la figura 12.3, tenemos una excelente rentabilidad utilizando la profundidad de 6 porque tenemos casi un 30% de rentabilidad anual. En cambio, cuando no precisamos la profundidad, el algoritmo puede profundizar todo lo que quiera, y no es bueno; podemos ver que los rendimientos del mismo modelo pero sin precisar la profundidad son del -80% en el periodo.

Figura 12.3: Backtest dependiendo de la profundidad máxima



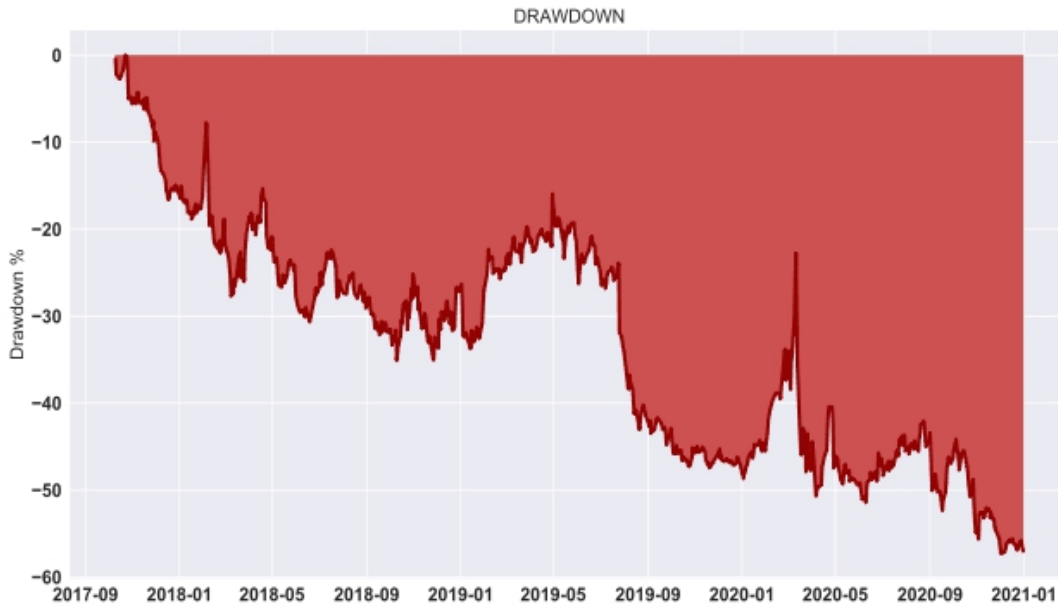


max_depth = None

Beta: -0.034 Alpha: -20.95 % Sharpe: -0.715. Sortino: -0.906

VaR: 91.56 % cVaR: 101.59 % VaR/cVaR: 1.11 drawdown: 57.31 %





En esta figura, podemos ver que la profundidad del árbol de decisión desempeña un papel crucial. De hecho, la diferencia de rentabilidad entre las dos estrategias es de casi el 160% en el periodo. Esto pone de manifiesto el problema de sobreajuste cuando dejamos que el algoritmo sin restricciones de profundidad.



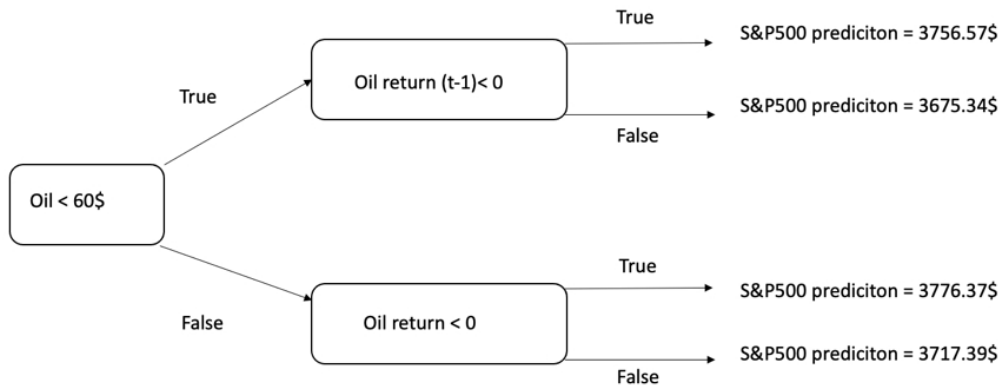
Los hiperparámetros más críticos en el árbol de decisión son la profundidad del árbol, que viene dada por el hiperparámetro `max_depth` en scikit-learn.

12.1.2. Regresor de árbol de decisión

Esta parte explicará cómo funciona un regresor de árbol de decisión, cómo implementar un regresor de árbol de decisión utilizando Python y cómo hacer un backtest de la estrategia.

El regresor de árbol de decisión sigue casi el mismo proceso que el clasificador de árbol de decisión. Veámoslo en un árbol sencillo en la figura 12.4.

Figura 12.4: Regresor de árbol de decision



Para implementar un regresor de árbol de decisión, utilizaremos la misma sintaxis que para el clasificador de árbol de decisión.

Código 12.2: Regresor de árbol de decisión

```
# Importar la clase
from sklearn.tree import DecisionTreeClassifier

# Inicializar la clase
trc = DecisionTreeClassifier(max_depth = 6)

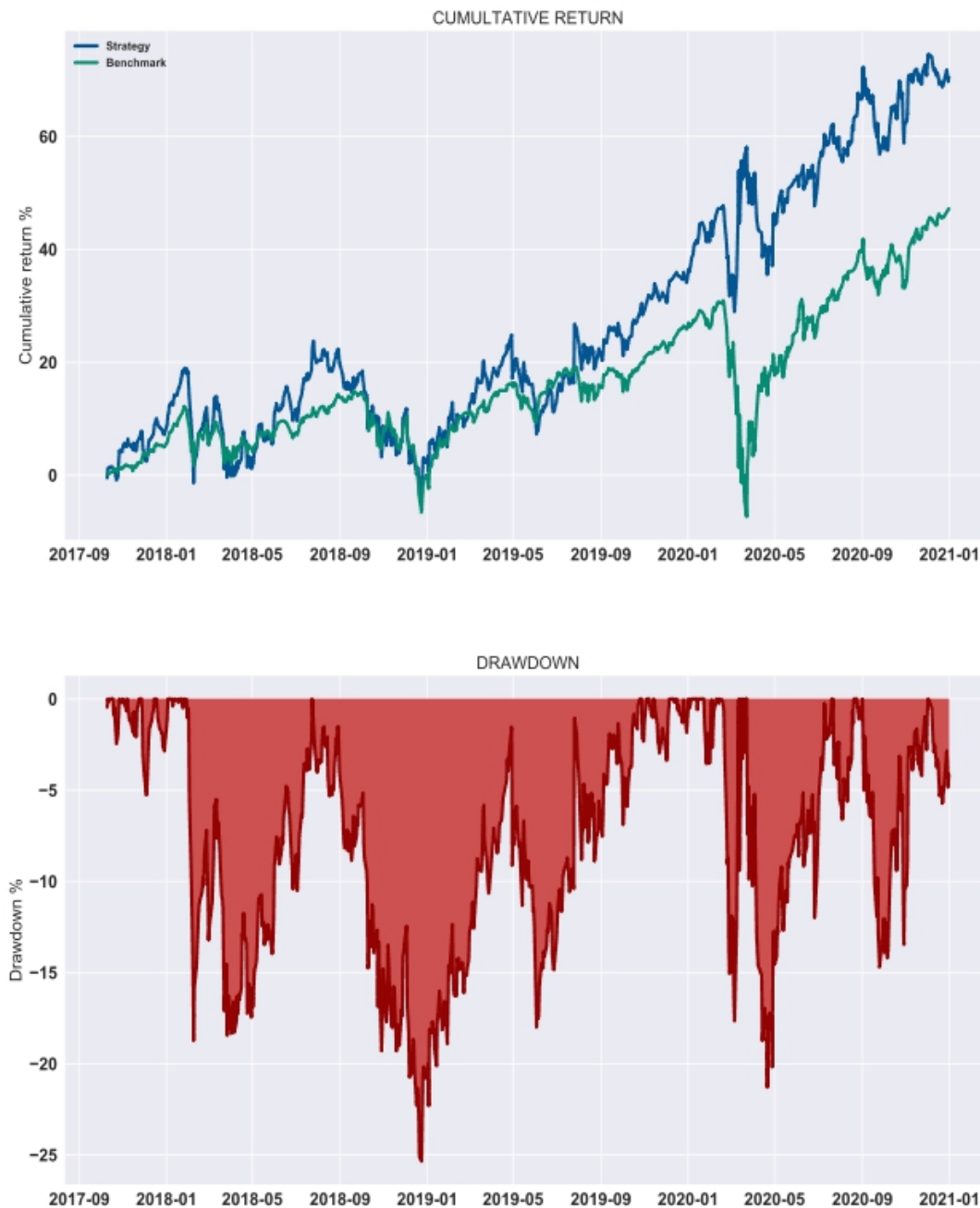
# Ajustar el modelo
trc.fit(X_train, y_train_cla)
```

Podemos ver los resultados de la estrategia en la figura 12.5 y como podemos ver con la restricción en la profundidad del algoritmo, también da buenos resultados.

Figura 12.5: Predicción de acciones de Google basada en estrategias de backtest

Beta: 0.243 Alpha: 18.27 % Sharpe: 0.728 Sortino: 0.96

VaR: 47.95 % cVaR: 57.81 % VaR/cVaR: 1.206 drawdown: 25.35 %



Es un backtest muy bueno. Tenemos un ratio de Sharpe de 0.72 y un alfa del 18.27%, lo que significa que superamos al mercado. Sin embargo, tenemos un drawdown considerable. La mejor manera de reducir el riesgo de una estrategia es combinarla con las demás, pero lo veremos más adelante.

12.1.3. Optimizar los hiperparámetros

En la última parte, hemos dicho que dejar la profundidad máxima del árbol sin restricción no es una excelente, pero no tenemos que decir cómo encontrar el óptimo.

Para encontrar el mejor valor para el parámetro `max_depth`, vamos a utilizar el algoritmo `GridSearchCV` de `scikit-learn`. Es un algoritmo sencillo. Simplemente prueba todas las posibilidades y toma la mejor.

Código 12.3: Grid Search CV

```
# Importar la clase
from sklearn.model_selection import GridSearchCV

# Crear el modelo
dtr = DecisionTreeRegressor()

# Elegir una lista de parámetros
param = {"max_depth": [3, 6, 15]} 1

# Crear el GridSearch
model = GridSearchCV(dtr, param_grid = param, cv = 3)

# Entrenar el modelo
model.fit(X_train.values, y_train_reg.values)

# Mostrar el mejor modelo
model.best_estimator_
```

Salida (mejor modelo):

```
DecisionTreeRegressor(ccp_alpha = 0.0, criterion = 'mse',
max_depth = 3, max_features = None, max_leaf_nodes = None,
min_impurity_decrease = 0.0, min_impurity_split = None, min_samples_leaf = 1,
min_samples_split = 2,
```

```
min_weight_fraction_leaf = 0.0, presort = 'deprecated',  
random_state = None, splitter = 'best')
```

- 1 Crea un diccionario que contenga todas las variables que queremos probar con los diferentes valores. Aquí, sólo queremos probar el modelo de prueba en función de la profundidad máxima del árbol (3, 6 o 15).

Como podemos ver, el mejor modelo es una longitud de 3. ¿Es el mejor modelo en qué criterio? Como no tenemos que especificar, un criterio es el criterio de entrenamiento del algoritmo que utilizará el MSE. Lo maravilloso es que podemos personalizar la función para encontrar el mejor algoritmo utilizando nuestro criterio. Vamos a crear algunos criterios para explicar esto. Aquí se propone la suma de los rendimientos y el ratio de Sharpe, pero podemos crear el ratio que queramos.

Código 12.4: Crear nuevas métricas

```
# Importar la función  
from sklearn.metrics import make_scorer  
  
# RETURNS  
# Crear criterio de returns  
def returns(y, y_pred): 1  
    return np.sum(np.sign(y_pred) * y)  
  
# Transformar el criterio en métrica  
returns_metric = make_scorer(returns, greater_is_better = True) 2  
  
# SHARPE  
# Crear el criterio sharpe  
def sharpe(y, y_pred):  
    r = np.sign(y_pred) * y  
    return np.mean(r) / np.std(r) 3  
  
# Transformar el criterio en métrica
```

```
sharpe_metric = make_scorer(sharpe, greater_is_better = True)
```

- 1 La función de criterio para la optimización de hiperparámetros sólo toma dos parámetros de entrada: el objetivo real y el objetivo predicho.
- 2 El parámetro `greater_is_better` indica al algoritmo si queremos maximizar o minimizar la métrica.
- 3 Aquí no es necesario anular el ratio de Sharpe porque la relación de tamaño no importa. Así pues, añadimos menos cosas posibles para optimizar el tiempo de cálculo.



La función `make_score` nos permite transformar una función simple en una métrica compatible con los algoritmos de `scikit-learn`.

La función `make_score` nos permite transformar una función simple en una métrica compatible con los algoritmos de `scikit-learn`.

Con la métrica `returns_metric` y la métrica `sharpe_metric`, tenemos una profundidad óptima de 3. Esto significa que con las 3 formas, tenemos una profundidad óptima de tres. Por lo tanto, podemos decir que en el conjunto de entrenamiento, la mejor profundidad para el algoritmo es 3.

12.2. Bosque aleatorio

En esta sección se analiza el bosque aleatorio. En primer lugar, veremos cómo calcular un clasificador de bosque aleatorio, luego un regresor de bosque aleatorio y cómo optimizar los hiperparámetros de un bosque aleatorio.

12.2.1. Clasificador de bosque aleatorio

En esta parte, aprenderemos a implementar un clasificador de bosque aleatorio. El bosque aleatorio es el primer método

ensemble que aprenderemos. Un método ensemble es un método que reagrupa algoritmos para tener una mejor predicción.

Naturalmente, como su nombre indica, el bosque aleatorio es un conjunto de árboles de decisión. De hecho, el bosque aleatorio no es más que un algoritmo que combina las predicciones de muchos árboles de decisión para tener mejores predicciones.

No vamos a profundizar en la explicación del bosque aleatorio porque la intuición es fácil de entender. Sin embargo, si queremos profundizar, necesitamos mucha matemática, que es irrelevante para este libro. Veamos el código de un clasificador de bosque aleatorio.

Código 12.5: Clasificador de bosque aleatorio

```
# Importar la clase
from sklearn.ensemble import RandomForestClassifier

# Inicializar la clase
lr = RandomForestClassifier()

# Ajustar el modelo
lr.fit(X_train, y_train_cla)
```



Observa que aquí no hay ningún hiperparámetro personalizado para mostrar la diferencia en la figura 12.6.

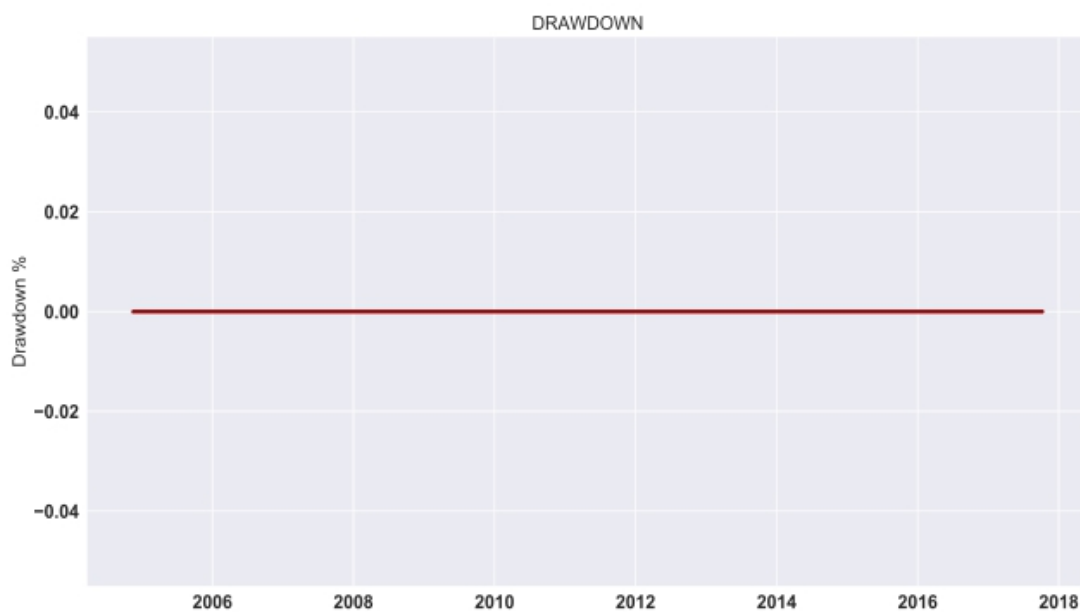
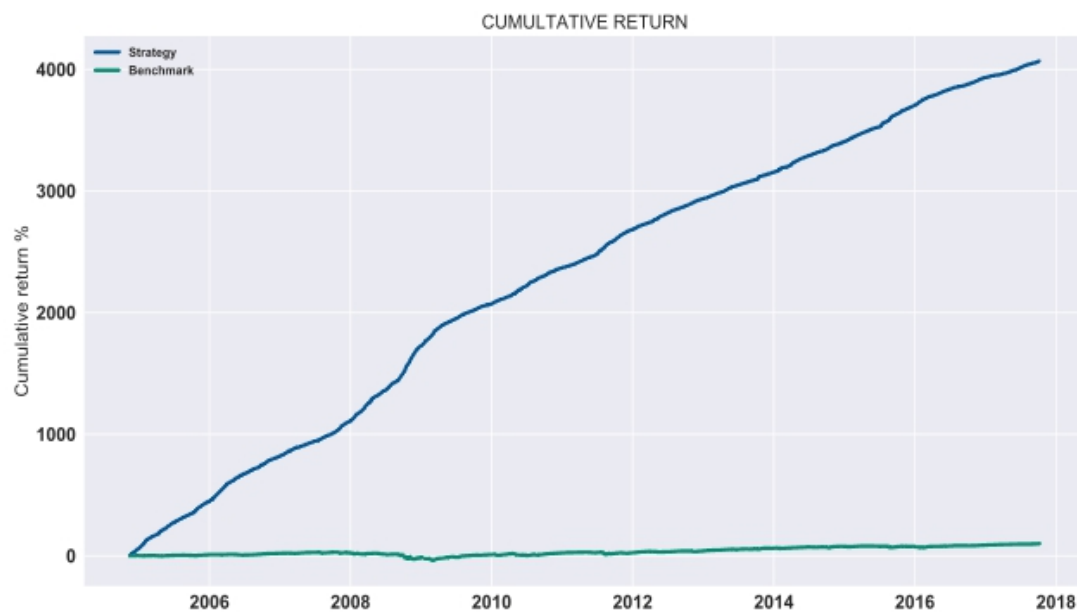
Destacaremos el problema con el bosque aleatorio (igual que un árbol de decisión) cuando no se precisa la profundidad y el número de estimadores (sólo para el bosque aleatorio). Hay un gran sobreajuste, como podemos ver en la figura 12.6.

Figura 12.6: Resaltado del sobreajuste

Backtest del conjunto de entrenamiento

Beta: -0.001 Alpha: 315.67 % Sharpe: 14.104 Sortino: nan

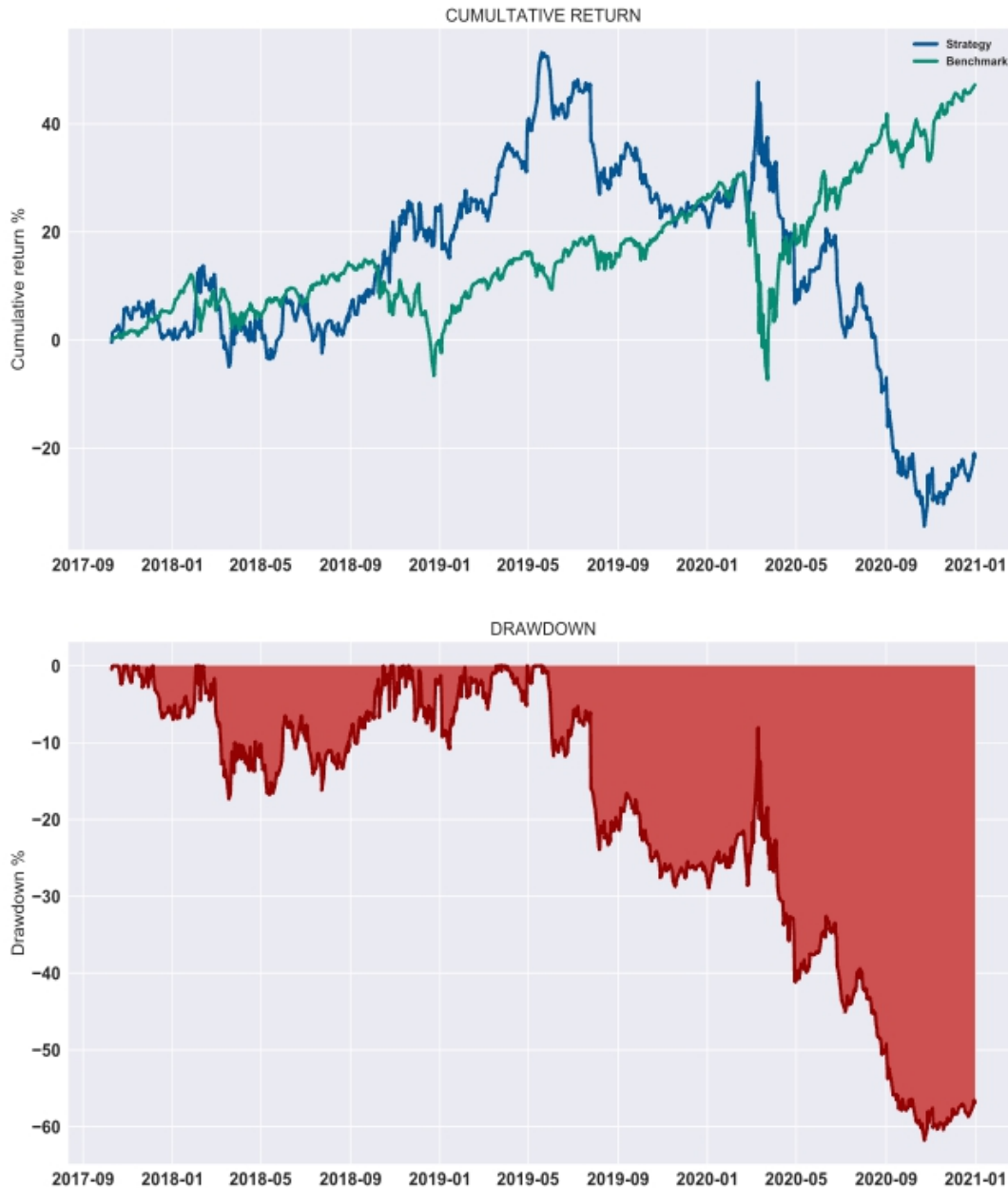
VaR: -263.64 % cVaR: -256.21 % VaR/cVaR: 0.972 drawdown: -0%



Backtest del conjunto de test

Beta: -0.018 Alpha: -6.45 Sharpe: -0.224 Sortino: -0.28

VaR: 76.65 % cVaR: 86.33 % VaR/cVaR: 1.126 drawdown: 61.75 %



Como podemos ver en la pantalla, hay sobreajuste porque, en el conjunto de entrenamiento (los datos que el algoritmo ya conoce), ganamos un 350% al año con un drawdown del 0%, en lugar de que el conjunto de test pierda dinero.

12.2.2. Regresor de bosque aleatorio

En esta parte, hablaremos de los regresores de bosque aleatorio. Naturalmente, un regresor de bosque aleatorio es un conjunto de regresores de árbol de decisión.

No explicamos las mismas cosas para evitar ser aburridos para este algoritmo. Podemos copiar exactamente lo que decimos para el clasificador de bosque aleatorio en el regresor de bosque aleatorio. Así pues, veamos cómo implementarlo y destaquemos de nuevo el problema de sobreajuste de estos algoritmos.

Código 12.6: Regresor de bosque aleatorio

```
# Importar la clase
from sklearn.ensemble import RandomForestRegressor

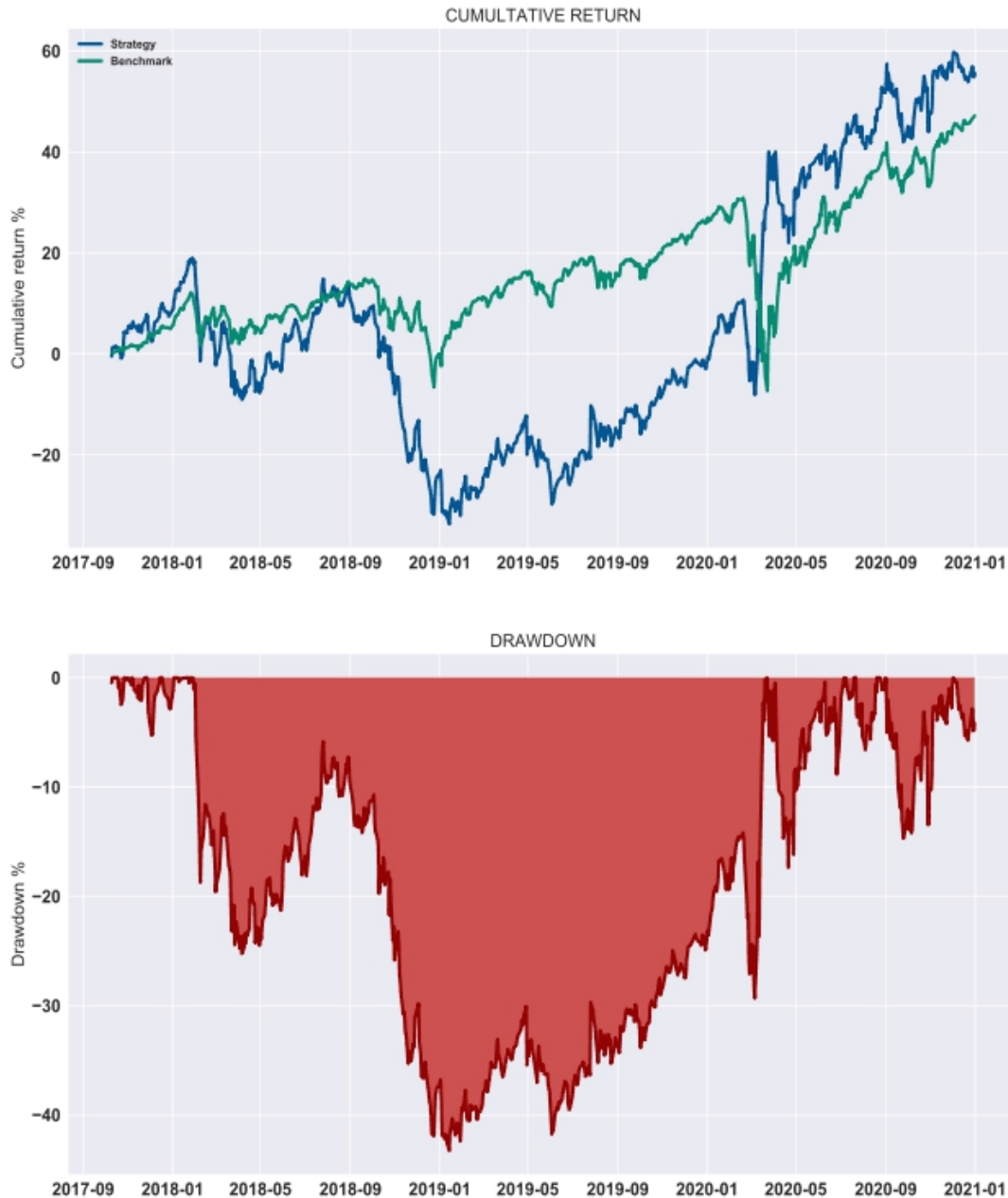
# Inicializar la clase
lr = RandomForestRegressor()

# Ajustar el modelo
lr.fit(X_train, y_train_reg)
```

Figura 12.7: Resultado del sobreajuste en el bosque aleatorio

Beta: 0.511	Alpha: 9.76 %	Sharpe: 0.574	Sortino: 0.777

VaR: 52.51 %	cVaR: 62.26 %	VaR/cVaR: 1.186	drawdown: 43.26 %



Es un buen rendimiento, pero hay una reducción significativa durante un periodo muy largo. Significa que la estrategia es peligrosa.

12.2.3. Optimizar los hiperparámetros

En esta parte, optimizaremos los hiperparámetros del árbol de decisión. Para ello, utilizaremos la métrica de Sharpe

empleada en la parte 12.1.3.

Código 12.7: Optimizar los hiperparámetros

```
# Importar la clase
from sklearn.model_selection import GridSearchCV

# Crear el modelo
dtr = RandomForestRegressor()

# Elegir una lista de parámetros
param = {"max_depth": [3, 6, 15], "n_estimators": [50, 100, 150]}

# Importar la función
from sklearn.metrics import make_scorer

# SHARPE
# Crear el criterio sharpe
def sharpe(y, y_pred):
    r = np.sign(y_pred) * y
    return np.mean(r) / np.std(r)

# Transformar el criterio en métrica
sharpe_metric = make_scorer(sharpe, greater_is_better = True)

model = GridSearchCV(dtr, param_grid = param,
                    scoring = sharpe_metric, cv = 3)

model.fit(X_train.values, y_train_reg.values)

model.best_estimator_
```



Los parámetros que influyen en un bosque aleatorio son el número de árboles y la profundidad de los árboles.

En la siguiente figura, podemos ver que el ajuste de los hiperparámetros influye considerablemente en el rendimiento.

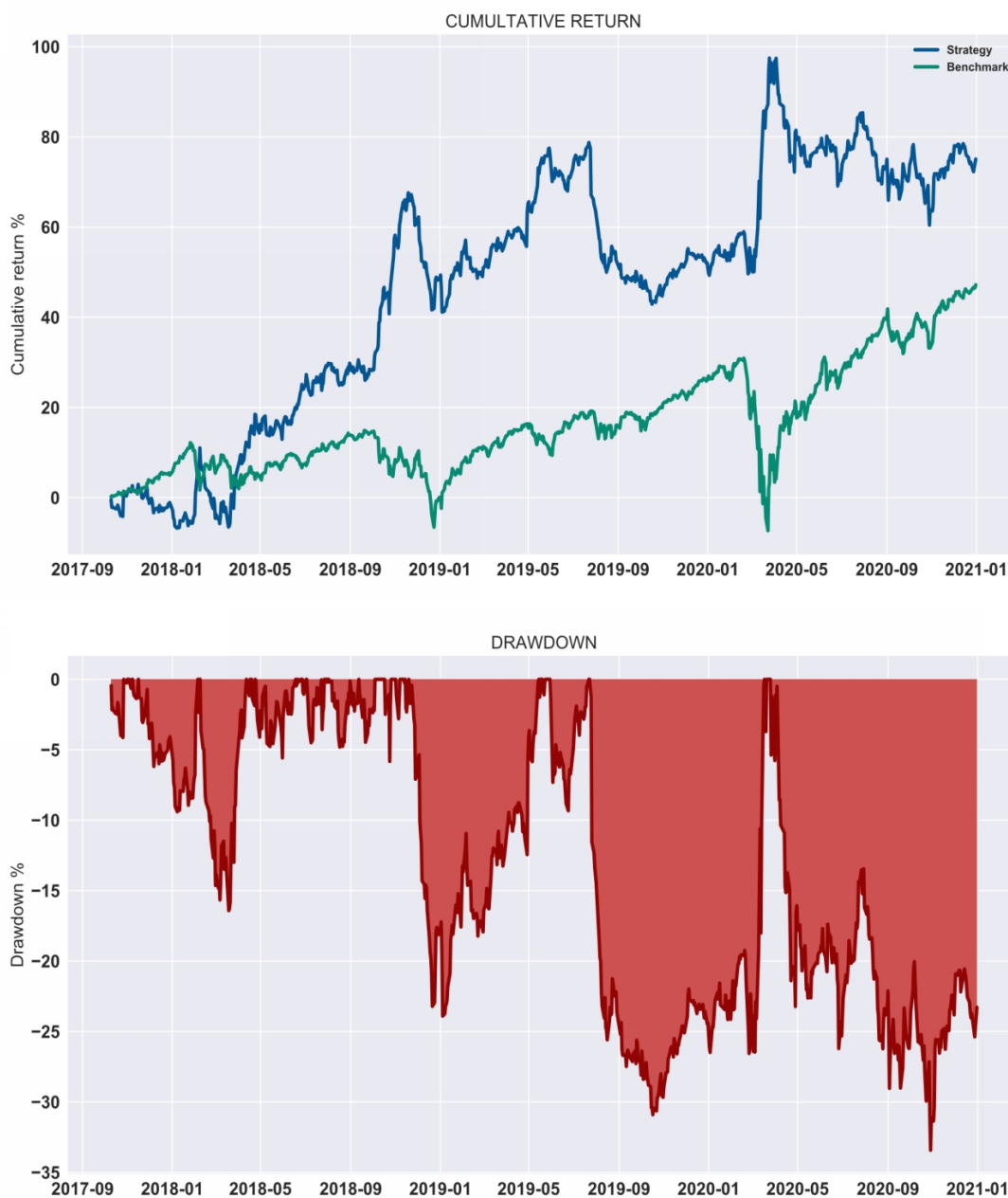
De hecho, esto nos permite tener una curva más suave y un mejor indicador de backtest.

Figura 12.8: Retornos con y sin ajuste

Sin modelo de ajuste

Beta: -0.056 Alpha: 24.09 % Sharpe: 0.776 Sortino: 1.143

VaR: 46.72 % cVaR: 56.63 % VaR/cVaR: 1.212 drawdown: 33.43 %



Con modelo de ajuste

Beta: 0.553

Alpha: 18.25 %

Sharpe: 0.879

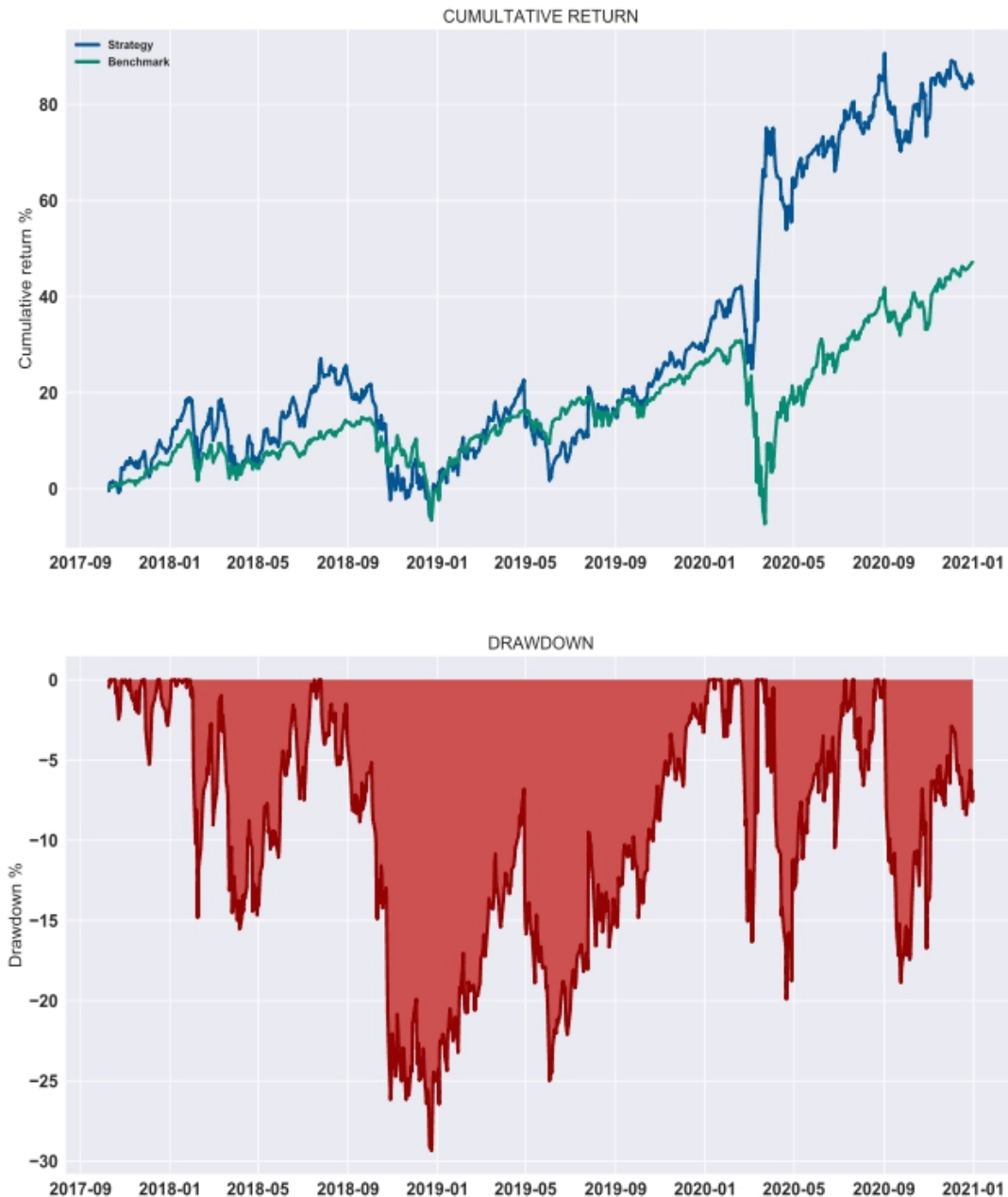
Sortino: 1.194

VaR: 44.05 %

cVaR: 54.37 %

VaR/cVaR: 1.234

drawdown: 29.33 %



Desafortunadamente, para el ejemplo, el modelo sin ajuste no está sobreajustado porque tiene rendimientos razonables. Sin

embargo, el modelo con ajuste es mejor porque tiene menos drawdown, un ratio Sharpe más alto y menos cVaR.

12.3. Métodos ensemble

En esta sección, aprenderemos algunos métodos de ensemble . En primer lugar, veremos los métodos de ensemble más sencillos, el voting, el bagging y el stacking.

12.3.1. Método voting

El método voting o de votación es el más sencillo de los métodos ensemble. Veamos cómo funciona en la figura 12.9.

Como podemos ver que los algoritmos son bien diferentes, la predicción es mejor cuando agregamos su predicción. Sin embargo, necesitamos algunos supuestos matemáticos como la independencia del estimador. Pero, es casi imposible tenerlo en la vida real porque los algoritmos se entrenan con los mismos datos.

Figura 12.9: Clasificador voting

True value	✗	✓	✗	✓	Count
SVC	✗	✗	✓	✓	2
Decision tree classifier	✗	✗	✗	✗	2
Random forest classifier	✓	✗	✗	✓	2
Voting classifier	✗	✗	✗	✓	3

Como podemos ver en la figura, la agregación de los estimadores es mejor que los estimadores por separado.

Cuando trabajamos con estimadores de regresión, seguimos el mismo proceso, salvo que la predicción es la media de todas las predicciones. Veamos un código para un regresor voting que utiliza LinearRegression, SVR, y RandomForestRegressor. Luego pondremos los resultados en la figura 12.10.

Código 12.8: Implementación del regresor voting

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestClassifier,
    VotingRegressor

# Inicializar los estimadores
clf1 = LinearRegression()
clf2 = RandomForestRegressor(n_estimators = 150, max_depth = 3)
clf3 = SVR(epsilon = 1.5)

# Crear el método ensemble
eclf1 = VotingRegressor(estimators = [
```

```

('lr', clf1), ('rf', clf2), ("svc", clf3)]) 1

# Entrenar el modelo
eclf1.fit(X_train, y_train_reg)

# Crear predicciones para el dataset completo
df["prediction"] = eclf1.predict(np.concatenate((X_train, X_test),
axis=0))

# Calcular la estrategia
df["strategy"] = np.sign(df["prediction"]) * df["returns"]

# Backtest
backtest_dynamic_portfolio(df["strategy"].iloc[split:])

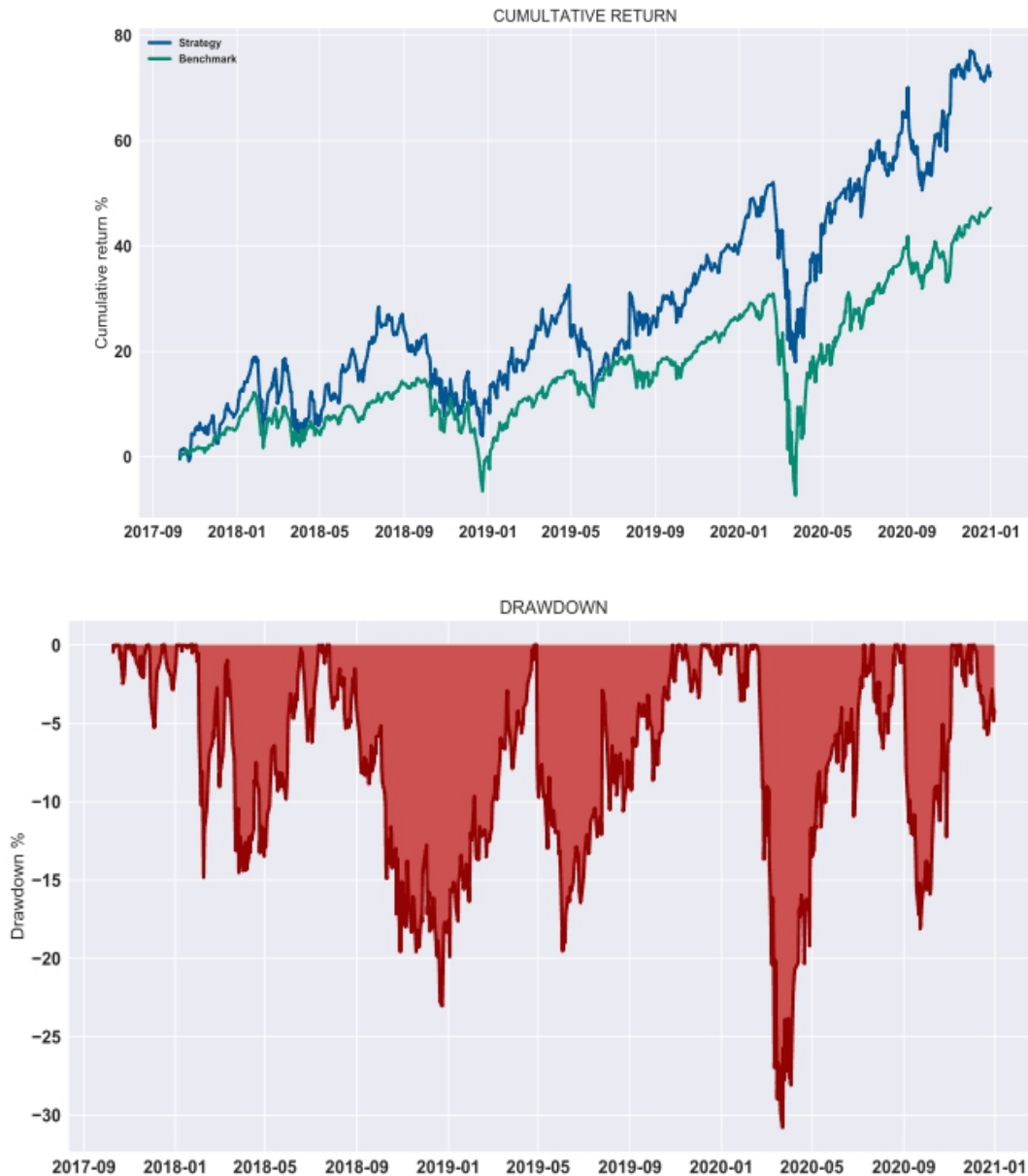
```

- 1 Los modelos voting o de votación se definen mediante una lista de los estimadores.

Figura 12.10: Rendimientos acumulados del regresor voting

Beta: 1.056	Alpha: 7.16 %	Sharpe: 0.754	Sortino: 0.968

VaR: 47.4 %	cVaR: 57.12 %	VaR/cVaR: 1.205	drawdown: 30.79 %



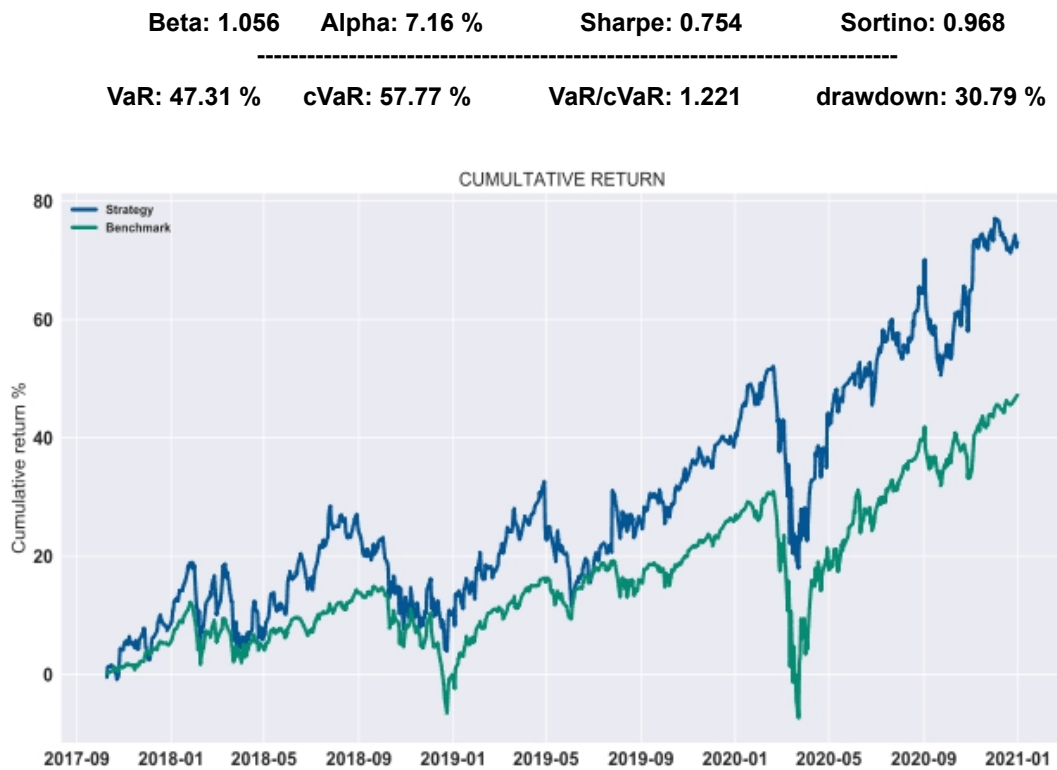
El método voting tiene resultados similares a los del bosque aleatorio, pero la rentabilidad acumulada es menos volátil.

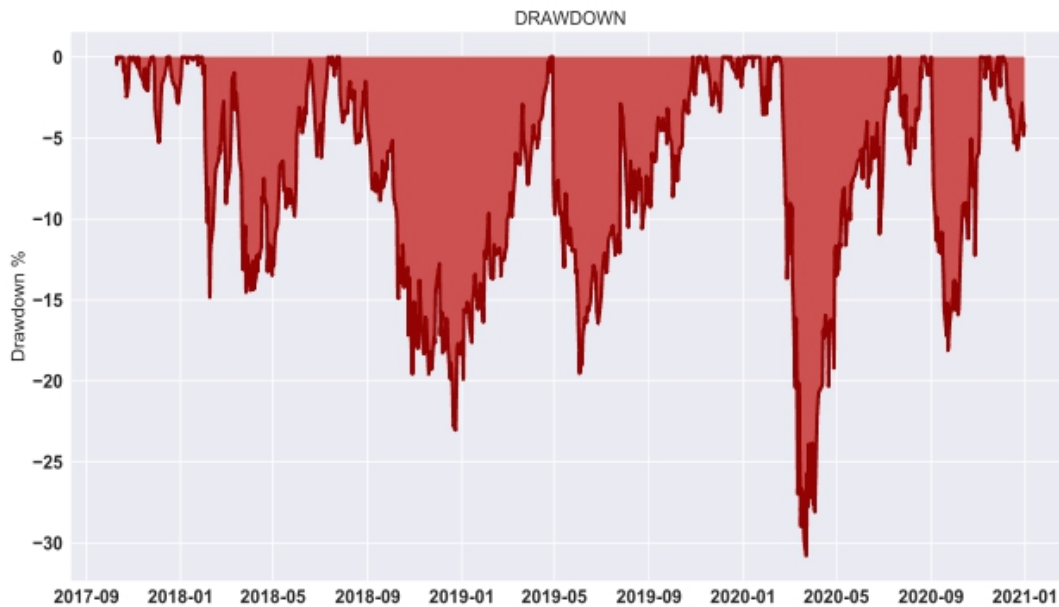
12.3.2. Método bagging

En esta parte, hablaremos del método bagging. Es una forma sencilla de tener un método ensemble en unas pocas líneas de código. De hecho, el bagging es un método que nos permite entrenar un método ensemble con n estimadores

iguales. Por ejemplo, un bosque aleatorio es un bagging con árboles de decisión. Ahora, vamos a crear un regresor bagging que utiliza 150 SVR.

Figura 12.11: Rendimiento acumulado del bagging utilizando 150 SVR





En esta figura, podemos ver el rendimiento acumulado de un algoritmo de bagging creado con 150 SVR.

Código 12.10: Bagging con SVR

```
from sklearn.svm import SVR
from sklearn.ensemble import BaggingRegressor

# Inicializar el bagging
bag_reg = BaggingRegressor(SVR(epsilon = 1.5), n_estimators = 150)

# Entrenar el método
bag_reg.fit(X_train, y_train_reg)

# Crear predicciones para el dataset completo
df["prediction"] = bag_reg.predict(
    np.concatenate((X_train, X_test), axis = 0))

# Calcular la estrategia
df["strategy"] = np.sign(df["prediction"]) * df["returns"]
```

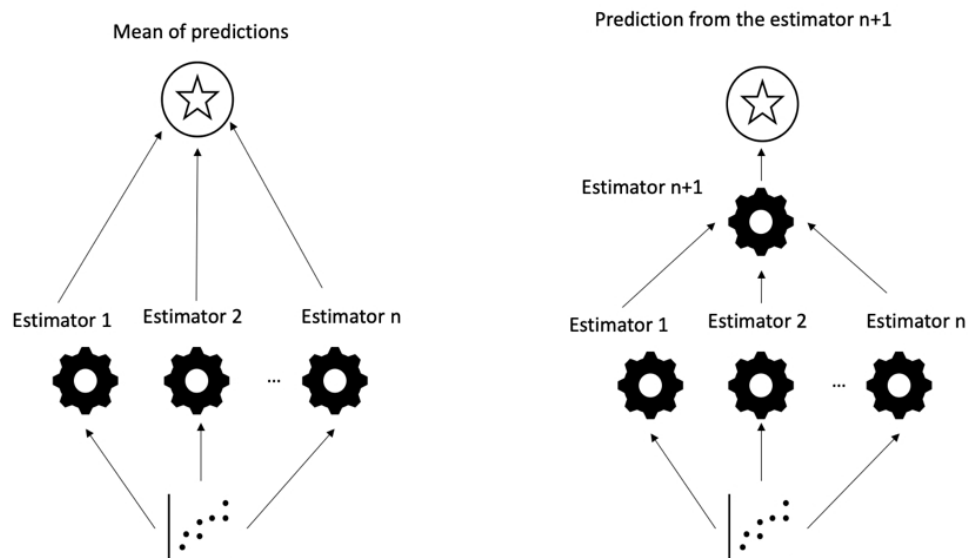
```
# Backtest
```

```
backtest_dynamic_portfolio(df["strategy"].iloc[split:])
```

12.3.3. Método stacking

En esta parte, hablaremos del método stacking. Este método es un poco diferente a los demás porque ahora, en vez de hacer una media, por ejemplo, daremos la predicción de los estimadores a otro algoritmo que dará la predicción.

Figura 12.12: Esquema del método stacking



Esta figura muestra la diferencia entre el voting y el bagging (izquierda), que hacen la media de la predicción del estimador en lugar del stacking (derecha), que hace un algoritmo para agregar mejor la predicción.



Normalmente, ponemos un árbol de decisión o un bosque aleatorio cuando hacemos un clasificador de stacking porque funcionan muy bien con la variable ficticia.

Código 12.11: Implementación del método

```

from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor,
    StackingRegressor

# Inicializar los estimadores
clf1 = LinearRegression()
clf2 = RandomForestRegressor(n_estimators = 150, max_depth = 3)
clf3 = SVR(epsilon = 1.5)

# Crear el método ensemble
eclf1 = StackingRegressor(estimators = [
    ('lr', clf1), ('rf', clf2), ('svc', clf3)])

# Entrenar el método
eclf1.fit(X_train, y_train_reg)

# Crear predicciones para el dataset completo
df["prediction"] = ec1f1.predict(np.concatenate((X_train, X_test),
    axis = 0))

# Calcular la estrategia
df["strategy"] = np.sign(df["prediction"]) * df["returns"]

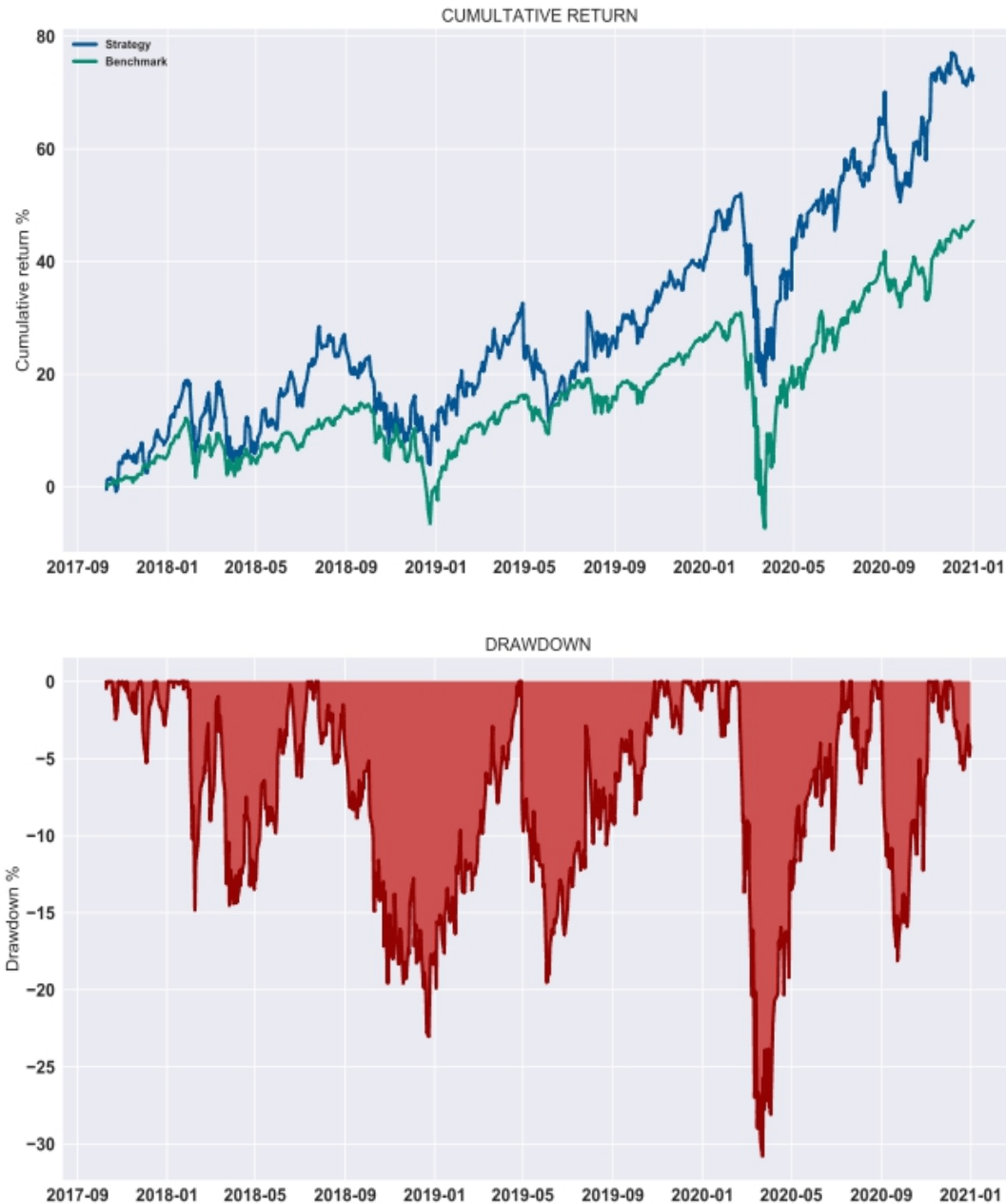
# Backtest
backtest_dynamic_portfolio(df["strategy"].iloc[split:])

```

Figura 12.11: Rendimiento acumulado de un método stacking

Beta: 1.056	Alpha: 7.16 %	Sharpe: 0.754	Sortino: 0.968

VaR: 46.46 %	cVaR: 56.64 %	VaR/cVaR: 1.219	drawdown: 30.79 %



En esta figura, podemos ver el rendimiento acumulado de un regresor stacking.

Como podemos ver, los resultados de los métodos ensemble son los mismos. Significa que no podemos hacer una predicción mejor usando estos datos con este tipo de algoritmo. Para evitar este problema, veremos algo más potente usando deep learning más adelante.

Resumen

- Para encontrar la mejor separación entre grupos, el árbol de decisión recorta el espacio con n hiperplanos (donde n es la profundidad del árbol).
- Un bosque aleatorio es un método ensemble (bagging) que utiliza diferentes árboles de decisión para tener mejores predicciones que un solo árbol de decisión.
- El árbol de decisión y el bosque aleatorio están muy expuestos al sobreajuste. Para evitarlo, tenemos que limitar la profundidad del árbol y el número de árboles para el bosque aleatorio.
- Los métodos voting combinan algunos algoritmos. Proporciona la media de las predicciones de los algoritmos individuales para la regresión y la clase más votada para la clasificación.
- Bagging es un método ensemble que combina el mismo algoritmo n veces (por ejemplo, random forest).
- El stacking es un método ensemble que utiliza un algoritmo con las predicciones de algoritmos en características para encontrar las mejores predicciones.

Aplicación Trading en vivo y Screener

Este código se basa en la clase creada en el anexo: MetaTrader.

Código 12.12: Aplicación Trading / Screener para regresor de árbol de decisión

```
from MT5 import *  
import numpy as np
```

```

import pandas as pd
import warnings
warnings.filterwarnings("ignore")
from sklearn.tree import DecisionTreeRegressor
import time
import pickle
from joblib import dump, load
import os
from sklearn.preprocessing import StandardScaler

path = ""
# Ex: C:/Desktop/Python_for_finance_and_algorithmic_trading/ChapterN/

def create_model_weights(symbol):
    """ Pesos para la regresión lineal sobre el cambio porcentual """
    # Importar los datos
    data = MT5.get_data(symbol, 3500)[["close"]].pct_change(1)

    # Crear una nueva variable
    data.columns = ["returns"]

    # Ingeniería de características
    data["returns t-1"] = data[["returns"]].shift(1)

    # Media de los retornos
    data["mean returns 15"] = data[["returns"]].rolling(15).mean().
        shift(1)
    data["mean returns 60"] = data[["returns"]].rolling(60).mean().
        shift(1)

    # Volatilidad de los retornos
    data["volatility returns 15"] = data[["returns"]].rolling(15).std().

```

```

        shift(1)

data["volatility returns 60"] = data[["returns"]].rolling(60).std().
        shift(1)

# Dividir los datos
data = data.dropna()
split = int(0.80 * len(data))

# Creación del conjunto de entrenamiento
X_train = data[["returns t-1", "mean returns 15", "mean returns 60",
               "volatility returns 15", "volatility returns 60"]].
        iloc[:split]
y_train = data[["returns"]].iloc[:split]

sc = StandardScaler()
X_train = sc.fit_transform(X_train)

# Creación del modelo
alg = DecisionTreeRegressor(max_depth = 6)

# Ajustar el modelo
alg.fit(X_train, y_train)

# Guardar el modelo
alg_var = pickle.dumps(alg)
alg_pickel = pickle.loads(alg_var)

dump(alg_pickel ,os.path.join(path, f"Models/{symbol}_reg.joblib"))

def tree_reg_sig(symbol):
    """
    Función para predecir el valor de mañana utilizando el modelo ARIMA
    """

```

```
# Crear los pesos si no los hay en la carpeta
```

```
try:
```

```
    alg = load(os.path.join(path, f"Models/{symbol}_reg.joblib"))
```

```
except:
```

```
    create_model_weights(symbol)
```

```
    alg = load(os.path.join(path, f"Models/{symbol}_reg.joblib"))
```

```
# Tomar el último porcentaje de cambio
```

```
data = MT5.get_data(symbol, 3500)[["close"]].pct_change(1)
```

```
# Crear una nueva variable
```

```
data.columns = ["returns"]
```

```
# Ingeniería de características
```

```
data["returns t-1"] = data["returns"].shift(1)
```

```
# Media de los retornos
```

```
data["mean returns 15"] = data["returns"].rolling(15).mean().  
    shift(1)
```

```
data["mean returns 60"] = data["returns"].rolling(60).mean().  
    shift(1)
```

```
# Volatilidad de los retornos
```

```
data["volatility returns 15"] = data["returns"].rolling(15).std().  
    shift(1)
```

```
data["volatility returns 60"] = data["returns"].rolling(60).std().  
    shift(1)
```

```
X = data["returns t-1", "mean returns 15", "mean returns 60",  
        "volatility returns 15", "volatility returns 60"]
```

```
iloc[-1:,:].values
```



```

# Encontrar la señal
prediction = alg.predict(X)
buy = prediction[0] > 0
sell = not buy

return buy, sell

# True = Trading en vivo y False = Screener
live = True

if live:
    current_account_info = mt5.account_info()
    print("-----")
    print("Date: ", datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
    print(f"Balance: {current_account_info.balance} USD, \t"
          f"Equity: {current_account_info.equity} USD, \t"
          f"Profit: {current_account_info.profit} USD")
    print("-----")

info_order = {
    "Google": ["Alphabet_Inc_C_(GOOG.O).a", 1.00]
}

start = datetime.now().strftime("%H:%M:%S") #"23:59:59"

while True:

    # Verificación para lanzamiento
    if datetime.now().weekday() not in (5, 6):
        is_time = datetime.now().strftime("%H:%M:%S") == start
    else:
        is_time = False

    # Lanzar el algoritmo

```

```

if is_time:

    # Abrir las transacciones
    for asset in info_order.keys():

        # Inicializar los inputs
        symbol = info_order[asset][0]
        lot = info_order[asset][1]

        # Crear las señales
        buy, sell = tree_reg_sig(symbol)

        # Ejecutar el algoritmo
        if live:
            MT5.run(symbol, buy, sell, lot)

        else:
            print(f"Symbol: {symbol}\n"
                  f"Buy: {buy}\n"
                  f"Sell: {sell}")
            time.sleep(1)

```



El parámetro live establece el modo trading en vivo (live = True) o el modo screener (live = False).

Capítulo 13: Red neuronal profunda (DNN)

En este capítulo, veremos nuestro primer algoritmo para el aprendizaje profundo. Hablaremos de la red neuronal profunda (DNN), o red neuronal artificial (ANN). El aprendizaje profundo es un campo del aprendizaje automático que utiliza los algoritmos más potentes. Sin embargo, exige muchos recursos para ejecutarse. Para explicar la DNN y crear una estrategia de trading utilizando este algoritmo, explicaremos la intuición que hay detrás de la DNN y cómo crear un clasificador DNN y un regresor DNN. Además, después de este capítulo, seremos capaces de crear nuestra función de pérdida. Implementaremos los algoritmos en el precio de las acciones de Apple.

13.1. Intuición tras la DNN

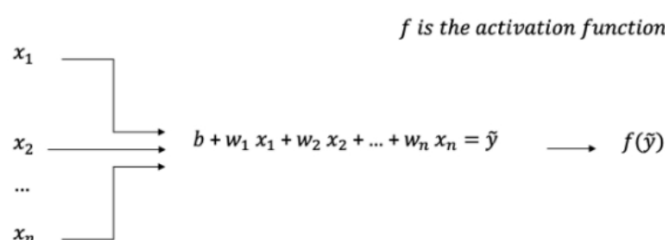
En esta parte hablaremos de la intuición que hay detrás de las redes neuronales profundas o DNN. Para ello, hablaremos de la propagación hacia delante de una red neuronal, luego del gradiente descendente y, por último, de la propagación hacia atrás.

13.1.1. Propagación hacia delante

En esta subsección, aprenderemos mucho sobre la propagación de la predicción, que es el proceso utilizado por el algoritmo para crear una predicción. Lo esencial en este proceso es la neurona. De hecho, es una parte esencial para entender el concepto de propagación de reenvío. Para empezar, explicamos cómo funciona una neurona.

Una neurona es como una regresión lineal. Tiene una entrada llamada X y un conjunto de parámetros para predecir una salida y . Los parámetros se llaman pesos en el aprendizaje profundo, y el intercepto (β_0 en la regresión lineal) se llama sesgo. La única diferencia entre la neurona y una regresión lineal es que ponemos una función de activación (hablaremos de las funciones de activación más adelante). Veamos el proceso detrás de una neurona en la figura 13.1.

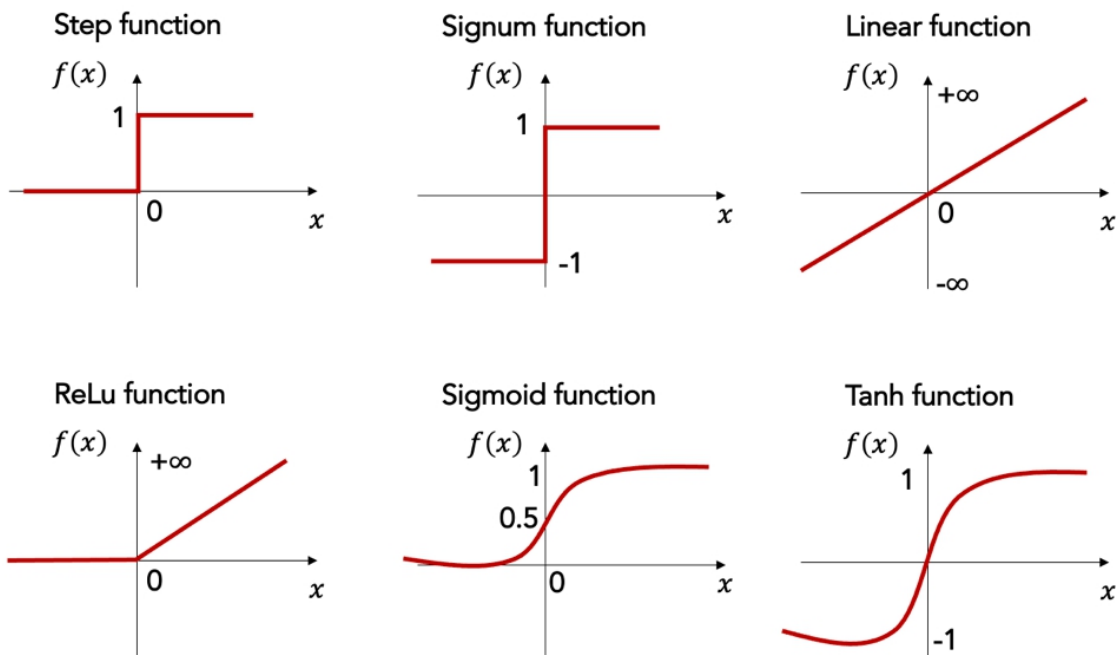
Figura 13.1: Proceso de una neurona



En esta figura, podemos ver el proceso que hay detrás de una neurona en una red neuronal profunda.

Hablemos ahora de la función de activación. Normalmente, es sólo una forma de contener el valor de la salida. Sin embargo, explicaremos más adelante que en la capa oculta, la elección de la función de activación no es "realmente" esencial, pero es crucial para la capa de salida. Veamos las distintas funciones de activación en la figura 13.2 para entender mejor la noción.

Figura 13.2: Algunas funciones de activación

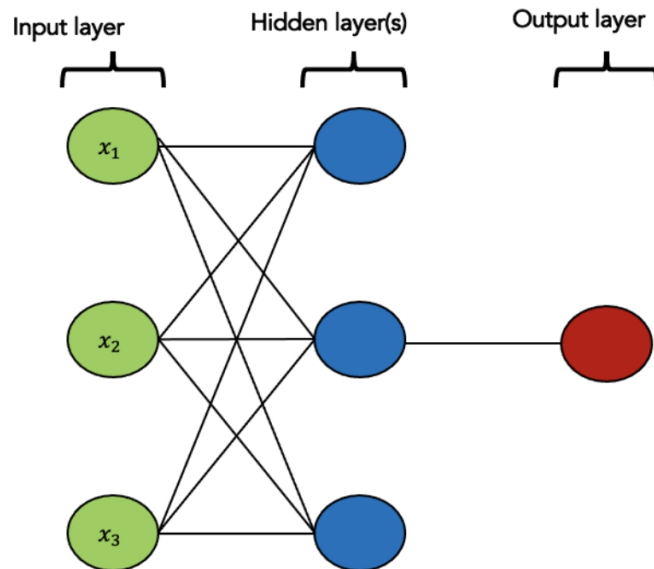


Esta figura muestra algunas funciones de activación, mientras que las más utilizadas son la función ReLu, la lineal y la sigmoidea.

Ahora, ya conocemos casi toda la intuición que hay detrás de la propagación de reenvío. Lo único que necesitamos saber es cómo combinar neuronas para crear una red neuronal.

La construcción de una red neuronal es sencilla. En la figura 13.1, hemos creado una neurona con entradas y salidas. Debemos conectar algunas neuronas para crear una red neuronal, como en la figura 13.3. Así, la salida de la primera neurona se convierte en la entrada de la siguiente y así sucesivamente hasta la neurona de salida.

Figura 13.3: Red neuronal profunda



En esta figura podemos ver el funcionamiento de una DNN. Vemos la capa de entrada con los datos, la capa oculta y la capa de salida.



Como se puede ver, la interpretabilidad del modelo es difícil porque hay mucho cálculo y peso. Por lo tanto, a algunas personas de finanzas no les gustan las DNN porque quieren saber en qué criterios invertirán. Sin embargo, a la mayoría no le importa.

Por lo tanto, la propagación de reenvío es tener los datos introducidos en la red neuronal y dar una salida, la predicción. Sin embargo, este proceso no entrena nuestro algoritmo. Así que, en las siguientes partes, veremos cómo entrenar la DNN.

13.1.2. Gradiente descendente

En esta parte, aprenderemos qué es el gradiente descendente. El gradiente descendente es el algoritmo más popular en el aprendizaje automático para optimizar el peso de un algoritmo. Es una forma de resolver un problema de optimización.

El gradiente descendente es un algoritmo basado en el gradiente de la función que se quiere optimizar. Este algoritmo es matemáticamente fácil de entender, pero necesitamos utilizar algunas matemáticas muy complejas para crearlo. Por lo tanto, aquí sólo veremos la intuición. Sin embargo, si queremos profundizar en el tema, podemos encontrar mucha documentación al respecto en internet.

Tomemos un ejemplo sencillo para entender cómo funciona el gradient descendente. Estamos en una montaña y hay una gran niebla. Queremos bajar de la montaña pero no vemos nada.

Para hacerlo, buscaremos sólo con los pies el camino con más pendiente descendente, y nos dirigimos en esta dirección durante algunos metros; continuaremos este proceso hasta que estemos abajo.

Es precisamente lo que hace el algoritmo. Cuando queremos comprobar qué camino tiene más pendiente descendente, el algoritmo utiliza el gradiente más negativo a medida que utilizamos nuestros pies para encontrar la pendiente más descendente. La elección de la tasa de aprendizaje es esencial. Veamos en la figura 13.4 la importancia de la tasa de aprendizaje.

Figura 13.4: Elección de la tasa de aprendizaje



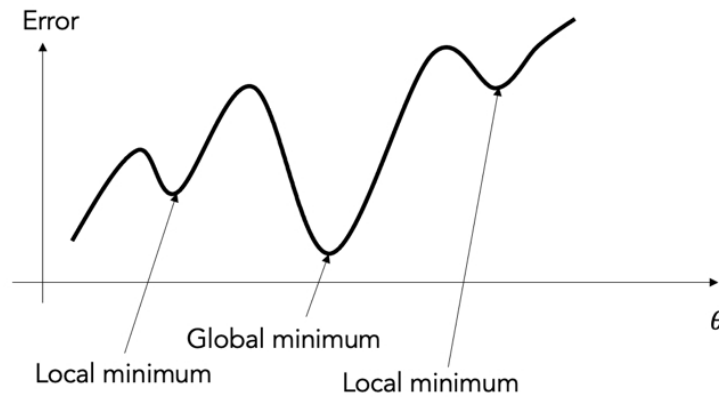
En esta figura, podemos ver la importancia del aprendizaje en el proceso de aprendizaje.



En el aprendizaje profundo, la letra θ representa el peso de todas las neuronas de la red. Son los parámetros de los modelos. Por lo tanto, si cambiamos θ , cambiamos algunos o todos los pesos y sesgos de las neuronas.

El único problema del gradiente descendente es que puede caer en un mínimo local. Para esquematizar el mínimo local y global, podemos pensar en un buscador de oro en una cueva. Hay un paquete de 1 kg de oro (el mínimo local) y sólo un paquete de 100 kg (el mínimo global). Supongamos que no sabemos que hay un paquete en la cueva. Hay muchas posibilidades de encontrar un paquete de 1 kg y salir de la cueva sin el paquete de 100 kg; este problema depende de la función de coste. Normalmente, para una tarea de regresión, utilizamos el MSE, que es convexo, por lo que no hay problema. Sin embargo, el algoritmo puede caer en un mínimo local si creamos nuestra función de pérdida como en la siguiente parte.

Figura 13.5: Mínimo gloal VS mínimo local



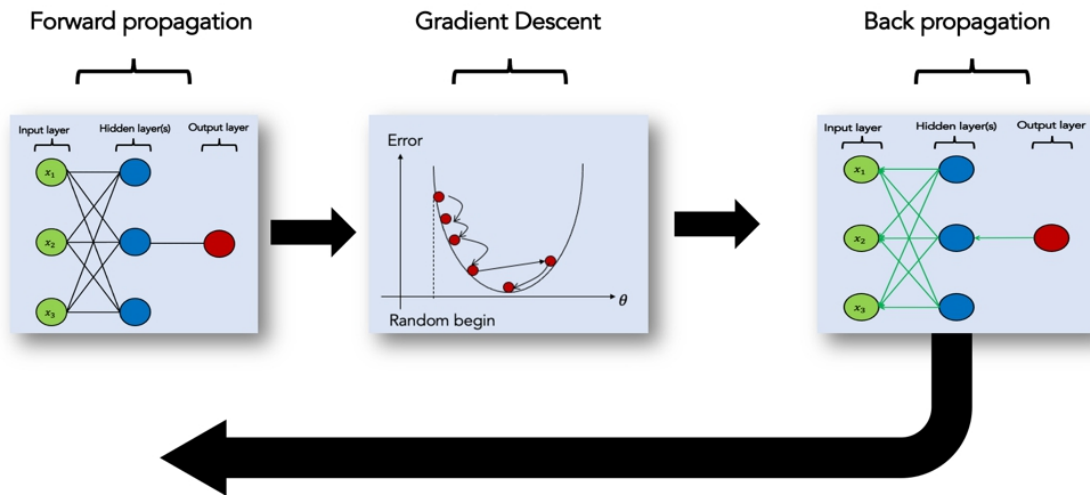
Esta figura muestra que si la función no es estrictamente convexa, el algoritmo puede caer en un mínimo local aunque exista un mínimo global en otro lugar.

Para resolver este problema, podemos utilizar el gradiente descendente estocástico u otra de estas derivadas.

13.1.3. Propagación hacia atrás

Hablaremos de la propagación hacia atrás. En este paso, sabemos todo lo que debemos hacer para explicar la propagación hacia atrás. Necesitamos ensamblar las partes.

Figura 13.6: Entrenamiento de una DNN



Después de ver esta figura, podemos entender la definición de propagación hacia atrás: cambiar el peso después de cada iteración según el gradiente descendente para optimizar el algoritmo y hacer mejores predicciones en el futuro.

13.2. DNN para clasificación

Esta sección explicará cómo hacer correctamente una DNN para una tarea de clasificación utilizando Python. Prepararemos los datos y destacaremos algunos puntos que hay que conocer. Luego implementaremos la DNN y crearemos una estrategia con la predicción.

13.3.1. Preparación de los datos

En esta subsección hablaremos de la preparación de los datos. Al igual que en los algoritmos anteriores, los datos se dividirán en conjuntos de entrenamiento y de test. Trabajamos con el precio de las acciones de Apple en un marco temporal diario.

Código 13.1: Preparación de los datos

```
# Importar los datos
df = yf.download("AAPL", end = "2021-01-01")[["Adj Close"]].
```

```

    pct_change(1)
df.columns = ["returns"]

# Ingeniería de características
df["returns t-1"] = df[["returns"]].shift(1)

# Media de los rendimientos
df["mean returns 15"] = df[["returns"]].rolling(15).mean().shift(1)
df["mean returns 60"] = df[["returns"]].rolling(60).mean().shift(1)

# Volatilidad de los rendimientos
df["volatility returns 15"] =
    df[["returns"]].rolling(15).std().shift(1)
df["volatility returns 60"] =
    df[["returns"]].rolling(60).std().shift(1)

# Eliminar los valores faltantes
df = df.dropna()

# Porcentaje del conjunto de entrenamiento
split = int(0.80 * len(df))

# Creación del conjunto de entrenamiento
X_train = df[["returns t-1", "mean returns 15", "mean returns 60",
    "volatility returns 15", "volatility returns 60"]].
    iloc[:split]

y_train_reg = df[["returns"]].iloc[:split]

y_train_cla = np.round(df[["returns"]].iloc[:split] + 0.5)

# Creación del conjunto de test

```

```

X_test = df[["returns t-1", "mean returns 15", "mean returns 60",
            "volatility returns 15", "volatility returns 60"]].
            iloc[split:]

y_test_reg = df[["returns"]].iloc[split:]
y_test_cla = np.round(df[["returns"]].iloc[split:] + 0.5)

# NORMALIZACIÓN
# Importar la clase
from sklearn.preprocessing import StandardScaler

# Inicializar la clase
sc = StandardScaler()

# Estandarizar los datos
X_train_scaled = sc.fit_transform(X_train)
X_test_scaled = sc.transform(X_test)

```

No explicamos esta parte porque está codificada desde los capítulos anteriores. Al mismo tiempo, es fundamental normalizar nuestros datos cuando trabajamos con una red neuronal porque es un algoritmo muy complejo.

13.2.2. Implementación de una DNN para una tarea de clasificación

En esta parte, implementaremos una DNN para una tarea de clasificación. Utilizaremos TensorFlow para crear el modelo. Los algoritmos de aprendizaje profundo exigen un poco de comprensión para crearlos. Por lo tanto, asegúrate de entender los capítulos anteriores antes de empezar.

Crear un modelo con TensorFlow nos da muchas posibilidades. Sin embargo, por el contrario, tenemos que entender lo que queremos hacer en lugar del algoritmo scikit-learn en el que tenemos que poner los datos. Sin embargo, no

te preocupes, vamos a explicar paso a paso la construcción los diferentes pasos de la DNN.

Código 13.2: Red neuronal profunda para tareas de clasificación

```
# Librerías
import tensorflow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

nb_hidden_layer = 1 1

# Inicialización del modelo secuencial
classifier = Sequential() 2

# Añadir las capas ocultas
for _ in range(nb_hidden_layer): 3
    classifier.add(Dense(75, input_shape = (X_train.shape[1], ),
                        activation = "relu")) 4

# Capa de salida densa
classifier.add(Dense(1, activation = "sigmoid")) 5

# Compilar el modelo
classifier.compile(loss = "binary_crossentropy",
                  optimizer = "adam") 6

# Entrenamiento
classifier.fit(X_train_scaled, y_train_cla, epochs = 15,
              batch_size = 32, verbose = 1) 7
```

1	Define el número de capas ocultas.
---	------------------------------------

2	Inicializa la red neuronal.
3	Crear un bucle que añade las capas ocultas.
4	<code>input_shape = (X_train.shape[1],)</code> solo es necesario para la primera capa.
5	Es necesario poner una sigmoide a la activación porque es un clasificador.
6	Compilación del modelo utilizando el gradiente estocástico y la entropía cruzada binaria como función de pérdida.
7	Ajustar el modelo usando solamente <code>verbose=1</code> para destacar el entrenamiento del modelo.

Ahora, vamos a explicar este código. En primer lugar, tenemos que inicializar el modelo como hacíamos para los modelos de scikit-learn. Después de eso, sólo tenemos un modelo vacío, tenemos que construirlo. El bucle nos permite crear tantas capas ocultas como queramos. La función Dense de TensorFlow crea una capa de redes neuronales. Tenemos que ser precisos sobre cuántas neuronas queremos, la forma de las entradas y la función de activación.

Luego tenemos que definir una capa particular para la salida porque tenemos que predecir la clase. Así, queremos predecir un 0 o un 1. Elegimos la sigmoidea para activar esta neurona porque el valor estará entre 0 y 1, y queremos una probabilidad.

Luego tenemos que explicar cómo queremos entrenar el algoritmo. Elegimos la función de entropía cruzada y un gradiente descendente estocástico llamado "adam". Para ajustar el modelo, debemos elegir el número de epochs y el batch_size.



Podemos poner callbacks en la función de ajuste para detener el algoritmo a un cierto nivel de pérdida de la

función, pero esto no es relevante para este libro. Puedes leer la documentación de TensorFlow para más detalles.

13.2.3. Predicción y Backtest

En esta parte, vamos a analizar el rendimiento de una estrategia creada utilizando una DNN. Vamos a utilizar precisamente el mismo código que en el backtest anterior.

Código 13.3: Cálculo de la estrategia de retorno utilizando las predicciones de la DNN

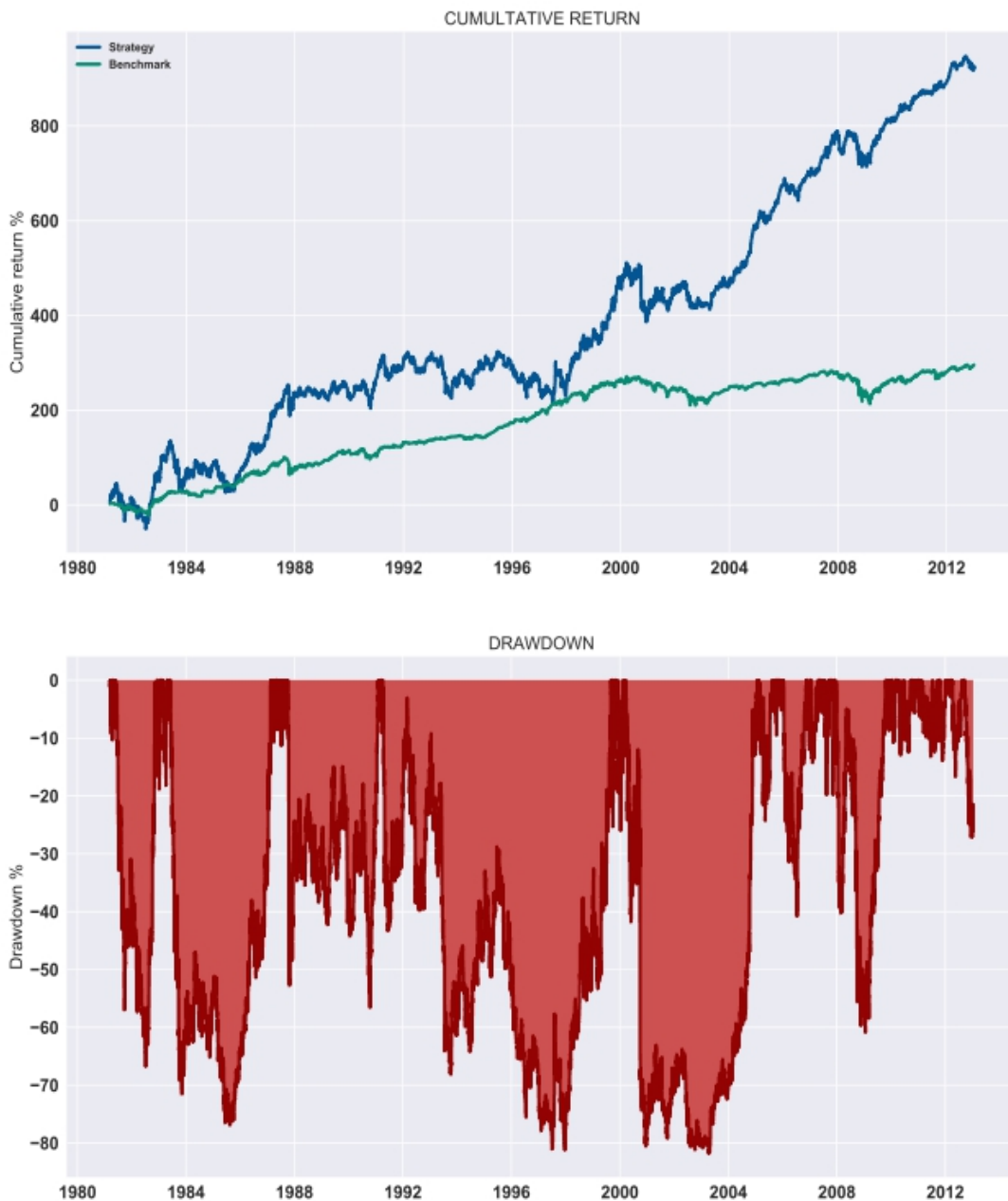
```
# Crear predicciones para el dataset completo
df["prediction"] =
    classifier.predict(np.concatenate((X_train, X_test),
                                     axis = 0))
df["prediction"] = np.where(df["prediction"] == 0, -1, 1)
# Calcular la estrategia
df["strategy"] = np.sign(df["prediction"]) * df["returns"]

# Backtest
backtest_dynamic_portfolio(df["strategy"].iloc[split:])
```

Con este código, podemos ver el backtest de una estrategia utilizando el clasificador DNN sobre acciones de Apple. Como podemos ver, hay un excelente rendimiento ya que ganamos más de un 30% anual. En comparación, tenemos que considerar que no hemos integrado las tasas.

Figura 13.7: Backtest de una estrategia de predicción DNN sobre las acciones de Apple

Beta: 1.23	Alpha: 17.49 %	Sharpe: 0.594	Sortino: 0.841
VaR: 84.73 %	cVaR: 100.5 %	VaR/cVaR: 1.186	drawdown: 81.8 %



Podemos ver una buena tendencia en el crecimiento del capital utilizando esta estrategia.

13.3. DNN para regresión

En esta sección, aprenderemos a crear una DNN para una tarea de regresión y a crear nuestra función de pérdida. A continuación, haremos un backtest de la estrategia.

13.3.1. Implementación de una DNN para una tarea de regresión

En esta parte, crearemos una DNN para una tarea de regresión. Es muy similar al clasificador, pero hay algunas sutilezas. Sólo tenemos que cambiar la función de pérdida para optimizar la activación de la neurona de salida. Veamos el código y luego lo explicaremos con más detalle.

Código 13.4: Aplicación del regresor DNN

```
nb_hidden_layer = 1

# Inicialización del modelo secuencial
regressor = Sequential()

# Añadir las capas ocultas
for _ in range(nb_hidden_layer):
    regressor.add(Dense(75, input_shape = (X_train.shape[1],),
                        activation = "relu"))

# Capa de salida densa
regressor.add(Dense(1, activation = "linear"))

# Compilar el modelo
regressor.compile(loss = "mse", optimizer = "adam")

# Entrenamiento
regressor.fit(X_train_scaled, y_train_reg, epochs = 15,
              batch_size = 32, verbose = 1)
```



Aquí la función de activación de la neurona de salida es esencial. Por ejemplo, si ponemos una sigmoidea, el algoritmo predice sólo un retorno positivo, lo que es imposible.

Hemos elegido entrenar el modelo utilizando el error cuadrático medio (MSE) como función de pérdida.



Si nos iniciamos en el aprendizaje profundo, se aconseja mantener la función de activación lineal para la salida de un regresor. Si entendemos bien el concepto, podemos probar con una función tanh u otras como las vistas anteriormente.

13.3.2. Función de pérdida personalizada

En la última parte hemos visto cómo entrenar el modelo con una métrica habitual. Al mismo tiempo, podemos crear nuestras métricas si queremos. La pregunta es, ¿por qué queremos hacerlo? Vamos a explicar algunas limitaciones:

- Invertimos lo mismo si el algoritmo predice un 15% o un -0.000001%.
- El error es casi el mismo para el MSE entre 0.000001 o -0.000001.

Por tanto, debemos incorporar en nuestra función de pérdida que si la predicción es errónea en signo, añadimos un malus. Necesitamos calcular la siguiente ecuación en Tensorflow.

$$\text{MSE}_{\alpha} = \begin{cases} (y - \tilde{y})^2 & \text{if } y * \tilde{y} > 0 \\ \alpha(y - \tilde{y})^2 & \text{if } y * \tilde{y} < 0 \end{cases}$$

Donde $\tilde{y}$ es la predicción, y es el valor real y $\alpha > 1$.

Código 13.5: Función MSE personalizada

```
def ALPHA_MSE(y_true, y_pred): 1
    y_true_roll = tf.roll(y_true, shift = 1, axis = 0) 2
    y_pred_roll = tf.roll(y_pred, shift = 1, axis = 0)
```

```

y_true_dif = tf.math.sign(y_true_roll - y_true) 3
y_pred_dif = tf.math.sign(y_pred_roll - y_pred)
booleen_vector = y_true_dif == y_pred_dif 4
alpha = tf.where(booleen_vector, 1, 3) 5
alpha = tf.cast(alpha, dtype = tf.float32) 6
mse = F.square(y_true - y_pred)
mse = tf.cast(mse, dtype = tf.float32)
scale_mse = tf.multiply(alpha, mse)
alpha_mse = F.mean(scale_mse)
return alpha_mse

```

- 1 La función de pérdida personalizada debe tener dos parámetros *y_true* e *y_pred*, que son tensores. Por lo tanto, todos los cálculos deben utilizar funciones de TensorFlow.
- 2 Crear una nueva variable que contenga el desplazamiento del objetivo en 1.
- 3 Tomar el signo de la variación diaria.
- 4 Crear un tensor booleano en el que cada valor es True si *y_true* e *y_pred* varían de la misma manera y False en caso contrario.
- 5 Crear un multiplicador, que es igual a 1 si los dos vectores varían de la misma manera y 3 para penalizar el tamaño incorrecto.
- 6 Transformar un tensor en un vector constante.



Esta parte no es obligatoria para el libro, pero debes saber que crear una función de pérdida personalizada es posible. Puedes consultar la documentación de TensorFlow si quieres profundizar en el tema.

Ten en cuenta que personalizar una función de pérdida y crear una métrica con scikit-learn son dos cosas distintas;

no las confundas. Con las métricas personalizadas de scikit-learn, entrenamos un modelo usando un MSE u otra función esencial y tomamos los mejores modelos siguiendo las métricas creadas. Sin embargo, la función de pérdida personalizada en TensorFlow es la función que calcula el error del modelo (como MSE en scikit-learn).

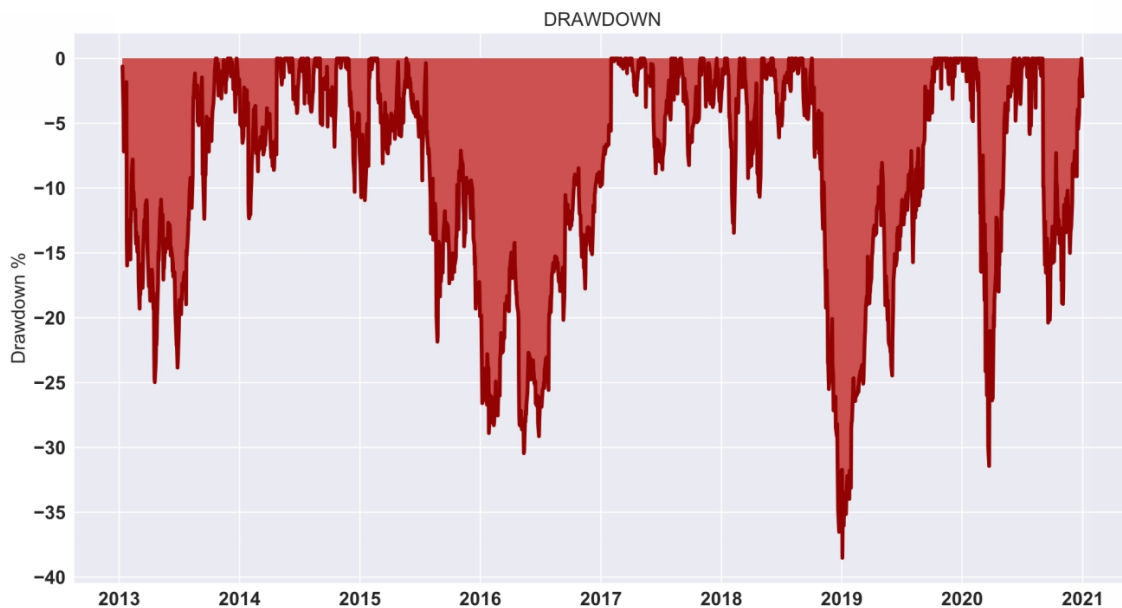
13.3.3. Predicción y Backtest

En esta parte, haremos un backtest de la estrategia creada usando la función de pérdida personalizada y la estrategia usando el entrenamiento DNN con pérdida MSE.

Como podemos ver en la figura y como hemos dicho en la última parte, la función de pérdida personalizada no es para principiantes. De hecho, la estrategia de pérdida personalizada es Buena, pero no mejor que la estrategia usando el MSE.

Figura 13.8: Backtest de la estrategia utilizando MSE y MSE alfa





MSE

Beta: 1.127

Alpha: 15.55 %

Sharpe: 1.065

Sortino: 1.396

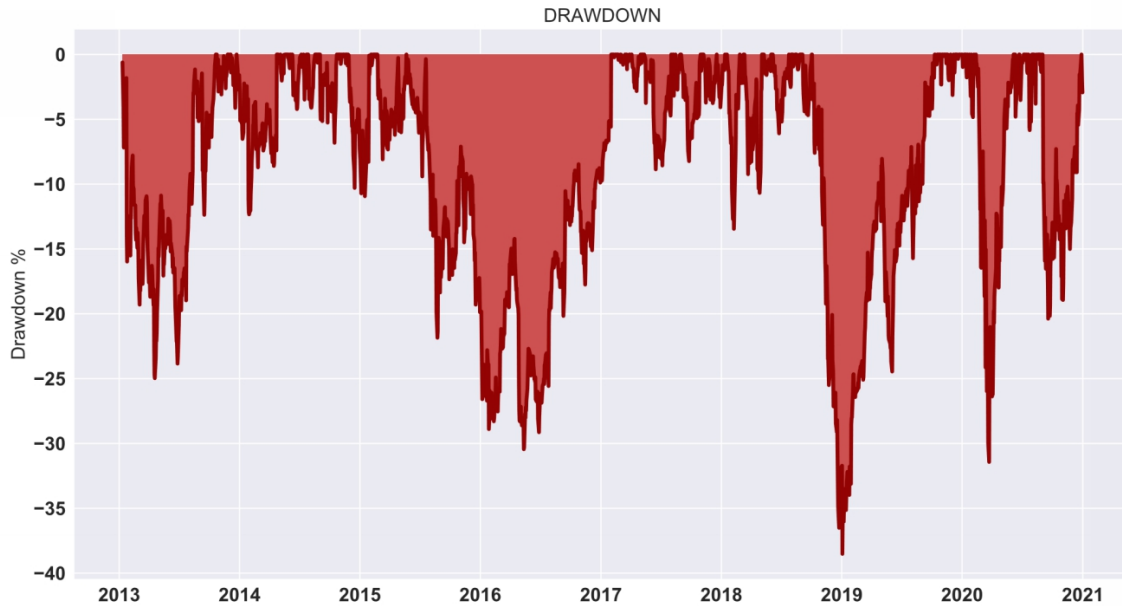
VaR: 36.33 %

cVaR: 45.6 %

VaR/cVaR: 1.255

drawdown: 38.52 %





Este gráfico muestra que la función de pérdida personalizada no influye en el rendimiento, ya que las dos pruebas retrospectivas son similares.

Con la DNN, tenemos excelentes resultados, pero la DNN no tiene en cuenta que trabajamos con una serie temporal.

Resumen

- El proceso detrás de una neurona es como la regresión lineal; en vez de poner la y en una función de activación.
- Existen muchas funciones de activación. Sin embargo, las más utilizadas son la función ReLu y la sigmoidea. La elección de la función es crucial en algunas situaciones.
- Lo esencial en el gradiente descendente es la tasa de aprendizaje. Además, utilizamos el gradiente descendente estocástico para evitar caer en un mínimo local.

- La propagación hacia atrás es el momento en el que cambiamos los pesos del modelo. Por lo tanto, sin esta parte, no podemos entrenar nuestro modelo.
- Podemos personalizar una función de pérdida. Sin embargo, se necesitan muchos conocimientos en aprendizaje profundo para hacerlo correctamente. Por lo tanto, si no nos sentimos cómodos con las nociones de los libros, no deberíamos entrar en este tema.

Aplicación Trading en vivo y Screener

Este código se basa en la clase creada en el anexo: MetaTrader.

Código 13.6: Aplicación Trading / Regresor ANN

```
from MT5 import *
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
import time
import pickle
from joblib import dump, load

import os
from sklearn.preprocessing import StandardScaler
import tensorflow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

path = ""
# Ex: C:/Desktop/Python_for_finance_and_algorithmic_trading/ChapterN/

def ANN():
```

```
# Crear el modelo
```

```
nb_hidden_layer = 1
```

```
# Inicialización del modelo secuencial
```

```
alg = Sequential()
```

```
# Añadir las capas ocultas
```

```
for _ in range(nb_hidden_layer):
```

```
    alg.add(Dense(75, input_shape = (5,), activation = "relu"))
```

```
# Capa de salida densa
```

```
alg.add(Dense(1, activation = "linear"))
```

```
# Compilar el modelo
```

```
alg.compile(loss = "mse", optimizer = "adam")
```

```
return alg
```

```
def create_model_weights(symbol):
```

```
    """ Pesos para la regresión lineal sobre el cambio porcentual """
```

```
    # Importar los datos
```

```
    data = MT5.get_data(symbol, 3500)[["close"]].pct_change(1)
```

```
    # Crear una nueva variable
```

```
    data.columns = ["returns"]
```

```
    # Ingeniería de características
```

```
    data["returns t-1"] = data[["returns"]].shift(1)
```

```
    # Media de los rendimientos
```

```
    data["mean returns 15"] = data[["returns"]].rolling(15).mean().  
        shift(1)
```

```
    data["mean returns 60"] = data[["returns"]].rolling(60).mean().  
        shift(1)
```



```
# Volatilidad de los rendimientos
```

```
data["volatility returns 15"] = data[["returns"]].rolling(15).std().  
    shift(1)
```

```
data["volatility returns 60"] = data[["returns"]].rolling(60).std().  
    shift(1)
```

```
# Dividir los datos
```

```
data = data.dropna()  
split = int(0.80 * len(data))
```

```
# Creación del conjunto de entrenamiento
```

```
X_train = data[["returns t-1", "mean returns 15", "mean returns 60",  
    "volatility returns 15", "volatility returns 60"]].  
    iloc[:split]  
y_train = data[["returns"]].iloc[:split]
```

```
# Inicializar la clase
```

```
sc = StandardScaler()
```

```
# Estandarizar los datos
```

```
X_train = sc.fit_transform(X_train)  
alg = ANN()
```

```
# Entrenamiento
```

```
alg.fit(X_train, y_train, epochs = 13, batch_size = 32, verbose = 1)
```

```
# Guardar el modelo
```

```
alg.save_weights(os.path.join(path, f"Models/ANN_reg_{symbol}"))
```

```
def ANN_cla_sig(symbol):
```

```
    """ Function for predict the value of tommorow using ARIMA model """
```

```
# Crear los pesos si no existen en la carpeta
```

```

try:
    alg = ANN()
    alg.load_weights(os.path.join(path, f"Models/ANN_reg_{symbol}"))
except:
    create_model_weights(symbol)
    alg = ANN()
    alg.load_weights(os.path.join(path, f"Models/ANN_reg_{symbol}"))

# Tomar el último porcentaje de cambio
data = MT5.get_data(symbol, 3500)[["close"]].pct_change(1)

# Crear una nueva variable
data.columns = ["returns"]

# Ingeniería de características
data["returns t-1"] = data[["returns"]].shift(1)

# Media de los rendimientos
data["mean returns 15"] = data[["returns"]].rolling(15).mean().
    shift(1)
data["mean returns 60"] = data[["returns"]].rolling(60).mean().
    shift(1)

# Volatilidad de los rendimientos
data["volatility returns 15"] = data[["returns"]].rolling(15).std().
    shift(1)
data["volatility returns 60"] = data[["returns"]].rolling(60).std().
    shift(1)

X = data[["returns t-1", "mean returns 15", "mean returns 60",
    "volatility returns 15", "volatility returns 60"]].
    iloc[-1:,:].values

```

```
# Encontrar la señal
```

```
prediction = alg.predict(X)
```

```
prediction = np.where(prediction == 0, -1, 1)
```

```
buy = prediction[0][0] > 0
```

```
sell = not buy
```

```
return buy, sell
```

```
# True = Trading en vivo y False = Screener
```

```
live = True
```

```
if live:
```

```
    current_account_info = mt5.account_info()
```

```
    print("-----")
```

```
    print("Date: ", datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
```

```
    print(f"Balance: {current_account_info.balance} USD, \t"
```

```
          f"Equity: {current_account_info.equity} USD, \t"
```

```
          f"Profit: {current_account_info.profit} USD")
```

```
    print("-----")
```

```
info_order = {
```

```
    "Apple": ["AAPL.a", 1.00]
```

```
}
```

```
start = datetime.now().strftime("%H:%M:%S")#"23:59:59"
```

```
while True:
```

```
    # Verificación para lanzamiento
```

```
    if datetime.now().weekday() not in (5, 1):
```

```
        is_time = datetime.now().strftime("%H:%M:%S") == start
```

```
    else:
```

```
        is_time = False
```

```
# Lanzar el algoritmo

if is_time:

    # Abrir las transacciones
    for asset in info_order.keys():

        # Inicializar los inputs
        symbol = info_order[asset][0]
        lot = info_order[asset][1]

        # Crear las señales
        buy, sell = ANN_cla_sig(symbol)

        # Ejecutar el algoritmo
        if live:
            MT5.run(symbol, buy, sell, lot)

        else:
            print(f"Symbol: {symbol}\n"
                  f"Buy: {buy}\n"
                  f"Sell: {sell}")

time.sleep(1)
```



El parámetro live establece el modo trading en vivo (live = True) o el modo screener (live = False).

Capítulo 14: Red neuronal recurrente (RNN)

En este capítulo hablaremos de las redes neuronales recurrentes (RNN). Este algoritmo especializado en series temporales puede ser un valioso aliado a la hora de crear una estrategia de trading porque los precios de los activos son series temporales. Además, es uno de los algoritmos de deep learning más complejos. Implementaremos las estrategias utilizando las acciones de Netflix.

14.1 Principios de la RNN

En esta sección aprenderemos cómo funciona una RNN. Aquí veremos las bases de la RNN, pero puedes consultar algunos artículos de internet o [cursos de la ruta de Inteligencia Artificial de Frogames Formación](#) si quieres profundizar en el tema.

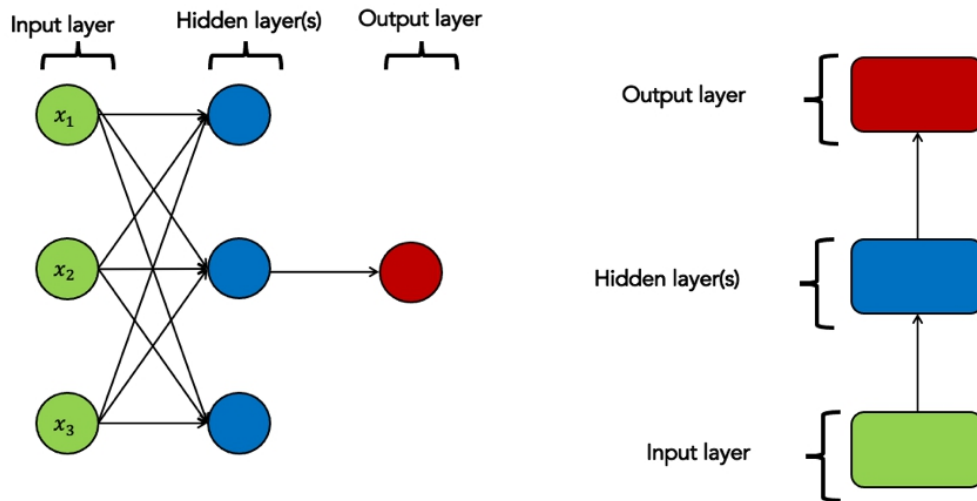
14.1.1. Cómo funciona una RNN

En esta parte se analizará la intuición que subyace a la RNN. Los modelos RNN están especializados en el modelado de series temporales. De hecho, la estructura de la red es ligeramente diferente de la ANN porque las capas ocultas están interconectadas.

La interconexión entre las capas ocultas permite al modelo "recordar" el pasado. En la RNN, hay dos tipos de memoria en lugar de la ANN. La RNN tiene su memoria a largo plazo pero también una memoria a corto plazo creada por la interconexión de la que hablamos anteriormente.

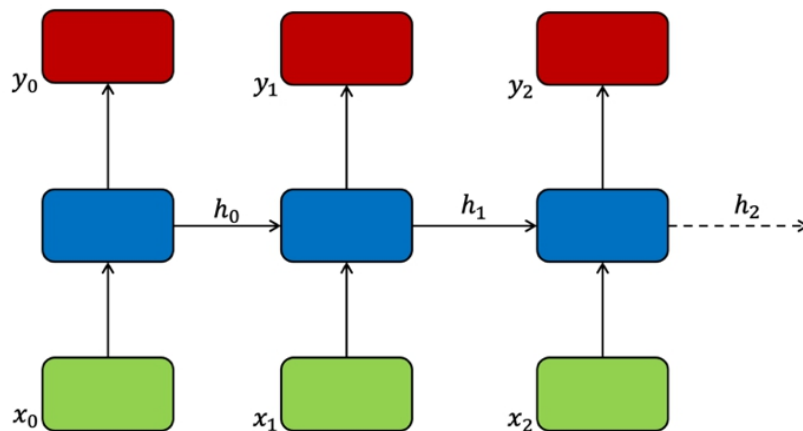
Necesitamos una pequeña representación para entender mejor cómo funciona una RNN. Para ello, necesitamos simplificar la representación de la ANN. Esquematizaremos la ANN sólo con su capa y no todas las neuronas. Esto nos permite visualizar el modelo de forma sencilla para crear la representación de la RNN más fácilmente. Veamos la figura 14.1.

Figura 14.1: Nueva esquematización de la ANN



Esta figura muestra la representación de una ANN utilizando sólo la capa con el cuadrado en lugar de la izquierda con todas las neuronas. Sin embargo, es la misma ANN.

Figura 14.2: Esquematización de una RNN



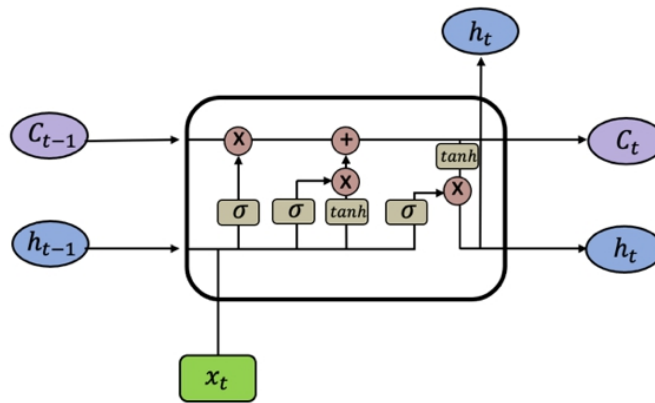
Como vemos, la RNN no es otra cosa que una ANN conectada. Vemos que la información de la capa oculta se da directamente a la siguiente capa oculta, y no se integra sólo con los pesos de los modelos.

El problema con la RNN es que el modelo puede tener algunos problemas con el gradiente, ya que hay muchas más derivaciones que en el caso de la ANN, lo que puede causar problemas de gradiente evanescente. Al mismo tiempo, los investigadores han encontrado una solución a este problema. Han creado nuevas neuronas denominadas neuronas de memoria a largo plazo (LSTM). Es ligeramente diferente de la estructura de una neurona que vimos en la última sección, por lo que tenemos que estudiarla.

14.1.2. Neurona LSTM

¿Cómo funciona una célula LSTM? Naturalmente, no se trata de un curso de aprendizaje profundo. Así que, si quieres entender mejor la célula LSTM, tienes que ir a internet porque este tema es irrelevante para este libro. Veamos cómo es una célula LSTM. A continuación, explicaremos la diferencia entre las células densas y las células LSTM.

Figura 14.3: Célula LSTM



Esta figura muestra el funcionamiento de una célula LSTM en la que la memoria en el tiempo t es C_t , y la información en el tiempo t es h_t . Así, podemos decir que hay una memoria a largo plazo y otra a corto plazo.

Para entender mejor la célula LSTM, pondremos un pequeño ejemplo. Supongamos que tenemos un algoritmo que predice la palabra siguiente, por ejemplo. Necesitamos C , que es la frase, y h , que es algún adjetivo, y x , la palabra anterior.

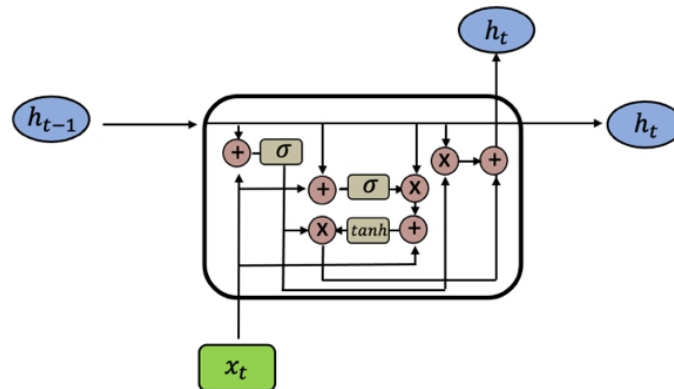
La sigma es la función de activación sigmoidea. Funciona como una válvula de fontanería: dejará pasar o no la información utilizando la propiedad de la función sigmoidea.

Así que no vamos más allá. Es comprensible que no se entienda esta noción. Sin embargo, esta sección solo nos da algo de información para entender mejor como vamos a crear la estrategia usando la RNN y no ser un curso completo de deep learning. Si no entiendes esta explicación, no te preocupes, podemos continuar el libro sin entenderla.

14.1.3. Célula GRU

En esta parte, veremos derivados de la neurona LSTM, la Unidad Recurrente Cerrada (GRU). Es una excelente alternativa a la LSTM porque sólo toma la información de la neurona anterior y no la memoria, lo que puede ser problemático en nuestro caso. En efecto, supongamos que recordamos cómo reaccionó el mercado hace 15 años. En ese caso, no será la misma que la reacción del mercado ahora porque las instituciones del mercado han cambiado, las tecnologías han evolucionado, etc.

Figura 14.4: Célula GRU



Como podemos ver en la célula GRU, sólo tenemos en cuenta la información h , no consideramos la memoria C_t al contrario de lo que hacíamos en la LSTM.



Aunque la GRU y la LSTM difieran, debes probar ambas cuando crees tu estrategia de trading en un modelo RNN.

En esta sección, hemos visto mucha información sobre RNN. Podemos releer esta sección si nos sentimos incómodos con esta noción o pasar a la sección siguiente porque la teoría es necesaria para entender lo que hacemos, pero no para crear el modelo. Al fin y al cabo, ya conocemos el funcionamiento de una ANN. La RNN es similar aunque tenga algunas especificidades. Ahora, estamos listos para comenzar el código de la RNN utilizando TensorFlow.

14.2. RNN para clasificación

Veremos cómo transformar nuestros datos en datos 3d (necesarios para la clasificación y regresión e implementar nuestro modelo) y cómo backtestar la estrategia creada.

14.2.1. Transformar datos 2d a datos 3d

Esta parte explicará por qué y cómo transformar nuestros datos 2d en datos 3d. Si hemos entendido muy bien la teoría detrás de la RNN (aunque sea muy difícil con la información que te dan), hemos entendido que esta red toma para cada x , una matriz de forma $(lags, m)$ donde lags es el número de lag y m es el número de características. Así, si tenemos n observaciones, tenemos una matriz de forma $(n, lags, m)$. La figura 14.5 representa la base de datos que necesitamos.

Figura 14.5: Esquematización de los datos para una RNN

Una observación
observaciones

	Features		
Lags	0.11	0.08	-0.01
	-0.01	0.07	0.17
	0.03	-0.10	0.02
	0.02	0.09	-0.06
	0.10	0.07	0.13
	-0.10	0.15	-0.10

Todas las

	Features		
Lags	0.11	0.08	-0.01
	-0.01	0.07	0.17
	0.03	-0.10	0.02
	0.02	0.09	-0.06
	0.10	0.07	0.13
	-0.10	0.15	-0.10
Number of observations			

Como vemos, la base de datos para una RNN tiene que ser tridimensional. Son las mismas reglas que para la ANN. Sin embargo, para la ANN, tenemos una línea como observación, por lo que una matriz de 1 dimensión, y cuando tenemos más de uno, la matriz se convierte en 2-dimensional, en lugar de la RNN, que necesita una matriz de 2 dimensiones POR observación. Naturalmente, si tenemos más de uno, la matriz será de 3 dimensiones.

Código 14.1: Transformar datos 2d en datos 3d

```
def X_3d_RNN(X_s, y_s, lag):
```

```

# Verificación simple
if len(X_s) != len(y_s):
    print("Warnings")

# Crear el X_train
X_train = []
for variable in range(0, X_s.shape[1]):
    X = []
    for i in range(lag, X_s.shape[0]):
        X.append(X_s[i-lag:i, variable])
    X_train.append(X)
X_train, np.array(X_train)
X_train = np.swapaxes(np.swapaxes(X_train, 0, 1), 1, 2)

# Crear el y_train
y_train = []

for i in range(lag, y_s.shape[0]):
    y_train.append(y_s[i, :].reshape(-1, 1).transpose())
y_train = np.concatenate(y_train, axis = 0)
return X_train, y_train

```



Para comprobar si `X_train` e `y_train` tienen el mismo tamaño, mostramos un mensaje, pero no lanzamos un error porque por alguna razón podemos usar `X_train` e `y_train` con longitudes diferentes.

Observamos que la forma `y_train` no cambiará, pero la introducimos en la función para desplazar el valor debido al desfase. Ahora, tenemos los datos en un formato adecuado. Podemos empezar a implementar el clasificador RNN utilizando TensorFlow.

14.2.2. Implementando el modelo

La creación de la red en Python será muy similar a la de la ANN. Por lo tanto, sólo destacaremos los puntos nuevos. Con la función LSTM, tenemos un nuevo argumento que es *return_sequences*. Para hacerlo fácil, si este parámetro es false, devolveremos un array de 2 dimensiones, y si es True, devolveremos un array de 3 dimensiones. Por lo tanto, tenemos que ponerlo en el lugar correcto. Si no usamos nuestra función de pérdida, no tendremos ningún problema con eso, pero si usamos nuestra función de pérdida, necesitamos usar este parámetro correctamente.

Código 14.2: Clasificador RNN

```
# Librerías
import tensorflow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM

# Inicialización del modelo
classifier = Sequential()

# Añadir capa LSTM
classifier.add(LSTM(units = 10, return_sequences = True,
    input_shape = (X_train_3d.shape[1], X_train_3d.shape[2], )))

# Bucle que añade la capa LSTM
for _ in range(1):
    classifier.add(LSTM(units = 10, return_sequences = True))
# Última capa LSTM, pero con return_sequences = False
classifier.add(LSTM(units = 10, return_sequences = False))

# Capa de salida densa
classifier.add(Dense(1, activation = "sigmoid"))

# Compilar el modelo
```

```
classifier.compile(loss = "binary_crossentropy",  
                  optimizer = "adam")
```

Entrenamiento

```
classifier.fit(X_train_3d, y_train_3d, epochs = 15,  
              batch_size = 32, verbose = 1)
```



Como vemos, la RNN necesita muchos recursos. Para evitar el cómputo, puedes tomar un modelo ya entrenado en internet y modificarlo un poco para adaptarlo a tus datos. También puedes usar GPU, CPU o TPU en Google Colab.

14.2.3. Predicción y backtest

Ahora que ya tenemos nuestro algoritmo, veremos las ligeras diferencias en el código cuando queramos hacer una predicción y backtest con una estrategia utilizando una RNN.

Código 14.3: Predicciones y backtest

```
# Crear predicciones para el dataset completo  
y_pred_train = np.concatenate((np.zeros([lag, 1]),  
                                classifier.predict(X_train_3d)), axis = 0)  
y_pred_test = np.concatenate((np.zeros([lag, 1]),  
                               classifier.predict(X_test_3d)), axis = 0)  
  
df["prediction"] = np.concatenate((y_pred_train, y_pred_test),  
                                  axis = 0)  
df["prediction"] = np.where(df["prediction"] == 0, -1, 1)  
  
# Calcular la estrategia  
df["strategy"] = np.sign(df["prediction"]) * df["returns"]  
# Backtest  
backtest_dynamic_portfolio(df["strategy"].iloc[split + lag:])
```

Tenemos que añadir un vector de 0 de las longitudes de



retardo para asegurar que los datos estarán en un buen lugar. Hemos borrado 15 datos cada día utilizando la función para transformar los datos en 3d.

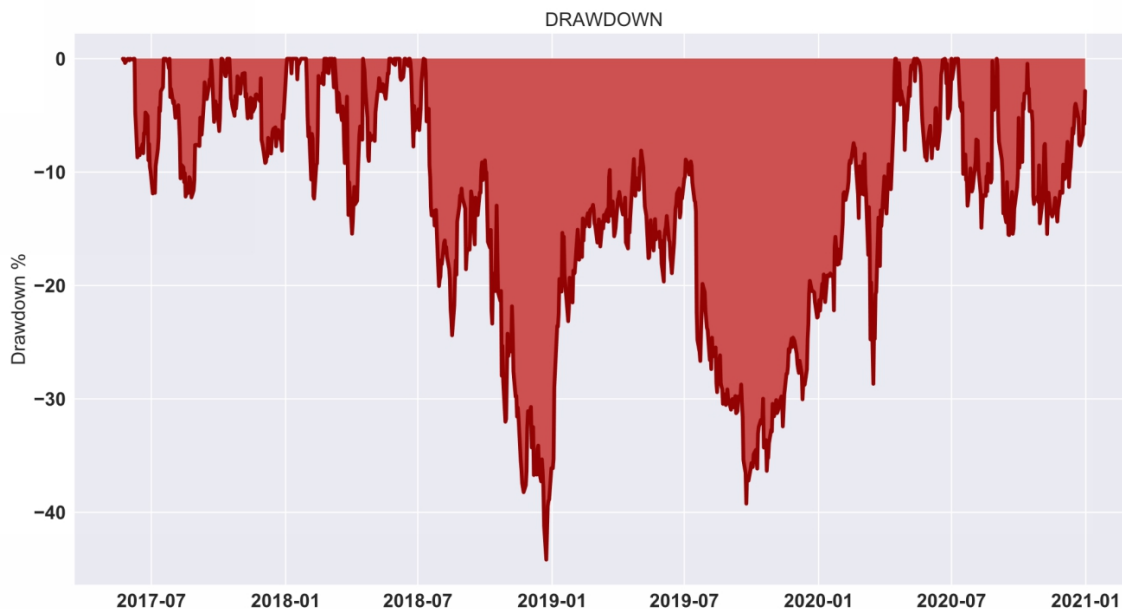
Como podemos ver en el backtest, es un algoritmo excelente porque el crecimiento del capital es muy estable. Tenemos excelentes indicadores como el ratio de Sharpe mayor que uno, pero tenemos un drawdown del 45%; por eso veremos en el capítulo en el que vamos a crear el proyecto, cómo podemos crear una cartera de estrategias para disminuir la volatilidad de la cartera (como lo anterior en el capítulo 3 sobre la optimización de la cartera estática).

Figura 14.6: Backtest de la estrategia utilizando predicciones RNN sobre las acciones de Netflix

Beta: 0.953 Alpha: 28.52 % Sharpe: 1.04 Sortino: 1.539

VaR: 52.82 % cVaR: 66.92 % VaR/cVaR: 1.267 drawdown: 44.18 %





En esta figura, podemos ver que el crecimiento del capital es muy bueno, excepto de 2018-07 a 2019-07 con una detracción del 45%.

14.3. Regresor RNN

Esta sección explicará cómo hacer un regresor RNN y crear una función para automatizar la construcción. Para ello, haremos una pequeña precisión sobre la normalización. Luego veremos una nueva función de aprendizaje profundo llamada dropout y cómo realizar nuestra estrategia de trading creada con el regresor RNN.

14.3.1. Precisión sobre la estandarización

Anteriormente, dijimos que la normalización no debe aplicarse al objetivo. ¡VERDADERO! Sin embargo, utilizando RNN, tanto X como y se utilizarán para entrenar el modelo. Por tanto, será mejor normalizar el vector y y para reducir el tiempo de cálculo. Por otra parte, más tarde tendremos que hacer la transformación inversa en el vector y y después de obtener los

valores de escala correcta para tomar nuestras decisiones de inversión.

Código 14.4: Estandarización del objetivo

```
# Estandarización
sc_y = StandardScaler()
y_train_scaled = sc_y.fit_transform(y_train_reg)
y_test_scaled = sc_y.transform(y_test_reg)
```



Es imprescindible crear una nueva variable `sc_y` para normalizar estos datos y mantenerla porque tenemos la transformada inversa una vez que el algoritmo da las predicciones.

14.3.2. Implementando el modelo

En esta parte explicaremos un nuevo concepto, la capa de abandono o capa dropout. Para una mejor utilización, creamos una función RNN que nos permite crear un modelo sólo tomando algunos parámetros necesarios.

En primer lugar, hablemos del dropout. Es algo apasionante y muy sencillo. Es una capa que añadimos al modelo, que apaga un porcentaje de las neuronas de una capa. Sin embargo, ¿por qué queremos hacer eso?

Añadir alguna capa dropout es muy útil de dos maneras principales:

- **Mejor entrenamiento:** Apaga algunas neuronas y obliga a las demás a aprender más de lo normal para que el modelo esté mejor entrenado.
- **Riesgo de sobreajuste reducido:** el dropout nos permite reducir el riesgo de sobreajuste, ya que apagar aleatoriamente algunas neuronas obliga a las demás a no centrarse en un punto.

Código 14.5: Factorizar la implementación de RNN en una función

```
def RNN(number_neurons, number_hidden_layer, shape,
        loss = "mse", metrics = ["mae"], activation = "linear",
        optimizer = "adam", pct_dropout = 0.5):

    # Librerías
    import tensorflow
    from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import Dense, LSTM, Dropout

    # Inicialización de los datos
    model = Sequential()

    # Añadir capa LSTM
    model.add(LSTM(units = number_neurons, return_sequences = True,
                   input_shape = shape))

    # Añadir capa dropout
    model.add(Dropout(pct_dropout))

    # Bucle que añade las capas LSTM y dropout
    for _ in range(number_hidden_layer):
        model.add(LSTM(units = number_neurons,
                       return_sequences = True))
        model.add(Dropout(pct_dropout))

    # Última capa LSTM, pero con return_sequences = False para
    # tener un array 2d
    model.add(LSTM(units = number_neurons, return_sequences = False))

    # Añadir capa dropout
    model.add(Dropout(pct_dropout))
```

```

# Capa de salida densa
model.add(Dense(1, activation = activation))

# Compilar el modelo
model.compile(loss = loss, optimizer = optimizer,
              metrics = metrics)
return model

regressor = RNN(15, 3,
               taille = (X_train_3d.shape[1],
                        X_train_3d.shape[2]),
               loss = "mse", metrics=["mae"],
               activation = "linear", optimizer = "adam",
               pct_dropout = 0.65)

regressor.fit(X_train_3d, y_train_3d, epochs = 1, batch_size = 32,
             verbose=1)

```

14.3.3. Predicciones y backtest

En esta parte veremos cómo realizar la transformada inversa de normalización y backtestar la estrategia de trading utilizando un regresor RNN.

Código 14.6: Transformada inversa de normalización y backtest

```

# Crear predicciones para el dataset completo

# Transformada inversa
y_train_sc = sc_y.inverse_transform(regressor.predict(X_train_3d))

# Predicciones
y_pred_train = np.concatenate((np.zeros([lag, 1]), y_train_sc),

```

```

axis = 0)

# Transformada inversa
y_test_sc = sc_y.inverse_transform(regressor.predict(X_test_3d))

# Predicciones
y_pred_test = np.concatenate((np.zeros([lag, 1]), y_test_sc),
                              axis=0)

df["prediction"] = np.concatenate((y_pred_train, y_pred_test),
                                   axis=0)

# Calcular la estrategia
df["strategy"] = np.sign(df["prediction"]) * df["returns"]

# Backtest
backtest_dynamic_portfolio(df["strategy"].iloc[split+lag:])

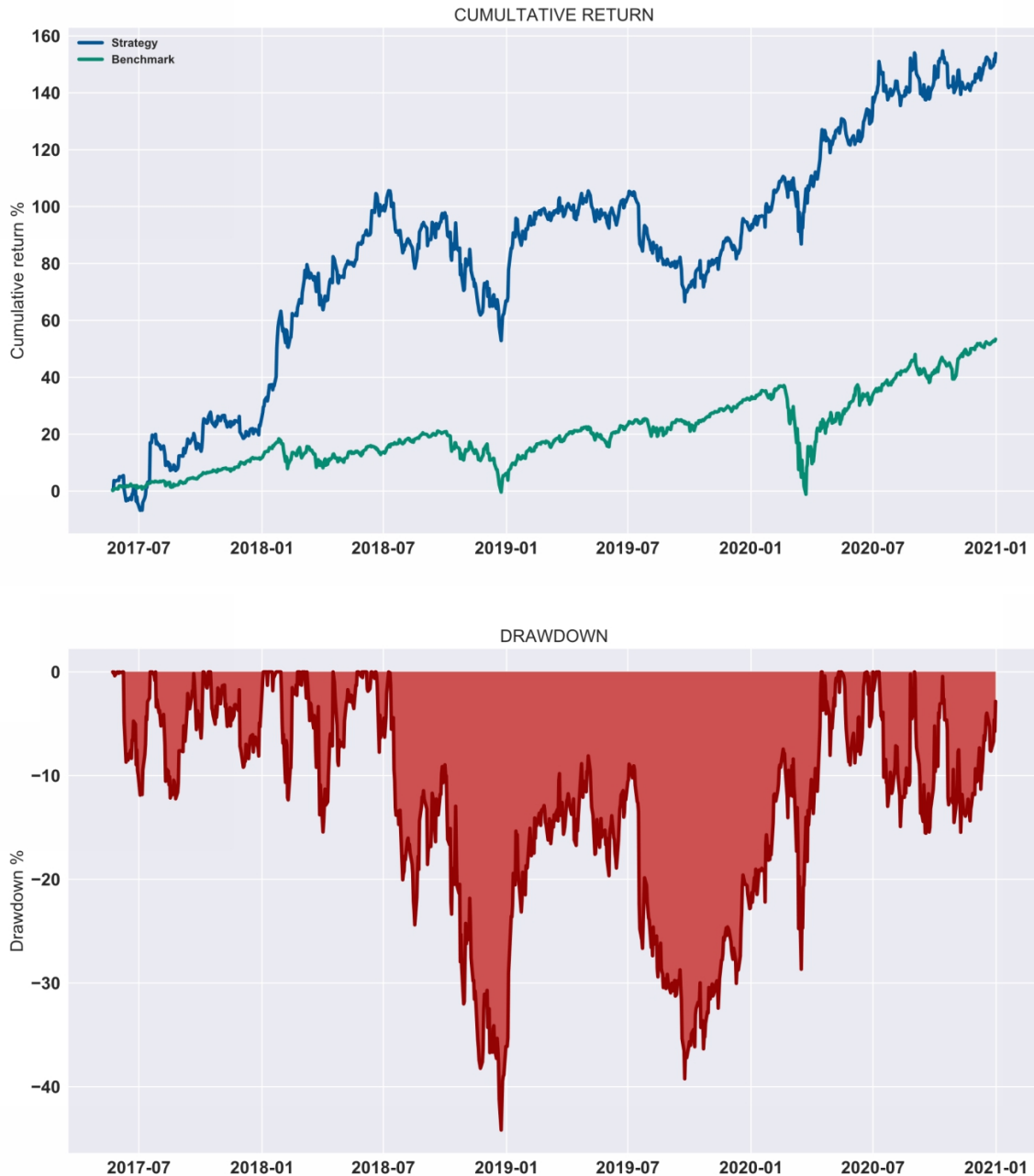
```

Como se muestra en la figura 14.7, es el mismo que para el clasificador en lugar del dropout. Eso significa que no hay sobreajuste en el clasificador.

Figura 14.7: Backtest de la estrategia utilizando predicciones RNN sobre las acciones de Netflix

Beta: 0.953	Alpha: 28.52 %	Sharpe: 1.04	Sortino: 1.539

VaR: 52.82 %	cVaR: 66.92 %	VaR/cVaR: 1.267	drawdown: 44.18 %



La estrategia tiene los mismos rendimientos que el clasificador RNN lo que puede ser posible porque se está entrenando con los mismos datos.

Antes de cerrar este capítulo, es necesario destacar algunos puntos sobre el aprendizaje profundo:

- Los algoritmos **necesitan muchos datos**, casi 50000 o 100000 como mínimo, para entrenarse, aunque aquí hayamos tomado una base de datos minúscula.
- Los algoritmos de aprendizaje profundo son **los más potentes**, pero también los más difíciles de entrenar debido al número de hiperparámetros.
- **El riesgo de sobreajuste** aumenta con el número de parámetros. Así que las redes neuronales están muy tocadas por estos problemas.

Resumen

- Los datos deben ser tridimensionales para entrenar una RNN, ya que cada observación es una matriz bidimensional.
- La red neuronal recurrente es útil cuando trabajamos con series temporales.
- La capa de abandono nos permite desactivar algunas neuronas para optimizar el entrenamiento y evitar el sobreajuste..

Aplicación Trading en vivo y Screener

Este código se basa en la clase creada en el anexo: MetaTrader.

Código 14.7: Aplicación Trading / Regresor RNN

```
from MT5 import *  
import numpy as np  
import pandas as pd  
import warnings  
warnings.filterwarnings("ignore")  
import time
```

```

import pickle
from joblib import dump, load
import os
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Dropout

path = ""
# Ex: C:/Desktop/Python_for_finance_and_algorithmic_trading/ChapterN/

def X_3d_RNN(X_s, y_s, lag):

    # Verificación simple
    if len(X_s) != len(y_s):
        print("Warnings")

    # Crear el X_train
    X_train = []

    for variable in range(0, X_s.shape[1]):
        X = []
        for i in range(lag, X_s.shape[0]):
            X.append(X_s[i-lag:i, variable])
        X_train.append(X)
    X_train, np.array(X_train)
    X_train = np.swapaxes(np.swapaxes(X_train, 0, 1), 1, 2)

    # Crear el y_train
    y_train = []
    for i in range(lag, y_s.shape[0]):
        y_train.append(y_s[i, :].reshape(-1, 1).transpose())
    y_train = np.concatenate(y_train, axis = 0)

    return X_train, y_train

```

```
def RNN():  
    # Crear el modelo  
    number_hidden_layer = 15  
    number_neurons = 10  
    loss = "mse"  
    metrics = ["mae"]  
    activation = "linear"  
    optimizer = "adam"  
    pct_dropout = 0.5  
  
    # Inicialización de los datos  
    model = Sequential()  
  
    # Añadir la capa LSTM  
    model.add(LSTM(units = number_neurons, return_sequences = True,  
                   input_shape = (15, 5, )))  
  
    # Añadir la capa dropout  
    model.add(Dropout(pct_dropout))  
  
    # Bucle que añade las capas LSTM y dropout  
    for _ in range(number_hidden_layer):  
        model.add(LSTM(units = number_neurons, return_sequences = True))  
        model.add(Dropout(pct_dropout))  
  
    # Última capa LSTM, pero con return_sequences = False  
    model.add(LSTM(units = number_neurons, return_sequences = False))  
  
    # Añadir la capa dropout  
    model.add(Dropout(pct_dropout))
```



```
# Capa de salida densa
```

```
model.add(Dense(1, activation = activation))
```

```
# Compilar el modelo
```

```
model.compile(loss = loss, optimizer = optimizer, metrics = metrics)
```

```
return model
```

```
def create_model_weights(symbol):
```

```
    """
```

```
    Ponderaciones para la regresión lineal sobre el cambio porcentual
```

```
    """
```

```
# Importar los datos
```

```
data = MT5.get_data(symbol, 3500)[["close"]].pct_change(1)
```

```
# Crear una nueva variable
```

```
data.columns = ["returns"]
```

```
# Ingeniería de características
```

```
data["returns t-1"] = data[["returns"]].shift(1)
```

```
# Media de los rendimientos
```

```
data["mean returns 15"] = data[["returns"]].rolling(15).mean().  
    shift(1)
```

```
data["mean returns 60"] = data[["returns"]].rolling(60).mean().  
    shift(1)
```

```
# Volatilidad de los rendimientos
```

```
data["volatility returns 15"] = data[["returns"]].rolling(15).std().  
    shift(1)
```

```
data["volatility returns 60"] = data[["returns"]].rolling(60).std().  
    shift(1)
```

```
# Dividir los datos
```

```

data = data.dropna()
split = int(0.80 * len(data))

# Creación del conjunto de entrenamiento
X_train = data[["returns t-1", "mean returns 15", "mean returns 60",
               "volatility returns 15", "volatility returns 60"]].
            iloc[:split]
y_train = np.round(data[["returns"]].iloc[:split] + 0.5)

# Inicializar la clase
sc = StandardScaler()

# Estandarizar los datos
X_train = sc.fit_transform(X_train)

lag = 15
X_train, y_train = X_3d_RNN(X_train, y_train.values, 15)

alg = RNN()

# Entrenamiento
alg.fit(X_train, y_train, epochs = 1, batch_size = 32, verbose = 1)

# Guardar el modelo
print("Train the model because there are no existed weights")
alg.save_weights(os.path.join(path, f"Models/RNN_reg_{symbol}"))

def RNN_reg_sig(symbol):
    """
    Función para predecir el valor de mañana utilizando el modelo ARIMA
    """

    # Crear los pesos si no hay en la carpeta
    try:

```

```

alg = RNN()
alg.load_weights(os.path.join(path, f"Models/RNN_reg_{symbol}"))
except:
    create_model_weights(symbol)
    alg = RNN()
    alg.load_weights(os.path.join(path, f"Models/RNN_reg_{symbol}"))

# Tomar el último porcentaje de cambio
data = MT5.get_data(symbol, 3500)[["close"]].pct_change(1)

# Crear nueva variable
data.columns = ["returns"]

# Ingeniería de características
data["returns t-1"] = data[["returns"]].shift(1)

# Media de los rendimientos
data["mean returns 15"] = data[["returns"]].rolling(15).mean().
    shift(1)
data["mean returns 60"] = data[["returns"]].rolling(60).mean().
    shift(1)

# Volatilidad de los rendimientos
data["volatility returns 15"] = data[["returns"]].rolling(15).std().
    shift(1)
data["volatility returns 60"] = data[["returns"]].rolling(60).std().
    shift(1)

X = data[["returns t-1", "mean returns 15", "mean returns 60",
    "volatility returns 15", "volatility returns 60"]]

# Inicializar la clase
sc = StandardScaler()

```

```
# Estandarizar los datos
```

```
X = sc.fit_transform(X)
```

```
y = data[["returns t-1"]]
```

```
X, _ = X_3d_RNN(X, y.values, 15)
```

```
X = X[-1:, :, :]
```

```
# Encontrar la señal
```

```
prediction = alg.predict(X)
```

```
buy = prediction[0][0] > 0
```

```
sell = not buy
```

```
return buy, sell
```

```
# True = Trading en vivo y False = Screener
```

```
live = True
```

```
if live:
```

```
    current_account_info = mt5.account_info()
```

```
    print("-----")
```

```
    print("Date: ", datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
```

```
    print(f"Balance: {current_account_info.balance} USD, \t"
```

```
          f"Equity: {current_account_info.equity} USD, \t"
```

```
          f"Profit: {current_account_info.profit} USD")
```

```
    print("-----")
```

```
info_order = {
```

```
    "Netflix": ["Netflix_Inc_(NFLX.O)", 1.00]
```

```
}
```

```
start = datetime.now().strftime("%H:%M:%S") #"23:59:59"
```

```

while True:

    # Verificación para lanzamiento

    if datetime.now().weekday() not in (5, 1):

        is_time = datetime.now().strftime("%H:%M:%S") == start

    else:

        is_time = False

    # Lanzar el algoritmo

    if is_time:

        # Abrir las transacciones

        for asset in info_order.keys():

            # Inicializar los inputs

            symbol = info_order[asset][0]

            lot = info_order[asset][1]

            # Crear las señales

            buy, sell = RNN_reg_sig(symbol)

            # Ejecutar el algoritmo

            if live:

                MT5.run(symbol, buy, sell, lot)

            else:

                print(f"Symbol: {symbol}\n"
                      f"Buy: {buy}\n"
                      f"Sell: {sell}")

            time.sleep(1)

```



El parámetro live establece el modo trading en vivo (live = True) o el modo screener (live = False).

Capítulo 15: Bonus - Red neuronal convolucional recurrente (RCNN)

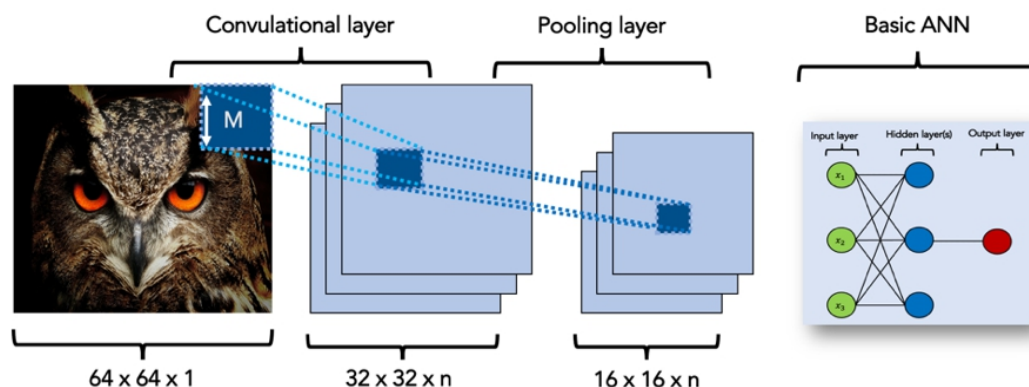
En este bonus, veremos un ejemplo de un algoritmo avanzado de aprendizaje profundo, la RCNN, que combina una CNN unidimensional y una RNN. Nos permite quedarnos con lo mejor de los dos modelos.

En efecto, la RNN es buena para encontrar relaciones en el tiempo porque es un algoritmo de análisis de series temporales. Al mismo tiempo, la CNN puede captar más características porque tiene un método de ajuste diferente.

15.1. Intuición de la CNN

La especialidad de las redes neuronales convolucionales (CNN) es que funcionan con filtros. Los filtros de una capa CNN son como el número de neuronas de una RNN. Cuantos más filtros haya, más complejo será el algoritmo. La CNN se utiliza principalmente para la detección de imágenes. Explicaremos un poco la noción utilizando el ejemplo de una imagen.

Figura 15.1: Intuición tras la CNN



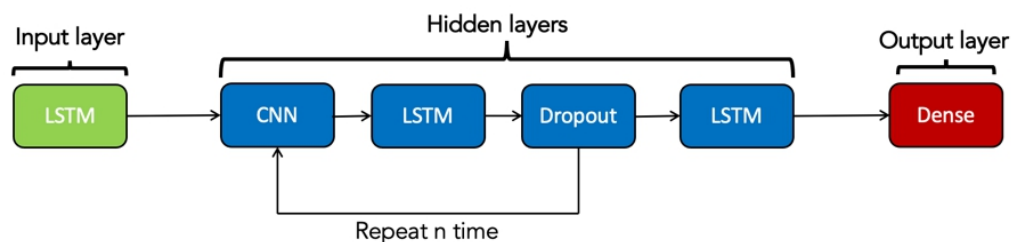
En esta figura, podemos ver la representación del funcionamiento de la CNN.

- M es la longitud de la franja, que es un parámetro importante. Si la franja es demasiado pequeña, nuestro algoritmo tarda demasiado tiempo en entrenarse, lo que aumenta el riesgo de sobreajuste. No hay que olvidar que cuanto mayor sea la complejidad, mayor será el riesgo de sobreajuste.
- N es el número de filtros. Cuantos más filtros haya, más potente y tardará más tiempo en entrenarse.
- La capa convolucional se crea mediante la función conv1D o conv2D de TensorFlow.
- La capa de pooling nos permite reducir la forma, mientras que no la utilizaremos en nuestro modelo.

15.2. Crear una RCNN

Cada red neuronal tiene muchas estructuras, en particular ésta, porque las combina. Usaremos una CNN unidimensional porque necesita datos tridimensionales como la RNN.

Figura 15.2: Descripción del modelo



Nuestro modelo es un poco más complejo que el otro. De hecho, combina RNN, CNN y dropout.

Código 15.1: RCNN

```
def RCNN(number_neurons, number_hidden_layer, shape,
         loss = "mse", metrics = ["mae"], activation = "linear",
         optimizer = "adam", pct_dropout = 0.5):

    # Librerías
    import tensorflow
    from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import Conv1D, Dense, LSTM, Dropout

    # Inicialización de los datos
    model = Sequential()

    # Añadir capa LSTM
    model.add(LSTM(units = number_neurons, return_sequences = True,
                   input_shape = shape))

    # Añadir capa dropout
    model.add(Dropout(pct_dropout))

    # Bucle que añade las capas LSTM y dropout
    for _ in range(number_hidden_layer):
        model.add(Conv1D(64, 3, activation = 'relu'))
        model.add(LSTM(units = number_neurons,
                       return_sequences = True))
        model.add(Dropout(pct_dropout))

    # Última capa LSTM, pero con return_sequences = False
    model.add(LSTM(units = number_neurons, return_sequences = False))

    # Capa de salida densa
    model.add(Dense(1, activation = activation))
```



```

# Compilar el modelo
model.compile(loss = loss, optimizer = optimizer,
              metrics = metrics)
return model

regressor = RNN(15, 3,
               shape = (X_train_3d.shape[1], X_train_3d.shape[2]),
               loss = "mse", metrics = ["mae"],
               activation = "linear", optimizer = "adam",
               pct_dropout = 0.65)

regressor.fit(X_train_3d, y_train_3d, epochs = 1, batch_size = 32,
             verbose = 1)

```

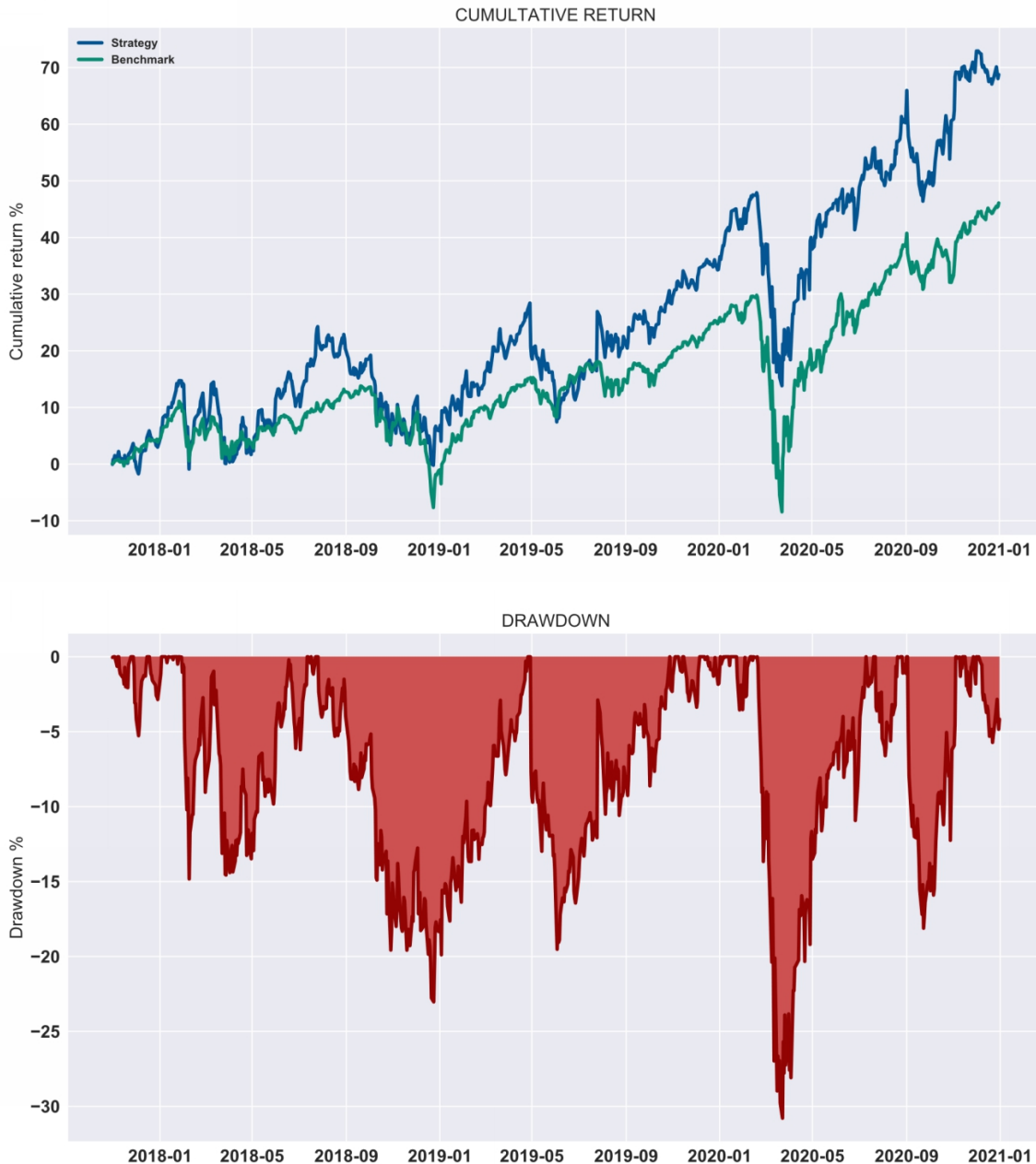
15.3. Backtest

En esta sección, haremos un backtest de la estrategia de trading basada en una RCNN. Para ello utilizaremos la misma función creada anteriormente en el capítulo 5.

Figura 15.3: Backtest de la estrategia utilizando las predicciones de Google de RCNN

Beta: 1.054	Alpha: 6.37 %	Sharpe: 0.721	Sortino: 0.926

VaR: 48.25 %	cVaR: 59.27 %	VaR/cVaR: 1.229	drawdown: 30.79 %



La estrategia es buena porque los ratios alfa, Sharpe y Sortino son positivos. La beta es cercana a 1, lo que significa que estamos correlacionados con los rendimientos del mercado. El drawdown es cercano al 30%, que es alto pero bueno para una estrategia sin combinar otras estrategias.

Capítulo 16: Proyecto completo de la vida real

En el capítulo anterior, hemos visto un montón de técnicas y algoritmos de muchos campos. Veremos que para poner en marcha un algoritmo de trading cuantitativo robusto, la clave está en utilizar todos estos campos simultáneamente para sacar provecho de cada uno de ellos. Combinaremos el aprendizaje automático para crear algoritmos predictivos, la estadística para hacer ingeniería de características y la gestión de carteras para combinar nuestras estrategias y disminuir el riesgo de esta inversión.

En primer lugar, buscaremos muchos activos para nuestra estrategia. A continuación, seleccionaremos los mejores activos utilizando un criterio de ratio de Sharpe. Además, encontraremos el stop loss, el take profit y el apalancamiento óptimos.

16.1. Preparación de los datos

En todos los proyectos de finanzas, los datos son lo más importante. El algoritmo más potente no puede predecir si le pedimos que prediga el precio de las acciones de Google utilizando el consumo de helados. Es exagerado, pero necesitamos preparar los datos con la mayor atención posible porque podemos perder dinero por un error en ellos. Normalmente, la preparación de los datos supone entre el 60% y el 80% del proyecto.

16.1.1. Importar los datos

El objetivo de la estrategia es probar algunos algoritmos en muchos activos. La cuestión es que ya sabemos cómo importar datos utilizando un símbolo. Entonces, ¿por qué hay una parte para explicar eso? Queremos trabajar con todos los activos disponibles en nuestro broker. Hay más de 2000. Por

lo tanto, necesitamos ejecutar un pequeño código para tenerlos todos.

Además, vamos a trabajar con los datos de nuestro broker directamente porque necesitamos estar lo más cerca posible del mercado. De hecho, en el capítulo anterior, exploramos cada técnica utilizando datos de Yahoo. De hecho, en los capítulos anteriores se trataba de explicar que las técnicas no crean una estrategia. Sin embargo, cuando trabajamos en un proyecto de la vida real, es mejor tener los datos del broker utilizando la plataforma MetaTrader 5 porque es más cercano a la realidad.

Código 16.1: Cómo scrapear todos los nombres de símbolos

```
# Inicializar el dispositivo
mt5.initialize()

# Crear listas vacías
symbols = []
sectors = []
descriptions = []

# Obtener la información de todos los símbolos
symbols_information = mt5.symbols_get() 1

# Tupla a lista
symbols_information_list = list(symbols_information) 2

# Extraer el nombre de los símbolos
for element in symbols_information_list:
    symbols.append(list(element)[-3]) 3
    sectors.append(list(element)[-1].split("\\")[0]) 4
    descriptions.append(list(element)[-7]) 5
```

```
# Crear un dataframe
```

```
informations = pd.DataFrame([symbols, sectors, descriptions],  
                             index = ["Symbol", "Sector", "Description"]).  
                             transpose() 6
```

- 1 La función `symbols_get()` nos da la información de todos los símbolos que el broker nos ofrece para operar.
- 2 `symbols_get()` devuelve una tupla, por lo que necesitamos transformarla en una lista para trabajar con este objeto.
- 3 Tomar el símbolo del activo.
- 4 Tomar el camino para tener el sector del activo.
- 5 Tomar una descripción del activo.
- 6 Poner toda la información en un dataframe.

Figura 16.1: Extracción de los símbolos y su sector

Symbol	Sector	Description
AUDUSD	FX Majors	Australian Vs US
EURUSD	FX Majors	US Vs Euro

Podemos ver que hemos scrapeado todos los símbolos con su sector. Por lo tanto, es fácil de hacer si queremos probar un algoritmo sólo en un sector.



Seleccionar un sector puede ser interesante si creamos un algoritmo que utilice bagging sobre activos.

Nuestra estrategia tomará sólo los activos con una dispersion exigua. Se trata de una elección personal. Podemos elegir otro criterio de selección de activos, como la volatilidad del activo.

Código 16.2: Encontrar el activo con menor dispersion

```

# Crear una lista vacía
spread = []

# Calcular la dispersión
for symbol in informations["Symbol"]:
    try:
        ask = mt5.symbol_info_tick(symbol).ask
        bid = mt5.symbol_info_tick(symbol).bid
        spread.append((ask - bid) / ask )
    except:
        spread.append(None)

# Tomar los activos con spread < 0.07%
informations["Spread"] = spread
lowest_spread_asset = informations.dropna().\
    loc[informations["Spread"] < 0.0007]

```

16.1.2. Ingeniería de características

Como ya se ha dicho, los datos son una parte esencial del proyecto. Así, no podemos dar al algoritmo únicamente los datos OHLC porque son insuficientes. Necesitamos ayudar a nuestro algoritmo. Para ello, utilizaremos el método de ingeniería de características anterior. En efecto, crearemos unas medias móviles y una volatilidad móvil en la función. A continuación, aplicaremos un algoritmo ACP al conjunto de datos para reducir el número de columnas.

Código 16.3: Crear nuevas características y aplicar ACP

```
def features_engeniring(df):  
    """  
    Esta función crea todos los conjuntos necesarios para los algoritmos  
    """  
  
    # Permite llamar a las variables fuera de la función  
    global X_train  
    global X_test  
    global y_train_reg  
    global y_train_cla  
    global X_train_scaled  
    global X_test_scaled  
    global split_train_test  
    global split_test_valid  
    global X_valid  
    global X_valid_scaled  
    global X_train_pca  
    global X_test_pca  
    global X_val_pca  
  
    # Crear nuestras propias métricas para calcular los rendimientos de  
    # la estrategia  
    df["returns"] = ((df["close"] - df["close"].shift(1)) / df["close"]).  
        shift(1)  
    df["sLow"] = ((df["low"] - df["close"].shift(1)) / df["close"].  
        shift(1)).shift(1)  
    df["sHigh"] = ((df["high"] - df["close"].shift(1)) / df["close"].  
        shift(1)).shift(1)  
  
    # Ingeniería de características  
    df["returns t-1"] = df["returns"].shift(1)
```

Media de los rendimientos

```
df["mean returns 15"] = df[["returns"]].rolling(15).mean().shift(1)
```

```
df["mean returns 60"] = df[["returns"]].rolling(60).mean().shift(1)
```

Volatilidad de los rendimientos

```
df["volatility returns 15"] = df[["returns"]].rolling(15).std().  
    shift(1)
```

```
df["volatility returns 60"] = df[["returns"]].rolling(60).std().  
    shift(1)
```

Descartar los valores faltantes

```
df = df.dropna()
```

Porcentaje del conjunto de entrenamiento

```
split = int(0.80 * len(df))
```

```
list_x = ["returns t-1", "mean returns 15", "mean returns 60",  
    "volatility returns 15", "volatility returns 60"]
```

```
split_train_test = int(0.70 * len(df))
```

```
split_test_valid = int(0.90 * len(df))
```

Creación del conjunto de entrenamiento

```
X_train = df[list_x].iloc[:split_train_test]
```

```
y_train_reg = df[["returns"]].iloc[:split_train_test]
```

```
y_train_cla = np.round(df[["returns"]].iloc[:split_train_test] + 0.5)
```

Creación del conjunto de test

```
X_test = df[list_x].iloc[split_train_test:split_test_valid]
```

Creación del conjunto de validación


```

X_val = df[list_x].iloc[split_test_valid:]

# NORMALIZACIÓN
# Importar la clase
from sklearn.preprocessing import StandardScaler

# Inicializar la clase
sc = StandardScaler()

# Estandarizar los datos
X_train_scaled = sc.fit_transform(X_train)
X_test_scaled = sc.transform(X_test)
X_val_scaled = sc.transform(X_val)

# ACP
# Importar la clase
from sklearn.decomposition import PCA

# Inicializar la clase
pca = PCA(n_components = 3)

# Aplicar ACP
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
X_val_pca = pca.transform(X_val_scaled)

```



Si la función modifica los valores de la variable global, debe declararse precediéndola de la palabra clave global.



En la siguiente sección, entrenaremos todos los algoritmos con los mismos datos, como en el capítulo 10. Es mejor entrenar diferentes modelos con diferentes características (independientes) para tener mejores resultados. Por lo tanto, en tu proyecto, puedes

encontrar las mejores características para cada activo, pero se necesita mucho tiempo.

16.1.3. Conjuntos de entrenamiento, test y validación

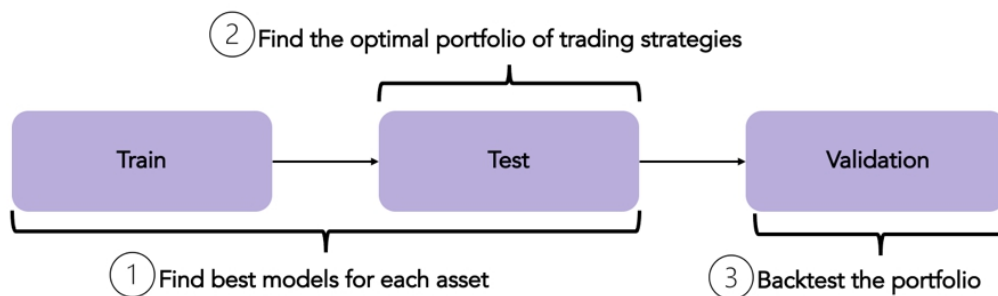
En los capítulos anteriores, para comprobar el rendimiento de nuestros algoritmos, hemos utilizado el conjunto de entrenamiento para entrenar el algoritmo y el conjunto de test para comprobar su rendimiento. En un proyecto más significativo, necesitamos crear tres conjuntos de datos: el conjunto de entrenamiento, el conjunto de test y el conjunto de validación.

Este proyecto debe tener tres conjuntos porque trabajamos con modelos predictivos, y necesitamos un conjunto para entrenarlo y dos conjuntos para trabajar con técnicas de gestión de carteras. Veamos en la figura 16.2 la utilidad de cada conjunto en nuestro proyecto.

- **Conjunto de entrenamiento:** Nos permite entrenar los modelos predictivos. No entrenamos la asignación óptima de la cartera con el conjunto de entrenamiento porque las predicciones sobre el conjunto de entrenamiento no son significativas (porque el algoritmo ya conoce los datos porque se ajusta a ellos).
- **Conjunto de test:** Nos permite encontrar los mejores modelos de predicción sobre los datos desconocidos. Entonces, como los rendimientos de la estrategia basada en las predicciones son significativos aquí (porque el algoritmo no conoce los datos antes), elegimos los mejores modelos (con el criterio de su elección). Entonces, tenemos nuestros mejores modelos. Podemos encontrar la asignación óptima de la cartera entre nuestras n mejores estrategias (sólo en los rendimientos del conjunto de test).

- **Conjunto de validación:** Ahora tenemos los mejores modelos y la mejor distribución de nuestro capital entre los modelos (la mejor distribución de la cartera). Podemos hacer un backtest de la cartera con estos datos anónimos.

Figura 16.2: Utilidad de cada conjunto de datos



Podemos ver un pequeño resumen del proceso de decisión para encontrar la mejor cartera de estrategias de trading.

16.2. Modelando la estrategia

Hemos preparado muchas funciones para los datos: importación, ingeniería de características y transformación. El objetivo de esta sección es encontrar activos con un diferencial bajo que no estén sujetos al mercado de eficiencia débil.

En primer lugar, encontraremos las mejores estrategias utilizando un criterio de ratio de Sharpe. A continuación, tomaremos las estrategias más rentables y utilizaremos un método de votación sobre el algoritmo. Además, utilizaremos un método de cartera en los datos del conjunto de test para encontrar la mejor asignación en nuestras estrategias de cartera.

16.2.1. Encontrar los mejores activos

Tomaremos los símbolos con un diferencial inferior a nuestro umbral de diferencial del 0.07%.

A continuación, calcularemos la ratio de Sharpe de una estrategia de trading creada utilizando cada algoritmo de aprendizaje automático. Además, tomaremos la estrategia con el mayor ratio de Sharpe. Para este proyecto sólo utilizaremos algoritmos clasificadores, pero podemos utilizar algoritmos de regresión en nuestro proyecto. Es lo mismo.



No podemos trabajar con algoritmos de aprendizaje profundo para este proyecto porque algunos activos solo tienen 1000 datos. Así que tenemos que utilizar los algoritmos adecuados: regresión lineal, SVR y árbol de decisión.

En primer lugar, para automatizar el funcionamiento, necesitamos crear una función que produzca una estrategia y calcule el ratio de Sharpe de esta estrategia en el conjunto de test. El único parámetro necesario es el modelo que queremos utilizar. Si vamos a probar un algoritmo de regresión, tenemos que utilizar el parámetro `reg=True`.

Código 16.4: Función de predicción

```
def predictor(model, reg = True, spread = 0.035):
    global df
    model.fit(X_train, y_train_cla)

    df = df.dropna()

    # Crear predicciones para el dataset completo
    df["prediction"] = model.predict(np.concatenate((X_train,
                                                    X_test, X_valid), axis = 0))

    if reg == False:
        df["prediction"] = np.where(df["prediction"] == 0, -1, 1)
```

```

# Calcular la estrategia
df["strategy"] = np.sign(df["prediction"]) * df["returns"]

returns = df["strategy"].
    iloc[split_train_test:split_test_valid]

return np.sqrt(252) * (returns.mean() - (spread / 100)) /
    returns.std()

```



Añadimos el diferencial del activo en el cálculo del ratio de Sharpe porque no todos los activos tienen el mismo diferencial. Por lo tanto, cuanto más significativo sea el diferencial del activo, más se penalizará.

Presta atención al diferencial, a veces está en porcentaje y a veces no. Por lo tanto, verifícalo antes de utilizarlo.

Con esta función, podemos automatizar el cálculo de 450 estrategias de trading. De hecho, tomaremos 150 activos y probaremos 3 algoritmos de aprendizaje automático.

Código 16.5: Automatización del testing

```

# Importar la clase
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.linear_model import LinearRegression

from tqdm import tqdm
# Modelos
tree = DecisionTreeRegressor(max_depth = 6)
svr = SVR(epsilon = 1.5)
lin = LinearRegression()

```

```

# Inicialización
symbols = symbols[:150] # ["EURUSD", "GBPAUD"]
lists = []

for symbol in tqdm(symbols):
    try:
        df = data(symbol, 3500)[["close"]].
        dropna().pct_change(1).dropna()
        df.columns = ["returns"]
        features_engeeniring(df)

        """ Regresor del árbol de decision """
        sharpe_tree = predictor(tree, reg = True)
        lists.append([symbol, "Tree", sharpe_tree])

        """ SVR """
        sharpe_svr = predictor(svr, reg = True)
        lists.append([symbol, "SVR", sharpe_svr])

        """ Regresión Lineal """
        sharpe_linreg = predictor(lin, reg = True)
        lists.append([symbol, "LinReg", sharpe_linreg])

    except:
        pass

```

Con este código, hemos encontrado el ratio de Sharpe para cada estrategia. Tomaremos el activo con mejor ratio Sharpe y tendremos buenos resultados con los tres algoritmos.



Los tres algoritmos elegidos son muy diferentes. La regresión lineal encuentra patrones lineales, el SVR los no lineales y el árbol de decisión también tiene otra forma de ajustarse. Por lo tanto, si estos tres algoritmos tienen buenos resultados, optimizamos nuestras posibilidades de obtener mejores resultados.

Figura 16.3: Los mejores activos elegidos

Asset	Bitcoin	JPN225	NAS100	US2000	XPTUSD
Sortino	2.9	1.6	1.2	1.1	0.7

Esta tabla muestra los mejores activos de nuestra selección con su ratio Sortino calculado en el conjunto de prueba.

16.2.2. Combinar los algoritmos

En el capítulo 10, hemos visto la potencia del método ensemble. Aquí aplicaremos el método de votación. Este método combinará los tres algoritmos anteriores: árbol de decisión, SVC y regresión logística.

Tenemos siete activos con un excelente ratio de Sharpe en el conjunto de prueba. Antes de crear una cartera utilizando estas estrategias, necesitamos crear un algoritmo para predecir cada activo. Para ello, utilizaremos un clasificador de votación.

Código 16.6: Aplicación del clasificador por votos

```
def voting(df, reg = True):
    """ Crear una estrategia utilizando un método de votación """
    # Importar la clase
    # Importar los modelos
    if reg:
        tree = DecisionTreeRegressor(max_depth = 6)
        svr = SVR(epsilon = 1.5)
        lin = LinearRegression()
        vot = VotingRegressor(
            estimators=[('lr', lin), ('tree', tree),
                        ('svr', svr)])
    else:
```

```

tree = DecisionTreeClassifier(max_depth = 6)
svr = SVC()
lin = LogisticRegression()

vot = VotingClassifier(
    estimators=[('lr', lin), ("tree", tree),
                ("svr", svr)])

# Entrenar el modelo
if reg == False:
    vot.fit(X_train_pca, y_train_cla)
else:
    vot.fit(X_train_pca, y_train_reg)

# Eliminar los valores faltantes
df = df.dropna()

# Crear predicciones para todo el conjunto de datos
df["prediction"] = vot.predict(
    np.concatenate((X_train_pca, X_test_pca,
                    X_val_pca), axis = 0))

# Eliminar valores faltantes
df = df.dropna()
if reg == False:
    df["prediction"] = np.where(df["prediction"] == 0, -1, 1)

# Calcular la estrategia
df["strategy"] = np.sign(df["prediction"]).shift(1) *
    df["returns"]
df["low_strategy"] = np.where(df["prediction"] > 0,
    df["sLow"], -df["sHigh"])
df["high_strategy"] = np.where(df["prediction"] > 0,
    df["sHigh"], -df["sLow"])

```



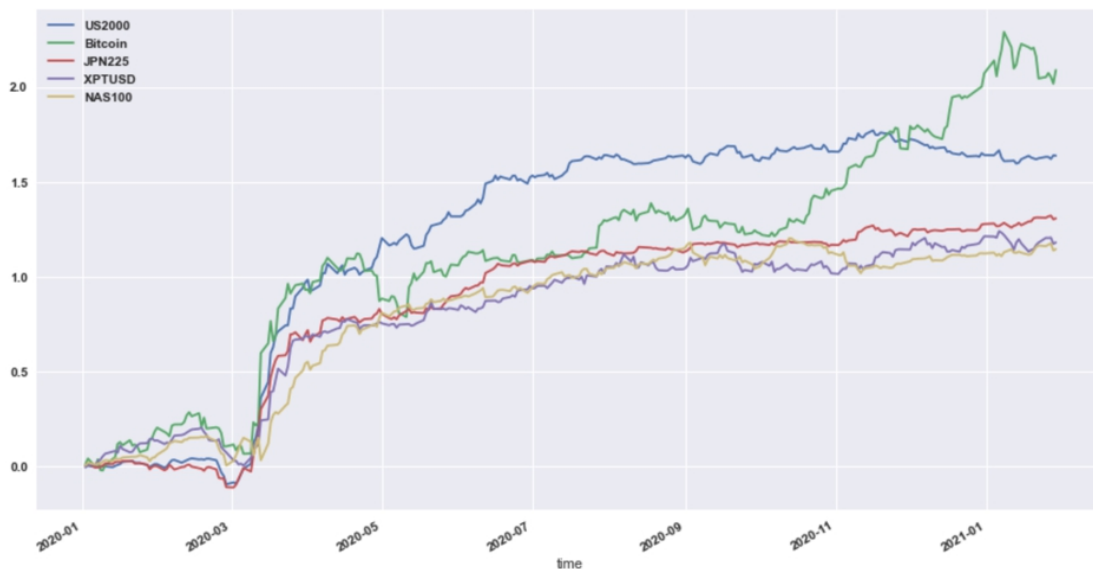
```
return vot, df["strategy"],  
        df["low_strategy"], df["high_strategy"]
```



También devolvemos el modelo por votación para guardarlo en una carpeta para la implementación del algoritmo de trading en vivo (Anexo: Cómo guardar un modelo en scikit-learn y Tensorflow).

Como podemos ver en la figura 16.4, el rendimiento de cada activo por sí solo no es muy rentable, teniendo en cuenta la volatilidad de estas estrategias. En la siguiente subsección se aplicará un método de cartera a las estrategias de trading para disminuir el riesgo de la inversión.

Figura 16.4.: Rendimiento acumulado de los activos de la cartera en el conjunto de test



Esta figura muestra la rentabilidad acumulada de cada activo de la cartera. Como vemos, las estrategias son poco rentables y muy arriesgadas por sí solas.

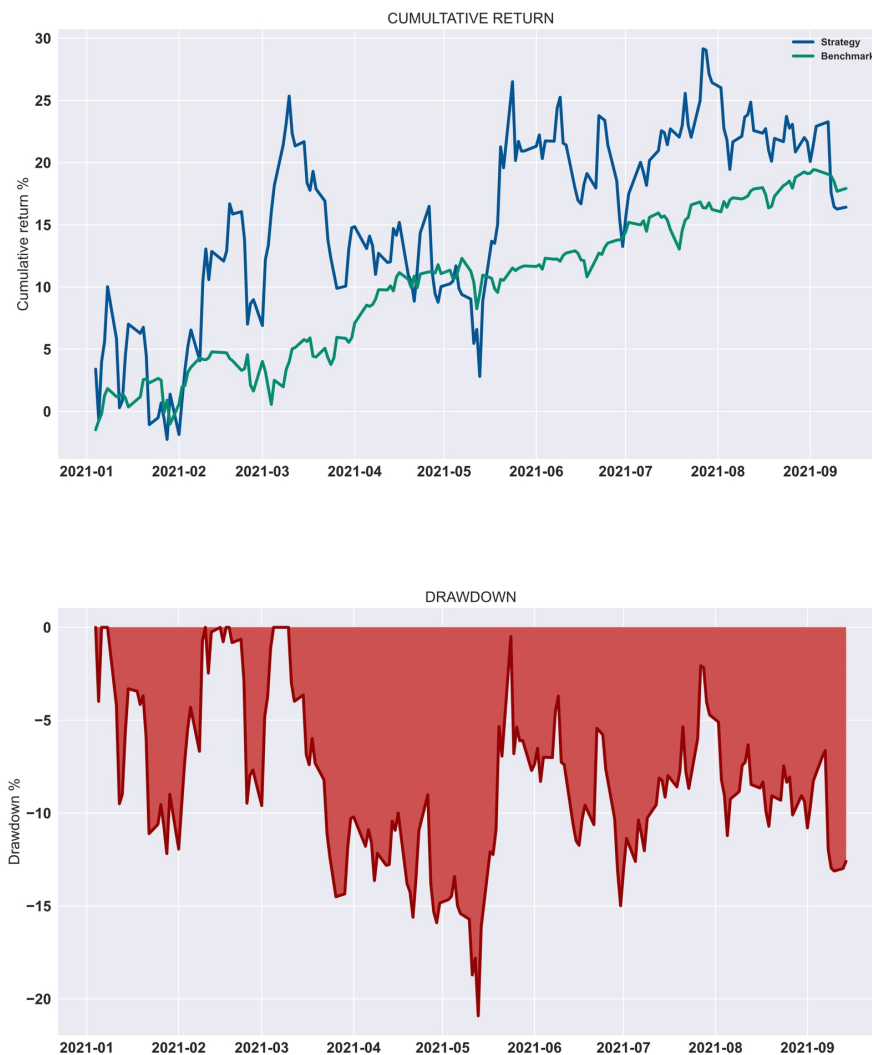
16.2.3. Aplicar técnicas de gestión de carteras

Tenemos siete estrategias para operar, y utilizaremos técnicas de gestión de carteras para crear una cartera con la que trabajar como una estrategia en la siguiente sección. Además, utilizaremos el criterio media-varianza-asimetría-curtosis porque nos permite tener una cartera menos arriesgada.

Figura 16.5: Backtest de la cartera en el conjunto de test

Beta: -0.144 Alpha: 27.36 % Sharpe: 0.603 Sortino: 0.954

VaR: 66.69 % cVaR: 79.23 % VaR/cVaR: 1.188 drawdown: 20.91 %



Esta figura nos muestra el backtest de la cartera sin apalancamiento, stop loss ni take profit. Como vemos, el backtest es bueno, con un alfa del 27%. Hay una buena tendencia en los beneficios pero con alta volatilidad.

La asignación de la cartera consiste en eliminar dos activos. Así, tomamos el 31% de nuestro presupuesto de Russel 2000, el 50% de Bitcoin y el 19% de JPN 225. Este criterio considera que los demás activos son demasiado arriesgados o poco rentables para entrar en la cartera. Además, el drawdown sigue siendo elevado porque la mayoría de los activos subyacentes son muy volátiles.

16.3. Encontrar el take profit, stop loss y apalancamiento óptimos

Como se muestra en la figura 16.6, el rendimiento de la cartera es bueno porque hemos ganado un 15% en 12 meses. La estrategia es arriesgada aunque hemos disminuido el riesgo (drawdown máximo: 20%). Hasta ahora, hemos trabajado con aprendizaje automático para predecir el comportamiento del activo. Hemos aplicado la gestión de carteras para reducir el riesgo. Utilizaremos un método estadístico para encontrar el mejor take profit, stop loss y apalancamiento para la estrategia.

16.3.1. Take profit óptimo (tp)

El umbral de toma de beneficios de una estrategia es fundamental. Puede convertir una estrategia no rentable en una estrategia rentable. En esta subsección veremos una técnica de búsqueda en cuadrícula para encontrar el mejor umbral de take profit. Crearemos una función que devuelva el ratio de Sharpe dependiendo de su umbral de take profit para encontrar los valores óptimos.

Código 16.7: Mejor función de umbral de take profit

```
def find_best_tp(tp):
    tp = tp / 100

    # Crear la cartera
    pf = pd.concat((low_portfolio, portfolio_return_test,
                    high_portfolio), axis = 1).dropna() - spread

    pf.columns = ["low", "Return", "high"]

    # Aplicar el tp
    pf["Return"] = np.where(pf["high"].values > tp, tp,
                           pf["Return"].values)
    pf["Return"] = np.where(pf["Return"].values > tp, tp,
                           pf["Return"].values)

    # Devolver el ratio sharpe
    return np.sqrt(252) * pf["Return"].mean() /
           pf["Return"].std()

pd.DataFrame([find_best_tp(tp) for tp in np.linspace(1.5,3,10)],
              index = np.linspace(1.5, 3, 10),
              columns = ["Sharpe"])
```



Reemplazamos el retorno por el take profit cuando el máximo del día está por encima del umbral tp. Hacemos lo mismo con los rendimientos porque a veces, en la vida real, hay problemas en los datos (high < close, por ejemplo).



Utilizando esta función, convergeremos a un tp de 0. Sin embargo, el umbral de mejor beneficio no tiene en cuenta el swap, una penalización o una bonificación que da el broker para incitar a la gente a ir de una determinada manera. Si el tp es muy bajo en el backtest, tendrá una estrategia rentable, pero el swap se llevará

todo su beneficio en la vida real. Es necesario mantener un umbral de take profit lo suficientemente alto.

Con nuestro cálculo, hemos encontrado un umbral óptimo de take profit en el 2.1% en el conjunto de test con un ratio de Sharpe de 0.7 en el periodo de validación, pero menor en la vida real porque hay comisiones adicionales que no podemos considerar con precisión. Por lo tanto, el 40% en 9 meses del periodo de validación no significa que tengamos que poner en producción con 100000\$ ahora. **Tenemos que poner en una cuenta demo para encontrar los mejores parámetros en la vida real para nuestras estrategias y asegurarnos de que no hay problemas en nuestros cálculos.**

16.3.2. Stop loss óptimo (sl)

Anteriormente, hemos visto cómo ganar dinero, pero la utilidad del stop loss es evitar perder dinero. En esta subsección, sólo crearemos una estrategia con un stop loss porque no podemos combinar tp y sl con los datos diarios (véase el capítulo 6). De hecho, no podemos decir qué umbral se supera primero.



Utilizando la función `find_best_sl`, vamos a encontrar la mejor sl. Además, veremos una convergencia del ratio de Sharpe después del 9.6% de sl. Esto significa que no hay valores después de este punto. Por lo tanto, podemos poner este umbral en nuestra estrategia porque necesitamos un sl en el comercio, y este umbral nunca se toca en el conjunto de test.

Código 16.7: Encontrar el mejor umbral de stop loss

```
def find_best_sl(sl):
    sl = sl / 100

    # Crear la cartera
    pf = pd.concat((low_portfolio, portfolio_return_test,
                    high_portfolio), axis = 1).dropna() - spread
    pf.columns = ["low", "Return", "high"]

    # Aplicar el sl
    pf["Return"] = np.where(pf["low"].values < -sl, -sl,
                            pf["Return"].values)
    pf["Return"] = np.where(pf["Return"].values < -sl, -sl,
                            pf["Return"].values)

    # Devolver el ratio sharpe
    return np.sqrt(252) * pf["Return"].mean() /
           pf["Return"].std()

pd.DataFrame([find_best_sl(sl) for sl in np.linspace(3,10,30)],
              index = np.linspace(3, 10, 30),
              columns = ["Sharpe"])
```

Así, tenemos un tp al 2.1% y un sl al 9.6% para nuestra estrategia. Sólo falta un apalancamiento para optimizar las ganancias de la estrategia.

16.3.3. Apalancamiento óptimo

El apalancamiento multiplica nuestro poder de inversión y nuestros beneficios. Sin embargo, también multiplica nuestro riesgo de la misma manera. Si somos nuevos en el trading, debemos dominar esta herramienta y entender cómo utilizarla eficazmente.

Dependiendo de nuestro nivel de aversión al riesgo, existen varias formas de determinar el mejor apalancamiento a utilizar. Uno de los más conocidos es el criterio de Kelly.

Nosotros utilizaremos otro método: el método drawdown. En finanzas, todas las personas son únicas, por lo que no puede haber un único método para encontrar el mejor apalancamiento, ya que depende de la aversión al riesgo de cada inversor.

El método drawdown es sencillo de calcular, tomamos el máximo drawdown de nuestra estrategia sin apalancamiento (Md_1), y debemos conocer el máximo drawdown que queremos ($Md_{leverage}$). Entonces el apalancamiento es:

$$\text{leverage} = \frac{Md_{leverage}}{Md_1}$$

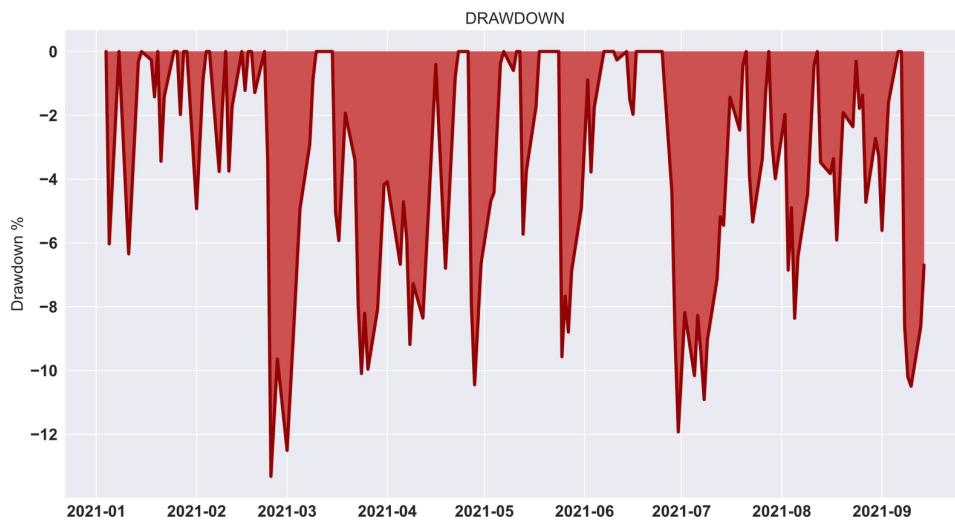
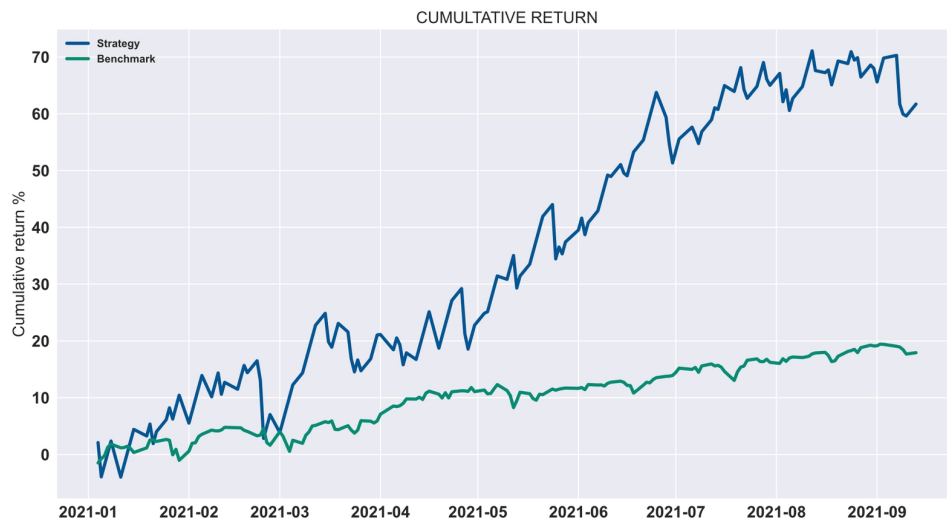
Así pues, supongamos que trabajamos con nuestra estrategia utilizando el 2.1% para el umbral tp y el 9% para el sl. En ese caso, queremos un drawdown máximo del 15% porque no somos amantes del riesgo, y el drawdown máximo es del 10%. Entonces, el apalancamiento de la estrategia es $15/10 = 1.5$.

Tenemos todos los hiperparámetros financieros de la estrategia. Un umbral de take profit al 2.1%, un umbral de stop loss al 9%, y un apalancamiento óptimo al 1.5.

Figura 16.6: Backtest de la cartera con apalancamiento, sl y tp

Beta: -0.174 Alpha: 93.37 % Sharpe: 2.178 Sortino: 2.529

VaR: 5.95 %. cVaR: 20.88 % VaR/cVaR: 3.508 drawdown: 13.32 %



¿Cómo mejorar el modelo?

- Utiliza ticks para realizar el backtest y encontrar mejores combinaciones entre la take profit y el umbral de stop loss.
- Utiliza ticks para calcular el diferencial en el tiempo y no a un día para seleccionar el activo porque no es representativo.

- Utiliza ticks para trabajar con stop loss posteriores.
- Intenta utilizar el bagging de activos en un algoritmo de aprendizaje profundo.
- Prueba otro criterio distinto del diferencial para seleccionar los activos.
- Ajusta el apalancamiento en función del cálculo de los intereses
- Stop loss dinámico para optimizar la ganancia

Aplicación Trading en vivo y Screener

Este código se basa en la clase creada en el anexo: MetaTrader.

Código 16.6: Aplicación Trading / Project

```
from MT5 import *
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
import time
from sklearn.ensemble import VotingClassifier
import pickle
from joblib import dump, load
import os
from sklearn.preprocessing import StandardScaler

path = ""
```

Ex: C:/Desktop/Python_for_finance_and_algorithmic_trading/ChapterN/

```
def create_model_weights(symbol):
```

```
    """ Pesos para la regresión lineal sobre el cambio porcentual """
```

```
    # Importar los datos
```

```
    data = MT5.get_data(symbol, 3500)[["close"]].pct_change(1)
```

```
    # Crear una nueva variable
```

```
    data.columns = ["returns"]
```

```
    # Ingeniería de características
```

```
    data["returns t-1"] = data[["returns"]].shift(1)
```

```
    # Media de los rendimientos
```

```
    data["mean returns 15"] = data[["returns"]].rolling(15).mean().  
                                shift(1)
```

```
    data["mean returns 60"] = data[["returns"]].rolling(60).mean().  
                                shift(1)
```

```
    # Volatilidad de los rendimientos
```

```
    data["volatility returns 15"] = data[["returns"]].rolling(15).std().  
                                shift(1)
```

```
    data["volatility returns 60"] = data[["returns"]].rolling(60).std().  
                                shift(1)
```

```
    # Dividir los datos
```

```
    data = data.dropna()
```

```
    split = int(0.80 * len(data))
```

```
    # Creación del conjunto de entrenamiento
```

```
    X_train = data[["returns t-1", "mean returns 15", "mean returns 60",  
                    "volatility returns 15", "volatility returns 60"]].  
                iloc[:split]
```

```
    y_train = np.round(data[["returns"]].iloc[:split] + 0.5)
```

```

sc = StandardScaler()
X_train = sc.fit_transform(X_train)

# Creación del modelo
tree = DecisionTreeClassifier(max_depth = 6)
svr = SVC()
lin = LogisticRegression()

alg = VotingClassifier(
    estimators=[('lr', lin), ('tree', tree), ('svr', svr)])

# Ajustar el modelo
alg.fit(X_train, y_train)

# Guardar el modelo
alg_var = pickle.dumps(alg)
alg_pickel = pickle.loads(alg_var)

dump(alg_pickel, os.path.join(path, f"Models/{symbol}_reg.joblib"))

def vot_cla_sig(symbol):
    """
    Función para predecir los valores de mañana usando un modelo ARIMA
    """

    # Crear los pesos si no hay en la carpeta
    try:
        alg = load(os.path.join(path, f"Models/{symbol}_reg.joblib"))
    except:
        create_model_weights(symbol)
        alg = load(os.path.join(path, f"Models/{symbol}_reg.joblib"))

    # Tomar el último porcentaje de cambio
    data = MT5.get_data(symbol, 3500)[["close"]].pct_change(1)

```

```
# Crear una nueva variable
```

```
data.columns = ["returns"]
```

```
# Ingeniería de características
```

```
data["returns t-1"] = data[["returns"]].shift(1)
```

```
# Media de los rendimientos
```

```
data["mean returns 15"] = data[["returns"]].rolling(15).mean().  
    shift(1)
```

```
data["mean returns 60"] = data[["returns"]].rolling(60).mean().  
    shift(1)
```

```
# Volatilidad de los rendimientos
```

```
data["volatility returns 15"] = data[["returns"]].rolling(15).std().  
    shift(1)
```

```
data["volatility returns 60"] = data[["returns"]].rolling(60).std().  
    shift(1)
```

```
X = data[["returns t-1", "mean returns 15", "mean returns 60",  
    "volatility returns 15", "volatility returns 60"]].  
    iloc[-1:,:].values
```

```
# Encontrar la señal
```

```
prediction = alg.predict(X)
```

```
prediction = np.where(prediction == 0, -1, 1)
```

```
buy = prediction[0] > 0
```

```
sell = not buy
```

```
return buy, sell
```

```
# True = Trading en vivo y False = Screener
```

```
live = True
```

```

if live:

    current_account_info = mt5.account_info()

    print("-----")
    print("Date: ", datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
    print(f"Balance: {current_account_info.balance} USD, \t"
          f"Equity: {current_account_info.equity} USD, \t"
          f"Profit: {current_account_info.profit} USD")
    print("-----")

info_order = {
    "RUSSEL 2000": ["US2000", 1.1],
    "Bitcoin": ["Bitcoin", 0.1],
    "Nasdaq 100": ["NAS100", 0.3]
}

start = datetime.now().strftime("%H:%M:%S")

while True:

    # Verificación for launch
    if datetime.now().weekday() not in (5,3):
        is_time = datetime.now().strftime("%H:%M:%S") == start
        # "23:59:59"
    else:
        is_time = False

    # Lanzar el algoritmo
    if is_time:

        # Abrir las transacciones
        for asset in info_order.keys():

            # Inicializar los inputs
            symbol = info_order[asset][0]
            lot = info_order[asset][1]

```

```
# Crear las señales

buy, sell = vot_cla_sig(symbol)

# Ejecutar el algoritmo

if live:

    MT5.run(symbol, buy, sell,lot)

else:

    print(f"Symbol: {symbol}\n")
        f"Buy: {buy}\n"
        f"Sell: {sell}")

time.sleep(1)
```

Capítulo 17: De la nada a trading en vivo

En este capítulo se analiza el proceso de creación y puesta en producción de una estrategia de trading, la gestión de una estrategia de trading en vivo y la combinación de estrategias entre ellas.

17.1 Directrices para la creación de estrategias de trading

En esta sección se recapitulará en primer lugar todo el proceso para crear una estrategia de trading correctamente con el fin de minimizar el riesgo de error. A continuación, hablaremos del plan de trading y del diario de trading.

17.1.1 El plan de trading

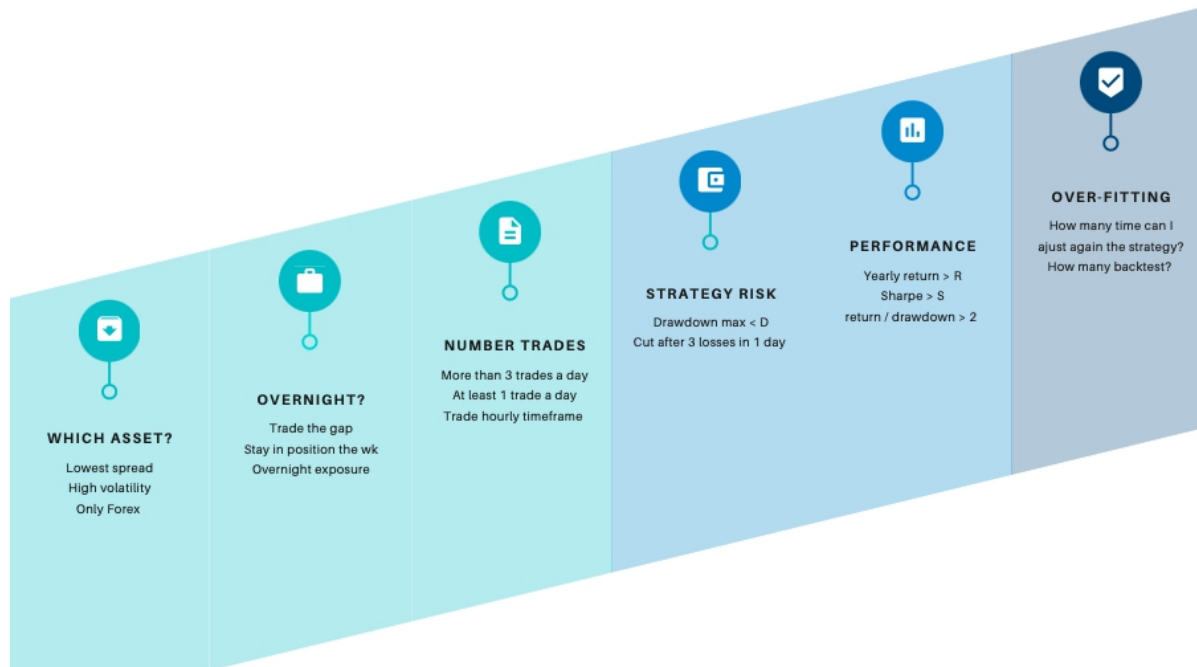
Mucha gente descuida el plan de trading, pero recuerda que mucha gente también pierde dinero. No es una relación perfecta, pero existe una relación negativa entre hacer un plan de trading y ser rentable. ¿Por qué? Porque el plan de trading se crea con la mente fresca antes de empezar a operar manualmente. Por lo tanto, será más fácil seguir el plan si tenemos nuestras pautas a la hora de buscar durante más de 50 horas una estrategia de trading rentable. Porque modificar el plan de trading según las dificultades no es bueno.

Como aquí no estamos haciendo trading manual, el plan de trading será ligeramente diferente, como ya sabemos. Un plan de trading nos dará el punto de entrada y salida con la gestión monetaria a seguir para cada estrategia. Sin embargo, si hemos automatizado esto, no necesitamos comprobarlo para cada operación. Entonces, en trading algorítmico, ¿cómo es un plan de trading?

Tenemos que poner todas las características de la estrategia en el mismo lugar, es decir, el rendimiento, el riesgo, cuántas operaciones queremos hacer (mínimo una operación y como máximo tres

operaciones en un día), qué plazo queremos utilizar (diario, horario, semanal), con qué activo queremos operar (activo con spread bajo, activo volátil), con qué mercado queremos operar (forex, acciones, cripto), si queremos estar expuestos durante la noche, etc.

Figura 17.1: Componente del plan de trading



Este es un ejemplo de las diferentes cosas que podemos incorporar a nuestro plan de trading; no es una lista exhaustiva. Siéntete libre de crear la tuya propia, según tus intentos.

Para concluir, el plan de trading nos permitirá planificar qué tipo de estrategia queremos para comenzar tranquilamente el proceso de investigación.



Tenemos que pensar en todas las características principales de nuestra estrategia antes de hacer la búsqueda. Es importante no cambiar el plan porque no encontremos lo que queremos; ¡sigue buscando! Encontrar una estrategia de trading exige, de media, 100 intentos.

17.1.2 El proceso de creación de una estrategia de trading

Crear una estrategia de trading requiere disciplina, creatividad y práctica. Hay que seguir varios pasos:

1. Crear un **plan de trading** que acepte o rechace una estrategia. Créalo siempre antes de hacer nada, así no tendremos que ajustar el intento en el futuro si la parte de codificación es demasiado dura.
2. **Importar los datos** parece fácil, pero es crucial; recuerda la cita: "basura entra, basura sale". Por tanto, cuidado con la fuente que utilizamos e importar siempre la mayor cantidad de datos posible del broker.
3. **La parte de preprocesamiento** contiene las características y la ingeniería de objetivo, la transformación como la normalización y el ACP. Ayudará al algoritmo a comprender mejor la relación entre los datos.
4. **Modelar** el comportamiento del objetivo utilizando algoritmos de machine learning o deep learning en nuestro caso o condiciones simples si utilizamos solo análisis técnico o price action.
5. **Backtestear** la estrategia dos veces como máximo para evitar sobreajustar los resultados y perder nuestro capital en el futuro.
6. Aplicar la predicción según el plan de trading durante la incubación (condiciones de entrada y salida, take-profit, stop-loss).
7. Comprueba los resultados de la incubación y compáralos con los resultados del backtest para ver si la distribución es similar (más detalles en la sección siguiente).
8. Repetir este proceso para cada estrategia.

Como vemos, el proceso parece sencillo, pero nos hemos tomado todo el libro para explicarlo. Así que, por favor, no subestimes la dificultad de este proceso; necesitaremos al menos 100 intentos para encontrar una buena estrategia (de media). No te rindas y se estable durante el trabajo; encontrarás estrategias rentables, pero requiere mucho tiempo.

17.1.3 Diario de trading

En el trading discrecional (manual), el diario de trading contiene información sobre cada operación que va a pasar en el mercado: entrada y salida, toma de beneficios, stop-loss, tamaño de la apuesta, gestión del riesgo, etc.

El objetivo es tener un diario de operaciones (la práctica) lo más parecido posible al plan de operaciones (la teoría). Sin embargo, cuando trabajemos en un proyecto de trading algorítmico, adoptaremos un enfoque diferente. ¿Por qué? Porque todos los parámetros se establecen con el algo, no pueden cambiar. Entonces, ¿qué escribiremos en nuestro diario de operaciones?

Cuando trabajamos en trading algorítmico, también necesitamos un diario de trading, pero para varias cosas más:

- Analiza la operación manualmente y escribe lo que piensas sobre ella: puntos positivos y negativos a corregir. He aquí una idea para comprender mejor la utilidad:
 - Stop-loss por encima del soporte demasiadas veces → Definir otra forma de colocar el stop-loss.
 - Señales falsas después de algunas noticias → No operes 1 hora antes de algunas noticias.
 - Comprobar la distribución de la rentabilidad entre la producción y el backtest y ver que la

rentabilidad de la producción es cada vez menor → Actualizar la estrategia o detenerla

- Mantener un historial de los problemas de codificación y cómo resolverlos; es valioso cuando estamos en otros proyectos para resolver nuestros problemas fácilmente.

17.2 No pongas todos los huevos en la misma cesta

Pensábamos que una vez que teníamos una estrategia rentable, el trabajo estaba hecho. No, una vez que tenemos una estrategia, necesitamos crear otras para combinarlas y encontrar una sólida cartera de estrategias de trading.

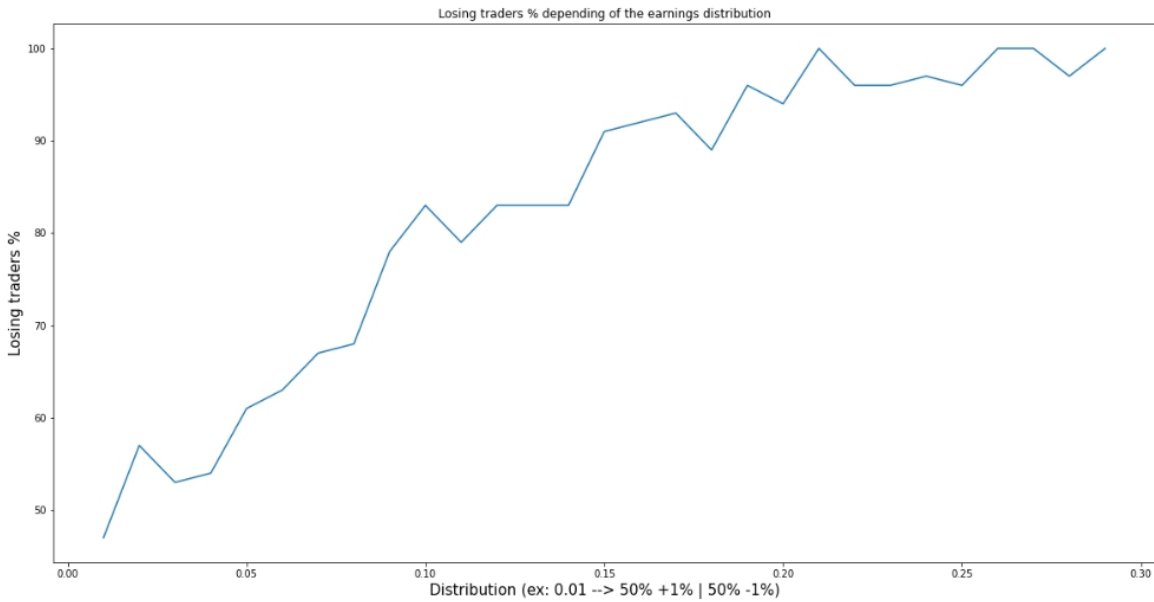
17.2.1 Tamaño de la apuesta

La primera regla es que NUNCA invertiremos todo nuestro capital en una sola estrategia. Es sencillo, pero es esencial. Además, hay varias reglas sencillas que debemos seguir para minimizar el riesgo de perder mucho dinero:

1. Nunca arriesgues más del 1% de tu capital en una operación (adapta el volumen)
2. Dependiendo de la estrategia, para el algo para el día después de 3 pérdidas o páralo para la semana después de 5% de pérdida, etc.

Si utilizamos el interés compuesto, veamos el peligro de tomar posiciones demasiado prominentes. En la figura 17.1, vemos que cuanto más aumenta el tamaño de la posición, más aumenta el riesgo de ser un operador perdedor.

Figura 17.1: Probabilidades de ser un operador perdedor en función del tamaño de la apuesta



Como podemos ver, si tenemos un 50% de posibilidades de ganar un 1% o un 50% de posibilidades de perder un 1%, con una operación al día durante un año, las posibilidades de ser un operador perdedor se sitúan en torno al 52%. Sin embargo, si aumentamos el tamaño de la apuesta del 1% al 30% por operación, vemos que las probabilidades de ser un operador perdedor se sitúan en torno al 100%. Es lógico, porque después de tres operaciones perdedoras consecutivas, sólo nos queda el 10% de nuestro capital, y necesitaremos el 900% de las ganancias para volver a nuestra cantidad inicial.

El punto principal del cálculo del tamaño de la apuesta es que es mejor utilizar el interés compuesto para obtener el beneficio del crecimiento exponencial, pero necesitamos gestionar el riesgo porque cuanto más significativa es la pérdida, más difícil es volver a la cantidad inicial como se muestra en la figura 17.2.

Figura 17.2: Beneficio necesario para recuperar una pérdida del x%

Losse	10%	20%	30%	40%	50%	60%	70%	80%	90%
Necessary profit (simple)	10%	20%	30%	40%	50%	60%	70%	80%	90%
Necessary profit (counpounded)	11%	25%	43%	66%	100%	150%	233%	400%	900%

Como podemos ver, utilizando el interés compuesto (representado por un lote dinámico en el trading), cuanto mayor es la pérdida, más difícil es volver a la cantidad inicial.

17.2.2 ¿Por qué crear una cartera de estrategias comerciales?

Hay muchas razones para crear una cartera de estrategia comercial. La mejor forma de crearla es comprobar activos no correlacionados o, mejor aún, una correlación negativa entre activos.

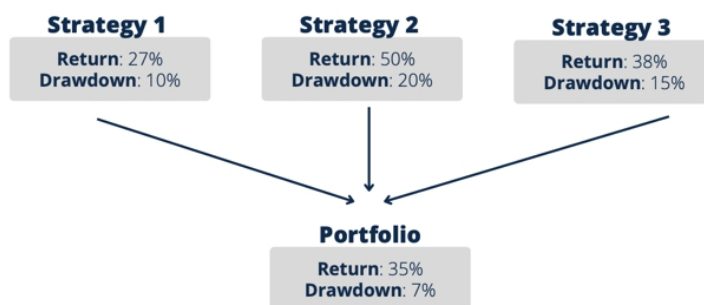


Sobre todo, la creación de una cartera de estrategias de trading es esencial para reducir el riesgo de su inversión: drawdown, VaR, etc.

Principalmente, supongamos que utilizamos el apalancamiento y el interés compuesto. En ese caso, necesitamos crear una cartera para diversificar nuestro riesgo y disminuir el riesgo de posición (lo que es posible aumentando el número de estrategias de trading). Además, la cartera y el tamaño de la apuesta encajan muy bien porque cuantas más estrategias de trading añadamos a la cartera, más podremos reducir el peso de una estrategia en la cartera y crear así una estrategia de inversión sólida.

El objetivo principal de crear una cartera de estrategias de trading es reducir la caída de la cartera manteniendo al mismo tiempo una rentabilidad cómoda. Veamos un ejemplo.

Figura 17.3: Ventaja de la cartera frente a la estrategia individual



Como vemos, combinando la estrategia, la rentabilidad de la cartera es la media entre las tres estrategias. Sin embargo, lo más importante es que el drawdown es muy bajo en comparación con una estrategia individual.

17.3. Proceso de live trading

El proceso de operar en vivo no es la parte más difícil, pero si hay un error en esto, perderemos mucho dinero. Por lo tanto, debemos ser lo más meticulosos posible.

17.3.1. La seguridad ante todo

Cuando ponemos en marcha nuestra estrategia, debemos prestar atención a la seguridad de nuestro capital. Por lo tanto, la mejor manera de hacerlo es colocar algunos valores como el siguiente ejemplo:

1. Detener el algoritmo después de X% de pérdida en un día: Será bueno hacerlo si el algoritmo hace algo mal, como tomar 100 posiciones simultáneamente. Sin embargo, necesitamos hacer un backtest de las pérdidas consecutivas en la distribución de un día para encontrar el mejor valor. Por ejemplo, supongamos que vemos que sólo hay un 1% de posibilidades de tener más de dos operaciones perdedoras al día. En ese caso, podemos poner el siguiente valor: "si tenemos

tres operaciones perdedoras al día, detenemos el algoritmo por ese día".

2. Detener el algoritmo después de X% de pérdida en un mes: a veces, las condiciones del mercado pueden ser difíciles para nuestro algoritmo; tratar de encontrar el mejor momento para detenerlo en el backtest y comprobar el rendimiento teniendo en cuenta que detener el algoritmo en algunas situaciones de mercado.
3. Detener el algoritmo si el drawdown del trading en vivo llega a ser dos veces superior al drawdown en incubación: es muy interesante saber cuándo hay que detener un algoritmo, pero lo discutiremos más adelante en el capítulo.

Estas tres reglas no son exhaustivas; hay muchas reglas similares para gestionar el riesgo y ser un operador rentable.

17.3.2 Fase de incubación

Esta parte es una de las más importantes. En esta subsección vamos a comprobar si nuestra estrategia de trading rentable teórica se puede poner en el comercio en vivo o no. Afortunadamente, es la parte más fácil del proyecto desde el punto de vista de la programación. Sin embargo, también es la más difícil desde el punto de vista psicológico. Veamos todo el proceso en detalle:

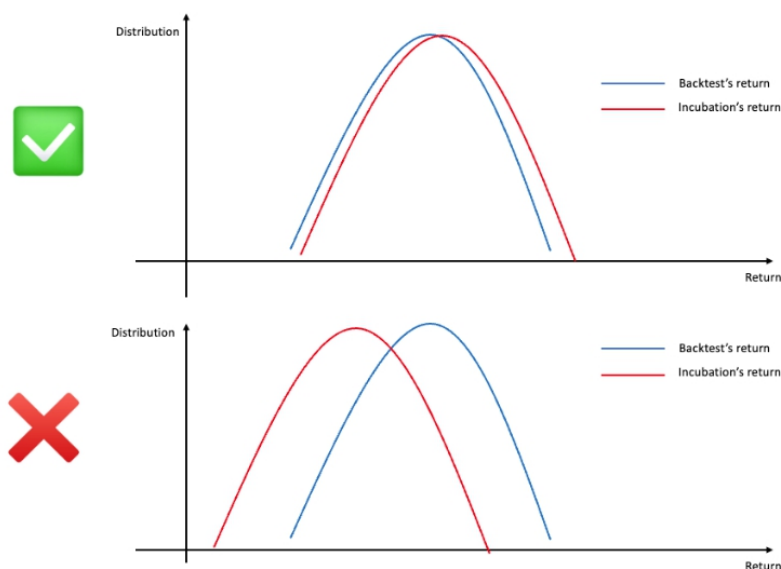
1. Ponemos nuestra estrategia en incubación en una cuenta demo o con un capital muy pequeño en operaciones reales para comprobar cómo funciona la estrategia en operaciones reales. Necesitamos entender si no hemos cometido errores u olvidado algo esencial para la rentabilidad de la estrategia.

2. NECESITAMOS ELEGIR SI MANTENEMOS LA ESTRATEGIA: parece fácil, pero no lo es. Después de un mes mínimo de trading en vivo, tenemos que comprobar si los resultados de la incubación son buenos o no.

¿Cómo saber si los rendimientos de incubación son buenos o no? Hay muchas maneras, pero ésta es la más sencilla. Para ello, podemos comparar la distribución entre los rendimientos del backtest y los rendimientos de la incubación. La mejor forma de hacerlo es comparar las estadísticas con los valores teóricos obtenidos en el backtest sobre los conjuntos de entrenamiento y test. A continuación, calcula el rendimiento en el periodo de incubación utilizando nuestra función de backtest.

Será emocionante entender si no cometemos ningún error porque deberíamos tener algo muy cercano. Veamos la siguiente figura con dos casos para entender cuando tenemos un problema o no.

Figura 17.4: Dos situaciones tras un periodo de incubación



Como podemos ver, las primeras distribuciones son similares, al contrario de lo que ocurre con las segundas, por lo que tenemos que revisar todo nuestro código para encontrar el problema (algo común en la fase de backtest).



Puedes utilizar un test estadístico de Kolmogorov para comparar dos distribuciones si necesitas automatizar el proceso en lugar de hacerlo sólo con el gráfico.

17.3.3 ¿Cuándo dejarlo?

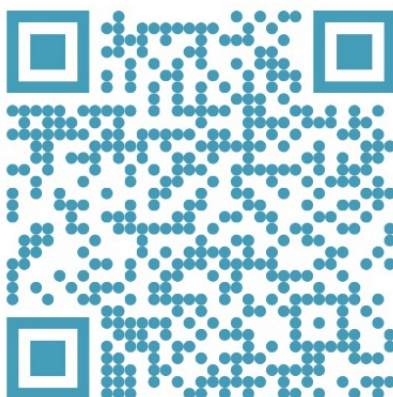
Es la eterna pregunta: ¿cuándo hay que parar el algoritmo? Es muy difícil decirlo y tenemos que adaptar nuestro análisis a las condiciones del mercado. Es una elección subjetiva.

Sin embargo, podemos encontrar algunos consejos para detener el algoritmo, especialmente el cálculo sobre el drawdown. El método se llama el método de gestión del drawdown.

Es sencillo de entender que detenemos el algoritmo cuando el drawdown cae por debajo de un umbral. Por lo tanto, el objetivo cuando hemos detenido el algoritmo es determinar si vamos a activar el algoritmo de nuevo o no. Para ello, no hay reglas. Sin embargo, tenemos que analizar el rendimiento de la estrategia de la A a la Z. Si los rendimientos fueron extraordinarios, pero durante la crisis COVID tenemos un drawdown anormal y el algoritmo se paró, entonces, yo personalmente pondré el algoritmo de nuevo en producción cuando vea que las condiciones del mercado son normales.... En el segundo caso, si tenemos un algoritmo con rentabilidades estables y desde hace 6 meses hasta ahora, el drawdown no para de aumentar, no lo activaré tras el cierre de la gestión del drawdown.

Espero que te haya gustado el libro. No dudes en unirme al curso gratuito para gozar de acceso a la comunidad exclusiva de Trading Algorítmico de Frogames por si tienes preguntas

sobre el libro o ver las preguntas de otros traders. ¡Te deseo una fantástica continuación!



Anexo: Interés compuesto frente a interés simple

En este artículo aprenderás la diferencia entre la estrategia de interés compuesto y la estrategia de interés simple. Explicaremos el porqué de la diferencia entre estos tipos de cálculo. A continuación, veremos cómo calcular los rendimientos acumulados utilizando estos métodos.

¿Cuál es la diferencia entre los métodos simple y compuesto?

La estrategia de interés simple es una estrategia de lote fijo. Significa que cada día invertiremos un capital C . Tenemos un capital $C \cdot r$. Si simulamos este comportamiento a lo largo de los días, tenemos la siguiente ecuación:

$$\Pi = C * r_1 + C * r_2 + \dots + C * r_n$$

$$\Pi = C * (r_1 + r_2 + \dots + r_n)$$

$$\Pi = C * \sum_{i=1}^n r_i$$

Donde Π es el beneficio de la estrategia, C es el capital invertido y r_i es el rendimiento en porcentaje del día i .

Sin embargo, el interés compuesto funciona con un coeficiente multiplicador global. Si tenemos un capital inicial de C , el día 1, tenemos $C * (1 + r_1)$ y el día 2, $C * (1 + r_1) * (1 + r_2)$. De este modo, el día n tenemos $C * (1 + r_1) * \dots * (1 + r_n)$. Con esta estrategia, se invertirá todo el capital en lugar del interés simple, que invierte constantemente el mismo capital:

$$\Pi = (C * (1 + r_1) * (1 + r_2) * \dots * (1 + r_n)) - C$$

$$\Pi = \left(\prod_{i=1}^n (1 + r_i) \right) * C - C$$

$$\Pi = C * \left(\prod_{i=1}^n (1 + r_i) - 1 \right)$$

Donde Π es el beneficio de la estrategia, C es el capital invertido y r_i es el rendimiento en porcentaje del día i .

Cómo calcular el interés simple y compuesto

Para calcular la rentabilidad acumulada de una estrategia de interés simple, necesitamos calcular la rentabilidad acumulada. Para ello, podemos utilizar la función `cumsum()` de `numpy`.

Código: Rentabilidad acumulada según el método del interés simple

```
cumulative_return = np.cumsum(returns)
```

Sin embargo, para calcular la rentabilidad acumulada de una estrategia de interés compuesto, es necesario utilizar el producto del coeficiente multiplicador. Para ello, utilizaremos la función `cumprod()` de numpy.

Código: Rentabilidad acumulada según el método del interés compuesto

```
cumulative_return = np.cumprod(1+returns) - 1
```

Anexo: Guardar y cargar modelos scikit-learn y Tensorflow

En este artículo, aprenderemos cómo guardar y cargar modelos scikit-learn y TensorFlow. Comenzaremos guardando un modelo scikit-learn. Para ello, utilizaremos la librería pickle y joblib utilizando el siguiente código.

Código: Guardar y cargar el modelo scikit-learn

```
from sklearn.svm import SVC
import pickle
from joblib import dump, load

# Guardar
svc = SVC()
alg_pickle = pickle.dumps(svc)
dump(alg_pickle, "svc.joblib")

# Cargar
alg = load("voting.joblib")
```

Para guardar modelos con TensorFlow, necesitamos usar el comando de TensorFlow.

Código: Cómo guardar y cargar modelos TensorFlow

```
def create_model():
    # Inicialización del modelo secuencial
    classifier = Sequential()

    # Añadir la capa densa y de dropout
    for _ in range(nb_hidden_layer):
        classifier.add(Dense(75, input_shape = (X_train.shape[1],),
```

```
activation = "relu"))
```

```
# Añadir la capa densa
```

```
classifier.add(Dense(1, activation = "sigmoid"))
```

```
# Compilación del modelo
```

```
classifier.compile(loss = "binary_crossentropy", optimizer = "adam")
```

```
return classifier
```

```
# Guardar los pesos
```

```
classifier.save_weights('classifier')
```

```
# Crear una nueva instancia del modelo
```

```
model = create_model()
```

```
# Restablecer los pesos
```

```
model.load_weights('classifier')
```


Anexo: Clase MetaTrader

Para más información sobre la configuración del dispositivo MetaTrader, consulta el archivo README.md del repositorio Github (enlace disponible en el capítulo 1).

```
import warnings
from datetime import datetime
import pandas as pd
import MetaTrader5 as mt5
warnings.filterwarnings("ignore")
mt5.initialize()

class MT5:
    def get_data(symbol, n, timeframe = mt5.TIMEFRAME_D1):
        """ Función para importar los datos del símbolo elegido """

        # Inicializar la conexión si no existe
        mt5.initialize()

        # Extracto de la fecha actual
        utc_from = datetime.now()

        # Importar los datos en una tupla
        rates = mt5.copy_rates_from(symbol, timeframe, utc_from, n)

        # De tupla a dataframe
        rates_frame = pd.DataFrame(rates)

        # Convertir el tiempo en segundos al formato datetime
        rates_frame['time'] = pd.to_datetime(rates_frame['time'],
                                             unit = 's')

        # Convertir la columna "hora" en el formato correcto
```

```

rates_frame['time'] = pd.to_datetime(rates_frame['time'],
                                     format = '%Y-%m-%d')

# Establecer la columna tiempo como índice del dataframe
rates_frame = rates_frame.set_index('time')

return rates_frame

def orders(symbol, lot, buy = True, id_position = None):
    """ Enviar las órdenes """

    # Inicializar la conexión si no existe
    if mt5.initialize() == False:
        mt5.initialize()

    # Modo de orden de llenado (hay que probar 0, 1 o 2
    # porque depende del broker)
    i = 1

    # Precio de compra
    ask_price = mt5.symbol_info_tick(symbol).ask

    # Precio de oferta
    bid_price = mt5.symbol_info_tick(symbol).bid

    # Tomar el punto del activo
    point = mt5.symbol_info(symbol).point

    deviation = 20 # mt5.getSlippage(symbol)
    #***** Abrir una operación *****

    if id_position == None:

        # Parámetros de orden de compra
        if buy:
            type_trade = mt5.ORDER_TYPE_BUY

```

```
sl = ask_price - 100 * point
tp = ask_price + 100 * point
price = ask_price
```

```
# Parámetros de orden de venta
```

```
else:
```

```
type_trade = mt5.ORDER_TYPE_SELL
sl = bid_price + 100 * point
tp = bid_price - 100 * point
price = bid_price
```

```
# Abrir la operación
```

```
request = {
    "action": mt5.TRADE_ACTION_DEAL,
    "symbol": symbol,
    "volume": lot,
    "type": type_trade,
    "price": price,
    "deviation": deviation,
    "sl": sl,
    "tp": tp,
    "magic": 234000,
    "comment": "python script order",
    "type_time": mt5.ORDER_TIME_GTC,
    "type_filling": i
}
```

```
# Enviar una solicitud de trading
```

```
result = mt5.order_send(request)
result_comment = result.comment
```

```
***** Cerrar la operación *****
```

```
else:
```

```
# Parámetros de orden de compra
if buy:
    type_trade = mt5.ORDER_TYPE_SELL
    price = bid_price

# Parámetros de orden de venta
else:
    type_trade = mt5.ORDER_TYPE_BUY
    price = ask_price

# Cerrar la operación
request = {
    "action": mt5.TRADE_ACTION_DEAL,
    "symbol": symbol,
    "volume": lot,
    "type": type_trade,
    "position": id_position,
    "price": price,
    "deviation": deviation,
    "magic": 234000,
    "comment": "python script order",
    "type_time": mt5.ORDER_TIME_GTC,
    "type_filling": i
}

# Enviar una solicitud de trading
result = mt5.order_send(request)
result_comment = result.comment
return result_comment
```

```

def resume():
    """ Devuelve las posiciones actuales. Posición = 0 --> Comprar """
    # Inicializar la conexión si no existe
    mt5.initialize()

    # Definir el nombre de las columnas que vamos a crear
    colonnes = ["ticket", "position", "symbol", "volume"]

    # Ir a las operaciones abiertas actuales
    current = mt5.positions_get()

    # Crear un dataframe vacío
    summary = pd.DataFrame()

    # Bucle para añadir cada fila al dataframe
    # (Puede mejorarse utilizando de lista de listas)
    for element in current:
        element_pandas = pd.DataFrame([element.ticket,
                                       element.type,
                                       element.symbol,
                                       element.volume],
                                       index = colonnes).transpose()
        summary = pd.concat((summary, element_pandas), axis = 0)

    return summary

def run(symbol, long, short, lot):

    # Inicializar la conexión si no existe
    if mt5.initialize() == False:

```

```

mt5.initialize()

# Elegir tu símbolo
print("-----")
print("Date: ", datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
print("SYMBOL:", symbol)

# Inicializar el dispositivo
current_open_positions = MT5.resume()

# Comprar o ir en corto
print(f"BUY: {long} \t SHORT: {short}")

""" Cerrar la operación eventualmente """
# Operación de tipo extracción
try:
    position = current_open_positions.
        loc[current_open_positions["symbol"] ==
            symbol].values[0][1]

    identifier = current_open_positions.
        loc[current_open_positions["symbol"] ==
            symbol].values[0][0]
except:
    position = None
    identifier = None

print(f"POSITION: {position} \t ID: {identifier}")

# Cerrar las operaciones
if long == True and position == 0:
    long = False

```

```

elif long == False and position == 0:
    res = MT5.orders(symbol, lot, buy = True,
                      id_position = identifier)
    print(f"CLOSE LONG TRADE: {res}")

elif short == True and position == 1:
    short = False

elif short == False and position == 1:
    res = MT5.orders(symbol, lot, buy = False,
                      id_position = identifier)
    print(f"CLOSE SHORT TRADE: {res}")

else:
    pass

""" Comprar o ir en corto """
if long == True:
    res = MT5.orders(symbol, lot, buy = True, id_position = None)
    print(f"OPEN LONG TRADE: {res}")

if short == True:
    res = MT5.orders(symbol, lot, buy = False, id_position = None)
    print(f"OPEN SHORT TRADE: {res}")

print("-----")

def close_all_night():
    result = MT5.resume()
    for i in range(len(result)):
        before = mt5.account_info().balance
        row = result.iloc[0+i:1+i, :]
        if row["position"][0] == 0:

```

```
res = MT5.orders(row["symbol"][0],
                 row["volume"][0],
                 buy = True,
                 id_position = row["ticket"][0])

else:
    res = MT5.orders(row["symbol"][0],
                    row["volume"][0],
                    buy = False,
                    id_position = row["ticket"][0])
```


Lecturas adicionales

Capítulo 3

- **Markowitz's "Portfolio Selection ": A Fifty-Year Retrospective, The University of Chicago Press**
<https://www.jstor.org/stable/269777>
- **Portfolio management: mean-variance analysis in the US asset market, Narela (Bajram) Spaseski,**
https://www.researchgate.net/publication/264423979_PORTFOLIO_MANAGEMENT_MEAN-VARIANCE_ANALYSIS_IN_THE_US_ASSET_MARKET
- **Mean-variance-skewness-kurtosis based portfolio optimization, KingKeung Lai, Shouyang Wand, Lean**
<https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.898.991&rep=rep1&type=pdf>

Capítulo 4

- **Tactical Asset Allocation (TAA), ADAM BARONE.**
<https://www.investopedia.com/terms/t/tacticalassetallocation.asp>

Capítulo 5

- **Optimization of conditional value-at-risk, R. Tyrrell Rockafellar.**

https://www.ise.ufl.edu/uryasev/files/2011/11/CVaR1_JOR.pdf

Capítulo 7

- **Stationarity and differencing.**
<https://people.duke.edu/~rnau/411diff.htm>

- **Cointegration, Niti Gupta.**
<https://www.wallstreetmojo.com/cointegration/>
- **Pairs Trading, James Chen.**
<https://www.investopedia.com/terms/p/pairstrade.asp>

Capítulo 8

- **What Is a Time Series?, Adam Hayes.**
<https://www.investopedia.com/terms/t/timeseries.asp>
- **Autoregressive–moving-average model, Wikipedia.**
https://en.wikipedia.org/wiki/Autoregressive–moving-average_model

Capítulo 9

- **Linear Regression for Machine Learning, Jason Brownlee.** <https://machinelearningmastery.com/linear-regression-for-machine-learning/>

Capítulo 11

- **Support-vector machine, Wikipedia.**
https://en.wikipedia.org/wiki/Support-vector_machine

Capítulo 12

- **Decision Trees in Machine Learning, Prashant Gupta.** <https://towardsdatascience.com/decision-trees-in-machine-learning-641b9c4e8052>
- **Ensemble Methods in Machine Learning: What are They and Why Use Them? Evan Lutins.**
<https://towardsdatascience.com/ensemble-methods-in-machine-learning-what-are-they-and-why-use-them-68ec3f9fef5f>

Capítulo 13

- **Stochastic Gradient Descent — Clearly Explained !!**, Aishwarya V Srinivasan.
<https://towardsdatascience.com/stochastic-gradient-descent-clearly-explained-53d239905d31>

Capítulo 14

- **Recurrent Neural Networks cheatsheet**, Afshine Amidi and Shervine Amidi.
<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks>

Capítulo 15

- **Introduction to Convolutional Neural Networks (CNN)**, Manav Mandal.
<https://www.analyticsvidhya.com/blog/2021/05/convolutional-neural-networks-cnn/>

[1] **Lectura adicional:** Markowitz's "Portfolio Selection ": A Fifty-Year Retrospective

[2] **Lectura adicional:** Tactical Asset Allocation (TAA), Adam BARONE

[3] **Lectura adicional:**

https://en.wikipedia.org/wiki/Capital_asset_pricing_model

[4] **Lectura adicional:** Stationarity and differencing

[5] **Lectura adicional :** Cointegration, Niti Gupta

[6] **Lectura adicional:** Pairs Trading, James Chen

[7] **Lectura adicional:** Autoregressive–moving-average model, Wikipedia